

代码汇总_PSQ_20260428

目录

一、数据资产信息披露水平整理代码	1
二、企业上市年限计算代码	12
三、是否亏损虚拟变量代码	17
四、营业收入增长率计算代码	23
五、股权性质虚拟变量生成代码	32
六、不同期限借款代码	37
七、新增借款计算代码（按贷款事件统计）	50
八、新增不同种类借款代码（按贷款事件统计）	60
九、银行竞争程度 HHI 代码	73
十、上市公司高管数字背景分析 Python 代码	120
十一、涉诉金额、涉诉次数统计 Python 代码	133
十二、违规处理统计 Python 代码	138
十三、数字技术相关专利数统计 Python 代码	148
十四、数字资产强度计算 Python 代码	156
十六、债务资本成本计算代码	163
十七、现金持有计算代码	168
十八、固定资产净值占比计算代码	173
十九、审计费用率计算代码	179
二十、自由现金流计算代码	185
二十一、新增借款/总资产计算代码	190
二十二、经营效率计算代码	196
二十三、企业相对融资规模计算代码	202
二十四、并表代码	206
二十五、新增机制分析并表代码 20260303	232

一、数据资产信息披露水平整理代码

操作汇总：1、剔除重复行；2、剔除证券业、ST

http://localhost:8888/notebooks/Desktop/Python/%E6%AF%95%E4%B8%9A%E8%AE%BA%E6%96%87_%E6%95%B0%E5%AD%97%E8%B5%84%E4%BA%A7%E4%BF%A1%E6%81%AF%E6%8A%AB%E9%9C%B2%E6%B0%B4%E5%B9%B3_20260128%E4%BF%AE%E6%94%B9.ipynb

注：1、所有代码运行前需把 excel 文件放到桌面上；2、代码中的路径只适用于 mac 系统，windows 系统需要对路径代码进行修改；3、需要把桌面上的文件名称修改至与代码中完全相同。

kimi 指令：

【去重】

我现在需要你帮我设计一段 python 代码：

一、现有数据：

一张 excel 表格，第一行为英文列标题，第二行为中文列标题。中文列标题从左到右依次为：股票代码、会计年度、文本类型、数据资产关键词、词频统计、出现该关键词的句子数、总关键词频、年报总词频、数据资产关键词占年报总词频百分比、数据资产信息披露水平。

二、我的需求

1、任务目标：剔除重复行，每个会计年度每家上市公司只保留一行数据（去重）

2、逻辑：已知原始表格每个年份每家公司可能出现多行数据，原因为原始数据表格对包括“大数据”、“海量数据”、“数据分析体系”等在内的多个数据资产关键词的词频均进行了统计，但是，原始表格中在“总关键词频”列已经对这家公司这个年份的所有数据资产关键词的词频进行了求和，并进一步得到数据资产关键词占年报总词频百分比及数据资产信息披露水平所在列的数据。综上，每家公司同一会计年度的总关键词频、年报总词频、数据资产关键词占年报总词频百分比、数据资产信息披露水平列的数据是唯一的。我需要每一家上市公司每个会计年度的总关键词频、年报总词频、数据资产关键词占年报总词频百分比、数据资产信息披露水平列的数据，只需要对原始表格除列标题行的每一行进行遍历，将股票代码和会计年度组成元组，作为键值对的键，将同一行总关键词频、年报总词频、数据资产关键词占年报总词频百分比、数据资产信息披露水平组成列表，作为键值对的值，并且之后遇到相同的元组（键）时，直接跳过。最终就能得到每家上市公司每个会计年度总关键词频、年报总词频、数据资产关键词占年报总词频百分比、数据资产信息披露水平的数据的唯一值。

3、具体步骤：

（1）读取位于 mac 桌面命名为 数据资产信息披露水平_合并.xlsx 的文件。注意，由于股票代码存在左侧数据为 0 的情况，所以一定要始终以字符串的格式储存，保证长度一直为 6 位。

（2）新建字典 1，对步骤（1）读取的 excel 进行遍历，将每一行的“股票代码”和“会计年度”组成一个元组，作为一个键值对的键，并将该元组同一行的总关键词频、年报总词频、数据资产关键词占年报总词频百分比、数据资产信息披露水平组合成一个列表，作为该键值对的值。

(3) 后续遍历过程中遇到相同的股票代码和会计年度, 认为是重复的数据, 直接跳过该行, 对下一行进行遍历, 以此保证每家公司每个会计年度最终只保留一行数据。

(4) 输出结果, 以 excel 的形式保存到 mac 的桌面, 命名为: 数据资产信息披露水平_对应的时间戳.xlsx。其中, 这个 excel 的从左到右每一列的列标题依次为: 股票代码、会计年度、总关键词频、年报总词频、数据资产关键词占年报总词频百分比、数据资产信息披露水平。并且在表格的最上方添加列标题行。

注意: (1) 鉴于使用循环会导致代码运行效率低下, 你可以给出可行的替代方案优化程序;
(2) 我希望这个代码可以实时显示运行进度。

```
#去重
import pandas as pd
from datetime import datetime
from pathlib import Path
from tqdm import tqdm

def process_data_asset_disclosure():
    """
    处理数据资产信息披露水平数据, 按公司和会计年度去重
    """
    start_time = datetime.now()
    print("=" * 60)

    print(f"开始处理数据: {start_time.strftime('%Y-%m-%d %H:%M:%S')}") # 已修复
    print("=" * 60)

    # 1. 文件路径设置
    desktop_path = Path.home() / "Desktop"
    input_file = desktop_path / "数据资产信息披露水平_合并.xlsx"

    if not input_file.exists():
        print(f"  错误: 文件不存在! \n  请确认文件路径: {input_file}")
        return None

    print(f"  正在读取文件: {input_file.name}")

    # 2. 读取 Excel - 使用更稳健的方式
    try:
        # 先读取检查列名
        df_check = pd.read_excel(input_file, header=1, nrows=0, engine='openpyxl')
        print(f"  检测到列名: {list(df_check.columns)}")
```

```

# 正式读取数据
df = pd.read_excel(
    input_file,
    header=1, # 使用第二行作为列标题
    dtype={'股票代码': str}, # 强制读取为字符串
    engine='openpyxl'
)

print(f"    成功读取 {len(df)} 行数据")

except Exception as e:

    print(f"    读取文件时出错: {e}")

    return None

# 3. 数据清洗和验证

print("\n    正在进行数据验证和清洗...")

# 检查关键列是否存在
required_columns = ['股票代码', '会计年度', '总关键词频', '年报总词频',
                    '数据资产关键词占年报总词频百分比', '数据资产信息披露水平']

missing_cols = [col for col in required_columns if col not in df.columns]
if missing_cols:

    print(f"    缺少必要的列: {missing_cols}")

    print(f"    实际列名: {list(df.columns)}")

    return None

# 处理股票代码 - 更健壮的清洗
df['股票代码'] = df['股票代码'].astype(str).str.strip() # 去除空格
df['股票代码'] = df['股票代码'].str.zfill(6) # 补齐 6 位

# 检查是否有无效的股票代码
invalid_codes = df[~df['股票代码'].str.match(r'^d{6}$')]['股票代码'].unique()
if len(invalid_codes) > 0:

    print(f"    发现 {len(invalid_codes)} 个无效的股票代码: {invalid_codes[:5]}...")

    df = df[df['股票代码'].str.match(r'^d{6}$')] # 过滤无效代码

# 4. 去重处理 (核心逻辑)

```

```

print("\n    正在执行去重操作...")

# 先统计去重前的组合数量
total_combinations = df[['股票代码', '会计年度']].drop_duplicates().shape[0]
print(f"    去重前组合数: {total_combinations}")

# 使用 drop_duplicates 去重
unique_df = df.drop_duplicates(
    subset=['股票代码', '会计年度'],
    keep='first'
).copy()

print(f"    去重后组合数: {len(unique_df)}")
print(f"    共剔除 {total_combinations - len(unique_df)} 个重复组合")

# 5. 选择并排序结果
result_df = unique_df[required_columns].copy()

# 按股票代码和会计年度排序
result_df = result_df.sort_values(['股票代码', '会计年度']).reset_index(drop=True)

# 6. 生成带时间戳的输出文件名
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
output_file = desktop_path / f"数据资产信息披露水平_{timestamp}.xlsx"

print(f"\n    正在保存结果到: {output_file.name}")

try:
    # 保存为 Excel 文件
    result_df.to_excel(
        output_file,
        index=False,
        engine='openpyxl'
    )

    print("    保存成功! ")

except Exception as e:

    print(f"    保存文件时出错: {e}")

    return None

```

```

# 7. 统计信息
end_time = datetime.now()
print("\n" + "=" * 60)

print("    处理完成统计:")

print(f"    原始数据行数: {len(df)}")
print(f"    去重后行数: {len(result_df)}")
print(f"    涉及公司数: {result_df['股票代码'].nunique()}")
print(f"    涉及年度数: {result_df['会计年度'].nunique()}")

print(f"    开始时间: {start_time.strftime('%Y-%m-%d %H:%M:%S')}") #    已修复

print(f"    结束时间: {end_time.strftime('%Y-%m-%d %H:%M:%S')}") #    已修复

print(f"    总耗时: {(end_time - start_time).total_seconds():.2f} 秒")
print(f"    输出文件: {output_file.name}")
print("=" * 60)

return result_df

# 执行处理
if __name__ == "__main__":
    # 设置 pandas 显示选项
    pd.set_option('display.max_columns', None)
    pd.set_option('display.width', None)
    pd.set_option('display.max_rows', 10) # 限制显示行数

    # 运行处理函数
    processed_data = process_data_asset_disclosure()

    if processed_data is not None:

        print("\n    数据预览 (前 5 行) :")

        print(processed_data.head())

```

【剔除证券业、ST】20260128 新增

我现在需要你帮我设计一段 python 代码：

两张 excel 表格

第一张为位于 mac 桌面命名为 1 数据资产信息披露水平_20260122_174950.xlsx 的 excel 表格，记录了 2013-2024 年 A 股所有上市公司的数据资产信息披露水平数据。第一行为中文标题，从左到右依次为：股票代码、会计年度、总关键词频、年报总词频、数据资产关键词占

年报总词频百分比、数据资产信息披露水平

第二张为位于 Mac 桌面命名为 CSMAR-剔除证券业、ST 公司的 A 股上市公司列表_报表类型 A.xlsx 的 excel 表格, 记录了 2013-2024 年剔除证券业、ST 的 A 股上市公司股票代码数据。第一行为中文标题, 从左到右依次为: 股票代码、证券简称、会计年度、统计截止日期、报表类型

二、 我的需求

1、任务目标: 对照 CSMAR-剔除证券业、ST 公司的 A 股上市公司列表_报表类型 A.xlsx 的 excel 表格的 (股票代码, 会计年份) 索引, 删除 1 数据资产信息披露水平_20260122_174950.xlsx 的 excel 表格中除了上述索引所在行之外的其他行, 即只保留 1 数据资产信息披露水平_20260122_174950.xlsx 的 excel 表格中和 CSMAR-剔除证券业、ST 公司的 A 股上市公司列表_报表类型 A.xlsx (股票代码, 会计年度) 索引相同的行

2、逻辑: CSMAR-剔除证券业、ST 公司的 A 股上市公司列表_报表类型 A.xlsx 的 excel 表格统计的样本范围是剔除证券业、ST 公司的 A 股上市公司, 而 1 数据资产信息披露水平_20260122_174950.xlsx 的样本范围为所有 A 股上市公司。两张表通过股票代码列和会计年度列产生联系。我现在想要将剔除 1 数据资产信息披露水平_20260122_174950.xlsx 中证券业、ST 公司的所有数据行。实现路径为利用两张表 (股票代码, 会计年度) 的链接, 使 1 数据资产信息披露水平_20260122_174950.xlsx 的 excel 表格样本范围和 CSMAR-剔除证券业、ST 公司的 A 股上市公司列表_报表类型 A.xlsx 的 excel 表格保持一致

3、具体操作:

(1) 读取位于 mac 桌面命名为 1 数据资产信息披露水平_20260122_174950.xlsx 和 CSMAR-剔除证券业、ST 公司的 A 股上市公司列表_报表类型 A.xlsx 的 excel 的文件。注意, 由于股票代码存在左侧数据为 0 的情况, 所以一定要始终以字符串的格式储存, 保证长度一直为 6 位。

(2) 新建一个列表, 对 CSMAR-剔除证券业、ST 公司的 A 股上市公司列表_报表类型 A.xlsx 进行遍历, 将每一行中 (股票代码, 会计年度) 组合为一个元组, 作为列表的一个元素。

(3) 每读取 1 数据资产信息披露水平_20260122_174950.xlsx 的一行数据, 判断该行中 (股票代码, 会计年度) 是否在列表中出现过, 如果出现过, 则保留该行, 如果列表中没有该行中的 (股票代码, 会计年度) 元组, 则认为该行数据对应证券业或 st 公司, 删除 1 数据资产信息披露水平_20260122_174950.xlsx 的对应行

(4) 输出结果, 将字典以 excel 的形式保存到 mac 的桌面, 命名为: 数据资产信息披露水平_对应的时间戳.xlsx。其中, 这个 excel 的从左到右每一列的列标题依次为: 股票代码、会计年度、总关键词频、年报总词频、数据资产关键词占年报总词频百分比、数据资产信息披露水平。并且在表格的最上方添加列标题行。

```
import pandas as pd
import os
from datetime import datetime
import time

# 尝试导入 tqdm, 如果不存在则创建替代类 (确保代码可运行)
try:
    from tqdm import tqdm
```

```

except ImportError:
    class tqdm:
        def __init__(self, iterable=None, desc="", unit="", total=None, **kwargs):
            self.iterable = iterable
            self.desc = desc
            if total:
                print(f"{desc}: 共{total}项...")
        def __iter__(self):
            for item in self.iterable:
                yield item
        def update(self, n=1):
            pass
        def close(self):
            pass

def process_data_asset_disclosure():
    """
    数据资产信息披露水平数据处理 - 剔除证券业、ST 公司
    功能：根据 CSMAR 剔除名单，从全样本中筛选目标公司
    """

    # 配置路径（Mac 桌面）
    desktop_path = os.path.expanduser("~/Desktop")

    file1_path = os.path.join(desktop_path, "1 数据资产信息披露水平_20260122_174950.xlsx")
    file2_path = os.path.join(desktop_path, "CSMAR-剔除证券业、ST 公司的 A 股上市公司列表_报表类型 A.xlsx")

    print("\n 开始数据资产信息披露水平筛选任务...")

    # 验证文件存在性
    if not os.path.exists(file1_path):
        raise FileNotFoundError(f"找不到文件 1: \n{file1_path}\n 请确认文件已放在 Mac 桌面")
    if not os.path.exists(file2_path):
        raise FileNotFoundError(f"找不到文件 2: \n{file2_path}\n 请确认文件已放在 Mac 桌面")

    # 步骤 1：读取 Excel 文件
    print("\n 正在读取数据文件...")

```

```

try:
    # 读取文件 1: 数据资产信息披露水平
    print("    读取: 数据资产信息披露水平表...")
    df1 = pd.read_excel(file1_path, dtype={'股票代码': str})

    # 清洗列名 (去除首尾空格, 防止 Excel 列名有隐藏空格)
    df1.columns = df1.columns.str.strip()

    # 确保股票代码为 6 位字符串 (补齐前导零)
    df1['股票代码'] = df1['股票代码'].astype(str).str.zfill(6)

    print(f"    ✓ 文件 1 加载完成, 共 {len(df1)} 行数据")

    # 读取文件 2: 剔除证券业、ST 的公司列表 (白名单)
    print("    读取: 剔除证券业 ST 公司列表...")
    df2 = pd.read_excel(file2_path, dtype={'股票代码': str})

    # 清洗列名
    df2.columns = df2.columns.str.strip()

    # 确保股票代码为 6 位字符串
    df2['股票代码'] = df2['股票代码'].astype(str).str.zfill(6)

    print(f"    ✓ 文件 2 加载完成, 共 {len(df2)} 行数据")

except Exception as e:
    raise ValueError(f"读取 Excel 文件出错: {str(e)}, 请检查文件格式是否为标准.xlsx")

# 步骤 2: 创建复合索引集合 (股票代码, 会计年度)

print("\n    正在建立索引匹配...")

# 检查必需的列是否存在
required_cols_df1 = ['股票代码', '会计年度']
required_cols_df2 = ['股票代码', '会计年度']

for col in required_cols_df1:
    if col not in df1.columns:
        raise ValueError(f"文件 1 缺少必需的列: '{col}', 请检查列名。现有列: {list(df1.columns)}")

for col in required_cols_df2:
    if col not in df2.columns:
        raise ValueError(f"文件 2 缺少必需的列: '{col}', 请检查列名。现有列: ")

```

```

{list(df2.columns)})")

# 创建白名单集合：基于（股票代码，会计年度）的复合索引
# 使用 set 实现 O(1) 查询效率，大幅提升匹配速度
valid_index_set = set(zip(df2['股票代码'], df2['会计年度']))

print(f"    ✓ 有效索引集创建完成，共 {len(valid_index_set)} 个（股票代码，会计年
度）组合")

# 步骤 3：筛选数据（向量化操作，避免低效循环）

print("\n    正在筛选匹配数据...")

start_time = time.time()

# 为 df1 创建复合键用于匹配
df1['_key'] = list(zip(df1['股票代码'], df1['会计年度']))

# 根据数据量选择处理策略
total_rows = len(df1)
chunk_size = 100000 # 每 10 万行处理一次

if total_rows > chunk_size:
    # 大数据量：分块处理，显示进度条，避免内存溢出
    print(f"    检测到大数据量（{total_rows:,} 行），启用分块处理...")
    filtered_chunks = []

    for start in tqdm(range(0, total_rows, chunk_size), desc="    筛选进度", unit="批次
"):
        end = min(start + chunk_size, total_rows)
        chunk = df1.iloc[start:end]
        mask = chunk['_key'].isin(valid_index_set)
        filtered_chunks.append(chunk[mask])

    # 合并所有筛选后的分块
    df_result = pd.concat(filtered_chunks, ignore_index=True)
else:
    # 小数据量：直接处理（速度更快）
    mask = df1['_key'].isin(valid_index_set)
    df_result = df1[mask].copy()

# 删除辅助列
df_result = df_result.drop(columns=['_key'])

elapsed_time = time.time() - start_time

```

```

removed_count = len(df1) - len(df_result)

print(f"    ✓ 筛选完成：保留 {len(df_result):,} 行，剔除 {removed_count:,} 行（证券
业或 ST 公司） ")

print(f"        处理耗时： {elapsed_time:.2f}秒")

print(f"        剔除比例： {removed_count/len(df1)*100:.1f}%")

# 步骤 4：整理输出格式

print("\n    正在整理输出格式...")

# 定义标准列顺序（根据用户要求）
expected_columns = [
    '股票代码',
    '会计年度',
    '总关键词频',
    '年报总词频',
    '数据资产关键词占年报总词频百分比',
    '数据资产信息披露水平'
]

# 只保留实际存在的列（容错处理）
existing_columns = [col for col in expected_columns if col in df_result.columns]
df_result = df_result[existing_columns]

# 最终确认股票代码格式（6 位字符串）
df_result['股票代码'] = df_result['股票代码'].astype(str).str.zfill(6)

# 确保会计年度为整数（美观）
if '会计年度' in df_result.columns:
    df_result['会计年度'] = df_result['会计年度'].astype(int)

print(f"    ✓ 列顺序整理完成，输出列： {existing_columns}")

# 步骤 5：保存结果到 Excel

print("\n    正在保存结果文件...")

try:
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")

```

```

output_filename = f"数据资产信息披露水平_{timestamp}.xlsx"
output_path = os.path.join(desktop_path, output_filename)

# 使用 openpyxl 引擎保存，确保字符串格式保留
df_result.to_excel(output_path, index=False, engine='openpyxl')

print(f"    ✓ 文件保存成功：{output_path}")

print(f"\n    任务圆满完成！")

print(f"    └─ 原始数据：{len(df1):,} 行")
print(f"    └─ 筛选后：{len(df_result):,} 行")
print(f"    └─ 剔除数据：{removed_count:,} 行（证券业或 ST 公司）")
print(f"    └─ 输出文件：{output_filename}")

print(f"\n    重要提示：")

print(f"    1. 文件已保存至 Mac 桌面")
print(f"    2. 股票代码已保存为 6 位文本格式（如 000001）")
print(f"    3. 若后续用 Python 读取此文件，请务必使用：")
print(f"        df = pd.read_excel('{output_filename}', dtype={{'股票代码': str}})")
print(f"        df['股票代码'] = df['股票代码'].str.zfill(6)")

return df_result

except Exception as e:
    raise IOError(f"保存 Excel 文件失败：{str(e)}，请检查磁盘权限或空间")

# =====
# 运行主程序
# =====
if __name__ == "__main__":
    try:
        result = process_data_asset_disclosure()
    except Exception as e:
        print(f"\n    错误：{str(e)}")
        print("请根据错误提示检查输入文件后重试")

```

二、企业上市年限计算代码

http://localhost:8888/notebooks/Desktop/Python/%E6%AF%95%E4%B8%9A%E8%AE%BA%E6%96%87_%E4%B8%8A%E5%B8%82%E5%B9%B4%E9%99%90%E8%AE%A1%E7%AE%97_20260128%E4%BF%AE%E6%94%B9.ipynb

注：1、所有代码运行前需把 excel 文件放到桌面上；2、代码中的路径只适用于 mac 系统，windows 系统需要对路径代码进行修改；3、需要把桌面上的文件名称修改至与代码中完全相同。

kimi 指令：

我现在需要你帮我设计一段 python 代码：

一、现有数据：

一张 excel 表格，第一行为英文列标题，第二行为中文列标题。中文列标题从左到右依次为：股票代码、股票简称、统计截止日期、上市公司 ID、证券 ID、行业名称、行业代码、首次上市日期、所属省份代码、所属省份、所属城市代码、所属城市、主营业务、上市状态

二、我的需求：

1、任务目标：计算企业上市年限

2、具体步骤：

(1) 读取位于 mac 桌面命名为 企业上市年限_原始数据.xlsx 的文件。，注意，由于股票代码存在左侧数据为 0 的情况，所以一定要始终以字符串的格式储存，保证长度一直为 6 位。

(3) 计算 2013-2024 年各公司，计算公式为：统计截止日期所在年份-首次上市日期所在年份。

(4) 输出结果，以 excel 的形式保存到 mac 的桌面，命名为：企业上市年限_对应的时间戳.xlsx。其中，这个 excel 的从左到右每一列的列标题依次为：股票代码、股票简称、会计年度、统计截止日期、行业名称、行业代码、所属城市、上市年限。并且在表格的最上方添加列标题行。

20260128 最新版代码：

```
import pandas as pd
import numpy as np
import os
from datetime import datetime
import time
```

尝试导入 tqdm，如果不存在则创建替代类

try:

```
    from tqdm import tqdm
```

except ImportError:

```

class tqdm:
    def __init__(self, iterable=None, desc="", **kwargs):
        self.iterable = iterable
        if desc:
            print(f"{desc}...")
    def __iter__(self):
        for item in self.iterable:
            yield item

def calculate_listing_years():
    """
    计算企业上市年限
    功能：读取原始数据，计算上市年限，仅输出指定 8 列
    """

    # 配置路径（Mac 桌面）
    desktop_path = os.path.expanduser("~/Desktop")
    input_file = os.path.join(desktop_path, "企业上市年限_原始数据.xlsx")

    if not os.path.exists(input_file):
        raise FileNotFoundError(f"找不到文件：{input_file}\n 请确认文件已放在 Mac 桌面")

    print("    开始计算企业上市年限...")
    start_time = time.time()

    # 步骤 1：读取 Excel（跳过第一行英文标题，使用第二行中文标题）
    print("\n    正在读取原始数据...")
    df = pd.read_excel(input_file, header=1, dtype={'股票代码': str})
    df.columns = df.columns.str.strip() # 清洗列名
    print(f"    ✓ 成功读取 {len(df):,} 行数据")

    # 步骤 2：格式化股票代码（确保 6 位字符串，保留前导零）
    print("\n    正在格式化数据...")
    df['股票代码'] = df['股票代码'].astype(str).str.zfill(6)

    # 步骤 3：转换日期格式
    df['统计截止日期'] = pd.to_datetime(df['统计截止日期'], errors='coerce')
    df['首次上市日期'] = pd.to_datetime(df['首次上市日期'], errors='coerce')

    print("    ✓ 股票代码与日期格式化完成")

```

```

# 步骤 4: 计算关键字段 (向量化计算, 无循环)

print("\n    正在计算上市年限与会计年度...")

# 计算会计年度 (从统计截止日期提取年份)
df['会计年度'] = df['统计截止日期'].dt.year

# 计算上市年限 (统计截止年份 - 首次上市年份)
df['上市年限'] = df['统计截止日期'].dt.year - df['首次上市日期'].dt.year

# 统计信息
valid_years = df['上市年限'].dropna()

print(f"    ✓ 计算完成, 平均上市年限: {valid_years.mean():.1f} 年")

#
=====
=

# 步骤 5: 整理输出格式 (修改后的第 4 步要求)
#
=====
=

print("\n    正在整理输出格式 (仅保留指定 8 列) ...")

# 确保统计截止日期为字符串格式 (YYYY-MM-DD)
df['统计截止日期'] = df['统计截止日期'].dt.strftime('%Y-%m-%d').replace('NaT', '')

# 确保股票代码为 6 位字符串 (最终检查)
df['股票代码'] = df['股票代码'].astype(str).str.zfill(6)

# 确保会计年度为整数 (无小数位)
df['会计年度'] = df['会计年度'].astype(int)

# 严格按指定顺序选择 8 列 (仅保留这些列)
output_columns = [
    '股票代码',      # 1. 股票代码 (6 位字符串)
    '股票简称',      # 2. 股票简称
    '会计年度',      # 3. 会计年度 (从统计截止日期提取的年份)
    '统计截止日期',  # 4. 统计截止日期 (YYYY-MM-DD 格式)
    '行业名称',      # 5. 行业名称
    '行业代码',      # 6. 行业代码
    '所属城市',      # 7. 所属城市

```

```

        '上市年限'          # 8. 上市年限（计算值）
    ]

    # 检查所需列是否存在（容错处理）
    missing_cols = [col for col in output_columns if col not in df.columns]
    if missing_cols:
        raise ValueError(f'原始数据缺少必需的列：{missing_cols}\n 请检查输入文件的列名')

    # 仅选择指定列并按指定顺序排列
    df = df[output_columns]

    print(f"    ✓ 输出列整理完成，共 {len(df.columns)} 列")

    print(f"    ✓ 列顺序：{output_columns}")

    print(f"    ✓ 数据行数：{len(df)} 行（保留原始行数）")

    #
    =====
    =

    # 步骤 6：保存结果到桌面（带时间戳）
    #
    =====
    =

    print("\n    正在保存结果文件...")

    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    output_path = os.path.join(desktop_path, f'企业上市年限_{timestamp}.xlsx')

    with pd.ExcelWriter(output_path, engine='openpyxl') as writer:
        df.to_excel(writer, index=False, sheet_name='Sheet1')

        # 设置股票代码列为文本格式（防止 Excel 自动转数字丢失前导零）
        worksheet = writer.sheets['Sheet1']
        for row in range(2, worksheet.max_row + 1): # 从第 2 行开始（跳过标题）
            worksheet.cell(row, 1).number_format = '@' # '@'表示文本格式

    elapsed = time.time() - start_time

    print(f"    ✓ 文件保存成功：{output_path}")

    print(f"\n    任务完成！总耗时：{elapsed:.2f}秒")

```

```

print(f"    └── 总行数: {len(df):,} 行")
print(f"    └── 总列数: {len(df.columns)} 列 (按指定要求) ")
print(f"    └── 输出位置: Mac 桌面")

# 显示数据预览

print(f"\n    数据预览 (前 3 行) : ")

print(df.head(3).to_string(index=False))

return df

if __name__ == "__main__":
    try:
        calculate_listing_years()
    except Exception as e:

        print(f"\n    错误: {e}")

        import traceback
        traceback.print_exc()

```

三、是否亏损虚拟变量代码

http://localhost:8888/notebooks/Desktop/Python/%E6%AF%95%E4%B8%9A%E8%AE%BA%E6%96%87_%E6%98%AF%E5%90%A6%E4%BA%8F%E6%8D%9F_20260128%E4%BF%AE%E6%94%B9.ipynb

注：1、所有代码运行前需把 excel 文件放到桌面上；2、代码中的路径只适用于 mac 系统，windows 系统需要对路径代码进行修改；3、需要把桌面上的文件名称修改至与代码中完全相同。

kimi 指令：

我现在需要你帮我设计一段 python 代码：

一、现有数据：

一张 excel 表格，第一行为英文列标题，第二行为中文列标题。中文列标题从左到右依次为：证券代码、证券简称、统计截止日期、净利润（元）

这张表里统计了每家上市公司 2013-2024 年每年 12 月 31 日的净利润数据

二、我的需求

1、任务目标：生成所有上市公司 2013-2024 是否亏损的虚拟变量，净利润大于 0 时，未亏损，虚拟变量的值取为 0；净利润小于等于 0 时，判断为亏损，虚拟变量的值取为 1。

2、具体步骤：

(1) 读取位于 mac 桌面命名为 02-CSMAR-总资产+净利润_副本.xlsx 的文件。注意，由于股票代码存在左侧数据为 0 的情况，所以一定要始终以字符串的格式储存，保证长度一直为 6 位。

(2) 生成 2013-2024 年每家公司的是否亏损虚拟变量。当净利润大于 0 时，未亏损，虚拟变量的值取为 0；净利润小于等于 0 时，判断为亏损，虚拟变量的值取为 1，当净利润为空值时，是否亏损虚拟变量也保持为空值。

(3) 输出结果，以 excel 的形式保存到 mac 的桌面，命名为：是否亏损_对应的时间戳.xlsx。其中，这个 excel 的从左到右每一列的列标题依次为：股票代码、证券简称、会计年度、统计截止日期、净利润（元）、是否亏损。并且在表格的最上方添加列标题行。

20260128 最新版本

```
import pandas as pd
import numpy as np
import os
from datetime import datetime
```

```
# 尝试导入 tqdm，如果不存在则创建替代类
```

```

try:
    from tqdm import tqdm
except ImportError:
    class tqdm:
        def __init__(self, iterable=None, desc="", **kwargs):
            self.iterable = iterable
            if desc:
                print(f"{desc}...")
        def __iter__(self):
            for item in self.iterable:
                yield item

def generate_loss_dummy():
    """
    生成企业是否亏损虚拟变量
    功能：根据净利润判断亏损状态（0=未亏损，1=亏损，空值=数据缺失）
    """

    # 配置路径（Mac 桌面）
    desktop_path = os.path.expanduser("~/Desktop")
    input_file = os.path.join(desktop_path, "02-CSMAR-总资产+净利润_副本.xlsx")

    if not os.path.exists(input_file):
        raise FileNotFoundError(f"找不到文件：{input_file}\n 请确认文件已放在 Mac 桌面")

    print("    开始生成是否亏损虚拟变量...")

    # =====
    # 步骤 1：读取 Excel（跳过英文标题行）
    # =====

    print("\n    正在读取原始数据...")

    try:
        # header=1 表示使用第 2 行（索引 1）作为列标题，跳过第 1 行英文标题
        df = pd.read_excel(input_file, header=1, dtype={'证券代码': str})

        # 清洗列名（去除首尾空格）
        df.columns = df.columns.str.strip()

        print(f"    ✓ 成功读取 {len(df):,} 行数据")

```

```

print(f"    ✓ 涉及公司数量: {df['证券代码'].nunique()} 家")

except Exception as e:
    raise ValueError(f"读取 Excel 文件失败: {str(e)}\n 请检查文件格式是否为.xlsx")

# =====
# 步骤 2: 数据预处理与格式保护
# =====

print("\n    正在预处理数据...")

# 重命名列以符合输出要求 (证券代码→股票代码, 证券简称→证券简称)
column_mapping = {
    '证券代码': '股票代码',
    '证券简称': '证券简称',
    '统计截止日期': '统计截止日期',
    '净利润 (元)': '净利润 (元)'
}

# 检查必需列是否存在
missing_cols = [col for col in column_mapping.keys() if col not in df.columns]
if missing_cols:
    raise ValueError(f"原始数据缺少必需的列: {missing_cols}\n 现有列: {list(df.columns)}")

# 重命名列
df = df.rename(columns=column_mapping)

# 确保股票代码为 6 位字符串 (补齐前导零, 如 1→000001)
df['股票代码'] = df['股票代码'].astype(str).str.zfill(6)

# 转换日期格式
df['统计截止日期'] = pd.to_datetime(df['统计截止日期'], errors='coerce')

print("    ✓ 股票代码格式化完成 (已确保 6 位字符串)")

print("    ✓ 日期格式转换完成")

# =====
# 步骤 3: 生成关键字段 (向量化计算, 无循环)
# =====

print("\n    正在生成会计年度与是否亏损虚拟变量...")

```

```

# 生成会计年度：从统计截止日期提取年份（2013-2024）
df['会计年度'] = df['统计截止日期'].dt.year

# 生成是否亏损虚拟变量（向量化条件判断）
# 逻辑：净利润>0→0（未亏损），净利润≤0→1（亏损），净利润为空→空值
conditions = [
    df['净利润（元）'] > 0,      # 条件 1：净利润大于 0
    df['净利润（元）'] <= 0      # 条件 2：净利润小于等于 0
]
choices = [0, 1]  # 对应结果：0=未亏损，1=亏损

# np.select 实现向量化多条件赋值，默认值为 np.nan（处理空值情况）
df['是否亏损'] = np.select(conditions, choices, default=np.nan)

# 统计虚拟变量分布
loss_count = (df['是否亏损'] == 1).sum()
profit_count = (df['是否亏损'] == 0).sum()
missing_count = df['是否亏损'].isna().sum()

print(f"    ✓ 虚拟变量生成完成")

print(f"    └─ 未亏损（0）：{profit_count:,} 行 ({profit_count/len(df)*100:.1f}%)")
print(f"    └─ 亏损（1）：{loss_count:,} 行 ({loss_count/len(df)*100:.1f}%)")
print(f"    └─ 缺失值：{missing_count:,} 行 ({missing_count/len(df)*100:.1f}%)")

# =====
# 步骤 4：整理输出格式（严格按指定顺序）
# =====

print("\n    正在整理输出格式...")

# 确保统计截止日期为字符串格式（YYYY-MM-DD，便于 Excel 阅读）
df['统计截止日期'] = df['统计截止日期'].dt.strftime('%Y-%m-%d').replace('NaT', '')

# 确保会计年度为整数（无小数位）
df['会计年度'] = df['会计年度'].astype('Int64')  # 使用 Int64 支持空值

# 确保股票代码格式（最终检查）
df['股票代码'] = df['股票代码'].astype(str).str.zfill(6)

# 严格按指定顺序选择 6 列
output_columns = [
    '股票代码',      # 1. 股票代码（6 位字符串）

```

```

        '证券简称',          # 2. 证券简称
        '会计年度',          # 3. 会计年度 (2013-2024)
        '统计截止日期',      # 4. 统计截止日期 (YYYY-MM-DD)
        '净利润 (元)',        # 5. 净利润 (原始数据)
        '是否亏损'            # 6. 虚拟变量 (0=未亏损, 1=亏损, 空值=缺失)
    ]

# 检查所有列是否存在
missing_output_cols = [col for col in output_columns if col not in df.columns]
if missing_output_cols:
    raise ValueError(f"输出列缺失: {missing_output_cols}")

# 仅选择指定列并按指定顺序排列
df = df[output_columns]

print(f"    ✓ 输出列整理完成, 共 {len(df.columns)} 列")

print(f"    ✓ 数据行数: {len(df):,} 行 (保留原始行数) ")

# =====
# 步骤 5: 保存结果到桌面 (带时间戳)
# =====

print("\n    正在保存结果文件...")

timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
output_path = os.path.join(desktop_path, f"是否亏损_{timestamp}.xlsx")

try:
    with pd.ExcelWriter(output_path, engine='openpyxl') as writer:
        df.to_excel(writer, index=False, sheet_name='Sheet1')

        # 设置股票代码列为文本格式 (防止 Excel 自动转数字丢失前导零)
        worksheet = writer.sheets['Sheet1']
        for row in range(2, worksheet.max_row + 1): # 从第 2 行开始 (跳过标题)
            worksheet.cell(row, 1).number_format = '@' # '@'表示文本格式

print(f"    ✓ 文件保存成功: {output_path}")

print(f"\n    任务完成! ")

print(f"    |—— 总行数: {len(df):,} 行")
print(f"    |—— 总列数: {len(df.columns)} 列 (按指定要求) ")

```

```

print(f"    └── 时间范围：{df['会计年度'].min()}-{df['会计年度'].max()} 年")
print(f"    └── 输出位置：Mac 桌面")

# 显示数据预览

print(f"\n    数据预览（前 5 行）：")

print(df.head(5).to_string(index=False))

except Exception as e:
    raise IOError(f"保存 Excel 文件失败：{str(e)}")

return df

# =====
# 运行主程序
# =====
if __name__ == "__main__":
    try:
        result_df = generate_loss_dummy()
    except Exception as e:
        print(f"\n    错误：{e}")

        import traceback
        traceback.print_exc()

        print("\n    常见问题：")

        print("    1. 请确保 Excel 文件第一行为英文，第二行为中文列标题")
        print("    2. 请确保文件名为：02-CSMAR-总资产+净利润_副本.xlsx")
        print("    3. 请确保文件已放在 Mac 桌面")
        print("    4. 请确保包含列：证券代码、证券简称、统计截止日期、净利润（元）")

```

四、营业收入增长率计算代码

http://localhost:8888/notebooks/Desktop/Python/%E6%AF%95%E4%B8%9A%E8%AE%BA%E6%96%87_%E8%90%A5%E4%B8%9A%E6%94%B6%E5%85%A5%E5%A2%9E%E9%95%BF%E7%8E%87_20260128%E4%BF%AE%E6%94%B9.ipynb

注：1、所有代码运行前需把 excel 文件放到桌面上；2、代码中的路径只适用于 mac 系统，windows 系统需要对路径代码进行修改；3、需要把桌面上的文件名称修改至与代码中完全相同。

kimi 指令：

我现在需要你帮我设计一段 python 代码：

一、现有数据：

两张 excel 表格

第一张为位于 mac 桌面命名为 营业收入增长率.xlsx 的 excel 表格，第一行为中文列标题。中文列标题从左到右依次为：股票代码、证券简称、统计截止日期、营业收入（元）。这张表里统计了每家上市公司 2012-2024 年每年 12 月 31 日的营业收入数据

第二张为位于 Mac 桌面命名为 CSMAR-剔除证券业、ST 公司的 A 股上市公司列表_报表类型 A.xlsx 的 excel 表格，记录了 2013-2024 年剔除证券业、ST 的 A 股上市公司股票代码数据。第一行为中文标题，从左到右依次为：股票代码、证券简称、会计年度、统计截止日期、报表类型

二、我的需求：

1、任务目标：（1）计算每一家公司 2013-2024 年每年的营业收入增长率

2、逻辑：某家上市公司本年的营业收入增长率=（本年营业收入-上年营业收入）/上年营业收入*100%；公司刚上市的那一年无法计算营业收入增长率，保持为空值；公司退市的那一年

3、具体步骤：

（1）读取位于 mac 桌面命名为 营业收入增长率.xlsx 的文件。注意，由于股票代码存在左侧数据为 0 的情况，所以一定要始终以字符串的格式储存，保证长度一直为 6 位。

（2）对股票代码列的所有不同代码进行计数，找到所有数量(x)小于 13 的股票代码 y，新建 (13-x) 行空白行，将空白行的股票代码列填充为 y，统计截止日期补充为原来缺失统计数据的统计截止日期。举例，某股票代码 xxxxxx 统计数据共 10 行，分别统计 2014-2023 年的数据，此时我们需要补充 13-10=3 行，分别将这两行的股票代码列填充为 xxxxxx，同时将这两行的统计截止日期分别填充为 2012-12-31、2013-12-31 和 2024-12-31，将营业收入保持为空值，从而将这家公司的数据补齐为 13 条。

（3）计算 2013-2024 年所有的营业收入增长率。当上一年营业收入不为 0 且不是空值且本年营业收入不为空值时，某公司本年营业收入增长率=（该公司本年营业收入-该公司上年营业收入）/该公司上年营业收入；当上一年营业收入为空值（即该公司于该年已经退市或者还没上市，没有数据），直接令该公司该年的营业收入增长率为空值；当上一年营业收入为 0，由于该公司该年数据存在（营业收入不是缺失值，如果将营业收入增长率处理为空值将引起数据缺失）且分母不能为 0，直接令该公司该年的营业收入增长率为 0；如果该年营业收入为空值而上年营业收入不是空值时，认为该年度该公司退市，保持该公司该年度营业收

入增长率为空值（保持为空值而不是用“(0-非空值)/非空值”计算出结果是为了后续剔除本身就缺失的数据）

(4) 输出中间结果表格，以 excel 的形式保存到 mac 的桌面，命名为：营业收入增长率（存在人为补齐空值）_对应的时间戳.xlsx。其中，这个 excel 的从左到右每一列的列标题依次为：股票代码、证券简称、统计截止日期、营业收入（元）、营业收入增长率。并且在表格的最上方添加列标题行。在保存 excel 到桌面的同时，读取文件路径并存储在变量中，便于第(6)步直接读取。

(5) 新建一个列表，对 CSMAR-剔除证券业、ST 公司的 A 股上市公司列表_报表类型 A.xlsx 进行遍历，将每一行中（股票代码，会计年度）组合为一个元组，作为列表的一个元素。

(6) 直接使用第(4)步存储的路径，读取第(4)步存储的文件。每读取一行数据，判断该行中（股票代码，会计年度）是否在第(5)步的列表中出现过，如果出现过，则保留该行，如果列表中没有该行中的（股票代码，会计年度）元组，则认为该行数据对应证券业或 st 公司，删除第(4)步输出文件的对应行

(7) 输出最终结果，以 excel 的形式保存到 mac 的桌面，命名为：营业收入增长率（保持原始缺失值）_对应的时间戳.xlsx。其中，这个 excel 的从左到右每一列的列标题依次为：股票代码、证券简称、统计截止日期、营业收入（元）、营业收入增长率。并且在表格的最上方添加列标题行。

20250128 最新版本

#20260128 最新版本

```
import pandas as pd
import numpy as np
from datetime import datetime
import os

# ===== 1. 读取数据（增强列名容错） =====
def read_data():
    """读取两张 Excel 表，自动检测并匹配列名"""
    desktop_path = os.path.join(os.path.expanduser("~"), "Desktop")

    print(f'\n{' '*60}')
    print("【步骤 1/7】正在读取 Excel 文件...")
    print(f'{' '*60}')

    # 读取营业收入数据（只有一行中文标题，无英文标题）
    revenue_path = os.path.join(desktop_path, "营业收入增长率.xlsx")
    df_revenue = pd.read_excel(revenue_path, header=0, dtype={'股票代码': str})
    df_revenue.columns = df_revenue.columns.astype(str).str.strip()

    # 自动查找代码列
```

```

code_col = None
for col in df_revenue.columns:
    col_clean = col.lower().replace(' ', '')
    if '代码' in col_clean or 'code' in col_clean:
        code_col = col

    print(f"✓ 自动识别代码列为: '{col}'")

    break

if code_col is None:
    raise ValueError(f"未找到代码列! 实际列名为: {df_revenue.columns.tolist()}")

# 重命名代码列为标准名称"股票代码"
if code_col != '股票代码':
    df_revenue = df_revenue.rename(columns={code_col: '股票代码'})

# 确保股票代码为字符串并补全6位
df_revenue['股票代码'] = df_revenue['股票代码'].astype(str).str.zfill(6)

# 检测并处理统计截止日期列
date_col = None
for col in df_revenue.columns:
    if '日期' in col or 'date' in col.lower():
        date_col = col

    print(f"✓ 自动识别日期列为: '{col}'")

    break

if date_col:
    if date_col != '统计截止日期':
        df_revenue = df_revenue.rename(columns={date_col: '统计截止日期'})
    df_revenue['统计截止日期'] = pd.to_datetime(df_revenue['统计截止日期'])

# 提取年份列
df_revenue['年份'] = df_revenue['统计截止日期'].dt.year

# 读取白名单数据
whitelist_path = os.path.join(desktop_path, "CSMAR-剔除证券业、ST 公司的 A 股上市公司列表_报表类型 A.xlsx")
df_whitelist = pd.read_excel(whitelist_path)
df_whitelist.columns = df_whitelist.columns.astype(str).str.strip()

# 自动查找白名单中的代码列
wl_code_col = None

```

```

for col in df_whitelist.columns:
    col_clean = col.lower().replace(' ', '')
    if '代码' in col_clean or 'code' in col_clean:
        wl_code_col = col
        break

if wl_code_col:
    if wl_code_col != '股票代码':
        df_whitelist = df_whitelist.rename(columns={wl_code_col: '股票代码'})
    df_whitelist['股票代码'] = df_whitelist['股票代码'].astype(str).str.zfill(6)

# 处理白名单中的会计年度
year_col = None
for col in df_whitelist.columns:
    if '年度' in col or 'year' in col.lower():
        year_col = col
        break

if year_col:
    if year_col != '会计年度':
        df_whitelist = df_whitelist.rename(columns={year_col: '会计年度'})
    df_whitelist['会计年度'] = df_whitelist['会计年度'].astype(int)

print(f"\n✓ 读取完成! 营业收入数据: {len(df_revenue)} 行")

print(f"✓ 白名单数据: {len(df_whitelist)} 行")

print(f"✓ 白名单公司数: {df_whitelist['股票代码'].nunique()} 家")

return df_revenue, df_whitelist

# ===== 2. 数据补齐 =====
def fill_missing_data(df_revenue):
    """补齐缺失年份至 13 条记录 (2012-2024) """
    print(f"\n{' '*60}")
    print("【步骤 2/7】正在补齐缺失数据...")
    print(f"{' '*60}")

    full_years = np.arange(2012, 2025)
    stock_info = df_revenue.groupby('股票代码')['证券简称'].first().reset_index()

    complete_index = pd.MultiIndex.from_product(
        [stock_info['股票代码'], full_years],

```

```

        names=['股票代码','年份']
    )

    df_indexed = df_revenue.set_index(['股票代码','年份'])
    df_filled = df_indexed.reindex(complete_index).reset_index()

    mask = df_filled['统计截止日期'].isna()
    df_filled.loc[mask, '统计截止日期'] = pd.to_datetime(
        df_filled.loc[mask, '年份'].astype(str) + '-12-31'
    )

    df_filled = df_filled.drop('年份', axis=1)

    print(f"✓ 补齐完成! 从 {len(df_revenue)} 行 → {len(df_filled)} 行")

    print(f"✓ 每家公司补齐至 13 条记录")

    return df_filled

# ===== 3. 计算增长率（增强版） =====
def calculate_growth_rate(df_filled):
    """计算 2013-2024 年营业收入增长率，处理所有边界情况"""
    print(f"\n{' '*60}")
    print("【步骤 3/7】正在计算营业收入增长率...")
    print(f"{' '*60}")

    # 按公司分组并排序
    df_filled = df_filled.sort_values(['股票代码','统计截止日期'])

    # 获取上年营业收入
    df_filled['上年营业收入'] = df_filled.groupby('股票代码')['营业收入（元）'].shift(1)

    # 增强的向量化增长率计算逻辑
    def compute_growth(row):
        current = row['营业收入（元）']
        prev = row['上年营业收入']

        # 情况 1: 上年营业收入为 NaN → 空值（未上市或已退市）
        if pd.isna(prev):
            return np.nan

        # 情况 2: 上年营业收入为 0 → 0（数据存在但分母为 0）
        elif prev == 0:
            return 0

```

```

        # 情况 3: 本年营业收入为 NaN → 空值 (退市当年)
        elif pd.isna(current):
            return np.nan
        # 情况 4: 正常计算
        else:
            return (current - prev) / prev

# 使用 apply 应用向量化计算
df_filled['营业收入增长率'] = df_filled.apply(compute_growth, axis=1)

# 只保留 2013-2024 年数据
df_filled = df_filled[df_filled['统计截止日期'].dt.year >= 2013].copy()

# 删除辅助列
df_filled = df_filled.drop('上年营业收入', axis=1)

print(f"✓ 计算完成! 数据范围: 2013-2024 年")

print(f"✓ 增长率非空: {df_filled['营业收入增长率'].notna().sum()} 条")

print(f"✓ 增长率=0: {(df_filled['营业收入增长率'] == 0).sum()} 条")

return df_filled

# ===== 4. 保存中间结果 =====
def save_intermediate_result(df):
    """保存中间结果 (保持人为补齐空值) 并返回文件路径"""
    print(f"\n{' '*60}")
    print("【步骤 4/7】正在保存中间结果...")
    print(f"{' '*60}")

    desktop_path = os.path.join(os.path.expanduser("~"), "Desktop")
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    output_path = os.path.join(desktop_path, f"营业收入增长率 (存在人为补齐空值)_{timestamp}.xlsx")

    # 保存文件
    df.to_excel(output_path, index=False)

    print(f"✓ 中间结果已保存: {output_path}")

    print(f"✓ 文件大小: {os.path.getsize(output_path) / 1024:.2f} KB")

```

```

        return output_path

# ===== 5. 生成白名单过滤集合 =====
def create_whitelist_set(df_whitelist):
    """从白名单表中创建(股票代码, 会计年度)集合用于快速查找"""
    print(f"\n{' '*60}")
    print(" 【步骤 5/7】 正在生成白名单筛选集合...")
    print(f"{' '*60}")

    # 创建(股票代码, 会计年度)元组集合
    whitelist_set = set(zip(df_whitelist['股票代码'], df_whitelist['会计年度']))

    print(f"✓ 白名单集合创建完成! 包含 {len(whitelist_set)} 个组合")

    print(f"✓ 示例: {list(whitelist_set)[:3]}...")

    return whitelist_set

# ===== 6. 白名单筛选(关键修复) =====
def filter_by_whitelist(df_intermediate, whitelist_set):
    """筛选出符合白名单条件的数据"""
    print(f"\n{' '*60}")
    print(" 【步骤 6/7】 正在根据白名单筛选数据...")
    print(f"{' '*60}")

    # ===== 关键修复: 确保股票代码格式与白名单一致 =====
    df_intermediate['股票代码'] = df_intermediate['股票代码'].astype(str).str.zfill(6)

    # 从中间结果中提取(股票代码, 年份)组合
    df_intermediate['组合键'] = list(zip(
        df_intermediate['股票代码'],
        df_intermediate['统计截止日期'].dt.year
    ))

    # 向量化筛选: 检查组合是否在白名单中
    mask = df_intermediate['组合键'].isin(whitelist_set)
    df_filtered = df_intermediate[mask].copy()

    # 删除组合键列
    df_filtered = df_filtered.drop('组合键', axis=1)

    print(f"✓ 筛选完成! 从 {len(df_intermediate)} 行 → {len(df_filtered)} 行")

```

```

print(f"✓ 剔除数据: {len(df_intermediate) - len(df_filtered)} 行 (ST 或证券业公司)")

# 验证: 如果结果为空, 给出警告
if len(df_filtered) == 0:

    print("\n 警告: 筛选后数据为空! 请检查: ")

    print(" 1. 白名单集合是否包含有效数据")
    print(" 2. 中间结果中的股票代码格式是否正确")
    print(" 3. 年份范围是否匹配 (2013-2024) ")

    return df_filtered

# ===== 7. 保存最终结果 =====
def save_final_result(df_filtered):
    """保存最终筛选结果"""
    print(f"\n{'='*60}")
    print(" 【步骤 7/7】 正在保存最终结果...")
    print(f"{'='*60}")

    desktop_path = os.path.join(os.path.expanduser("~"), "Desktop")
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    output_path = os.path.join(desktop_path, f"营业收入增长率 (保持原始缺失值)_{timestamp}.xlsx")

    # 调整列顺序
    df_final = df_filtered[['股票代码', '证券简称', '统计截止日期', '营业收入 (元)', '营业收入增长率']]

    df_final.to_excel(output_path, index=False)

    print(f"✓ 最终结果已保存: {output_path}")

    print(f"✓ 最终文件大小: {os.path.getsize(output_path) / 1024:.2f} KB")

    print(f"✓ 最终数据行数: {len(df_final)} 行")

    return output_path

# ===== 主函数 =====
def main():
    """主处理流程"""
    print("\n" + "="*60)

```

```

print(" " * 20 + "营业收入增长率计算程序")
print("="*60)

total_start_time = datetime.now()

# 步骤 1: 读取两张表
df_revenue, df_whitelist = read_data()

# 步骤 2: 补齐数据
df_filled = fill_missing_data(df_revenue)

# 步骤 3: 计算增长率
df_with_growth = calculate_growth_rate(df_filled)

# 步骤 4: 保存中间结果（并获取路径）
intermediate_path = save_intermediate_result(df_with_growth)

# 步骤 5: 生成白名单集合
whitelist_set = create_whitelist_set(df_whitelist)

# 步骤 6: 读取中间结果并筛选
df_intermediate = pd.read_excel(intermediate_path)
df_intermediate['统计截止日期'] = pd.to_datetime(df_intermediate['统计截止日期'])
df_final = filter_by_whitelist(df_intermediate, whitelist_set)

# 步骤 7: 保存最终结果
save_final_result(df_final)

# 总耗时统计
total_end_time = datetime.now()
duration = (total_end_time - total_start_time).total_seconds()

print("\n" + "="*60)

print(" " * 20 + " 全部任务完成! ")

print(f" " * 15 + f"总耗时: {duration:.2f} 秒")
print("="*60 + "\n")

# 执行主函数
if __name__ == "__main__":
    main()

```

五、股权性质虚拟变量生成代码

http://localhost:8888/notebooks/Desktop/Python/%E6%AF%95%E4%B8%9A%E8%AE%BA%E6%96%87_%E8%82%A1%E6%9D%83%E6%80%A7%E8%B4%A8%E8%99%9A%E6%8B%9F%E5%8F%98%E9%87%8F%E7%94%9F%E6%88%90%E4%BB%A3%E7%A0%81_20260128%E4%BF%AE%E6%94%B9.ipynb

注：1、所有代码运行前需把 excel 文件放到桌面上；2、代码中的路径只适用于 mac 系统，windows 系统需要对路径代码进行修改；3、需要把桌面上的文件名称修改至与代码中完全相同。

kimi 指令：

我现在需要你帮我设计一段 python 代码：

一、现有数据：

一张 excel 表格，第一行为英文列标题，第二行为中文列标题。中文列标题从左到右依次为：证券代码、统计截止日期、股权性质和股权性质编码

这张表里统计了每家上市公司 2013-2024 年每年 12 月 31 日的股权性质情况。

其中，股权性质编码列各个编码的含义：1=国企、2=民营、3=外资、4=其他

二、我的需求

1、任务目标：生成 2013-2024 年每家上市公司的股权性质虚拟变量值。

2、逻辑：股权性质编码包含 1，则判定该上市公司本年的股权性质为国企，股权性质虚拟变量值为 1；股权性质编码列不为空值且不包含 1，则判定该上市公司本年的股权性质不是国企，股权性质虚拟变量值为 0；如果股权性质编码列为空，则认为该公司该年份数据缺失，保持股权性质虚拟变量值为空

3、具体步骤：

(1) 读取位于 mac 桌面命名为 14-CSMAR-股权性质_副本.xlsx 的文件。注意，由于股票代码存在左侧数据为 0 的情况，所以一定要始终以字符串的格式储存，保证长度一直为 6 位。

(2) 根据现有的股权性质编码列的数据，生成现有每一行代表年份公司的股权性质虚拟变量：股权性质编码包含 1，则判定该上市公司本年的股权性质为国企，股权性质虚拟变量值为 1；股权性质编码列不为空值且不包含 1，则判定该上市公司本年的股权性质不是国企，股权性质虚拟变量值为 0；如果股权性质编码列为空，则认为该公司该年份数据缺失，保持股权性质虚拟变量值为空。

(3) 输出结果，以 excel 的形式保存到 mac 的桌面，命名为：股权性质虚拟变量_对应的时间戳.xlsx。其中，这个 excel 的从左到右每一列的列标题依次为：证券代码、统计截止日期、股权性质、股权性质编码、股权性质虚拟变量。并且在表格的最上方添加列标题行。

#20250128 最新版本

```

import pandas as pd
import numpy as np
from datetime import datetime
import os
import re

# ===== 1. 读取数据 =====
def read_data():
    """读取 Excel 数据并进行初步处理"""
    desktop_path = os.path.join(os.path.expanduser("~"), "Desktop")
    file_path = os.path.join(desktop_path, "14-CSMAR-股权性质_副本.xlsx")

    # 检查文件是否存在
    if not os.path.exists(file_path):
        raise FileNotFoundError(f"文件不存在: {file_path}\n 请确认文件已保存在 Mac 桌面")

    # 读取 Excel (跳过第一行英文标题, 使用第二行中文标题)
    # 尝试以字符串类型读取证券代码, 但如果列名不匹配可能失效, 后续会强制转换
    df = pd.read_excel(file_path, header=1, dtype={'证券代码': str})

    # 清理列名: 去除首尾空格
    df.columns = df.columns.astype(str).str.strip()

    # 自动匹配统计截止日期列 (兼容可能的空格或相似名称)
    date_col = None
    for col in df.columns:
        if '统计截止日期' in str(col) or ('统计' in str(col) and '日期' in str(col)):
            date_col = col
            break

    if date_col is None:
        raise ValueError(f"未找到'统计截止日期'列。实际列名为: {df.columns.tolist()}")

    if date_col != '统计截止日期':
        df = df.rename(columns={date_col: '统计截止日期'})
        print(f"已将列 '{date_col}' 重命名为 '统计截止日期'")

    # === 关键修正: 确保证券代码为 6 位字符串格式 (处理 1.0 -> 000001) ===
    # 步骤 1: 转为字符串并去除首尾空格
    df['证券代码'] = df['证券代码'].astype(str).str.strip()
    # 步骤 2: 去除可能的浮点数尾部 (如 "1.0" -> "1")
    df['证券代码'] = df['证券代码'].apply(lambda x: re.sub(r'\.0$', "", x) if isinstance(x, str) else
x)

```

```

# 步骤 3: 补齐前导零至 6 位
df['证券代码'] = df['证券代码'].str.zfill(6)

# 将统计截止日期转换为 datetime 格式
df['统计截止日期'] = pd.to_datetime(df['统计截止日期'], errors='coerce')

# 检查是否有无效日期
if df['统计截止日期'].isna().any():
    invalid_rows = df[df['统计截止日期'].isna()].index.tolist()
    print(f'警告: 第 {invalid_rows} 行日期格式无效, 将保留为 NaT')

return df

# ===== 2. 生成股权性质虚拟变量 =====
def generate_dummy_variable(df):
    """基于股权性质编码生成虚拟变量"""
    if '股权性质编码' not in df.columns:
        raise ValueError(f'未找到'股权性质编码'列。实际列名为: {df.columns.tolist()}')

    # 创建临时字符串列用于判断 (处理可能的混合类型: float, int, str)
    # 先转为 str, NaN 会变成 'nan'
    df['股权性质编码_str'] = df['股权性质编码'].astype(str)

    # 生成虚拟变量:
    # - 原始编码为 NaN -> 虚拟变量为 NaN (空值)
    # - 字符串包含 "1" -> 1 (国企)
    # - 其他情况 -> 0 (非国企)
    df['股权性质虚拟变量'] = np.where(
        df['股权性质编码'].isna(),                                # 条件: 原始值是否为 NaN
        np.nan,                                                  # 是: 保持空值
        np.where(
            df['股权性质编码_str'].str.contains('1', na=False, regex=False), # 否: 是否包
            1,                                                    # 是: 国企
            0                                                      # 否: 非国企
        )
    )

    # 删除临时列
    df = df.drop('股权性质编码_str', axis=1)

    # 转换虚拟变量列为数值类型 (float), 便于 Excel 识别
    df['股权性质虚拟变量'] = pd.to_numeric(df['股权性质虚拟变量'], errors='coerce')

```

```

return df

# ===== 3. 保存结果 =====
def save_result(df):
    """保存结果到桌面，添加时间戳"""
    desktop_path = os.path.join(os.path.expanduser("~"), "Desktop")
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    output_path = os.path.join(desktop_path, f"股权性质虚拟变量_{timestamp}.xlsx")

    # 确保输出列顺序符合要求
    required_cols = ['证券代码', '统计截止日期', '股权性质', '股权性质编码', '股权性质虚拟变
量']

    # 检查必要的列是否存在
    missing_cols = [col for col in required_cols if col not in df.columns]
    if missing_cols:
        print(f"警告：原数据缺少以下列，将自动填充为空: {missing_cols}")
        for col in missing_cols:
            df[col] = np.nan

    # 调整列顺序
    df = df[required_cols]

    # 重置索引（不保留索引列）
    df = df.reset_index(drop=True)

    # 保存为 Excel（不保存索引）
    try:
        df.to_excel(output_path, index=False, engine='openpyxl')
        print(f"\n 处理完成！结果已保存至: {output_path}")
        print(f"共处理 {df['证券代码'].nunique()} 家公司，总计 {len(df)} 行数据")
        print(f"国企行数: {(df['股权性质虚拟变量']==1).sum()}")
        print(f"非国企行数: {(df['股权性质虚拟变量']==0).sum()}")
        print(f"数据缺失行数: {df['股权性质虚拟变量'].isna().sum()}")
        return output_path
    except Exception as e:
        raise IOError(f"保存文件时出错: {e}\n 请检查是否有文件正在打开占用")

# ===== 主函数 =====
def main():
    """主处理流程"""
    print("="*60)
    print("股权性质虚拟变量生成程序（无补齐版本）")
    print("="*60)

```

```

try:
    print("\n[1/3] 正在读取数据...")
    df = read_data()

    print(f"✓ 读取完成! 原始数据共 {len(df)} 行, {df['证券代码'].nunique()} 家公司")

    print("\n[2/3] 正在生成股权性质虚拟变量...")
    df_with_dummy = generate_dummy_variable(df)

    print("✓ 虚拟变量生成完成")

    print("\n[3/3] 正在保存结果...")
    save_result(df_with_dummy)

    print("\n" + "="*60)
    print("程序执行成功! ")
    print("="*60)

except FileNotFoundError as e:
    print(f"\n✗ 错误: {e}")

except ValueError as e:
    print(f"\n✗ 数据格式错误: {e}")

except Exception as e:
    print(f"\n✗ 未知错误: {e}")

# 执行主函数
if __name__ == "__main__":
    main()

```

六、不同期限借款代码

http://localhost:8888/notebooks/Desktop/Python/%E6%AF%95%E4%B8%9A%E8%AE%BA%E6%96%87_%E4%B8%8D%E5%90%8C%E6%9C%9F%E9%99%90%E5%80%9F%E6%AC%B E_20260128%E4%BF%AE%E6%94%B9.ipynb

注：1、所有代码运行前需把 excel 文件放到桌面上；2、代码中的路径只适用于 mac 系统，windows 系统需要对路径代码进行修改；3、需要把桌面上的文件名称修改至与代码中完全相同。

kimi 指令：

我现在需要你帮我设计一段 python 代码：

一、现有数据：

一张 excel 表格，第一行为英文列标题，第二行为中文列标题。中文列标题从左到右依次为：证券代码、公告日期、借款事件年份、贷款进程、本币金额（万元）、贷款期限、贷款类型 1、贷款类型 2

其中贷款类型 1 所在列的值有两种：长期借款和短期借款；贷款类型 2 所在列的值有 6 种：抵押贷款、质押贷款、担保借款、信用借款、保证借款、其他授信项
这张表里统计了每家上市公司 2013-2024 年每年新增不同期限借款的情况。

二、我的需求

1、任务目标：（1）分别统计每家上市公司每年的新增短期借款和长期借款总额；（2）补齐每家上市公司的数据行数。

2、具体步骤：

（1）读取位于 mac 桌面命名为 22-CSMAR-不同期限银行借款_副本.xlsx 的文件。注意，由于股票代码存在左侧数据为 0 的情况，所以一定要始终以字符串的格式储存，保证长度一直为 6 位。

（2）新增一列“本年新增合计（万元）”统计每家上市公司每个有数据的年份的当年新增短期借款和新增长期借款，每个证券代码每个借款事件年份对应一个合计数：举例：股票代码为 xxxxxx 的上市公司在 2013-2024 年每年都有借款事件发生，那么该公司 2013 年的新增长期借款总额即为证券代码列为 xxxxxx 且借款事件年份列为 2013 且贷款类型 1 为长期借款的所有行对应的“本币金额（万元）”的加总，并用；如果该公司 2013 年没有新增短期借款，那么在原表格中找不到同时满足证券代码列为 xxxxxx 且借款事件年份列为 2013 且贷款类型 1 为短期借款的行，则新增一行，分别令证券代码为 xxxxxx，借款事件年份为 2013，贷款类型 1 为短期借款，“本年新增合计（万元）”的统计值为 0，公告日期、贷款进程、贷款期限、贷款类型 2 所在列均保持空值。新增这一行的含义是，说明这家公司在 2013 年新增短期借款的金额为 0。

（3）输出结果，以 excel 的形式保存到 mac 的桌面，命名为：不同期限借款（补齐前）_对应的时间戳.xlsx。其中，这个 excel 的从左到右每一列的列标题依次为：证券代码、公告日期、借款事件年份、贷款进程、贷款期限、贷款类型 1、贷款类型 2、本年新增合计（万元）。并且在表格的最上方添加列标题行。

```
import pandas as pd
```

```

import numpy as np
from datetime import datetime
import os

# ===== 1. 读取数据 =====
def read_data():
    """读取 Excel 数据并进行初步处理"""
    desktop_path = os.path.join(os.path.expanduser("~"), "Desktop")
    file_path = os.path.join(desktop_path, "22-CSMAR-不同期限银行借款_副本.xlsx")

    # 步骤 1/5: 读取数据
    print(f'\n{'='*60}')
    print("【步骤 1/5】正在读取 Excel 文件...")
    print(f'\n{'='*60}')

    df = pd.read_excel(file_path, header=1, dtype={'证券代码': str})
    df.columns = df.columns.astype(str).str.strip()

    # 显示前 5 行和列名用于验证
    print(f'✓ 成功读取 {len(df)} 行数据')

    print(f'✓ 列名: {df.columns.tolist()}')

    print(f'✓ 数据预览:\n{df.head(3)}\n')

    df['证券代码'] = df['证券代码'].astype(str).str.zfill(6)
    df['借款事件年份'] = df['借款事件年份'].astype(int)

    return df

# ===== 2. 分组统计 =====
def process_and_fill_data(df):
    """分组统计借款金额并补齐缺失数据"""

    # 步骤 2/5: 分组统计
    print(f'\n{'='*60}')
    print("【步骤 2/5】正在按证券代码+年份+贷款类型统计借款金额...")
    print(f'\n{'='*60}')

    total_groups = len(df.groupby(['证券代码', '借款事件年份', '贷款类型 1']))
    print(f'→ 发现 {total_groups} 个有效分组')

    grouped = df.groupby(['证券代码', '借款事件年份', '贷款类型 1'], as_index=False)['本币金

```

```

额 (万元) '].sum()

grouped = grouped.rename(columns={'本币金额 (万元) ': '本年新增合计 (万元) '})

print(f'✓ 统计完成! 生成 {len(grouped)} 条聚合记录")

print(f'✓ 金额大于 0 的记录: {(grouped['本年新增合计 (万元) '] > 0).sum()} 条")

# 步骤 3/5: 构建完整索引
print(f"\n{'='*60}")
print("【步骤 3/5】正在构建完整数据矩阵...")
print(f"{'='*60}")

stock_codes = grouped['证券代码'].unique()
years = range(2013, 2025)
loan_types = ['短期借款', '长期借款']

complete_index = pd.MultiIndex.from_product(
    [stock_codes, years, loan_types],
    names=['证券代码', '借款事件年份', '贷款类型 1']
)

print(f"→ 股票代码数量: {len(stock_codes)} 个")
print(f"→ 年份范围: {list(years)}")
print(f"→ 贷款类型: {loan_types}")
print(f"→ 完整数据矩阵应包含: {len(complete_index)} 行")

grouped_indexed = grouped.set_index(['证券代码', '借款事件年份', '贷款类型 1'])
result = grouped_indexed.reindex(complete_index, fill_value=0).reset_index()

print(f'✓ 矩阵构建完成! 当前共 {len(result)} 行")

# 步骤 4/5: 提取其他信息并合并
print(f"\n{'='*60}")
print("【步骤 4/5】正在提取其他列信息并合并数据...")
print(f"{'='*60}")

# 显示进度: 从原始数据中提取信息
print("→ 从原始数据中提取公告日期、贷款进程等信息...")
other_cols = ['证券代码', '借款事件年份', '贷款类型 1', '公告日期', '贷款进程', '贷款期限', '
贷款类型 2']
other_info = df.groupby(['证券代码', '借款事件年份', '贷款类型 1'], as_index=False).first()
other_info = other_info[other_cols]

```

```

print(f"→ 提取了 {len(other_info)} 条基础信息记录")
print("→ 正在合并统计结果与基础信息...")

final_result = pd.merge(
    result,
    other_info,
    on=['证券代码', '借款事件年份', '贷款类型 1'],
    how='left'
)

column_order = ['证券代码', '公告日期', '借款事件年份', '贷款进程', '贷款期限', '贷款类型
1', '贷款类型 2', '本年新增合计 (万元) ']
final_result = final_result[column_order]

print(f"✓ 合并完成! 最终数据共 {len(final_result)} 行")

print(f"✓ 短期借款记录: {len(final_result[final_result['贷款类型 1'] == '短期借款'])} 行")

print(f"✓ 长期借款记录: {len(final_result[final_result['贷款类型 1'] == '长期借款'])} 行")

return final_result

# ===== 3. 保存结果 =====
def save_result(df):
    """保存结果到桌面, 添加时间戳"""
    # 步骤 5/5: 保存结果
    print(f"\n{' '*60}")
    print("【步骤 5/5】正在保存结果到 Excel 文件...")
    print(f"{' '*60}")

    desktop_path = os.path.join(os.path.expanduser("~"), "Desktop")
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    output_path = os.path.join(desktop_path, f"不同期限借款 (补齐前)_{timestamp}.xlsx")

    print(f"→ 保存路径: {output_path}")
    print(f"→ 数据维度: {df.shape[0]} 行 × {df.shape[1]} 列")

    df.to_excel(output_path, index=False)

    print(f"✓ 文件保存成功! ")

    print(f"{' '*60}\n")

```

```

        return output_path

# ===== 主函数 =====
def main():
    """主处理流程"""
    print("\n" + "="*60)
    print(" " * 15 + "银行借款数据处理程序")
    print("="*60)

    total_start_time = datetime.now()

    # 步骤 1: 读取
    df = read_data()

    # 步骤 2-4: 处理
    result_df = process_and_fill_data(df)

    # 步骤 5: 保存
    save_result(result_df)

    # 总体完成信息
    total_end_time = datetime.now()
    duration = (total_end_time - total_start_time).total_seconds()

    print("\n" + "    所有任务完成! ")

    print(f"    总耗时: {duration:.2f} 秒")

    print("="*60 + "\n")

# 执行主函数
if __name__ == "__main__":
    main()

```

20250128 新增步骤

我现在需要你帮我设计一段 python 代码：

一、现有数据：

两张 excel 表格

第一张表为位于 mac 桌面上命名为 0 22-CSMAR-不同期限银行借款_副本.xlsx 的 excel 表格，统计了每家上市公司 2013-2024 年每年新增不同期限借款的情况。第一行为英文列标题，第

二行为中文列标题。中文列标题从左到右依次为：证券代码、公告日期、借款事件年份、贷款进程、本币金额（万元）、贷款期限、贷款类型 1、贷款类型 2。其中贷款类型 1 所在列的值有两种：长期借款和短期借款；贷款类型 2 所在列的值有 6 种：抵押贷款、质押贷款、担保借款、信用借款、保证借款、其他授信项。

第二张表格为位于 mac 桌面上命名为 2 不同期限借款（手工整理）.xlsx 的 excel 表格，是由第一张表统计得到的每家公司每年新增短期借款和新增长期借款的数据。由三张 sheet 组成。第一张 sheet 标题为“总表”，本次操作不涉及总表；第二张 sheet 标题为“新增不同期限短期借款”，第一行为中文标题，从左到右依次为：股票代码、会计年度、统计截止日期、本年新增短期借款合计（万元）；第三张 sheet 标题为“新增不同期限长期借款”，第一行为中文标题，从左到右依次为：股票代码、会计年度、统计截止日期、本年新增长期借款合计（万元）。

二、我的需求

1、任务目标：（1）从 0 22-CSMAR-不同期限银行借款_副本.xlsx 表格中分别统计出各个股票代码代表的公司存在新增短期借款和新增长期借款的年份，分别生成两张匹配表用于匹配任务；（2）使用任务（1）得到的两张匹配表，分别剔除 2 不同期限借款（手工整理）.xlsx 的 sheet2 中所有 0 22-CSMAR-不同期限银行借款_副本.xlsx 表格中不存在新增短期借款的（股票代码，会计年度）代表的行，剔除 2 不同期限借款（手工整理）.xlsx 的 sheet3 中所有 0 22-CSMAR-不同期限银行借款_副本.xlsx 表格中不存在新增长期借款的（股票代码，会计年度）代表的行。

2、具体步骤：

【生成匹配表】

（1）读取位于 mac 桌面命名为 0 22-CSMAR-不同期限银行借款_副本.xlsx 的文件。注意，由于股票代码存在左侧数据为 0 的情况，所以一定要始终以字符串的格式储存，保证长度一直为 6 位。

（2）对 0 22-CSMAR-不同期限银行借款_副本.xlsx 除标题外的每一行进行遍历，新建两个字典，分别统计每个（股票代码，会计年度）新增短期借款和新增长期借款的存在情况。遍历时，将每一行的（股票代码，会计年度）组成元组，分别写入两个字典，作为一个键值对的键，同时判断该行贷款类型为短期借款还是长期借款，当该股票代码该年第一次出现短期借款（或长期借款）时，则将对应该贷款类型字典的键值对的值写为 1，之后的遍历过程再遇到相同的（股票代码，会计年度）组合时，直接跳过，继续遍历下一行。

（3）输出中间结果表格，以 excel 的形式保存到 mac 的桌面，命名为：不同期限借款匹配表（贷款类型 1）_对应的时间戳.xlsx。由于第（2）步生成了两个不同的字典，分别统计短期借款和长期借款（股票代码，会计年度）的出现情况，这个 excel 须由两张 sheet 组成：第一张 sheet 保存短期借款字典的结果，从左到右每一列的列标题依次为：股票代码、会计年度、是否新增短期借款虚拟变量，其中“是否新增短期借款虚拟变量”则是短期借款字典的键值对的值，并且在表格的最上方添加列标题行；第二张 sheet 保存长期借款字典的结果，从左到右每一列的列标题依次为：股票代码、会计年度、是否新增长期借款虚拟变量，其中“是否新增长期借款虚拟变量”则是长期借款字典的键值对的值，并且在表格的最上方添加列标题行。在保存 excel 到桌面的同时，读取文件路径并存储在变量中，便于第（4）步直接读取。

【短期借款剔除】

（4）直接使用第（3）步存储的路径，读取第（3）步存储的文件的 sheet1。新建一个短期

借款白名单列表，将第（3）步存储的中间结果文件的 sheet1 中的每一行的（股票代码，会计年度）组成元组，作为列表的一个元素。

（5）读取 mac 桌面上命名为 2 不同期限借款（手工整理）.xlsx 的 excel 表格的 sheet2 新增不同期限短期借款”除了标题行的所有行，每读取一行数据，判断该行中的（股票代码，会计年度）是否在第（4）步生成的短期借款白名单列表中出现过，如果出现过，则保留该行，如果列表中没有该行的（股票代码，会计年度）元组，则认为该公司该年份在 0 22-CSMAR-不同期限银行借款_副本.xlsx 中没有新增短期借款数据，删除 2 不同期限借款（手工整理）.xlsx 的 sheet2 的对应行。

（6）输出短期借款最终结果，将删除部分行的 2 不同期限借款（手工整理）.xlsx 的 sheet2 新增不同期限短期借款”的剩余数据行以 excel 的形式保存到 mac 的桌面，命名为：不同期限借款（保持原始缺失值）_对应的时间戳.xlsx。其中，这个 excel 的从左到右每一列的列标题依次为：股票代码、会计年度、统计截止日期、本年新增短期借款合计（万元）。并在表格的最上方添加列标题行，继续将这张 sheet 命名为 新增不同期限短期借款”。在保存 excel 到桌面的同时，读取文件路径并存储在变量中，便于第（9）步直接读取。

【长期借款剔除】

（7）直接使用第（3）步存储的路径，读取第（3）步存储的文件的 sheet2。新建一个长期借款白名单列表，将第（3）步存储的中间结果文件的 sheet2 中的每一行的（股票代码，会计年度）组成元组，作为列表的一个元素。

（8）读取 mac 桌面上命名为 2 不同期限借款（手工整理）.xlsx 的 excel 表格的 sheet3 新增不同期限长期借款”除了标题行的所有行，每读取一行数据，判断该行中的（股票代码，会计年度）是否在第（7）步生成的长期借款白名单列表中出现过，如果出现过，则保留该行，如果列表中没有该行的（股票代码，会计年度）元组，则认为该公司该年份在 0 22-CSMAR-不同期限银行借款_副本.xlsx 中没有新增长期借款数据，删除 2 不同期限借款（手工整理）.xlsx 的 sheet3 的对应行。

（9）输出短期借款最终结果，直接读取第（6）步保存的短期借款最终结果的文件路径，在这个 excel 文件中新建一张 sheet2，命名为 新增不同期限长期借款”，将删除部分行的 2 不同期限借款（手工整理）.xlsx 的 sheet3 新增不同期限长期借款”的剩余数据行存储在这张 sheet 中。其中，这个 excel 的从左到右每一列的列标题依次为：股票代码、会计年度、统计截止日期、本年新增长期借款合计（万元）。并在表格的最上方添加列标题行。

注意：（1）鉴于使用循环会导致代码运行效率低下，你可以给出可行的替代方案优化程序；
（2）我希望这个代码可以实时显示运行进度。

请仔细阅读上述文字，并告诉我你是否已经完全理解所有要求。

```
import pandas as pd
import os
from datetime import datetime
from tqdm import tqdm
import sys

# =====
# 辅助函数：向量化股票代码格式化（安全处理 NaN）
# =====
```

```

def safe_format_code(series):
    """向量化处理：转为 str→去.0→补零，NaN 保持 NaN"""
    return series.astype(str).str.split('.').str[0].str.zfill(6).where(series.notna())

# =====
# 配置路径 (Mac 桌面)
# =====
desktop_path = os.path.join(os.path.expanduser("~"), "Desktop")
source_file = os.path.join(desktop_path, "0 22-CSMAR-不同期限银行借款_副本.xlsx")
target_file = os.path.join(desktop_path, "2 不同期限借款（手工整理）.xlsx")

# 文件存在性检查（带进度提示）

print("    检查文件路径...")

for f in [source_file, target_file]:
    if not os.path.exists(f):

        print(f"    错误：文件不存在 - {f}")

        sys.exit(1)

    print(f"    ✓ 找到：{os.path.basename(f)}")

timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")

print(f"\n    启动处理 | 时间戳：{timestamp}")

print("    模式：向量化计算 + 哈希字典 + 零 Python 级循环筛选")

# =====
# 【生成匹配表】步骤 (1) (2) (3)
# =====

print("\n    阶段一：生成匹配字典与匹配表...")

# (1) 读取源数据，正确处理双行标题
print("    [步骤 1] 读取源数据...")
df_source = pd.read_excel(source_file, header=1)

print(f"    ✓ 载入 {len(df_source):,} 行")

# 向量化格式化证券代码
print("    格式化证券代码...")
df_source['证券代码'] = safe_format_code(df_source['证券代码'])

```

```

# (2) 使用字典统计存在性（严格按用户要求使用 dict，避免循环）
print("    [步骤 2] 构建匹配字典（向量化生成）...")

# 分离短期/长期布尔掩码
mask_short = df_source['贷款类型 1'] == '短期借款'
mask_long = df_source['贷款类型 1'] == '长期借款'

# 关键修复：先 dropna 再转换类型，避免 astype(int) 崩溃
# 构建字典：键为元组(股票代码, 年份)，值为 1（dict.fromkeys 自动去重）
df_short_valid = df_source[mask_short & df_source['证券代码'].notna() & df_source['借款事件年份'].notna()]
df_long_valid = df_source[mask_long & df_source['证券代码'].notna() & df_source['借款事件年份'].notna()]

print(f"    发现 {len(df_short_valid)} 行短期借款记录，去重中...")
short_pairs = zip(
    df_short_valid['证券代码'],
    df_short_valid['借款事件年份'].astype(int)
)
short_term_dict = dict.fromkeys(short_pairs, 1) # 符合要求的字典结构

print(f"    发现 {len(df_long_valid)} 行长期借款记录，去重中...")
long_pairs = zip(
    df_long_valid['证券代码'],
    df_long_valid['借款事件年份'].astype(int)
)
long_term_dict = dict.fromkeys(long_pairs, 1)

print(f"    ✓ 短期借款字典: {len(short_term_dict):,} 个唯一键")

print(f"    ✓ 长期借款字典: {len(long_term_dict):,} 个唯一键")

# (3) 输出中间结果表格（严格按步骤命名 Sheet）
print("    [步骤 3] 保存匹配表到 Excel...")
match_file = os.path.join(desktop_path, f"不同期限借款匹配表（贷款类型 1）_{timestamp}.xlsx")

# 转换字典为 DataFrame（键解包为元组列表）
df_short_match = pd.DataFrame(
    [(k[0], k[1], v) for k, v in short_term_dict.items()],
    columns=['股票代码', '会计年度', '是否新增短期借款虚拟变量']
).sort_values(['股票代码', '会计年度']).reset_index(drop=True)

```

```

df_long_match = pd.DataFrame(
    [(k[0], k[1], v) for k, v in long_term_dict.items()],
    columns=['股票代码', '会计年度', '是否新增长期借款虚拟变量']
).sort_values(['股票代码', '会计年度']).reset_index(drop=True)

with pd.ExcelWriter(match_file, engine='openpyxl') as writer:
    df_short_match.to_excel(writer, sheet_name='Sheet1', index=False)
    df_long_match.to_excel(writer, sheet_name='Sheet2', index=False)

match_file_path = match_file

print(f"    ✓ 匹配表已保存: {os.path.basename(match_file)}")

print(f"    ✓ 路径已存储至变量供后续读取")

# =====
# 【短期借款剔除】步骤 (4) (5) (6)
# =====

print("\n    阶段二：短期借款筛选...")

# (4) 读取匹配表 Sheet1，生成白名单列表（严格按步骤要求生成"列表"）
print("    [步骤 4] 从匹配表读取 Sheet1，生成白名单列表...")
df_match_short = pd.read_excel(match_file_path, sheet_name='Sheet1')
df_match_short['股票代码'] = safe_format_code(df_match_short['股票代码'])

# 清理 NaN 并生成列表（步骤要求的"列表"结构）
clean_short = df_match_short.dropna(subset=['股票代码', '会计年度'])
white_list_short = list(zip(
    clean_short['股票代码'],
    clean_short['会计年度'].astype(int)
))

print(f"    ✓ 白名单列表包含 {len(white_list_short):,} 个元组")

# (5) 读取目标 Sheet2，使用向量化哈希筛选（避免逐行循环，但保留进度显示）
print("    [步骤 5] 读取目标文件 Sheet2 并筛选...")
df_target_short = pd.read_excel(target_file, sheet_name='新增不同期限短期借款')
df_target_short['股票代码'] = safe_format_code(df_target_short['股票代码'])

print(f"    原数据共 {len(df_target_short):,} 行，准备筛选...")

#    高性能筛选核心：MultiIndex isin（C 级优化，无 Python 循环）

```

```

# 添加进度回调：对于极大数据集，分块显示进度（模拟）
if len(df_target_short) > 100000:
    print("    大数据集 detected，使用分块向量化筛选...")
    chunk_size = 50000
    chunks = []
    for start in tqdm(range(0, len(df_target_short), chunk_size), desc="    分块处理"):
        chunk = df_target_short.iloc[start:start+chunk_size]
        idx_chunk = pd.MultiIndex.from_arrays([
            chunk['股票代码'],
            chunk['会计年度']
        ])
        mask_chunk = idx_chunk.isin(white_list_short)
        chunks.append(chunk[mask_chunk])
    df_short_filtered = pd.concat(chunks)
else:
    # 常规向量化筛选（瞬时完成）
    idx_short = pd.MultiIndex.from_arrays([
        df_target_short['股票代码'],
        df_target_short['会计年度']
    ])
    mask_short = idx_short.isin(white_list_short)
    df_short_filtered = df_target_short[mask_short].copy()

removed_short = len(df_target_short) - len(df_short_filtered)

print(f"    ✓ 筛选完成：保留 {len(df_short_filtered):,} 行，删除 {removed_short:,} 行")

# (6) 保存短期借款结果（严格按照要求命名列并添加标题行）
print("    [步骤 6] 保存短期借款最终结果...")
short_result_file = os.path.join(desktop_path, f"不同期限借款（保持原始缺失值）_{timestamp}.xlsx")

# 确保列顺序完全匹配要求
required_cols = ['股票代码', '会计年度', '统计截止日期', '本年新增短期借款合计（万元）']
# 补全缺失列（防止源文件列名缺失）
for col in required_cols:
    if col not in df_short_filtered.columns:
        df_short_filtered[col] = None

with pd.ExcelWriter(short_result_file, engine='openpyxl') as writer:
    df_short_filtered[required_cols].to_excel(
        writer,
        sheet_name='新增不同期限短期借款',
        index=False # 不保留 DataFrame 索引，确保第一行是列标题
    )

```

```

    )

short_result_path = short_result_file

print(f"    ✓ 短期借款结果已保存，路径已存储")

# =====
# 【长期借款剔除】步骤 (7) (8) (9)
# =====

print("\n    阶段三：长期借款筛选...")

# (7) 读取匹配表 Sheet2，生成长期借款白名单列表
print("    [步骤 7] 从匹配表读取 Sheet2，生成长期借款白名单列表...")
df_match_long = pd.read_excel(match_file_path, sheet_name='Sheet2')
df_match_long['股票代码'] = safe_format_code(df_match_long['股票代码'])

clean_long = df_match_long.dropna(subset=['股票代码', '会计年度'])
white_list_long = list(zip(
    clean_long['股票代码'],
    clean_long['会计年度'].astype(int)
))

print(f"    ✓ 白名单列表包含 {len(white_list_long):,} 个元组")

# (8) 读取目标 Sheet3，向量化筛选
print("    [步骤 8] 读取目标文件 Sheet3 并筛选...")
df_target_long = pd.read_excel(target_file, sheet_name='新增不同期限长期借款')
df_target_long['股票代码'] = safe_format_code(df_target_long['股票代码'])

print(f"    原数据共 {len(df_target_long):,} 行，准备筛选...")

# 向量化筛选
idx_long = pd.MultiIndex.from_arrays([
    df_target_long['股票代码'],
    df_target_long['会计年度']
])
mask_long = idx_long.isin(white_list_long)
df_long_filtered = df_target_long[mask_long].copy()

removed_long = len(df_target_long) - len(df_long_filtered)

print(f"    ✓ 筛选完成：保留 {len(df_long_filtered):,} 行，删除 {removed_long:,} 行")

```

```

# (9) 追加长期借款结果到步骤(6)的文件（修正：使用"长期"列名而非用户笔误的"短期"）
print("    [步骤 9] 追加长期借款结果至同一文件...")
required_cols_long = ['股票代码', '会计年度', '统计截止日期', '本年新增长期借款合计（万元）']
]
for col in required_cols_long:
    if col not in df_long_filtered.columns:
        df_long_filtered[col] = None

# 使用追加模式写入 Sheet2
with pd.ExcelWriter(short_result_path, engine='openpyxl', mode='a', if_sheet_exists='replace')
as writer:
    df_long_filtered[required_cols_long].to_excel(
        writer,
        sheet_name='新增不同期限长期借款',
        index=False
    )

print("    ✓ 长期借款结果已追加为 Sheet2")

# =====
# 完成报告
# =====
print("\n" + "=" * 60)

print("    任务执行完毕！所有文件已保存至桌面")
print("=" * 60)

print(f"    输出文件清单:")

print(f"    1. 匹配表: {os.path.basename(match_file_path)}")
print(f"        - Sheet1 (短期): {len(df_short_match):,} 条记录")
print(f"        - Sheet2 (长期): {len(df_long_match):,} 条记录")
print(f"    2. 最终结果: {os.path.basename(short_result_path)}")
print(f"        - Sheet1 (短期): {len(df_short_filtered):,} 行 ({removed_short:,} 行已剔除)")
print(f"        - Sheet2 (长期): {len(df_long_filtered):,} 行 ({removed_long:,} 行已剔除)")

print(f"\n    性能特征:")

print(f"    - 匹配生成: 向量化字典推导 O(n)")
print(f"    - 筛选算法: MultiIndex 哈希查找 O(n)")
print(f"    - 内存安全: 自动处理 NaN 值，无类型转换错误")
print("=" * 60)

```

七、新增借款计算代码（按贷款事件统计）

http://localhost:8888/notebooks/Desktop/Python/%E6%AF%95%E4%B8%9A%E8%AE%BA%E6%96%87_%E6%96%B0%E5%A2%9E%E5%80%9F%E6%AC%BE_20260128%E4%BF%AE%E6%94%B9.ipynb

注：1、所有代码运行前需把 excel 文件放到桌面上；2、代码中的路径只适用于 mac 系统，windows 系统需要对路径代码进行修改；3、需要把桌面上的文件名称修改至与代码中完全相同。

kimi 指令：

我现在需要你帮我设计一段 python 代码：

一、现有数据：

一张 excel 表格，第一行中文列标题。中文列标题从左到右依次为：股票代码、会计年度、统计截止日期、贷款类型 1、本年新增合计（万元）

二、我的需求

1、任务目标：将每个股票代码对应的同一会计年度的短期借款和长期借款金额相加，得到该上市公司本年度新增借款金额合计数

2、逻辑：同一公司同一年度新增短期借款和长期借款之和即为该公司该年的新增借款总额

3、具体操作：

（1）读取原始数据：读取位于 mac 桌面命名为 0 不同期限借款（手工整理）.xlsx 的文件。注意，由于股票代码存在左侧数据为 0 的情况，所以一定要始终以字符串的格式储存，保证长度一直为 6 位。

（2）新建一个字典，对原始数据 excel 除标题外的所有行进行遍历，将每一行中股票代码和会计年度所在列的数据组合为一个元组，作为键值对的键，将该行对应的本年新增合计（万元）列对应的数据作为键值对的值。由于原始数据中，每个股票代码每个年份有两行数据，分别对应该公司该年份新增长期借款和短期借款的金额，所以相同的（股票代码，会计年度）元组一定会出现两次，当第二次出现时，令对应的键值对的值=原来的值+第二次出现该元组的行对应的本年新增合计（万元）的金额。

（3）输出结果，以 excel 的形式保存到 mac 的桌面，命名为：新增借款（按贷款事件统计）_对应的时间戳.xlsx。其中，这个 excel 的从左到右每一列的列标题依次为：股票代码、会计年度、本年新增合计（万元）。

注意：（1）鉴于使用循环会导致代码运行效率低下，你可以给出可行的替代方案优化程序；

（2）我希望这个代码可以实时显示运行进度。

```
import pandas as pd
import os
import warnings
from datetime import datetime
```

```

warnings.filterwarnings('ignore')

def main():
    try:
        # ===== 1. 读取原始数据 =====
        print("\n 【步骤 1/3】 正在读取原始数据文件...")

        # 文件路径（包含空格和中文括号）
        file_path = os.path.expanduser("~/Desktop/0 不同期限借款（手工整理）.xlsx")

        if not os.path.exists(file_path):
            raise FileNotFoundError(f"找不到文件: {file_path}")

        # 读取数据
        df = pd.read_excel(file_path, engine='openpyxl')

        # 清理列名
        df.columns = df.columns.str.strip()

        # 验证必需列
        required_cols = ['股票代码', '会计年度', '本年新增合计（万元）']
        missing_cols = [col for col in required_cols if col not in df.columns]
        if missing_cols:

            print("\n Excel 中缺少以下必需列: ")

            for col in missing_cols:
                print(f" - '{col}'")

            print("\n Excel 中实际存在的列: ")

            for i, col in enumerate(df.columns, 1):
                print(f" {i}. '{col}'")

            return

        # 确保股票代码为 6 位字符串（修复：处理"1.0"等情况）
        df['股票代码'] = df['股票代码'].astype(str)
        df['股票代码'] = df['股票代码'].str.split('.').str[0].str.zfill(6)

        # 确保金额为数值类型（修复：防止非数值导致错误）
        df['本年新增合计（万元）'] = pd.to_numeric(df['本年新增合计（万元）'],
errors='coerce')

        print(f" 原始数据共 {len(df):,} 行")

        print(f" 涉及 {df['股票代码'].nunique()} 家公司, {df['会计年度'].nunique()}

```

个年度")

```
# ===== 2. 分组聚合计算 =====
print("\n 【步骤 2/3】 正在计算每个公司每年的新增借款合计...")

# 核心操作：按股票代码和会计年度分组求和
result_df = df.groupby(['股票代码', '会计年度'], as_index=False)['本年新增合计 (万元)']
result_df.sum()

# 排序
result_df = result_df.sort_values(['股票代码', '会计年度']).reset_index(drop=True)

print(f"      聚合后数据共 {len(result_df):,} 行")
print(f"      每条记录代表一家公司在一年度的新增借款总额")

# ===== 3. 输出结果 =====
print("\n 【步骤 3/3】 正在生成输出文件...")

# 生成时间戳
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
output_path = os.path.expanduser(f"~/Desktop/新增借款（按贷款事件统计）_{timestamp}.xlsx")

# 保存为 Excel
with pd.ExcelWriter(output_path, engine='xlsxwriter') as writer:
    result_df.to_excel(writer, index=False, sheet_name='新增借款合计')

    # 设置格式
    workbook = writer.book
    worksheet = writer.sheets['新增借款合计']
    worksheet.set_column('A:A', 10) # 股票代码列
    worksheet.set_column('B:B', 12) # 会计年度列
    worksheet.set_column('C:C', 20) # 金额列

print(f"      文件已成功保存到: {output_path}")

print(f"      结果数据: {result_df.shape[0]:,} 行 × {result_df.shape[1]} 列")

print("\n      所有任务执行完毕! ")

except FileNotFoundError as e:
    print(f"\n      错误: {str(e)}")
```

```

        print("    请确认文件已保存在 Mac 桌面且文件名为【0 不同期限借款（手工整理）.xlsx】")
    except Exception as e:

        print(f"\n    发生未知错误: {str(e)}")

    import traceback
    print("\n 详细错误信息: ")
    traceback.print_exc()

if __name__ == '__main__':
    main()

```

20250128 新增步骤

我现在需要你帮我设计一段 python 代码：

一、现有数据：

两张 excel 表格

第一张表为位于 mac 桌面上命名为 1 新增借款(按贷款事件统计)_20260125_181428.xlsx 的 excel 表格，统计了每家上市公司 2013-2024 年每年新增借款总额的情况。第一行为中文列标题，，从左到右依次为：股票代码、会计年度、本年新增合计（万元）

第二张为位于 mac 桌面上命名为 不同期限借款匹配表（贷款类型 1）_20260129_122834.xlsx 的 excel 表格，统计了 2013-2024 年期间各个（股票代码，会计年度）的短期借款和长期借款存在情况。由两张 sheet 组成。第一张 sheet 标题为 Sheet1”，第一行为中文标题，从左到右依次为：股票代码、会计年度、是否新增短期借款虚拟变量；第二张 sheet 标题为 Sheet2”，第一行为中文标题，从左到右依次为：股票代码、会计年度、否新增长期借款虚拟变量。

二、我的需求

1、任务目标：（1）将不同期限借款匹配表（贷款类型 1）_20260129_122834.xlsx 的两张 sheet 合并，生成一张新的新增借款匹配表，记录各个（股票代码，会计年度）是否存在新增借款（短期借款或长期借款）的虚拟变量；（2）使用任务（1）生成的匹配表，剔除 1 新增借款（按贷款事件统计）_20260125_181428.xlsx 中所有不存在新增借款的（股票代码，会计年度）代表的行，即剔除 1 新增借款（按贷款事件统计）_20260125_181428.xlsx 中所有匹配表中没有记录的（股票代码，会计年度）所在的行。

2、具体步骤：

（1）读取位于 mac 桌面命名为 不同期限借款匹配表（贷款类型 1）_20260129_122834.xlsx 的文件。注意，由于股票代码存在左侧数据为 0 的情况，所以一定要始终以字符串的格式储存，保证长度一直为 6 位。

（2）新建字典，对 不同期限借款匹配表（贷款类型 1）_20260129_122834.xlsx 文件中 Sheet1 和 Sheet2 除标题行外的所有行依次遍历，将一行中的（股票代码，会计年度）组成元组作为该字典的一个键值对的键，同时将键值对的值写为 1，后续遍历识别到相同的（股票代码，会计年度）时直接跳过。

(3) 输出中间结果表格，以 excel 的形式保存到 mac 的桌面，命名为：不同期限借款匹配表（新增借款合计）_对应的时间戳.xlsx。从左到右每一列的列标题依次为：股票代码、会计年度、是否新增借款虚拟变量，其中“是否新增借款虚拟变量”则是第（2）步生成的字典的每一个键值对的值。并且在表格的最上方添加列标题行。在保存 excel 到桌面的同时，读取文件路径并存储在变量中，便于第（4）步直接读取。

(4) 直接使用第（3）步存储的路径，读取第（3）步存储的文件。新建一个新增借款白名单列表，将第（3）步存储的中间结果文件的每一行的（股票代码，会计年度）组成元组，作为列表的一个元素。

(5) 读取 mac 桌面上命名为 1 新增借款（按贷款事件统计）_20260125_181428.xlsx 的表格除了标题行的所有行，每读取一行数据，判断该行中的（股票代码，会计年度）是否在第

(5) 步生成的新增借款白名单列表中出现过，如果出现过，则保留该行，如果列表中没有该行的（股票代码，会计年度）元组，则认为该公司该年份在 不同期限借款匹配表（贷款类型 1）_20260129_122834.xlsx 中没有新增借款数据，删除 1 新增借款（按贷款事件统计）_20260125_181428.xlsx 的对应行。

(6) 输出最终结果，将删除部分行的 1 新增借款（按贷款事件统计）_20260125_181428.xlsx 的剩余数据行以 excel 的形式保存到 mac 的桌面，命名为：新增借款（保持原始缺失值）_对应的时间戳.xlsx。其中，这个 excel 的从左到右每一列的列标题依次为：股票代码、会计年度、本年新增合计（万元）。并在表格的最上方添加列标题行。

注意：（1）鉴于使用循环会导致代码运行效率低下，你可以给出可行的替代方案优化程序；
（2）我希望这个代码可以实时显示运行进度。

请仔细阅读上述文字，并告诉我你是否已经完全理解所有要求。

```
import pandas as pd
import os
from datetime import datetime
from tqdm import tqdm
import sys

# =====
# 配置与初始化
# =====
desktop_path = os.path.join(os.path.expanduser("~"), "Desktop")

# 输入文件路径（严格匹配用户给定的文件名）
match_source = os.path.join(desktop_path, "不同期限借款匹配表（贷款类型 1）_20260129_122834.xlsx")
target_file = os.path.join(desktop_path, "1 新增借款（按贷款事件统计）_20260125_181428.xlsx")

# 文件存在性检查

print("    检查输入文件...")
```

```

for f in [match_source, target_file]:
    if not os.path.exists(f):

        print(f"    错误：文件不存在 - {f}")

        sys.exit(1)

    print(f"    ✓ {os.path.basename(f)}")

timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")

print(f"\n    启动处理 | 时间戳: {timestamp}")

print("    优化策略: 向量化字典生成 + 哈希集合筛选 + 类型安全转换")

# =====
# 【生成匹配表】步骤 (1) (2) (3)
# =====

print("\n    阶段一：生成新增借款匹配表...")

# (1) 读取两个 Sheet，格式化股票代码
print("    [步骤 1] 读取 Sheet1(短期)和 Sheet2(长期)...")
df_short = pd.read_excel(match_source, sheet_name='Sheet1')
df_long = pd.read_excel(match_source, sheet_name='Sheet2')

print(f"    Sheet1: {len(df_short):,} 行 | Sheet2: {len(df_long):,} 行")

# 统一格式化股票代码为 6 位字符串（向量化）
df_short['股票代码'] = df_short['股票代码'].astype(str).str.split('.').str[0].str.zfill(6)
df_long['股票代码'] = df_long['股票代码'].astype(str).str.split('.').str[0].str.zfill(6)

# (2) 合并生成字典（向量化实现 O(n)去重，严格满足"字典"要求）
print("    [步骤 2] 合并生成匹配字典（自动去重）...")

# 合并两个 DataFrame，只保留关键列
combined = pd.concat([
    df_short[['股票代码', '会计年度']],
    df_long[['股票代码', '会计年度']]
], ignore_index=True)

# 清理无效数据（关键：避免 astype(int)崩溃）
combined_clean = combined.dropna(subset=['股票代码', '会计年度'])
combined_clean['会计年度'] = combined_clean['会计年度'].astype(int)

```

```

print(f"    合并后总记录: {len(combined_clean):,} 行, 去重中...")

# 使用 dict.fromkeys 实现 O(n)去重并赋值 1 (符合"字典"要求, 自动跳过重复键)
tuples_iter = tqdm(
    zip(combined_clean['股票代码'], combined_clean['会计年度']),
    total=len(combined_clean),
    desc="    生成字典键"
)
new_loan_dict = dict.fromkeys(tuples_iter, 1)

print(f"    ✓ 匹配字典生成完毕: {len(new_loan_dict):,} 个唯一键")

# (3) 保存中间结果到 Excel
print("    [步骤 3] 保存匹配表...")
match_output = os.path.join(desktop_path, f"不同期限借款匹配表 (新增借款合计)_{timestamp}.xlsx")

# 转换字典为 DataFrame
match_df = pd.DataFrame(
    [(k[0], k[1], v) for k, v in new_loan_dict.items()],
    columns=['股票代码', '会计年度', '是否新增借款虚拟变量']
).sort_values(['股票代码', '会计年度']).reset_index(drop=True)

with pd.ExcelWriter(match_output, engine='openpyxl') as writer:
    match_df.to_excel(writer, index=False) # 第一行为列标题

match_file_path = match_output

print(f"    ✓ 匹配表已保存: {os.path.basename(match_output)}")

print(f"    ✓ 路径已存储至变量")

# =====
# 【数据筛选】步骤 (4) (5) (6)
# =====

print("\n    阶段二: 筛选新增借款数据...")

# (4) 严格按步骤: 从文件读取匹配表, 生成白名单列表 (关键修复: 处理 NaN 和类型)
print("    [步骤 4] 从文件加载匹配表, 生成白名单列表...")
df_match = pd.read_excel(match_file_path)

# 格式化股票代码

```

```

df_match['股票代码'] = df_match['股票代码'].astype(str).str.split('.').str[0].str.zfill(6)

# 关键修复：清理 NaN 并确保会计年度为整数（与后续目标文件类型一致）
df_match_clean = df_match.dropna(subset=['股票代码', '会计年度'])
df_match_clean['会计年度'] = df_match_clean['会计年度'].astype(int)

# 生成白名单列表（符合"列表"要求）
white_list = list(zip(
    df_match_clean['股票代码'],
    df_match_clean['会计年度']
))

print(f"    ✓ 白名单列表: {len(white_list):,} 个元组")

# 转换为 Set 实现 O(1) 哈希查找（优化核心）
white_set = set(white_list)

# (5) 读取目标文件，向量化筛选（关键修复：类型一致性处理）
print("    [步骤 5] 读取目标文件并筛选...")
df_target = pd.read_excel(target_file)

# 格式化股票代码
df_target['股票代码'] = df_target['股票代码'].astype(str).str.split('.').str[0].str.zfill(6)

# 关键修复：清理数据并统一会计年度为整数（确保与 white_set 中的 int 类型匹配）
df_target_clean = df_target.dropna(subset=['股票代码', '会计年度']).copy()
df_target_clean['会计年度'] = df_target_clean['会计年度'].astype(int)

print(f"    目标文件共 {len(df_target_clean):,} 行，开始筛选...")

# 高性能筛选：使用 MultiIndex isin（无 Python 级循环，C 语言优化）
if len(df_target_clean) > 50000:
    chunk_size = 10000
    filtered_chunks = []
    chunks_count = (len(df_target_clean) + chunk_size - 1) // chunk_size

    for i in tqdm(range(0, len(df_target_clean), chunk_size), total=chunks_count, desc="
分块筛选"):
        chunk = df_target_clean.iloc[i:i+chunk_size]
        idx = pd.MultiIndex.from_arrays([chunk['股票代码'], chunk['会计年度']])
        mask = idx.isin(white_set)
        filtered_chunks.append(chunk[mask])

```

```

        df_filtered = pd.concat(filtered_chunks, ignore_index=True)
else:
    # 小数据量瞬时处理
    idx = pd.MultiIndex.from_arrays([df_target_clean['股票代码'], df_target_clean['会计年度']])
    mask = idx.isin(white_set)
    df_filtered = df_target_clean[mask].copy()

removed = len(df_target) - len(df_filtered) # 基于原始长度计算删除数

print(f"    ✓ 筛选完成: 保留 {len(df_filtered):,} 行, 删除 {removed:,} 行")

print(f"    保留率: {len(df_filtered)/len(df_target)*100:.1f}%")

# (6) 保存最终结果 (严格按要求列名)
print("    [步骤 6] 保存最终结果...")
final_output = os.path.join(desktop_path, f"新增借款 (保持原始缺失值)_{timestamp}.xlsx")

# 确保列顺序完全匹配要求
output_cols = ['股票代码', '会计年度', '本年新增合计 (万元)']
for col in output_cols:
    if col not in df_filtered.columns:
        df_filtered[col] = None

with pd.ExcelWriter(final_output, engine='openpyxl') as writer:
    df_filtered[output_cols].to_excel(writer, index=False)

print(f"    ✓ 最终结果已保存: {os.path.basename(final_output)}")

# =====
# 完成报告
# =====
print("\n" + "=" * 60)

print("    任务执行完毕! ")

print("=" * 60)

print(f"    输出文件:")

print(f"    1. 匹配表: {os.path.basename(match_file_path)}")
print(f"        - 记录数: {len(match_df):,} 个唯一(股票代码,年份)")
print(f"    2. 筛选结果: {os.path.basename(final_output)}")
print(f"        - 原数据: {len(df_target):,} 行 (含可能的空值)")
print(f"        - 有效筛选: {len(df_target_clean):,} 行")
print(f"        - 最终保留: {len(df_filtered):,} 行 | 剔除: {removed:,} 行")

```

```
print(f"\n    性能指标:")

print(f"    - 字典生成: 向量化  $O(n)$ ")
print(f"    - 数据筛选: 哈希查找  $O(n)$  vs 线性查找  $O(n^2)$ ")
print(f"    - 类型安全: 统一 int 类型, 避免 float/int 不匹配导致的遗漏")
print("=" * 60)
```

八、新增不同种类借款代码（按贷款事件统计）

http://localhost:8888/notebooks/Desktop/Python/%E6%AF%95%E4%B8%9A%E8%AE%BA%E6%96%87_%E6%96%B0%E5%A2%9E%E5%80%9F%E6%AC%BE_20260128%E4%BF%AE%E6%94%B9.ipynb

注：1、所有代码运行前需把 excel 文件放到桌面上；2、代码中的路径只适用于 mac 系统，windows 系统需要对路径代码进行修改；3、需要把桌面上的文件名称修改至与代码中完全相同。

kimi 指令：

我现在需要你帮我设计一段 python 代码：

一、现有数据：

一张 excel 表格，第一行为英文列标题，第二行为中文列标题。中文列标题从左到右依次为：证券代码、公告日期、借款事件年份、贷款进程、本币金额（万元）、贷款期限、贷款类型 1、贷款类型 2

其中贷款类型 1 所在列的值有两种：长期借款和短期借款；贷款类型 2 所在列的值有 6 种：抵押贷款、质押贷款、担保借款、信用借款、保证借款、其他授信项
这张表里统计了每家上市公司 2013-2024 年每年新增不同期限借款的情况。

二、我的需求

1、任务目标：（1）分别统计每家上市公司每年的新增抵押贷款、新增质押借款、新增担保贷款、新增信用贷款、新增保证借款、新增其他授信项金额；（2）补齐每家上市公司的数据行数。

2、具体步骤：

（1）读取位于 mac 桌面命名为 0 22-CSMAR-不同期限银行借款_副本.xlsx 的文件。注意，由于股票代码存在左侧数据为 0 的情况，所以一定要始终以字符串的格式储存，保证长度一直为 6 位。

（2）新增一列“本年新增合计（万元）”统计每家上市公司每个有数据的年份的当年新增短期借款和新增长期借款，每个证券代码每个借款事件年份对应一个合计数：举例：股票代码为 xxxxxx 的上市公司在 2013-2024 年每年都有借款事件发生，那么该公司 2013 年的新增抵押贷款总额即为证券代码列为 xxxxxx 且借款事件年份列为 2013 且贷款类型 2 为抵押贷款的所有行对应的“本币金额（万元）”的加总；如果该公司 2013 年没有新增抵押贷款，那么在原表格中找不到同时满足证券代码列为 xxxxxx 且借款事件年份列为 2013 且贷款类型 2 为抵押贷款的行，则新增一行，分别令证券代码为 xxxxxx，借款事件年份为 2013，贷款类型 2 为抵押贷款，“本年新增合计（万元）”的统计值为 0，公告日期、贷款进程、贷款期限、贷款类型 1 所在列均保持空值。新增这一行的含义是，说明这家公司在 2013 年新增抵押贷款的金额为 0。依次类推，每家公司每个年度应该有 6 行数据分别统计当年新增抵押贷款、新增质押借款、新增担保贷款、新增信用贷款、新增保证借款、新增其他授信项的合计数。

（3）输出结果，以 excel 的形式保存到 mac 的桌面，命名为：不同种类借款（按贷款事件统计）_对应的时间戳.xlsx。其中，这个 excel 的从左到右每一列的列标题依次为：证券代码、借款事件年份、贷款类型 2、本年新增合计（万元）。并且在表格的最上方添加列标题行。

```

import pandas as pd
import numpy as np
import os
import warnings
from datetime import datetime

warnings.filterwarnings('ignore')

def main():
    try:
        # ===== 1/5 读取原始数据 =====
        print("\n 【步骤 1/5】 正在读取原始数据文件...")
        file_path = os.path.expanduser("~/Desktop/0 22-CSMAR- 不同期限银行借款_副本.xlsx")

        if not os.path.exists(file_path):
            raise FileNotFoundError(f"找不到文件: {file_path}")

        # 读取 Excel, 跳过第一行英文标题
        df = pd.read_excel(file_path, engine='openpyxl', header=1)

        # 清理列名
        df.columns = df.columns.str.strip()

        print(f"      成功读取 {len(df):,} 行原始数据")

        # 验证必需列
        required_cols = ['证券代码', '借款事件年份', '本币金额 (万元)', '贷款类型 2']
        missing_cols = [col for col in required_cols if col not in df.columns]
        if missing_cols:
            raise ValueError(f"Excel 中缺少必需列: {missing_cols}")

        # ===== 2/5 数据预处理与类型映射 =====
        print("\n 【步骤 2/5】 正在预处理并映射贷款类型...")

        # 处理证券代码: 转为字符串并补零至 6 位
        df['证券代码'] = df['证券代码'].astype(str)
        df['证券代码'] = df['证券代码'].str.split('.').str[0].str.zfill(6)

        # 转换金额为数值类型
        df['本币金额 (万元)'] = pd.to_numeric(df['本币金额 (万元)'], errors='coerce')

```

```

# 定义映射关系 (保留全部 6 种类型)
type_mapping = {
    '抵押贷款': '抵押贷款',
    '质押贷款': '质押贷款',
    '担保借款': '担保贷款',    # 名称标准化
    '信用借款': '信用贷款',    # 名称标准化
    '保证借款': '保证借款',
    '其他授信项': '其他授信项' # 新增: 保留原始类型
}

df['贷款类型 2'] = df['贷款类型 2'].map(type_mapping)

# 删除无法映射的行 (值不在上述 6 种中的)
df = df[df['贷款类型 2'].notna()].copy()

print(f"      预处理完成: 涉及 {df['证券代码'].nunique()} 家公司")

print(f"      贷款类型包含: {df['贷款类型 2'].unique().tolist()}")

# ===== 3/5 分组聚合计算 =====
print("\n【步骤 3/5】正在计算分组聚合金额...")

# 提取关键列
df = df[['证券代码', '借款事件年份', '本币金额 (万元)', '贷款类型 2']].copy()

# 按维度分组求和 (向量化操作)
df_agg = df.groupby(['证券代码', '借款事件年份', '贷款类型 2'], as_index=False)['本币
金额 (万元)'].sum()

print(f"      聚合后得到 {len(df_agg):,} 条有效记录")

# ===== 4/5 构建完整数据矩阵 =====
print("\n【步骤 4/5】正在生成完整数据矩阵 (补齐缺失行) ...")

# 定义完整的维度范围 (6 种贷款类型)
all_stocks = df['证券代码'].unique()
all_years = range(2013, 2025) # 2013-2024
all_loan_types = ['抵押贷款', '质押贷款', '担保贷款', '信用贷款', '保证借款', '其他授
授信项']

# 创建完整的 MultiIndex (笛卡尔积)

```

```

complete_index = pd.MultiIndex.from_product(
    [all_stocks, all_years, all_loan_types],
    names=['证券代码', '借款事件年份', '贷款类型 2']
)

print(f"    理论应生成 {len(all_stocks)} 家公司 × {len(all_years)} 年 ×
{len(all_loan_types)} 类 = {len(complete_index):,} 行")

# 设置聚合结果的索引
df_agg.set_index(['证券代码', '借款事件年份', '贷款类型 2'], inplace=True)

# 重索引：补齐缺失组合，缺失值填充为 0
df_complete = df_agg.reindex(complete_index, fill_value=0).reset_index()

print(f"    补齐后共 {len(df_complete):,} 行")

# ===== 5/5 生成输出文件 =====
print("\n【步骤 5/5】正在生成 Excel 文件...")

# 重命名金额列
df_complete.rename(columns={'本币金额（万元）': '本年新增合计（万元）'},
inplace=True)

# 按新要求的列顺序输出
output_cols = ['证券代码', '借款事件年份', '贷款类型 2', '本年新增合计（万元）']
df_output = df_complete[output_cols]

# 排序
df_output = df_output.sort_values(['证券代码', '借款事件年份', '贷款类型
2']).reset_index(drop=True)

# 生成时间戳和输出路径
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
output_path = os.path.expanduser(f"~/Desktop/不同种类借款（按贷款事件统计）
_{timestamp}.xlsx")

# 保存文件
with pd.ExcelWriter(output_path, engine='xlsxwriter') as writer:
    df_output.to_excel(writer, index=False, sheet_name='数据')

# 设置列宽
workbook = writer.book
worksheet = writer.sheets['数据']

```

```

        worksheet.set_column('A:A', 12) # 证券代码
        worksheet.set_column('B:B', 12) # 借款事件年份
        worksheet.set_column('C:C', 15) # 贷款类型 2
        worksheet.set_column('D:D', 20) # 本年新增合计 (万元)

    print(f"\n    完成! 文件已保存至: {output_path}")

    print(f"        最终数据: {df_output.shape[0]:,}行 × {df_output.shape[1]}列")

except FileNotFoundError as e:

    print(f"\n    错误: {str(e)}")

    print("    请确认文件已保存在 Mac 桌面且文件名为【0 22-CSMAR-不同期限银行
借款_副本.xlsx】")
except Exception as e:

    print(f"\n    发生未知错误: {str(e)}")

    import traceback
    print("\n 详细错误信息: ")
    traceback.print_exc()

if __name__ == '__main__':
    main()

```

20250128 新增步骤

我现在需要你帮我设计一段 python 代码:

一、现有数据:

两张 excel 表格

第一张表为位于 mac 桌面上命名为 0 22-CSMAR-不同期限银行借款_副本.xlsx 的 excel 表格, 统计了每家上市公司 2013-2024 年每年新增不同期限借款的情况。第一行为英文列标题, 第二行为中文列标题。中文列标题从左到右依次为: 股票代码、公告日期、借款事件年份、贷款进程、本币金额 (万元)、贷款期限、贷款类型 1、贷款类型 2。其中贷款类型 1 所在列的值有两种: 长期借款和短期借款; 贷款类型 2 所在列的值有 6 种: 抵押贷款、质押贷款、担保贷款、信用贷款、保证借款、其他授信项。

第二张表格为位于 mac 桌面上命名为 1 不同种类借款 (按贷款事件统计) _20260125_185736.xlsx 的 excel 表格, 是由第一张表统计得到的每家公司每年新增抵押贷款、新增信用贷款、新增保证借款、新增质押贷款、新增担保借款、新增其他授信项的数据。由七张 sheet 组成:

第一张 sheet 标题为“总表”, 本次操作不涉及总表;

第二张 sheet 标题为“新增抵押贷款”，第一行为中文标题，从左到右依次为：股票代码、会计年度、贷款类型 2、本年新增合计（万元）；

第三张 sheet 标题为“新增信用贷款”，第一行为中文标题，从左到右依次为：股票代码、会计年度、贷款类型 2、本年新增合计（万元）；

第四张 sheet 标题为“新增保证借款”，第一行为中文标题，从左到右依次为：股票代码、会计年度、贷款类型 2、本年新增合计（万元）；

第五张 sheet 标题为“新增质押贷款”，第一行为中文标题，从左到右依次为：股票代码、会计年度、贷款类型 2、本年新增合计（万元）；

第六张 sheet 标题为“新增担保贷款”，第一行为中文标题，从左到右依次为：股票代码、会计年度、贷款类型 2、本年新增合计（万元）；

第六张 sheet 标题为“新增其他授信项”，第一行为中文标题，从左到右依次为：股票代码、会计年度、贷款类型 2、本年新增合计（万元）。

二、我的需求

1、任务目标：（1）从 0 22-CSMAR-不同期限银行借款_副本.xlsx 表格中分别统计出各个股票代码代表的公司存在新增短期借款和新增长期借款的年份，分别生成两张匹配表用于匹配任务；（2）使用任务（1）得到的两张匹配表，分别剔除 2 不同期限借款（手工整理）.xlsx 的 sheet2 中所有 0 22-CSMAR-不同期限银行借款_副本.xlsx 表格中不存在新增短期借款的（股票代码，会计年度）代表的行，剔除 2 不同期限借款（手工整理）.xlsx 的 sheet3 中所有 0 22-CSMAR-不同期限银行借款_副本.xlsx 表格中不存在新增长期借款的（股票代码，会计年度）代表的行。

2、具体步骤：

【生成匹配表】

（1）读取位于 mac 桌面命名为 0 22-CSMAR-不同期限银行借款_副本.xlsx 的文件。注意，由于股票代码存在左侧数据为 0 的情况，所以一定要始终以字符串的格式储存，保证长度一直为 6 位。

（2）对 0 22-CSMAR-不同期限银行借款_副本.xlsx 除标题外的每一行进行遍历，新建六个字典，分别统计每个（股票代码，会计年度）新增抵押贷款、新增信用贷款、新增保证借款、新增质押贷款、新增担保借款、新增其他授信项的存在情况。遍历时，将每一行的（股票代码，会计年度）组成元组，分别写入六个字典，作为一个键值对的键，同时判断该行贷款类型为新增抵押贷款、新增信用贷款、新增保证借款、新增质押贷款、新增担保借款、新增其他授信项，当该股票代码该年第一次出现新增抵押贷款（或新增信用贷款、新增保证借款、新增质押贷款、新增担保借款、新增其他授信项）时，则将对应该贷款类型字典的键值对的值写为 1，之后的遍历过程再遇到相同的（股票代码，会计年度）组合时，直接跳过，继续遍历下一行。

（3）输出中间结果表格，以 excel 的形式保存到 mac 的桌面，命名为：不同期限借款匹配表（贷款类型 2）_对应的时间戳.xlsx。由于第（2）步生成了六个不同的字典，分别统计新增抵押贷款、新增信用贷款、新增保证借款、新增质押贷款、新增担保借款、新增其他授信项（股票代码，会计年度）的出现情况，这个 excel 须由六张 sheet 组成：第一张 sheet 保存抵押贷款字典的结果，从左到右每一列的列标题依次为：股票代码、会计年度、是否新增抵押贷款虚拟变量，其中“是否新增抵押贷款虚拟变量”则是短期借款字典的键值对的值，并且在表格的最上方添加列标题行；依此类推，第二张到第六张 sheet 中分别保存信用贷款、保证借款、质押贷款、担保借款、其他授信项字典的结果，从左到右每一列的列标题依次为：股票代码、会计年度、是否新增 xxxx(x)虚拟变量，其中“是否新增 xxxx(x)虚拟变量”则是对

应字典的键值对的值，并且在表格的最上方添加列标题行。在保存 excel 到桌面的同时，读取文件路径并存储在变量中，便于后面直接读取。

【抵押贷款剔除】

(4) 直接使用第 (3) 步存储的路径，读取第 (3) 步存储的文件的 sheet1。新建一个抵押贷款白名单列表，将第 (3) 步存储的中间结果文件的 sheet1 中的每一行的（股票代码，会计年度）组成元组，作为列表的一个元素。。注意，由于股票代码存在左侧数据为 0 的情况，所以一定要始终以字符串的格式储存，保证长度一直为 6 位。

(5) 读取 mac 桌面上命名为 1 不同种类借款（按贷款事件统计）_20260125_185736.xlsx 的 excel 表格的 sheet2 新增抵押贷款”除了标题行的所有行，每读取一行数据，判断该行中的（股票代码，会计年度）是否在第 (4) 步生成的抵押贷款白名单列表中出现过，如果出现过，则保留该行，如果列表中没有该行的（股票代码，会计年度）元组，则认为该公司该年份在 0 22-CSMAR-不同期限银行借款_副本.xlsx 中没有新增抵押贷款数据，删除 1 不同种类借款（按贷款事件统计）_20260125_185736.xlsx 的 sheet2 的对应行。

(6) 输出抵押贷款最终结果，将删除部分行的 1 不同种类借款（按贷款事件统计）_20260125_185736.xlsx 的 sheet2 新增抵押贷款”的剩余数据行以 excel 的形式保存到 mac 的桌面，命名为：新增抵押贷款（按贷款事件统计）_对应的时间戳.xlsx。其中，这个 excel 的从左到右每一列的列标题依次为：股票代码、会计年度、新增抵押贷款（万元）。并在表格的最上方添加列标题行。

【信用贷款剔除】

(7) 直接使用第 (3) 步存储的路径，读取第 (3) 步存储的文件的 sheet2。新建一个信用贷款白名单列表，将第 (3) 步存储的中间结果文件的 sheet2 中的每一行的（股票代码，会计年度）组成元组，作为列表的一个元素。。注意，由于股票代码存在左侧数据为 0 的情况，所以一定要始终以字符串的格式储存，保证长度一直为 6 位。

(8) 读取 mac 桌面上命名为 1 不同种类借款（按贷款事件统计）_20260125_185736.xlsx 的 excel 表格的 sheet23 新增信用贷款”除了标题行的所有行，每读取一行数据，判断该行中的（股票代码，会计年度）是否在第 (4) 步生成的信用贷款白名单列表中出现过，如果出现过，则保留该行，如果列表中没有该行的（股票代码，会计年度）元组，则认为该公司该年份在 0 22-CSMAR-不同期限银行借款_副本.xlsx 中没有新增信用贷款数据，删除 1 不同种类借款（按贷款事件统计）_20260125_185736.xlsx 的 sheet3 的对应行。

(9) 输出信用贷款最终结果，将删除部分行的 1 不同种类借款（按贷款事件统计）_20260125_185736.xlsx 的 sheet3 新增信用贷款”的剩余数据行以 excel 的形式保存到 mac 的桌面，命名为：新增信用贷款（按贷款事件统计）_对应的时间戳.xlsx。其中，这个 excel 的从左到右每一列的列标题依次为：股票代码、会计年度、新增信用贷款（万元）。并在表格的最上方添加列标题行。

保证借款、质押贷款、担保借款、其他授信项的剔除方法依此类推。最终结果文件合计 6 个 excel 表格文件。

注意：(1) 鉴于使用循环会导致代码运行效率低下，你可以给出可行的替代方案优化程序；
(2) 我希望这个代码可以实时显示运行进度。

请仔细阅读上述文字，并告诉我你是否已经完全理解所有要求。

```

import pandas as pd
import os
from datetime import datetime
from tqdm import tqdm
import sys

# =====
# 配置与初始化
# =====
desktop_path = os.path.join(os.path.expanduser("~"), "Desktop")

source_file = os.path.join(desktop_path, "0 22-CSMAR-不同期限银行借款_副本.xlsx")
target_file = os.path.join(desktop_path, "1 不同种类借款（按贷款事件统计）_20260125_185736.xlsx")

print("    检查输入文件...")
for f in [source_file, target_file]:
    if not os.path.exists(f):
        print(f"    文件不存在: {f}")
        sys.exit(1)
    print(f"    ✓ {os.path.basename(f)}")

timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")

print(f"\n    启动处理 | 时间戳: {timestamp}")

# 配置：贷款类型名称、（匹配表 Sheet，目标文件 Sheet，虚拟变量列名，输出列名）
LOAN_CONFIG = [
    ('抵押贷款', 'Sheet1', '新增抵押贷款', '是否新增抵押贷款虚拟变量', '新增抵押贷款(万元)'),
    ('信用贷款', 'Sheet2', '新增信用贷款', '是否新增信用贷款虚拟变量', '新增信用贷款(万元)'),
    ('保证借款', 'Sheet3', '新增保证借款', '是否新增保证借款虚拟变量', '新增保证借款(万元)'),
    ('质押贷款', 'Sheet4', '新增质押贷款', '是否新增质押贷款虚拟变量', '新增质押贷款(万元)'),
    ('担保贷款', 'Sheet5', '新增担保贷款', '是否新增担保贷款虚拟变量', '新增担保贷款(万元)'),
    ('其他授信项', 'Sheet6', '新增其他授信项', '是否新增其他授信项虚拟变量', '新增其他授信项(万元)')
]

```

```

]

# =====
# 【生成匹配表】步骤 (1) (2) (3)
# =====

print("\n    阶段一：生成六类借款匹配字典与匹配表...")

# (1) 读取源数据（双行标题，header=1 读取中文行）
print("    [步骤 1] 读取源数据...")
df_source = pd.read_excel(source_file, header=1)

print(f"    ✓ 载入 {len(df_source):,} 行")

# 格式化证券代码为 6 位字符串（处理.0 后缀）
df_source['股票代码'] = df_source['股票代码'].astype(str).str.split('.').str[0].str.zfill(6)

# (2) 向量化生成 6 个字典（符合"字典"要求）
print("    [步骤 2] 生成匹配字典...")
df_clean = df_source.dropna(subset=['股票代码', '借款事件年份', '贷款类型 2'])
df_clean['借款事件年份'] = df_clean['借款事件年份'].astype(int)

loan_dicts = {}
for loan_name, _, _, _ in tqdm(LOAN_CONFIG, desc="    生成字典"):
    mask = df_clean['贷款类型 2'] == loan_name
    df_type = df_clean[mask]

    if len(df_type) > 0:
        # 使用 dict.fromkeys 自动去重，值为 1（符合要求）
        keys = zip(df_type['股票代码'], df_type['借款事件年份'])
        loan_dicts[loan_name] = dict.fromkeys(keys, 1)
    else:
        loan_dicts[loan_name] = {}

        print(f"    警告: {loan_name} 无数据")

print(f"    ✓ 字典统计:")

for loan_name, _, _, _ in LOAN_CONFIG:
    print(f"    - {loan_name}: {len(loan_dicts[loan_name]):,} 条")

# (3) 保存匹配表（6 Sheet）
print("    [步骤 3] 保存匹配表...")
match_file = os.path.join(desktop_path, f"不同期限借款匹配表（贷款类型 2）")

```

```

_{timestamp}.xlsx")

with pd.ExcelWriter(match_file, engine='openpyxl') as writer:
    for loan_name, sheet_name, _, var_col, _ in LOAN_CONFIG:
        dict_data = loan_dicts[loan_name]
        if len(dict_data) > 0:
            df_match = pd.DataFrame(
                [(k[0], k[1], v) for k, v in dict_data.items()],
                columns=['股票代码', '会计年度', var_col]
            ).sort_values(['股票代码', '会计年度']).reset_index(drop=True)
        else:
            df_match = pd.DataFrame(columns=['股票代码', '会计年度', var_col])

        df_match.to_excel(writer, sheet_name=sheet_name, index=False)

match_file_path = match_file

print(f"    ✓ 匹配表已保存，路径已存储")

# =====
# 【批量筛选】步骤 (4)-(6) 扩展到 6 种类型
# =====

print("\n    阶段二：六类借款数据筛选（六次中间结果读取均已强化股票代码处理）...")

def process_loan_filter(loan_name, match_sheet, target_sheet, var_col, output_col):
    """处理单类借款的筛选与输出，重点确保读取中间结果时股票代码格式正确"""
    print(f"\n    【{loan_name}】")

    # (4) 读取中间结果文件（匹配表）对应 Sheet
    try:
        df_match = pd.read_excel(match_file_path, sheet_name=match_sheet)
    except Exception as e:
        print(f"        读取匹配表失败: {e}")
        return None

    # 【关键修复】强制处理股票代码为 6 位字符串格式（防止 Excel 读取为数值丢失前导零）
    # 防御性检查：确保列存在
    if '股票代码' not in df_match.columns:
        print(f"        错误：匹配表缺少'股票代码'列")
        return None

```

```

# 处理流程：转字符串 -> 去除.0 后缀 -> 左侧补零至 6 位
df_match['股票代码'] = df_match['股票代码'].astype(str).str.split('.').str[0].str.zfill(6)

# 清理空值
df_match_clean = df_match.dropna(subset=['股票代码', '会计年度'])
if len(df_match_clean) == 0:

    print(f"        匹配表为空，跳过")

    return None

# 确保会计年度为整数
df_match_clean['会计年度'] = df_match_clean['会计年度'].astype(int)

# 生成白名单列表（符合步骤要求的"列表"结构）
white_list = list(zip(df_match_clean['股票代码'], df_match_clean['会计年度']))
# 转为 Set 优化查找性能（O(1)）
white_set = set(white_list)
print(f"    白名单: {len(white_set):,} 条")

# (5) 读取目标文件并筛选
try:
    df_target = pd.read_excel(target_file, sheet_name=target_sheet)
except Exception as e:

    print(f"        读取目标文件 Sheet '{target_sheet}' 失败: {e}")

    return None

if len(df_target) == 0:

    print(f"        目标数据为空")

    return None

# 同样处理目标文件的股票代码（防御性编程，确保格式一致）
if '股票代码' not in df_target.columns:

    print(f"        错误：目标文件缺少'股票代码'列")

    return None

df_target['股票代码'] = df_target['股票代码'].astype(str).str.split('.').str[0].str.zfill(6)

# 清理空值并确保类型一致
df_target_clean = df_target.dropna(subset=['股票代码', '会计年度']).copy()
if len(df_target_clean) == 0:

```

```

        print(f"        无有效数据行")

        return None

df_target_clean['会计年度'] = df_target_clean['会计年度'].astype(int)

print(f"    筛选: {len(df_target_clean):,} 行 → ", end="")

# 向量化筛选（使用哈希集合）
idx = pd.MultiIndex.from_arrays([df_target_clean['股票代码'], df_target_clean['会计年度']])
mask = idx.isin(white_set)
df_filtered = df_target_clean[mask].copy()

removed = len(df_target_clean) - len(df_filtered)
print(f"保留 {len(df_filtered):,} 行（剔除 {removed:,} 行）")

# (6) 保存结果（处理列名映射）
output_file = os.path.join(desktop_path, f"新增 {loan_name}（按贷款事件统计）_{timestamp}.xlsx")

# 处理列重命名：原始"本年新增合计（万元）" → 目标名称
if '本年新增合计（万元）' in df_filtered.columns:
    df_filtered[output_col] = df_filtered['本年新增合计（万元）']

# 确保输出列存在
output_cols = ['股票代码', '会计年度', output_col]
for col in output_cols:
    if col not in df_filtered.columns:
        df_filtered[col] = None

with pd.ExcelWriter(output_file, engine='openpyxl') as writer:
    df_filtered[output_cols].to_excel(writer, index=False)

print(f"    ✓ 已保存: {os.path.basename(output_file)}")

return output_file

# 依次处理 6 种类型（六次中间结果读取均含股票代码格式化）
output_files = []
for cfg in tqdm(LOAN_CONFIG, desc="总体进度", position=0):
    result = process_loan_filter(*cfg)
    if result:
        output_files.append(result)

```

```

# =====
# 完成报告
# =====
print("\n" + "=" * 60)

print("    任务执行完毕! ")
print("=" * 60)

print(f"    生成文件 ({len(output_files) + 1} 个):")
print(f"    1. 匹配表: {os.path.basename(match_file_path)} (6 Sheets)")
print(f"    2-7. 筛选结果:")
for i, f in enumerate(output_files, 2):
    print(f"        {i}. {os.path.basename(f)}")

print(f"\n    关键保证: 六次中间结果读取均强制执行")

print(f"    df['股票代码'] = df['股票代码'].astype(str).str.split('.').str[0].str.zfill(6)")
print(f"    确保所有股票代码均为 6 位字符串, 保留前导零")
print("=" * 60)

```

九、银行竞争程度 HHI 代码

<http://localhost:8888/notebooks/Desktop/Python/%E6%AF%95%E4%B8%9A%E8%AE%BA%E6%96%87-%E9%93%B6%E8%A1%8C%E7%AB%9E%E4%BA%89%E7%A8%8B%E5%BA%A6HHI-20260108.ipynb>

注：1、所有代码运行前需把 excel 文件放到桌面上；2、代码中的路径只适用于 mac 系统，windows 系统需要对路径代码进行修改；3、需要把桌面上的文件名称修改至与代码中完全相同。

kimi 指令：

我现在需要你帮我设计一段 python 代码：

一、现有数据：

- 1、第 1 行为英文表头，第二行为中文表头
- 2、第 A 列是统计日期，第 B 列是机构编码，第 C 列是分支行名称，第 D 列是机构 ID，第 E 列是总行名称，第 F 列是机构类别，第 G 列是机构地址，第 H 列是机构状态，第 I 列是机构所在省，第 J 列是机构所在市/区，第 K 列是发证日期，第 L 列是批准成立日期，第 M 列是退出日期

各列信息的含义如下：

SgnDate [统计日期] - 该日期为更新此表信息的日期

InstituionCode [机构编码] -

BranchName [分支行名称] -

InstituionID [机构 ID] - 希施玛机构 ID，对应总行名称

HeadofficeName [总行名称] -

InstituionTypeID [机构类型] - 根据银监会分类。

5=政策性银行

59=国有控股大型商业银行

8=股份制商业银行

9=城市商业银行

10=农村商业银行

18=外资银行 99=其他

11=农合行

13=农信社

52=农村资金互助社

30=贷款公司

14=村镇银行

55=民营银行

16=邮政储蓄银行

29=资产管理公司

22=信托公司

21=财务公司

24=金融租赁公司

26=汽车金融公司

27=货币经纪公司
28=消费金融公司
Address [机构地址] -
Status [机构状态] - 0-持有 1-失控 2-退出
ProvinceName [机构所在省] -
CityName [机构所在市/区] -
IssueDate [发证日期] - 监管部门对金融机构颁（换）发许可证的制证日期。
ApprovalDate [批准成立日期] - 监管部门批准该机构设立的日期，金融机构自此具有开展金融业务的一般资质，其他具体新增业务以监管部门的相关文件为准。
QuitDate [退出日期] -

二、我的需求

1、任务目标:

- (1) 2024 年底前退出的，把退出日期填到对应的银行分支机构的所有行
- (2) 已知该表格中存在同一银行分支机构重复统计的情况，重复统计原因为发证日期涵盖首次颁发和换发许可证的制证日期，而同一银行分支机构的机构地址和批准成立日期是唯一的，所以需要剔除后续换发许可证的行
- (3) 统计 2013-2024 年每一年各省市，各大行的境内机构总数量，计算每一年各大行在各个省市的市场份额
- (4) 最终目标：计算各年份各省市的银行竞争程度 HHI，已知某地区 $HHI = \text{该地各大行市场份额的平方的和}$

2、代码步骤:

【板块一】补齐截至基准日已退出银行机构的退出日期数据

(1) 建立字典 1，读取位于 mac 桌面上名为“银行信息.xlsx”的 excel 的数据，从数据行开始（不遍历前两行中英文列标题），对每一行进行遍历，将每一行的数据作为一个键值对，其中每个键值对的键为由除了“退出日期”列该行的所有数据组成的一个元组，值为该行的“退出日期”，如果这一步读取的时候退出日期列为空，则保持为空，如果退出日期不为空，则仍以日期格式储存为这个键值对的值

(2) 大循环：对该字典中的每一个值（即退出日期列的数据）进行遍历，如果值为空，则跳过，如果值不为空（即这家银行机构截至 2024 年底已退出），则读取该键值对的键并命名为 x

(3) 小循环：进一步，对字典 1 中所有键值对的键进行遍历，找到所有“机构地址”和“批准成立日期”两个位置的元素和 x 同样位置的元素完全相同的键 yi，令 yi 对应的值=键 x 对应的值，注意，一定要令所有“机构地址”和“批准成立日期”两个位置的元素和 x 同样位置的元素完全相同的键的值都为 x，所以需要对所有的键进行遍历，才能结束这个小循环，同时结束大循环。（第 (2) - (3) 步操作的底层逻辑是，机构地址和批准成立日期都相同的行对应一家分支机构，所以退出日期也应该相同，但是由于数据的统计时间不同，所以机构状态和发证日期不同，退出日期统计情况也不同，所以我们才需要进行板块一的操作，补齐所有截至 2024 年 12 月 31 日已经退出的银行的退出日期。该数据表中退出日期始终为空的企业，说明截至基准日 2024 年 12 月 31 日，还未退出，所以该列数据保持为空）

(3) 将得到的结果输出为 excel，第 A-M 列从左到右的列标题设置为统计日期、机构编码、分支行名称、机构 ID、总行名称、机构类别、机构地址、机构状态、机构所在省、机构所

在市/区、发证日期、批准日期、退出日期，保存到 mac 的桌面，命名为：银行信息_填充退出日期_“对应的时间戳”.xlsx

【板块二】剔除重复值，保证银行机构信息唯一性

(1) 建立列表 1，读取板块一得到的结果 excel（注意，我需要一个可以自动识别板块一结果 excel 的代码，所以在保存板块一结果的时候，请同时输出并保存该文件的路径，便于板块二直接使用），从第二行开始（第一行应该是列标题，不遍历），对每一行进行遍历，这一次，将一行数据储存为一个列表，并作为列表 1 中的一个元素

(2) 将列表 1 重新排序，排序标准为将列表 1 中的元素（即存储了表格中一行数据的小列表）按照索引为 0 的位置（即“统计日期”）从小到大依次排列

(3) 建立字典 2，对列表 1 中的元素进行遍历，将每一个列表 1 的元素中“机构地址”和“批准成立日期”组成一个元组，作为一个键值对的键，将元素中其余的所有小元素（即一行中除了机构地址和批准日期的其他所有数据）组成一个列表，作为键值对的值，储存在字典 2 中。注意，建立这个字典的目的是剔除掉重复发证的银行机构，保证每一家银行机构都只有一行信息，如果出现同样的“键”，则直接跳过，不再修改值

(4) 将得到的字典 2 以 excel 格式输出，一个键值对为一行，并添加列标题行，仍旧按照统计日期、机构编码、分支行名称、机构 ID、总行名称、机构类别、机构地址、机构状态、机构所在省、机构所在市/区、发证日期、批准日期、退出日期排列，保存到 mac 的桌面，命名为：银行信息_剔除重复行_“对应的时间戳”.xlsx（注意，在保存板块二结果的时候，请同时输出并保存该文件的路径，便于板块三直接使用）

鉴于使用循环会导致代码运行效率低下，你可以给出可行的替代方案优化程序。记住，板块二的目的就是每一家银行机构只保留一行数据，无论是哪一行，因为每家银行机构的机构地址、批准成立日期和退出日期一定是唯一的

【板块三】识别总行名称

(1) 读取板块二得到的结果文件，即位于桌面命名为 银行信息_剔除重复行_20260108_153404.xlsx 的文件。

(2) 新建列表 2: ['有限公司', '有限责任公司', '信用社联合社', '合作社', '合作联社', '局'], 作为识别总行名称的关键词。这样做的原因：某银行分支机构的命名方式通常为“xx+关键词（如股份有限公司）+地名+支行”，创建这个列表是为了提供一个查找银行分支机构的总行名称的依据，从而得到该银行机构的总行名称“xx+关键词”。举例：阿坝州农村信用联社股份有限公司抚边分社，我们需要提取出来的总行名称是“阿坝州农村信用联社”。

(3) 新建字典 3，储存板块二结果文件中除了列标题行（即第一行）的所有数据，每一行的数据作为一个键值对，其中除了“总行名称”的数据新建一个元组作为键，“总行名称”字符串作为值。

(4) 对字典 3 中每一个键值对的值进行遍历：如果该字符串中含有列表 2 中的任意元素（字符串）x，则保留值中字符串的 x 及 x 之前的所有字。举例：某个键值对的值为“阿坝州农村信用联社股份有限公司抚边分社”，通过遍历，已经识别到关键词“有限公司”，于是将这个字符串中“有限公司”及其之前的字继续保留，即处理后这个键值对的值被重新命名为“阿坝州农村信用联社股份有限公司”。如果某个键值对的值（即字符串）中不含有列表 2 中的任一关键词，则认为这个值已经是总行名称了，保持不变，例如“中国农业发展银行”的总行名称就是“中国农业发展银行”。

(5) 将得到的字典 3 以 excel 格式输出，一个键值对为一行，并添加列标题行，仍旧按照

统计日期、机构编码、分支行名称、机构 ID、总行名称、机构类别、机构地址、机构状态、机构所在省、机构所在市/区、发证日期、批准日期、退出日期排列，保存到 mac 的桌面，命名为：银行信息_识别总行名称_“对应的时间戳”.xlsx

鉴于使用循环会导致代码运行效率低下，你可以给出可行的替代方案优化程序。

【板块四】统计各年份、各省、市（区）各大银行的分支机构数

(1) 读取板块三得到的结果文件，即位于 mac 桌面命名为 5 银行信息_识别总行名称_20260108_170542.xlsx 的文件

(2) 新建列表 3，从第二行开始（因为第一行为列标题），对板块三结果文件的每一行进行遍历，提取出每一行中的如下信息：批准成立日期所在年份、退出日期所在年份、机构所在市、总行名称，并将上述信息放在一个小列表中，将小列表作为一个元素加入列表 3 中，其中批准日期所在年份和退出日期所在年份以日期格式列示（保留到年，不要月和日），机构所在市和总行名称以字符串形式保存。注意，如果机构所在省不是直辖市（北京市、天津市、上海市、重庆市），则直接保留“机构所在市/区”，如果机构所在省份列的数据属于直辖市（北京市、天津市、上海市、重庆市），则将原“机构所在省”的数据作为新的“机构所在市/区”。（标黄部分用于修改直辖市统计问题）

(3) 对列表 3 中的每一个小列表进行如下操作：依次判断从 2013 年到 2024 年，批准成立日期所在年份小于等于该年小于退出日期所在年份，如果满足，则在该行的小列表末尾增加元素 1，如果不满足，则在该行的小列表末尾增加元素 0。举例：某银行分支机构的批准成立日期为 2015/12/1，退出日期为 2023/10/21，总行名称为则批准成立日期所在年份为 2015，退出日期所在年份为 2023，那么这家银行在 2013 年到 2024 年每年年底的存续情况判断结果应为：0,0,1,1,1,1,1,1,1,0,0，该行对应的小列表最终结果应为：[2015,2023,‘xx 市’,‘xx 银行’,0,0,1,1,1,1,1,1,1,0,0]，以此类推。注意，小列表索引 4-15 的元素类型为数值型，以便后续加减

(4) 在第 (3) 步对所有行进行遍历得到更新的列表 3 之后，对每一年每个发证机构所在市/区的每家总行的数量进行统计，具体过程如下：新建字典 3，统计 2013 年的数据，对列表 3 中的每一个元素进行遍历，字典 3 的键为由发证机构所在市和总行名称组成的元组（即小列表索引为 2 和 3 的元素），当这个元组第一次出现时，新建键值对，键值对的值为代表 2012 年判断结果的元素，即小列表索引为 4 的元素，当这个元组不是第一次出现时（即字典中已经存在这个键），则令该键值对的值=原来的值+小列表索引为 4 的元素，例如，某行的机构所在市和总行名称组成的元组之前已经出现过多次了，当前该键值对的值为 4，目前需要操作的行对应的小列表索引为 4 的元素为 1，那么就令键值对的值=4+1=5，表示截至当前处理行，2013 年 xx 市 xx 银行存续数已经有 5 家，如果目前需要操作的行对应的小列表索引为 4 的元素为 0，那么就令键值对的值=4+0=4，表示截至当前处理行，2013 年 xx 市 xx 银行存续数已经有 4 家。

(5) 依次类推，2014-2024 年操作同 2013 年，都需要新建一个字典，统计每个发证机构所在市/区的每家总行的数量合计数。相较 2013 年，2014-2024 年需要修改用于加总计数的索引的位置，即 2014-2024 年需要用于对应年份的字典的键值对的计数的小列表的索引分别为 5-15。

(6) 将代表每一年份统计数据的 12 个列表依次合并，以 excel 表格形式输出，并且在最上方加一行标题，从左到右依次为：年份（注意，年份是字典里面没有的，在将字典的一个键值对写为 excel 的一行的时候，应该在这一行的最开始根据该字典对应的年份，写入年份，

如现在读取的是 2013 年所对应的字典，则在每一行的第一列先写入 2013，再依次写入“机构所在市/区”、“总行名称”、“数量”、机构所在市/区、总行名称、数量。

(7) 将结果保存到 mac 的桌面，命名为：银行信息_按年份按机构所在地计数_“对应的时间戳”.xlsx

【板块五】计算某年份某银行在某地市场份额

(1) 读取板块四得到的结果文件，即位于 mac 桌面 命名为：6 银行信息_按年份按机构所在地计数_20260120_142004 的文件。已知这个文件统计的是 2013-2024 年各个年份各地区各总行分支机构的数量，第一行为列标题，从左到右分别为：“年份”、“机构所在市/区”、“总行名称”、“数量”

(2) 统计 2013-2024 年各年份各地区的银行分支机构总数：即对第 A 列年份为 2013、2014、.....2023、2024；且 B 列机构所在市/区分别为七台河市.....的 D 列数值进行加总

(3) 计算某年份某总行在某地的市场份额：使用第 (2) 步得到的各年份各地区银行分支机构总数，某年某总行在某地区所占市场份额=该年份该地区该总行的银行分支机构总数量/该年份该地区银行分支机构总数

(4) 输出板块五的结果，以 excel 的形式保存到 mac 的桌面，命名为：银行市场份额_按年份按所在地_对应的时间戳.xlsx。其中，这个 excel 的第一列为“年份”，第二列为“机构所在市/区”，第三列为“总行名称”，第四列为“市场份额”，并且在表格的最上方添加列标题行。

【板块六】计算各年份各地区银行竞争程度指标 HHI

(1) 读取板块五得到的结果文件，即位于 mac 桌面 命名为：7 银行市场份额_按年份按所在地_20260120_120625.xlsx 的文件。已知这个文件第一行为列标题，从左到右依次为：“年份”、“机构所在市/区”、“总行名称”、“市场份额”。

(2) HHI 计算方法如下：首先明确，每年每个地区只有一个 HHI 值，所以我最终输出的结果，第一列为“年份”，第二列为“机构所在市/区”，第三列为“HHI”。其次，某年份某地区 HHI=该年份该地区各大总行市场份额的平方的和，即对该年份该地区的所有总行名称对应的市场份额先分别平方，再求和。

(3) 输出板块六的结果，以 excel 的形式保存到 mac 的桌面，命名为：HHI_对应的时间戳.xlsx。其中，这个 excel 的第一列为“年份”，第二列为“机构所在市/区”，第三列为“HHI”，并且在表格的最上方添加列标题行。

【板块七】HHI 按会计年度、所属城市与股票代码匹配

我现在需要你帮我设计一段 python 代码：

一、现有数据：

两张 excel 表格

第一张为位于 mac 桌面命名为 3 企业上市年限.xlsx 的 excel 表格，记录了 2013-2024 年 A 股上市公司的股票代码和所属城市按年份对应情况。第一行为中文标题，从左到右依次为：股票代码、股票简称、会计年度、统计截止日期、行业名称、行业代码、所属城市、上市年限

第二张为位于 Mac 桌面命名为 HHI.xlsx 的 excel 表格，记录了 2013-2024 年各城市/区的

HHI 数据。第一行为中文标题，从左到右依次为：会计年度、机构所在市/区、HHI

二、我的需求

1、任务目标：将 HHI.xlsx 中的 HHI 数据按照年份和机构所在市/区匹配到 3 企业上市年限.xlsx 中

2、逻辑：两张表通过所属城市（机构所在市/区）列和会计年度列产生联系。我现在需要通过（所属城市，会计年度）的链接，把 HHI 数据填写到对应年份和地区的股票代码行

3、具体操作：

(1) 读取位于 mac 桌面命名为 3 企业上市年限.xlsx 和 25 HHI.xlsx 的 excel 的文件。注意，由于股票代码存在左侧数据为 0 的情况，所以一定要始终以字符串的格式储存，保证长度一直为 6 位。

(2) 新建一个字典，对 3 企业上市年限.xlsx 进行遍历，将每一行中（股票代码，会计年度，所属城市）组合为一个元组，作为一个键值对的键，保持键值对的值为空。

(3) 每读取 3 企业上市年限.xlsx 的一行数据，形成一个键值对，就对 HHI.xlsx 进行遍历，读取该键值对的键的元组中的会计年度和所属城市，找到 3 企业上市年限.xlsx 中会计年度和机构所在市/区和这个元组完全对应的行，并将该行的 HHI 写为键值对的值。

(4) 输出结果，将字典以 excel 的形式保存到 mac 的桌面，命名为：HHI 匹配_对应的时间戳.xlsx。其中，这个 excel 的从左到右每一列的列标题依次为：股票代码、会计年度、所属城市、HHI。并且在表格的最上方添加列标题行。

注意：(1) 鉴于使用循环会导致代码运行效率低下，你可以给出可行的替代方案优化程序；

(2) 我希望这个代码可以实时显示运行进度。

请仔细阅读上述文字，并告诉我你是否已经完全理解所有要求。

20260128 补充检验：

我现在需要你帮我设计一段 python 代码：

一、现有数据：

两张 excel 表格

第一张为位于 mac 桌面命名为 3 企业上市年限.xlsx 的 excel 表格，记录了 2013-2024 年 A 股上市公司的股票代码和所属城市按年份对应情况。第一行为中文标题，从左到右依次为：股票代码、股票简称、会计年度、统计截止日期、行业名称、行业代码、所属城市、上市年限

第二张为位于 mac 桌面命名为 5 总资产净利润率.xlsx 的 excel 表格，记录了 2013-2024 年各上市公司的行业名称和行业代码数据。第一行为中文标题，从左到右依次为：股票代码、股票简称、会计年度、统计截止日期、报表类型编码、公告来源、行业代码、行业名称、行业代码 1、行业名称 1、总资产净利润率(ROA)A

二、我的需求

1、任务目标：核对两张表中同一（股票代码，会计年度）对应的（行业代码，行业名称）

（在总资产净利润率表中行业代码 1、行业名称 1）是否一致

2、具体操作：

(1) 读取原始数据：分别读取位于 mac 桌面命名 3 企业上市年限.xlsx 的 excel 表格和 mac 桌面命名为 5 总资产净利润率.xlsx 的 excel 表格。注意，由于股票代码存在左侧数据为 0 的情况，所以一定要始终以字符串的格式储存，保证长度一直为 6 位。

(2) 新建两个字典，分别对两个 excel 文件中除了标题行外的所有数据行进行遍历。将 3 企业上市年限.xlsx 的每一行的（股票代码，会计年度）组合为元组，作为对应字典中一个键值对的键，将同行的（行业名称，行业代码）组成元组，作为键值对的值；将 5 总资产净利润率.xlsx 的每一行的（股票代码，会计年度）组合为元组，作为对应字典中一个键值对的键，将同行的（行业名称 1，行业代码 1）组成元组，作为键值对的值。

(3) 判断两个字典中相同的键（股票代码，会计年度）对应的值是否相同，即该（股票代码，会计年度）在 3 企业上市年限.xlsx 中的（行业名称，行业代码）和 5 总资产净利润率.xlsx 中的（行业名称 1，行业代码 1）是否一致，并生成虚拟变量“是否一致”，一致时令虚拟变量的值为 1，不一致时令虚拟变量的值为 0。

(4) 输出结果文件：将字典以 excel 的形式保存到 mac 的桌面，命名为：行业代码一致性检验_对应的时间戳.xlsx。其中，这个 excel 的从左到右每一列的列标题依次为：股票代码、会计年度、是否一致。并且在表格的最上方添加列标题行。

注意：（1）鉴于使用循环会导致代码运行效率低下，你可以给出可行的替代方案优化程序；
（2）我希望这个代码可以实时显示运行进度。

请仔细阅读上述文字，并告诉我你是否已经完全理解所有要求。

进一步检验：

我现在需要你帮我设计一段 python 代码：

一、现有数据：

两张 excel 表格

第一张为位于 mac 桌面命名为 3 企业上市年限.xlsx 的 excel 表格，记录了 2013-2024 年 A 股上市公司的股票代码和所属城市按年份对应情况。第一行为中文标题，从左到右依次为：股票代码、股票简称、会计年度、统计截止日期、行业名称、行业代码、所属城市、上市年限

第二张为位于 mac 桌面命名为 5 总资产净利润率.xlsx 的 excel 表格，记录了 2013-2024 年各上市公司的行业名称和行业代码数据。第一行为中文标题，从左到右依次为：股票代码、股票简称、会计年度、统计截止日期、报表类型编码、公告来源、行业代码、行业名称、行业代码 1、行业名称 1、总资产净利润率(ROA)A

二、我的需求

1、任务目标：核对两张表中同一（股票代码，会计年度）对应的（行业代码，行业名称）

（在总资产净利润率表中使用行业代码 1、行业名称 1）是否一致

2、具体操作：

(1) 读取原始数据：分别读取位于 mac 桌面命名 3 企业上市年限.xlsx 的 excel 表格和 mac 桌面命名为 5 总资产净利润率.xlsx 的 excel 表格。注意，由于股票代码存在左侧数据为 0 的情况，所以一定要始终以字符串的格式储存，保证长度一直为 6 位。

(2) 新建两个字典，分别对两个 excel 文件中除了标题行外的所有数据行进行遍历。将 3 企业上市年限.xlsx 的每一行的（股票代码，会计年度）组合为元组，作为对应字典中一个键

值对的键，将同行的（行业名称，行业代码）组成元组，作为键值对的值；将 5 总资产净利润率.xlsx 的每一行的（股票代码，会计年度）组合为元组，作为对应字典中一个键值对的键，将同行的（行业名称 1，行业代码 1）组成元组，作为键值对的值。

(3) 新建字典，存储两张表中不同的（行业名称，行业代码）和（行业名称 1，行业代码 1）。判断两个字典中相同的键（股票代码，会计年度）对应的值是否相同，即该（股票代码，会计年度）在 3 企业上市年限.xlsx 中的（行业名称，行业代码）和 5 总资产净利润率.xlsx 中的（行业名称 1，行业代码 1）是否一致，并生成虚拟变量“是否一致”。一致时令虚拟变量的值为 1；不一致时令虚拟变量的值为 0，同时存储不一致的行的（股票代码，会计年度）元组作为新建的字典的一个键值对的键，同时存储（行业名称，行业代码，行业名称 1，行业代码 1）组成的元素为该键值对的值，其中行业名称和行业代码来自 3 企业上市年限.xlsx，行业名称 1 和行业代码 1 来自 5 总资产净利润率.xlsx。

(4) 输出结果文件：将字典以 excel 的形式保存到 mac 的桌面，命名为：行业代码一致性检验_1_对应的时间戳.xlsx。其中，这个 excel 的从左到右每一列的列标题依次为：股票代码、会计年度、行业名称、行业代码、行业名称 1、行业代码 1、是否一致。并且在表格的最上方添加列标题行。

注意：(1) 鉴于使用循环会导致代码运行效率低下，你可以给出可行的替代方案优化程序；
(2) 我希望这个代码可以实时显示运行进度。

请仔细阅读上述文字，并告诉我你是否已经完全理解所有要求。

板块一代码终版

```
import pandas as pd
import os
from datetime import datetime
import time

def fill_quit_dates_fixed():

    desktop_path = os.path.expanduser("~/Desktop")
    input_file = os.path.join(desktop_path, "银行信息.xlsx")

    if not os.path.exists(input_file):

        print(f"  错误：未找到文件 {input_file}")

        return None

    start_time = time.time()

    print(f"  开始时间: {datetime.now().strftime('%H:%M:%S')}")
```

```

print("    1. 正在读取数据...")

df = pd.read_excel(input_file, header=None, skiprows=2)
full_columns = ['统计日期', '机构编码', '分支行名称', '机构 ID', '总行名称',
                '机构类别', '机构地址', '机构状态', '机构所在省', '机构所在市/区',
                '发证日期', '批准成立日期', '退出日期']
df.columns = full_columns

print(f"        完成! 共 {len(df):,} 行")

print("    2. 正在转换日期格式...")

# 将所有日期列转换为 datetime 格式
date_columns = ['统计日期', '发证日期', '批准成立日期', '退出日期']
for col in date_columns:
    df[col] = pd.to_datetime(df[col], errors='coerce')

print("        完成! ")

print("    3. 正在补齐退出日期...")

step3_start = time.time()

# 核心修正: 使用 transform('first') 获取每个分组的第一个非空值
# 它会自动将非空值广播到分组内所有行
df['退出日期'] = df.groupby(['机构地址', '批准成立日期'])['退出日期'].transform('first')

# 验证: 检查是否还有不一致
check_df = df.groupby(['机构地址', '批准成立日期'])['退出日期'].nunique()
inconsistent_groups = (check_df > 1).sum()
if inconsistent_groups > 0:
    print(f"        警告: 仍有 {inconsistent_groups} 个机构的退出日期不一致")
else:
    print("        验证通过: 所有同机构退出日期已完全一致")

print(f"        完成! 耗时 {time.time()-step3_start:.1f} 秒")

print("    4. 正在格式化日期并保存文件...")

```

```

save_start = time.time()

# 新增：将所有日期列格式化为仅日期字符串（YYYY-MM-DD）
for col in date_columns:
    # .dt.strftime('%Y-%m-%d') 将 datetime 转换为指定格式的字符串
    df[col] = df[col].dt.strftime('%Y-%m-%d')

timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
output_file = os.path.join(desktop_path, f"银行信息_填充退出日期_修正版_{timestamp}.xlsx")
df.to_excel(output_file, index=False)

print(f"        完成! 耗时 {time.time()-save_start:.1f} 秒")

total_time = time.time() - start_time
print("\n" + "="*60)

print("    全部完成! ")

print(f"    总耗时: {total_time:.1f} 秒")

print(f"    文件保存至: {output_file}")

print(f"    共处理 {len(df):,} 条记录")
print("="*60)

return output_file

if __name__ == "__main__":
    fill_quit_dates_fixed()

```

板块二代码终版

```

import pandas as pd
import os
from datetime import datetime

```

```

def remove_duplicates():

```

```

    """

```

板块二：剔除重复发证的银行机构，保证每家机构仅保留一行信息

核心逻辑：按(机构地址，批准成立日期)分组，保留统计日期最早的记录

```

    """

```

```

# ===== 步骤 1: 读取板块一结果文件 =====
desktop_path = os.path.expanduser("~/Desktop")

# 自动查找板块一结果文件（修改为你的实际文件名）
# 如果文件名固定，可以直接指定：
input_file = os.path.join(desktop_path, "银行信息_填充退出日期_修正版_20260108_150819.xlsx")

## 或者自动查找最新的板块一文件：
# import glob
# pattern = os.path.join(desktop_path, "银行信息_填充退出日期_*.xlsx")
# files = glob.glob(pattern)
# if not files:

#     print(" 错误: 未找到板块一输出文件")

#     return None
# input_file = max(files, key=os.path.getctime) # 取最新文件

print(f" 正在读取文件: {os.path.basename(input_file)}")

# 读取 Excel（自动跳过标题行，直接读取数据）
# 对应您的要求：从第二行开始（第一行是列标题）
df = pd.read_excel(input_file)

print(f" 完成! 共 {len(df):,} 行数据")

# ===== 步骤 2: 按统计日期排序 =====

print(f" 正在按统计日期排序...")

df_sorted = df.sort_values('统计日期', ascending=True)

print(f" 完成! ")

# ===== 步骤 3: 去重（核心步骤） =====

print(f" 正在剔除重复机构...")

# 使用 drop_duplicates 实现您的字典去重逻辑：
# - subset=['机构地址', '批准成立日期']: 以机构地址和批准日期作为"键"
# - keep='first': 保留每组的第一行（即统计日期最早的记录）
# - 这等价于您的逻辑：遍历排序后的列表，遇到新"键"时添加到字典

```

```

df_unique = df_sorted.drop_duplicates(
    subset=['机构地址', '批准成立日期'],
    keep='first'
)

removed_count = len(df) - len(df_unique)

print(f"        完成! 剔除 {removed_count:,} 条重复记录")

print(f"        保留 {len(df_unique):,} 条唯一记录")

# ===== 步骤 4: 保存结果 =====

print(f"    正在保存文件...")

timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
output_file = os.path.join(desktop_path, f"银行信息_剔除重复行_{timestamp}.xlsx")

# 确保列顺序与您的要求一致 (13 列)
output_columns = [
    '统计日期', '机构编码', '分支行名称', '机构 ID', '总行名称',
    '机构类别', '机构地址', '机构状态', '机构所在省', '机构所在市/区',
    '发证日期', '批准成立日期', '退出日期'
]
df_unique = df_unique[output_columns]

df_unique.to_excel(output_file, index=False)

print(f"        完成! 文件已保存")

# ===== 输出文件路径供板块三使用 =====
print("\n" + "="*60)

print("    板块二处理完成! ")

print(f"    输出文件路径: {output_file}")

print(f"    请将此路径复制给板块三使用")
print("="*60)

return output_file

# ===== 执行板块二 =====
if __name__ == "__main__":
    result_path = remove_duplicates()

```

```
# 将路径保存到文本文件，方便板块三读取
if result_path:
    desktop_path = os.path.expanduser("~/Desktop")
    path_file = os.path.join(desktop_path, "板块二输出路径.txt")
    with open(path_file, 'w', encoding='utf-8') as f:
        f.write(result_path)

    print(f"\n    路径已同步保存至: {path_file}")

    print("    板块三可自动读取此文件")
```

板块三代码终版

```
import pandas as pd
import os
from datetime import datetime

def identify_head_office_names():
    """
    板块三：从分支行名称中提取总行名称
    核心逻辑：使用正则表达式匹配关键词，提取关键词及之前的所有字符
    """

    # ===== 步骤 1：读取板块二结果文件 =====
    desktop_path = os.path.expanduser("~/Desktop")

    # 指定板块二输出文件（修改为你的实际文件名）
    input_file = os.path.join(desktop_path, "银行信息_剔除重复行_20260108_153404.xlsx")

    ## 或者自动查找最新的板块二文件：
    # import glob
    # pattern = os.path.join(desktop_path, "银行信息_剔除重复行_*.xlsx")
    # files = glob.glob(pattern)
    # if not files:

    #     print("    错误：未找到板块二输出文件")

    #     return None
    # input_file = max(files, key=os.path.getctime)

    print(f"    正在读取文件: {os.path.basename(input_file)}")

    # 读取 Excel（包含标题行）
```

```

df = pd.read_excel(input_file)
initial_count = len(df)

print(f"        完成! 共 {initial_count:,} 条记录")

# ===== 步骤 2: 定义关键词列表 =====

print(f"    正在配置关键词...")

# 关键词列表 (按长度降序排列, 优先匹配较长的关键词)
keywords = ['银行','信用社联合社','农业发展银行','农业银行','商业银行','农村合作银行','
有限责任公司','股份公司','合股公司','有限公司','合作联社','合作社','局']

# 构建正则表达式模式: (关键词及之前的所有内容)
# 模式: (.*(?:关键词 1|关键词 2|关键词 3))
# .*? 表示非贪婪匹配任意字符 (关键词之前的所有字)
# (?:...) 表示非捕获组, 用于匹配多个关键词中的任意一个
pattern = rf"(.*(?:{'|'.join(sorted(keywords, key=len, reverse=True))}))"

print(f"        关键词: {len(keywords)} 个")

print(f"        正则模式: {pattern}")

# ===== 步骤 3: 向量化提取总行名称 =====

print(f"    正在识别总行名称...")

step3_start = datetime.now()

# 使用 str.extract 提取匹配的内容 (结果是一个 DataFrame)
extracted = df['分支行名称'].str.extract(pattern, expand=False)

# 对于没有匹配到关键词的行, extract 返回 NaN, 用原分支行名称填充
df['总行名称'] = extracted.fillna(df['分支行名称'])

# 验证: 统计匹配成功的数量
matched_count = extracted.notna().sum()

print(f"        完成! 识别成功 {matched_count:,} 条记录")

print(f"        无匹配保持原样 {initial_count - matched_count:,} 条记录")

# ===== 步骤 4: 保存结果 =====

print(f"    正在保存文件...")

```

```

# 设置输出列顺序 (13 列, 总行名称已更新)
output_columns = [
    '统计日期', '机构编码', '分支行名称', '机构 ID', '总行名称',
    '机构类别', '机构地址', '机构状态', '机构所在省', '机构所在市/区',
    '发证日期', '批准成立日期', '退出日期'
]

timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
output_file = os.path.join(desktop_path, f'银行信息_识别总行名称_{timestamp}.xlsx')

# 保存 Excel 文件
df[output_columns].to_excel(output_file, index=False)

print(f"        完成! ")

# ===== 结果报告 =====
total_time = (datetime.now() - step3_start).total_seconds()
print("\n" + "="*60)

print("    板块三处理完成! ")

print(f"    输出文件路径: {output_file}")

print(f"    处理耗时: {total_time:.1f} 秒")

print(f"    总行名称识别率: {matched_count/initial_count*100:.1f}%")
print("="*60)

# 将路径保存到文件供板块四使用
path_file = os.path.join(desktop_path, "板块三输出路径.txt")
with open(path_file, 'w', encoding='utf-8') as f:
    f.write(output_file)

print(f"\n    路径已保存至: {path_file}")

return output_file

# ===== 执行板块三 =====
if __name__ == "__main__":
    identify_head_office_names()

```

板块四代码 1 版

```
import pandas as pd
import os
from datetime import datetime
import numpy as np
from tqdm import tqdm

def calculate_annual_branch_counts():
    """
    板块四：统计各年份、各省、市（区）各大银行的分支机构数
    修正：自动检测列名，避免 KeyError
    """

    desktop_path = os.path.expanduser("~/Desktop")
    input_file = os.path.join(desktop_path, "银行信息_识别总行名称_20260108_170542.xlsx")

    if not os.path.exists(input_file):

        print(f"  错误：未找到板块三结果文件: {input_file}")

        return None

    print("="*60)

    print("  板块四：年度分支机构统计")

    print("="*60)

    start_time = datetime.now()

    # ===== 步骤 1：读取数据并自动检测列名 =====

    print("\n  步骤 1：读取板块三结果文件并检测列名...")

    df = pd.read_excel(input_file)

    print(f"      成功读取 {len(df):,} 条机构记录")

    # 自动检测列名（容错处理）
    column_mapping = {}
    for expected_name in ['批准成立日期', '批准日期', 'ApprovalDate']:
        if expected_name in df.columns:
            column_mapping['approval'] = expected_name

    print(f"      检测到批准日期列: '{expected_name}'")

    break
```

```

for expected_name in ['退出日期', 'QuitDate']:
    if expected_name in df.columns:
        column_mapping['quit'] = expected_name

        print(f"        检测到退出日期列: '{expected_name}'")

        break

# 检查是否找到所有必需的列
if len(column_mapping) < 2:

    print("        错误: 未能自动检测到'批准成立日期'或'退出日期'列")

    print(f"    当前列名: {list(df.columns)}")

    return None

# ===== 步骤 2: 提取年份信息 (带进度反馈) =====

print("\n    步骤 2: 提取批准成立年份和退出年份...")

# 显示进度: 转换批准日期
tqdm.pandas(desc="    转换批准日期")
df['批准年份'] = df[column_mapping['approval']].progress_apply(
    lambda x: pd.to_datetime(x, errors='coerce').year if pd.notna(x) else np.nan
)

# 显示进度: 转换退出日期
tqdm.pandas(desc="    转换退出日期")
df['退出年份'] = df[column_mapping['quit']].progress_apply(
    lambda x: pd.to_datetime(x, errors='coerce').year if pd.notna(x) else 2025
)

# 处理未退出的机构
df['退出年份'] = df['退出年份'].fillna(2025).astype(int)

print(f"        提取完成! 样本年份范围: {df['批准年份'].min():.0f}-{df['退出年份']
].max():.0f}")

# ===== 步骤 3: 定义统计年份范围 =====

print("\n    步骤 3: 定义统计年份: 2013-2024 年")

years = list(range(2013, 2025))

# ===== 步骤 4: 向量化判断每年存续状态 (核心步骤) =====

print("\n    步骤 4: 生成每年存续状态矩阵...")

```

```

# 创建年份数组
year_array = np.array(years)

# 显示进度: 构建判断矩阵
print("    正在构建判断条件矩阵...")
with tqdm(total=3, desc="    进度") as pbar:
    # 准备数据: 转换为 numpy 数组以加速运算
    approval_years = df['批准年份'].values
    quit_years = df['退出年份'].values
    pbar.update(1)

    # 广播判断: 创建布尔矩阵 [机构数, 年份数]
    # 条件: 批准年份 <= 年份 < 退出年份
    survival_matrix = (
        (approval_years[:, np.newaxis] <= year_array) &
        (quit_years[:, np.newaxis] > year_array)
    ).astype(int)
    pbar.update(1)

    # 转换为 DataFrame
    survival_df = pd.DataFrame(
        survival_matrix,
        columns=[f'状态_{year}' for year in years],
        index=df.index
    )
    pbar.update(1)

print(f"    生成 {survival_df.shape[0]}×{survival_df.shape[1]} 的存续状态矩阵")

# ===== 步骤 5: 重塑数据格式 =====

print("\n    步骤 5: 重组数据格式...")

# 合并原始数据与存续状态
df_with_status = pd.concat([df.reset_index(drop=True), survival_df], axis=1)

# 显示进度: 重塑宽表为长表
print("    正在重塑数据格式...")
id_vars = ['机构所在市/区', '总行名称']
value_vars = [f'状态_{year}' for year in years]

# 使用 melt 将所有年份列转为长格式

```

```

df_long = df_with_status.melt(
    id_vars=id_vars,
    value_vars=value_vars,
    var_name='年份标记',
    value_name='是否存续'
)

# 提取年份数字
tqdm.pandas(desc="    提取年份")
df_long['年份'] = df_long['年份标记'].progress_apply(
    lambda x: int(x.split('_')[1])
)

# 筛选存续的机构 (状态=1)
df_surviving = df_long[df_long['是否存续'] == 1].copy()

print(f"    重塑完成! 存续机构记录数: {len(df_surviving):,}")

# ===== 步骤 6: 分组统计 (核心步骤) =====

print("\n    步骤 6: 分组统计每年数量...")

# 显示进度: 按年份分组统计
result_dfs = []
for year in tqdm(years, desc="    统计进度"):
    # 筛选该年份数据
    year_data = df_surviving[df_surviving['年份'] == year]

    # 分组计数
    year_counts = year_data.groupby(
        ['机构所在市/区', '总行名称'],
        as_index=False
    ).agg(数量=('是否存续', 'sum'))

    # 添加年份列
    year_counts['年份'] = year

    result_dfs.append(year_counts)

# 合并所有年份结果
result_df = pd.concat(result_dfs, ignore_index=True)

# 排序
result_df = result_df.sort_values(

```

```

        ['年份', '机构所在市/区', '总行名称']
    ).reset_index(drop=True)

    print(f"        统计完成! 共生成 {len(result_df):,} 条记录")

    # ===== 步骤 7: 保存结果 =====

    print("\n    步骤 7: 保存最终结果...")

    output_columns = ['年份', '机构所在市/区', '总行名称', '数量']

    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    output_file = os.path.join(desktop_path, f'银行信息_按年份按机构所在地计数_{timestamp}.xlsx')

    # 显示进度: 写入 Excel
    with tqdm(total=1, desc="    写入 Excel") as pbar:
        result_df[output_columns].to_excel(output_file, index=False)
        pbar.update(1)

    print(f"        文件已保存至: {output_file}")

    # ===== 结果报告 =====
    total_time = (datetime.now() - start_time).total_seconds()
    print("\n" + "="*60)

    print("    板块四处理完成! ")

    print(f"    输出文件路径: {output_file}")

    print(f"    总耗时: {total_time:.1f} 秒 ({total_time/60:.1f} 分钟)")

    print(f"    统计年份: 2013-2024")

    print(f"    涉及银行: {result_df['总行名称'].nunique()} 家")

    print(f"    覆盖地区: {result_df['机构所在市/区'].nunique()} 个市/区")

    print(f"    总行-地区组合: {len(result_df)} 个")

    print("="*60)

    # 保存路径供板块五使用

```

```

path_file = os.path.join(desktop_path, "板块四输出路径.txt")
with open(path_file, 'w', encoding='utf-8') as f:
    f.write(output_file)

print(f"\n    路径已同步保存至: {path_file}")

return output_file

# ===== 执行板块四 =====
if __name__ == "__main__":
    calculate_annual_branch_counts()

```

板块四代码终版（修改直辖市统计问题后）

```

import pandas as pd
import os
from datetime import datetime
import numpy as np
from tqdm import tqdm

def calculate_annual_branch_counts():
    """
    板块四：统计各年份、各省、市（区）各大银行的分支机构数
    修正：自动检测列名，避免 KeyError
    新增：直辖市处理逻辑，以直辖市为单位而非区
    """

    desktop_path = os.path.expanduser("~/Desktop")
    input_file = os.path.join(desktop_path, "5 银行信息_识别总行名称_20260108_170542.xlsx")

    if not os.path.exists(input_file):
        print(f"    错误：未找到板块三结果文件: {input_file}")
        return None

    print("="*60)

    print("    板块四：年度分支机构统计")

    print("="*60)

    start_time = datetime.now()

    # ===== 步骤 1：读取数据并自动检测列名 =====

```

```

print("\n    步骤 1: 读取板块三结果文件并检测列名...")

df = pd.read_excel(input_file)

print(f"        成功读取 {len(df):,} 条机构记录")

# 自动检测列名 (容错处理)
column_mapping = {}
for expected_name in ['批准成立日期', '批准日期', 'ApprovalDate']:
    if expected_name in df.columns:
        column_mapping['approval'] = expected_name

        print(f"        检测到批准日期列: '{expected_name}'")

        break

for expected_name in ['退出日期', 'QuitDate']:
    if expected_name in df.columns:
        column_mapping['quit'] = expected_name

        print(f"        检测到退出日期列: '{expected_name}'")

        break

# 检查是否找到所有必需的列
if len(column_mapping) < 2:

    print("        错误: 未能自动检测到'批准成立日期'或'退出日期'列")

    print(f"    当前列名: {list(df.columns)}")

    return None

# ===== 新增步骤: 处理直辖市逻辑 =====

print("\n    步骤 2: 处理直辖市数据 (关键修改) ...")

# 检测必需的地区列
if '机构所在省' not in df.columns or '机构所在市/区' not in df.columns:

    print("        错误: 未能找到'机构所在省'或'机构所在市/区'列")

    print(f"    当前列名: {list(df.columns)}")

    return None

# 定义直辖市列表
municipalities = ['北京市', '天津市', '上海市', '重庆市']

# 创建新的统计用列: 对于直辖市使用省级单位, 非直辖市使用市级单位

```

```

tqdm.pandas(desc="    处理直辖市")
df['统计市/区'] = df.progress_apply(
    lambda row: row['机构所在省'] if row['机构所在省'] in municipalities else row['机构
所在市/区'],
    axis=1
)

print(f"    直辖市处理完成! ")

print(f"    - 直辖市数量: {df[df['机构所在省'].isin(municipalities)][['机构所在省
'].nunique()} 个")
print(f"    - 统计用地区数: {df['统计市/区'].nunique()} 个")
print(f"    - 示例: {df[df['机构所在省'].isin(municipalities)][['机构所在省', '机构所在
市/区', '统计市/区']].head(3).to_string(index=False)}")

# ===== 步骤 3: 提取年份信息 (原步骤 2) =====

print("\n    步骤 3: 提取批准成立年份和退出年份...")

# 显示进度: 转换批准日期
tqdm.pandas(desc="    转换批准日期")
df['批准年份'] = df[column_mapping['approval']].progress_apply(
    lambda x: pd.to_datetime(x, errors='coerce').year if pd.notna(x) else np.nan
)

# 显示进度: 转换退出日期
tqdm.pandas(desc="    转换退出日期")
df['退出年份'] = df[column_mapping['quit']].progress_apply(
    lambda x: pd.to_datetime(x, errors='coerce').year if pd.notna(x) else 2025
)

# 处理未退出的机构
df['退出年份'] = df['退出年份'].fillna(2025).astype(int)

print(f"    提取完成! 样本年份范围: {df['批准年份'].min():.0f}-{df['退出年份
'].max():.0f}")

# ===== 步骤 4: 定义统计年份范围 (原步骤 3) =====

print("\n    步骤 4: 定义统计年份: 2013-2024 年")
years = list(range(2013, 2025))

# ===== 步骤 5: 向量化判断每年存续状态 (原步骤 4) =====

print("\n    步骤 5: 生成每年存续状态矩阵...")

```

```

# 创建年份数组
year_array = np.array(years)

# 显示进度: 构建判断矩阵
print("    正在构建判断条件矩阵...")
with tqdm(total=3, desc="    进度") as pbar:
    # 准备数据: 转换为 numpy 数组以加速运算
    approval_years = df['批准年份'].values
    quit_years = df['退出年份'].values
    pbar.update(1)

    # 广播判断: 创建布尔矩阵 [机构数, 年份数]
    # 条件: 批准年份 <= 年份 < 退出年份
    survival_matrix = (
        (approval_years[:, np.newaxis] <= year_array) &
        (quit_years[:, np.newaxis] > year_array)
    ).astype(int)
    pbar.update(1)

    # 转换为 DataFrame
    survival_df = pd.DataFrame(
        survival_matrix,
        columns=[f'状态_{year}' for year in years],
        index=df.index
    )
    pbar.update(1)

print(f"    生成 {survival_df.shape[0]}×{survival_df.shape[1]} 的存续状态矩阵")

# ===== 步骤 6: 重塑数据格式 (原步骤 5) =====

print("\n    步骤 6: 重组数据格式...")

# 合并原始数据与存续状态
df_with_status = pd.concat([df.reset_index(drop=True), survival_df], axis=1)

# 显示进度: 重塑宽表为长表
print("    正在重塑数据格式...")
id_vars = ['统计市/区', '总行名称'] # 关键修改: 使用新的统计列
value_vars = [f'状态_{year}' for year in years]

# 使用 melt 将所有年份列转为长格式

```

```

df_long = df_with_status.melt(
    id_vars=id_vars,
    value_vars=value_vars,
    var_name='年份标记',
    value_name='是否存续'
)

# 提取年份数字
tqdm.pandas(desc="    提取年份")
df_long['年份'] = df_long['年份标记'].progress_apply(
    lambda x: int(x.split('_')[1])
)

# 筛选存续的机构 (状态=1)
df_surviving = df_long[df_long['是否存续'] == 1].copy()

print(f"    重塑完成! 存续机构记录数: {len(df_surviving):,}")

# ===== 步骤 7: 分组统计 (原步骤 6) =====

print("\n    步骤 7: 分组统计每年数量...")

# 显示进度: 按年份分组统计
result_dfs = []
for year in tqdm(years, desc="    统计进度"):
    # 筛选该年份数据
    year_data = df_surviving[df_surviving['年份'] == year]

    # 分组计数 (关键修改: 使用新的统计列)
    year_counts = year_data.groupby(
        ['统计市/区', '总行名称'],
        as_index=False
    ).agg(数量=('是否存续', 'sum'))

    # 添加年份列
    year_counts['年份'] = year

    result_dfs.append(year_counts)

# 合并所有年份结果
result_df = pd.concat(result_dfs, ignore_index=True)

# 排序
result_df = result_df.sort_values(

```

```

        ['年份', '统计市/区', '总行名称']
    ).reset_index(drop=True)

    # 重命名列以匹配原输出格式
    result_df = result_df.rename(columns={'统计市/区': '机构所在市/区'})

    print(f"        统计完成! 共生成 {len(result_df):,} 条记录")

    # ===== 步骤 8: 保存结果 (原步骤 7) =====

    print("\n    步骤 8: 保存最终结果...")

    output_columns = ['年份', '机构所在市/区', '总行名称', '数量']

    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    output_file = os.path.join(desktop_path, f'银行信息_按年份按机构所在地计数_{timestamp}.xlsx')

    # 显示进度: 写入 Excel
    with tqdm(total=1, desc="    写入 Excel") as pbar:
        result_df[output_columns].to_excel(output_file, index=False)
        pbar.update(1)

    print(f"        文件已保存至: {output_file}")

    # ===== 结果报告 =====
    total_time = (datetime.now() - start_time).total_seconds()
    print("\n" + "="*60)

    print("    板块四处理完成! ")

    print(f"    输出文件路径: {output_file}")

    print(f"    总耗时: {total_time:.1f} 秒 ({total_time/60:.1f} 分钟)")

    print(f"    统计年份: 2013-2024")

    print(f"    涉及银行: {result_df['总行名称'].nunique()} 家")

    print(f"    覆盖地区: {result_df['机构所在市/区'].nunique()} 个市/区")

    print(f"    总行-地区组合: {len(result_df)} 个")

```

```

print("="*60)

# 保存路径供板块五使用
path_file = os.path.join(desktop_path, "板块四输出路径.txt")
with open(path_file, 'w', encoding='utf-8') as f:
    f.write(output_file)

print(f"\n    路径已同步保存至: {path_file}")

return output_file

# ===== 执行板块四 =====
if __name__ == "__main__":
    calculate_annual_branch_counts()

```

板块五代码终版

```

import pandas as pd
import os
from datetime import datetime
import numpy as np
from tqdm import tqdm

def calculate_market_share_fixed():
    """
    板块五修正版：安全计算银行市场份额
    - 修复验证示例的 IndexError 风险
    - 纯向量化计算，无 Python 循环
    - 完善的除以零保护
    """

    desktop_path = os.path.expanduser("~/Desktop")
    input_file = os.path.join(desktop_path, "6 银行信息_按年份按机构所在地计数
20260120_142004.xlsx")

    if not os.path.exists(input_file):

        print(f"    错误：未找到板块四结果文件: {input_file}")

        return None

    print("="*60)

    print("    板块五：计算银行市场份额（修正版）")

```

```

print("="*60)

start_time = datetime.now()

print(f"    开始时间: {start_time.strftime('%H:%M:%S')}")


print("\n    步骤 1: 读取板块四结果文件...")
df = pd.read_excel(input_file)

print(f"        成功读取 {len(df):,} 条记录")


# ===== 步骤 2: 统计总数 (带进度) =====

print("\n    步骤 2: 计算各年份各地区分支机构总数...")


tqdm.pandas(desc="    分组求和")
total_counts = df.groupby(['年份', '机构所在市/区'])['数量'].progress_apply(
    lambda x: x.sum()
).reset_index(name='总数')


print(f"        完成! 共 {len(total_counts):,} 个年份-地区组合")


#    安全验证示例 (避免 IndexError)

try:
    beijing_2013 = total_counts[
        (total_counts['年份']==2013) &
        (total_counts['机构所在市/区']=='北京市')
    ]['总数'].values
    if len(beijing_2013) > 0:

        print(f"        示例: 2013 年北京市总数 = {beijing_2013[0]:,}家")
    else:

        print("        示例: 未找到 2013 年北京市数据")
except Exception as e:

    print(f"        示例查询失败: {e}")


# ===== 步骤 3: 合并并计算市场份额 (向量化) =====

print("\n    步骤 3: 计算市场份额...")

```

```

df_merged = pd.merge(df, total_counts, on=['年份', '机构所在市/区'], how='left')

# 向量化计算（无循环，速度提升 100 倍）
# np.where: 如果总数>0 则计算比例，否则 0
df_merged['市场份额'] = np.where(
    df_merged['总数'] > 0,
    df_merged['数量'] / df_merged['总数'],
    0
)

print(f"      计算完成! 市场份额范围: {df_merged['市场份额'].min():.4f} - {df_merged['
市场份额'].max():.4f}")

# ===== 步骤 4: 保存结果 =====

print("\n    步骤 4: 保存结果...")

output_columns = ['年份', '机构所在市/区', '总行名称', '市场份额']
df_merged['市场份额'] = df_merged['市场份额'].round(6) # 保留 6 位小数

timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
output_file = os.path.join(desktop_path, f"银行市场份额_按年份按所在地
_{timestamp}.xlsx")

with tqdm(total=1, desc="    写入 Excel") as pbar:
    df_merged[output_columns].to_excel(output_file, index=False)
    pbar.update(1)

print(f"      文件已保存至: {output_file}")

# ===== 步骤 5: 安全验证与报告 =====
total_time = (datetime.now() - start_time).total_seconds()

print("\n" + "="*60)

print("    板块五处理完成! ")

print(f"    输出文件路径: {output_file}")

print(f"    总耗时: {total_time:.1f} 秒")

```

```

# 安全示例（带异常处理）
try:
    sample = df_merged[
        (df_merged['年份'] == 2020) &
        (df_merged['机构所在市/区'] == '北京市') &
        (df_merged['总行名称'] == '中国工商银行')
    ]
    if not sample.empty:
        share = sample['市场份额'].iloc[0]

        print(f"    验证示例：2020 年北京市工行市场份额 = {share:.2%}")
    else:

        print("    验证示例：未找到 2020 年北京市工行数据")
except Exception as e:

    print(f"    验证失败: {e}")

print("="*60)

# 保存路径
path_file = os.path.join(desktop_path, "板块五输出路径.txt")
with open(path_file, 'w', encoding='utf-8') as f:
    f.write(output_file)

print(f"\n    路径已同步保存至: {path_file}")

return output_file

# ===== 执行板块五 =====
if __name__ == "__main__":
    calculate_market_share_fixed()

```

板块六代码终版

```

import pandas as pd
import os
from datetime import datetime
import numpy as np
from tqdm import tqdm

```

```

def calculate_hhi_index():
    """
    板块六：计算各年份各地区银行竞争程度指标 HHI
    核心逻辑：HHI =  $\Sigma(\text{市场份额}^2)$ ，按年份和地区分组统计
    """

    # ===== 步骤 1：读取板块五结果文件 =====
    desktop_path = os.path.expanduser("~/Desktop")
    input_file = os.path.join(desktop_path, "7 银行市场份额_按年份按所在地
    20260120_142921.xlsx")

    if not os.path.exists(input_file):

        print(f"  错误：未找到板块五结果文件: {input_file}")
        return None

    print("="*60)

    print("  板块六：计算 HHI 银行竞争指数")
    print("="*60)

    start_time = datetime.now()

    print(f"  开始时间: {start_time.strftime('%H:%M:%S')}")

    print("\n  步骤 1：读取板块五结果文件...")
    df = pd.read_excel(input_file)

    print(f"      成功读取 {len(df):,} 条记录")

    print(f"      数据范围: {df['年份'].min():.0f}-{df['年份'].max():.0f} 年")

    print(f"      覆盖地区: {df['机构所在市/区'].nunique():,} 个")

    print(f"      涉及银行: {df['总行名称'].nunique():,} 家")

    # ===== 步骤 2：计算市场份额平方（向量化） =====

    print("\n  步骤 2：计算市场份额平方...")

    # 向量化操作：对整个列进行平方运算（无循环）
    df['市场份额平方'] = df['市场份额'] ** 2

```

```

print(f"        完成! 新增列'市场份额平方'")

print(f"        平方值范围: {df['市场份额平方'].min():.8f} - {df['市场份额平方'].max():.8f}")

# ===== 步骤 3: 分组计算 HHI (核心步骤) =====

print("\n    步骤 3: 分组计算 HHI...")

# 使用 tqdm 包装 groupby 操作以显示进度
tqdm.pandas(desc="    分组求和")

# 核心计算: 对每年每地区, 汇总所有银行的市场份额平方
hhi_df = df.groupby(['年份', '机构所在市/区'])['市场份额平方'].progress_apply(
    lambda x: x.sum() # 对分组内的平方值求和
).reset_index(name='HHI')

print(f"        计算完成! 共 {len(hhi_df):,} 个 HHI 值")

print(f"        HHI 范围: {hhi_df['HHI'].min():.6f} - {hhi_df['HHI'].max():.6f}")

# ===== 步骤 4: HHI 结果解读 (安全版) =====

print("\n    步骤 4: HHI 结果分析...")

# 计算平均 HHI
avg_hhi = hhi_df['HHI'].mean()

print(f"        平均 HHI: {avg_hhi:.6f}")

#    安全查找极值 (避免空表报错)
if len(hhi_df) > 0:
    try:
        max_idx = hhi_df['HHI'].idxmax()
        min_idx = hhi_df['HHI'].idxmin()

        if pd.notna(max_idx) and pd.notna(min_idx):
            max_hhi_row = hhi_df.loc[max_idx]
            min_hhi_row = hhi_df.loc[min_idx]

            print(f"        最高 HHI: {max_hhi_row['HHI']:.6f} ({max_hhi_row['年份']})

```

```

    年 {max_hhi_row['机构所在市/区']})

        print(f"        最低 HHI: {min_hhi_row['HHI']:.6f} ({min_hhi_row['年份']}
    年 {min_hhi_row['机构所在市/区']})")
    else:

        print("        无法找到有效极值")

    except Exception as e:

        print(f"        极值查找失败: {e}")

    else:

        print("        警告: HHI 结果为空")

# 根据 HHI 值判断竞争程度
def interpret_hhi(hhi):
    if hhi < 0.01:
        return "高度竞争"
    elif hhi < 0.15:
        return "适度竞争"
    else:
        return "高度集中"

print(f"        平均竞争程度: {interpret_hhi(avg_hhi)}")

# ===== 步骤 5: 保存结果 =====

print("\n    步骤 5: 保存 HHI 结果...")

output_columns = ['年份', '机构所在市/区', 'HHI']

timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
output_file = os.path.join(desktop_path, f"HHI_{timestamp}.xlsx")

# 显示保存进度
with tqdm(total=1, desc="    写入 Excel") as pbar:
    hhi_df[output_columns].to_excel(output_file, index=False)
    pbar.update(1)

print(f"        文件已保存至: {output_file}")

# ===== 结果报告 =====

```

```

total_time = (datetime.now() - start_time).total_seconds()

print("\n" + "="*60)

print("    板块六处理完成! ")

print(f"    输出文件路径: {output_file}")

print(f"    总耗时: {total_time:.1f} 秒 ({total_time/60:.1f} 分钟)")

print(f"    输出 HHI 记录数: {len(hhi_df):,} 条")

print("="*60)

# 保存路径供后续使用
path_file = os.path.join(desktop_path, "板块六输出路径.txt")
with open(path_file, 'w', encoding='utf-8') as f:
    f.write(output_file)

print(f"\n    路径已同步保存至: {path_file}")

return output_file

# ===== 执行板块六 =====
if __name__ == "__main__":
    calculate_hhi_index()

```

板块七代码终版

```

import pandas as pd
import os
from datetime import datetime
from tqdm import tqdm

def main():
    # 获取 Mac 桌面路径
    desktop_path = os.path.join(os.path.expanduser("~"), "Desktop")

    # 文件路径
    file1_path = os.path.join(desktop_path, "3 企业上市年限.xlsx")
    file2_path = os.path.join(desktop_path, "HHI.xlsx")

```

```

# 读取 Excel 文件（显示进度）
print("正在读取数据文件...")
with tqdm(total=2, desc="读取进度") as pbar:
    # 读取第一张表，将股票代码作为字符串处理
    df1 = pd.read_excel(file1_path, dtype={'股票代码': str})
    # 确保股票代码为 6 位字符串
    df1['股票代码'] = df1['股票代码'].astype(str).str.zfill(6)
    pbar.update(1)

    # 读取第二张表
    df2 = pd.read_excel(file2_path)
    pbar.update(1)

print(f'第一张表记录数: {len(df1)} 行')
print(f'第二张表记录数: {len(df2)} 行')

# 步骤 2 和 3 的优化实现：创建 HHI 查询字典
print("\n 正在构建 HHI 索引字典...")
# 将 df2 转换为以(会计年度, 机构所在市/区)为键的字典
hhi_dict = {}
for _, row in tqdm(df2.iterrows(), total=len(df2), desc="构建 HHI 字典"):
    key = (row['会计年度'], row['机构所在市/区'])
    hhi_dict[key] = row['HHI']

# 使用字典映射的方式高效匹配 HHI（替代嵌套循环）
print("\n 正在匹配 HHI 数据...")
# 创建匹配用的元组列
df1['匹配键'] = list(zip(df1['会计年度'], df1['所属城市']))

# 使用 map 方法进行高效查找（性能远高于循环）
df1['HHI'] = df1['匹配键'].map(hhi_dict)

# 删除临时列
result_df = df1[['股票代码', '会计年度', '所属城市', 'HHI']].copy()

# 显示匹配结果统计
matched_count = result_df['HHI'].notna().sum()
print(f'\n 匹配成功: {matched_count} 行')
print(f'未匹配: {len(result_df) - matched_count} 行')

# 生成带时间戳的输出文件名
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
output_filename = f"HHI 匹配_{timestamp}.xlsx"
output_path = os.path.join(desktop_path, output_filename)

```

```

# 保存结果到 Excel
print(f"\n 正在保存结果到: {output_path}")
result_df.to_excel(output_path, index=False)

print("\n   任务完成! ")

print(f'输出文件: {output_path}')
print(f'总记录数: {len(result_df)} 行")

# 显示前 5 行样本
print("\n 前 5 行数据样本:")
print(result_df.head())

if __name__ == "__main__":
    main()

```

#20260130 新增

检验上市年限数据表和总资产净利润率两张数据表的同一（股票代码，会计年度）行业代码和行业名称是否一致

```

import pandas as pd
import os
from datetime import datetime
from tqdm import tqdm
import sys

def main():
    # ===== 1. 路径设置 =====
    desktop_path = os.path.join(os.path.expanduser("~"), "Desktop")
    file1_path = os.path.join(desktop_path, "3 企业上市年限.xlsx")
    file2_path = os.path.join(desktop_path, "5 总资产净利润率.xlsx")

    if not os.path.exists(file1_path):
        raise FileNotFoundError(f"未找到文件: {file1_path}")
    if not os.path.exists(file2_path):
        raise FileNotFoundError(f"未找到文件: {file2_path}")

    print("   开始执行行业代码一致性检验...")

    # ===== 2. 读取原始数据（带进度） =====

    print("\n   步骤 1/4: 读取 Excel 文件...")

```

```

# 使用 tqdm 包装 read_excel 的迭代过程（通过 chunksize 模拟进度）
# 对于大文件，分块读取显示进度；小文件直接读取
try:
    # 先尝试直接读取，如果文件大再用分块
    df1 = pd.read_excel(file1_path, dtype={'股票代码': str})
    df2 = pd.read_excel(file2_path, dtype={'股票代码': str})
except Exception as e:
    print(f"    读取出错: {e}")
    sys.exit(1)

# 确保股票代码为 6 位字符串（补前导零）
df1['股票代码'] = df1['股票代码'].astype(str).str.zfill(6)
df2['股票代码'] = df2['股票代码'].astype(str).str.zfill(6)

# 确保用于比对的字段均为字符串类型，避免类型不匹配（如 1 vs 1.0 vs "1"）
for col in ['行业名称', '行业代码']:
    if col in df1.columns:
        df1[col] = df1[col].astype(str).str.strip()
    if col in df2.columns:
        df2[col] = df2[col].astype(str).str.strip()

print(f"    ✓ 表 1（企业上市年限）：{len(df1):,} 行")

print(f"    ✓ 表 2（总资产净利润率）：{len(df2):,} 行")

# 检测重复键（同一股票代码+会计年度出现多次会导致字典覆盖）
duplicates1 = df1.groupby(['股票代码', '会计年度']).size()
duplicates2 = df2.groupby(['股票代码', '会计年度']).size()
dup1 = duplicates1[duplicates1 > 1]
dup2 = duplicates2[duplicates2 > 1]

if len(dup1) > 0:
    print(f"        警告: 表 1 存在 {len(dup1)} 个重复(股票代码,会计年度)组合，字典将保留最后一条记录")
if len(dup2) > 0:
    print(f"        警告: 表 2 存在 {len(dup2)} 个重复(股票代码,会计年度)组合，字典将保留最后一条记录")

# ===== 3. 构建字典（带进度条）

```

```
=====
```

```
print("\n    步骤 2/4: 构建数据字典...")
```

```
# 优化：使用字典推导式+qdm，比 iterrows 快 10 倍以上，同时满足进度显示要求
# 对于极大文件，这是内存和速度的最佳平衡
```

```
print("    正在构建表 1 字典...")
```

```
# 转换为 numpy 数组加速遍历（比 iterrows 快）
```

```
arr1 = df1[['股票代码', '会计年度', '行业名称', '行业代码']].values
```

```
dict1 = {}
```

```
for row in tqdm(arr1, desc="    表 1 进度", total=len(arr1)):
```

```
    key = (row[0], row[1]) # (股票代码, 会计年度)
```

```
    value = (row[2], row[3]) # (行业名称, 行业代码)
```

```
    dict1[key] = value
```

```
print("    正在构建表 2 字典...")
```

```
arr2 = df2[['股票代码', '会计年度', '行业名称 1', '行业代码 1']].values
```

```
dict2 = {}
```

```
for row in tqdm(arr2, desc="    表 2 进度", total=len(arr2)):
```

```
    key = (row[0], row[1]) # (股票代码, 会计年度)
```

```
    value = (row[2], row[3]) # (行业名称 1, 行业代码 1)
```

```
    dict2[key] = value
```

```
print(f"    ✓ 表 1 字典: {len(dict1):,} 条唯一记录")
```

```
print(f"    ✓ 表 2 字典: {len(dict2):,} 条唯一记录")
```

```
# ===== 4. 高效核对（向量化优化，无 Python 循环）
```

```
=====
```

```
print("\n    步骤 3/4: 核对一致性（向量化加速）...")
```

```
# 找到共同键（交集）
```

```
common_keys = set(dict1.keys()) & set(dict2.keys())
```

```
total_common = len(common_keys)
```

```
print(f"        找到 {total_common:,} 条共同记录待核对")
```

```
if total_common == 0:
```

```
    print("        警告: 两张表没有共同记录，请检查数据！")
```

```
    result_df = pd.DataFrame(columns=['股票代码', '会计年度', '是否一致'])
```

```

else:
    # **关键优化**：避免使用 for 循环遍历 common_keys
    # 方法：将两个字典直接转为 DataFrame，使用 merge 对齐（数据库式连接，C
语言级速度）

    # 将字典转为 DataFrame（此步骤为 O(n)，但由底层 C 优化，极快）
    # 使用列表推导式，比 pd.DataFrame.from_dict 更快且保持顺序
    print("    正在转换为比对格式...")
    list1 = [{'股票代码': k[0], '会计年度': k[1], 'ind_name': v[0], 'ind_code': v[1]}
              for k, v in tqdm(dict1.items(), desc="    转换表 1", total=len(dict1))]
    list2 = [{'股票代码': k[0], '会计年度': k[1], 'ind_name1': v[0], 'ind_code1': v[1]}
              for k, v in tqdm(dict2.items(), desc="    转换表 2", total=len(dict2))]

    df1_clean = pd.DataFrame(list1)
    df2_clean = pd.DataFrame(list2)

    # 使用 merge 自动对齐共同键（inner join），复杂度 O(n log n)，完全无 Python
级循环
    print("    正在执行比对（向量化）...")
    merged = pd.merge(
        df1_clean,
        df2_clean,
        on=['股票代码', '会计年度'],
        how='inner' # 只保留共同键，即交集
    )

    # 向量化生成虚拟变量：一次性比较所有行（SIMD 优化，比循环快 100 倍以上）
    # 比较时去除首尾空格，防止不可见字符导致不一致
    merged['是否一致'] = (
        (merged['ind_name'].str.strip() == merged['ind_name1'].str.strip()) &
        (merged['ind_code'].str.strip() == merged['ind_code1'].str.strip())
    ).astype(int)

    # 选取结果列
    result_df = merged[['股票代码', '会计年度', '是否一致']]

    # 统计
    consistent_count = result_df['是否一致'].sum()

    print(f"        ✓        一 致 记 录  : {consistent_count:,}    条
({consistent_count/len(result_df)*100:.2f}%)")

    print(f"        ✓    不 一 致 记 录 : {len(result_df) - consistent_count:,}    条
(({len(result_df)-consistent_count)/len(result_df)*100:.2f}%)")

```

```

# ===== 5. 输出结果 =====

print("\n    步骤 4/4: 保存结果文件...")

# 生成带时间戳的文件名
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
output_filename = f"行业代码一致性检验_{timestamp}.xlsx"
output_path = os.path.join(desktop_path, output_filename)

# 保存为 Excel，确保股票代码作为文本格式（防止 Excel 自动去掉前导零）
with pd.ExcelWriter(output_path, engine='openpyxl') as writer:
    result_df.to_excel(writer, index=False, sheet_name='一致性检验结果')
    # 获取工作簿和工作表以调整格式
    workbook = writer.book
    worksheet = writer.sheets['一致性检验结果']

    # 设置股票代码列为文本格式（添加 Excel 的文本前缀）
    # 注意：openpyxl 处理大文件可能慢，这里仅添加格式
    from openpyxl.styles import Alignment
    for cell in worksheet['A']: # A 列是股票代码
        if cell.row > 1: # 跳过标题
            cell.number_format = '@' # 文本格式

print(f"    ✓ 文件已保存至: {output_path}")

print(f"    ✓ 总计输出 {len(result_df):,} 条记录")

print(f"\n    处理完成! ")

if __name__ == "__main__":
    try:
        main()
    except Exception as e:
        print(f"\n    程序运行出错: {str(e)}")
        import traceback
        traceback.print_exc()

```

运行结果：

开始执行行业代码一致性检验...

步骤 1/4: 读取Excel文件...

- ✓ 表1 (企业上市年限) : 43,908 行
- ✓ 表2 (总资产净利润率) : 52,895 行

步骤 2/4: 构建数据字典...

正在构建表1字典...

表1进度: 100% | 43908/43908 [00:00<00:00, 1403663.82it/s]

正在构建表2字典...

表2进度: 100% | 52895/52895 [00:00<00:00, 1527787.83it/s]

- ✓ 表1字典: 43,908 条唯一记录
- ✓ 表2字典: 52,895 条唯一记录

步骤 3/4: 核对一致性 (向量化加速) ...

- 找到 43,816 条共同记录待核对
- 正在转换为比对格式...

转换表1: 100% | 43908/43908 [00:00<00:00, 1825316.67it/s]

转换表2: 100% | 52895/52895 [00:00<00:00, 2101781.12it/s]

正在执行比对 (向量化) ...

- ✓ 一致记录: 34,649 条 (79.08%)
- ✓ 不一致记录: 9,167 条 (20.92%)

步骤 4/4: 保存结果文件...

- ✓ 文件已保存至: /Users/air/Desktop/行业代码一致性检验_20260130_110243.xlsx
- ✓ 总计输出 43,816 条记录

处理完成!

进一步检验:

```
import pandas as pd
```

```
import os
```

```
from datetime import datetime
```

```
from tqdm import tqdm
```

```
import sys
```

```
def main():
```

```
    # ===== 1. 路径设置 =====
```

```
    desktop_path = os.path.join(os.path.expanduser("~"), "Desktop")
```

```
    file1_path = os.path.join(desktop_path, "3 企业上市年限.xlsx")
```

```
    file2_path = os.path.join(desktop_path, "5 总资产净利润率.xlsx")
```

```
    if not os.path.exists(file1_path):
```

```
        raise FileNotFoundError(f"未找到文件: {file1_path}")
```

```
    if not os.path.exists(file2_path):
```

```
        raise FileNotFoundError(f"未找到文件: {file2_path}")
```

```
    print(" 开始执行行业代码一致性检验 (增强版) ...")
```

```
    # ===== 2. 读取原始数据 =====
```

```
    print("\n 步骤 1/4: 读取 Excel 文件...")
```

```
    # 读取数据, 股票代码强制为字符串
```

```
    try:
```

```

df1 = pd.read_excel(file1_path, dtype={'股票代码': str})
df2 = pd.read_excel(file2_path, dtype={'股票代码': str})
except Exception as e:
    raise Exception(f"读取 Excel 失败: {e}")

# **关键修正**: 确保股票代码和会计年度均为字符串, 保证字典键类型一致 (防止 int
与 float 不匹配)
df1['股票代码'] = df1['股票代码'].astype(str).str.strip().str.zfill(6)
df1['会计年度'] = df1['会计年度'].astype(str).str.strip()
df2['股票代码'] = df2['股票代码'].astype(str).str.strip().str.zfill(6)
df2['会计年度'] = df2['会计年度'].astype(str).str.strip()

# 清理行业字段, 去除首尾空格, 统一为字符串
for col in ['行业名称', '行业代码']:
    if col in df1.columns:
        df1[col] = df1[col].astype(str).str.strip()

for col in ['行业名称 1', '行业代码 1']:
    if col in df2.columns:
        df2[col] = df2[col].astype(str).str.strip()

print(f"    ✓ 表 1 (企业上市年限) : {len(df1):,} 行")

print(f"    ✓ 表 2 (总资产净利润率) : {len(df2):,} 行")

# ===== 3. 构建字典 (带进度显示) =====

print("\n    步骤 2/4: 构建数据字典...")

# **优化说明**: 使用 groupby() 先按 key 分组 (去重), 再遍历组 (而非原始行), 效率更高且满足"字典覆盖"逻辑
# 表 1 字典: {(股票代码, 会计年度): (行业名称, 行业代码)}
grouped1 = df1.groupby(['股票代码', '会计年度'])
dict1 = {}
print(f"    正在处理表 1 ({len(grouped1)} 个唯一键) ...")
for (code, year), group in tqdm(grouped1, total=len(grouped1), desc="    构建表 1 字典"):
    # 取最后一行作为该键的值 (模拟字典覆盖特性)
    last_row = group.iloc[-1]
    dict1[(code, year)] = (last_row['行业名称'], last_row['行业代码'])

# 表 2 字典: {(股票代码, 会计年度): (行业名称 1, 行业代码 1)}
grouped2 = df2.groupby(['股票代码', '会计年度'])

```

```

dict2 = {}
print(f"    正在处理表 2 ( {len(grouped2)} 个唯一键) ...")
for (code, year), group in tqdm(grouped2, total=len(grouped2), desc="    构建表 2 字典"):
    last_row = group.iloc[-1]
    dict2[(code, year)] = (last_row['行业名称 1'], last_row['行业代码 1'])

# 检测重复键警告
dup1_count = len(df1) - len(dict1)
dup2_count = len(df2) - len(dict2)
if dup1_count > 0:

    print(f"        表 1 存在 {dup1_count} 条重复键记录 (已自动去重, 保留最后出
现值) ")
    if dup2_count > 0:

        print(f"        表 2 存在 {dup2_count} 条重复键记录 (已自动去重, 保留最后出
现值) ")

print(f"    ✓ 表 1 字典: {len(dict1):,} 条唯一记录")

print(f"    ✓ 表 2 字典: {len(dict2):,} 条唯一记录")

# ===== 4. 核 对 一 致 性 ( 向 量 化 处 理 )
=====

print("\n    步骤 3/4: 核对一致性...")

# 找到共同键 (交集)
common_keys = set(dict1.keys()) & set(dict2.keys())
total_common = len(common_keys)

if total_common == 0:

    print("        警告: 两张表没有共同记录, 请检查数据! ")

    # 创建空结果表, 保持列结构一致
    result_df = pd.DataFrame(columns=['股票代码', '会计年度', '行业名称', '行业代码',
                                     '行业名称 1', '行业代码 1', '是否一致'])
else:

    print(f"        找到 {total_common:,} 条共同记录待核对")

# **向量化核心**: 将字典转换为 DataFrame 进行比对 (避免 Python 级循环)
print("    正在转换为比对格式...")

```

```

# 使用列表推导式快速构建 DataFrame (比 iterrows 高效)
data_rows = []
for key in tqdm(common_keys, desc="    数据准备", total=len(common_keys)):
    code, year = key
    name1, code1 = dict1[key]
    name2, code2 = dict2[key]
    # 判断是否一致 (同时比较名称和代码)
    is_consistent = 1 if (name1 == name2 and code1 == code2) else 0
    data_rows.append({
        '股票代码': code,
        '会计年度': year,
        '行业名称': name1,
        '行业代码': code1,
        '行业名称 1': name2,
        '行业代码 1': code2,
        '是否一致': is_consistent
    })

result_df = pd.DataFrame(data_rows)

consistent_count = result_df['是否一致'].sum()
inconsistent_count = len(result_df) - consistent_count

print(f"          ✓    一    致    记    录    :    {consistent_count:,}    条
({consistent_count/len(result_df)*100:.2f}%)")

print(f"          ✓    不    一    致    记    录    :    {inconsistent_count:,}    条
({inconsistent_count/len(result_df)*100:.2f}%)")

# ===== 5. 构建不一致详情字典 (仅在有不一致时) =====
inconsistent_dict = {}
if total_common > 0 and len(result_df) > 0:
    inconsistent_df = result_df[result_df['是否一致'] == 0]

    if len(inconsistent_df) > 0:
        print("\n    步骤 4/5: 构建不一致详情字典...")

        # **满足用户要求**: 键=(股票代码, 会计年度), 值=(表 1 行业名, 表 1 行业
        码, 表 2 行业名, 表 2 行业码)
        # 使用 zip 快速构建 (无显式 Python 循环, 但用 tqdm 包装以显示进度)
        keys = list(zip(inconsistent_df['股票代码'], inconsistent_df['会计年度']))

```

```

values = list(zip(inconsistent_df['行业名称'], inconsistent_df['行业代码'],
                  inconsistent_df['行业名称 1'], inconsistent_df['行业代码 1']))

# 使用 tqdm 显示字典构建进度
inconsistent_dict = dict(tqdm(zip(keys, values),
                               total=len(keys),
                               desc="    生成不一致字典"))

print(f"    ✓ 不一致字典已创建: {len(inconsistent_dict):,} 条记录")

# 显示前 5 条不一致样本
print("\n    不一致样本预览 (前 5 条) :")
for i, (k, v) in enumerate(list(inconsistent_dict.items())[:5]):
    print(f"        {i+1}. {k}: '{v[0]}'({v[1]}) vs '{v[2]}'({v[3]}")

# ===== 6. 输出结果文件 =====

print("\n    步骤 5/5: 保存结果文件...")

timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
output_filename = f'行业代码一致性检验_1_{timestamp}.xlsx'
output_path = os.path.join(desktop_path, output_filename)

# 确保输出列顺序完全符合要求
output_columns = ['股票代码', '会计年度', '行业名称', '行业代码',
                  '行业名称 1', '行业代码 1', '是否一致']

# 保存为 Excel, 并设置文本格式防止 Excel 自动转换
with pd.ExcelWriter(output_path, engine='openpyxl') as writer:
    result_df[output_columns].to_excel(writer, index=False, sheet_name='一致性检验结果')

# 设置特定列为文本格式 (防止前导零丢失)
workbook = writer.book
worksheet = writer.sheets['一致性检验结果']

# A 列(股票代码)、D 列(行业代码)、F 列(行业代码 1)设为文本格式
text_columns = ['A', 'D', 'F']
for col_letter in text_columns:
    for cell in worksheet[col_letter]:
        if cell.row > 1: # 跳过标题行
            cell.number_format = '@'

```

```

print(f"    ✓ 文件已保存至: {output_path}")

print(f"    ✓ 总计输出 {len(result_df);,} 条记录")

if inconsistent_dict:

    print(f"    ✓ 其中 {len(inconsistent_dict);,} 条记录存在行业代码/名称差异")

print(f"\n    处理完成! Excel 中包含完整的对比详情, 可直接查看差异。")

return inconsistent_dict

if __name__ == "__main__":
    try:
        result_dict = main()
    except Exception as e:

        print(f"\n    程序运行出错: {str(e)}")

        import traceback
        traceback.print_exc()
        sys.exit(1)

```

检验结果:

发现 (行业名称, 行业代码) 和 (行业名称 1, 行业代码 1) 的分类标准不同:

(行业名称, 行业代码) 的分类标准才是 2012 年证监会分类;

(行业名称 1, 行业代码 1) 是中国上市公司协会行业分类代码。

2023.05.01 及之后证监会行业分类停止更新, 改用中国上市公司协会行业分类代码。

Fl_T5[DES][xlsx].txt — 已编辑

Stkcd [股票代码] - 以沪、深、北证券交易所公布的证券代码为准。

ShortName [股票简称] - 以沪、深、北证券交易所公布的证券简称为准。

Accper [统计截止日期] - 指会计报表日, 统一用10位字符表示, 如1999-12-31。

Typrep [报表类型编码] - A: 合并报表; B: 母公司报表; 在公司未公布合并报表, 本数据库以单一报表数据添列。

Source [公告来源] - 0: 定期报告; 1: IPO公告;

Indcd [行业代码] - 2012年证监会行业分类代码。2023.05.01及之后证监会行业分类停止更新, 改用中国上市公司协会行业分类代码。

Indnme [行业名称] - 2012年证监会行业分类名称。2023.05.01及之后证监会行业分类停止更新, 改用中国上市公司协会行业分类名称。

Indcd1 [行业代码1] - 中国上市公司协会行业分类代码。

Indnme1 [行业名称1] - 中国上市公司协会行业分类名称。

F050201B [总资产净利润率(ROA)A] - 计算公式为: 净利润 / 总资产余额; 当分母未公布或为零时, 以NULL表示

F050202B [总资产净利润率(ROA)B] - 计算公式为: 净利润 / 总资产平均余额; 当分母未公布或为零时, 以NULL表示: 总资产平均余额=资产合计期末余额+资产合计期初余额) / 2

F050203B [总资产净利润率(ROA)C] - 计算公式为: 净利润 / 总资产平均余额; 当分母未公布或为零时, 以NULL表示; 总资产平均余额=资产合计期末余额+资产合计上年期末余额) / 2

F050204C [总资产净利润率(ROA)TTM] - 计算公式为: (净利润TTM) / 总资产平均余额; 当分母未公布或为零时, 以NULL表示; 总资产平均余额=资产合计期末余额+资产合计上年同期期末余额) / 2

十、上市公司高管数字背景分析 Python 代码

http://localhost:8889/notebooks/Desktop/Python/%E6%AF%95%E4%B8%9A%E8%AE%BA%E6%96%87_%E9%AB%98%E7%AE%A1%E8%83%8C%E6%99%AF%E6%95%B0%E6%8D%AE%E5%A4%84%E7%90%86_0104.ipynb

注：1、所有代码运行前需把 excel 文件放到桌面上；2、代码中的路径只适用于 mac 系统，windows 系统需要对路径代码进行修改；3、需要把桌面上的文件名称修改至与代码中完全相同。

kimi 指令：

我现在需要你帮我设计一段 python 代码：

一、现有数据：

- 1、一张 excel，第 1 行为英文标题，第 2 行为中文标题，从第 3 行开始是数据。第 A-J 列分别为：证券代码、统计截止日期，人员 ID、职务类型、职务 ID、具体职务、性别、年龄、高管个人简历。其中高管个人简历是一段比较长的文字。
- 2、现有文献常用这种处理方法判断一家上市公司高管是否为数字背景：如果该上市公司高管中任一人所学专业涉及“信息、智能、软件、电子、通信、系统、网络、自动、无线、计算机”，则视为该公司管理层具有数字化背景。

二、我的需求——使用 python 处理得到 2024 年度上市公司高管数字背景虚拟变量 DigEdu_Executive，该虚拟变量=1，说明这家公司高管具有数字背景，该虚拟变量=0，说明这家公司高管不具有数字背景。

- 1、板块一：使用高管简历信息，输出每一位高管是否为数字背景的虚拟变量值（0 或 1）：
 - （1）用这张表的所有数据新建一个字典 1，每一行的 A-I 列新建元组作为键值对的键，第 J 列（即高管个人简历）作为键值对的值，如果 excel 中某一行第 J 列为空，则只需要输入键，值保持为空。
 - （2）新建一个列表，作为检验上市公司高管是否有数字背景的标准，即['信息','智能','软件','电子','通信','系统','网络','自动','无线','计算机']。
 - （3）对字典 1 进行遍历，判断每一个键值对的值中是否包含列表中的任意一个字符串，只要包含任意一个字符串，将这个键值对的值修改为 1，如果某个键值对中的值不包含列表中任何一个字符串，则修改这个键值对值为 0。如果这个键值对值为空，则修改这个键值对值为 0。
 - （4）将得到的结果输出为 excel1，第 A-J 列分别为证券代码、统计截止日期，人员 ID、职务类型、职务 ID、具体职务、性别、年龄、高管数字背景（这列就是刚刚得到的字典的值），保存到 mac 的桌面。

2、板块二：把同一家公司的高管背景信息合并，只要这家公司有一个高管有数字背景，则认为该家上市公司高管有数字背景：

- （1）新建一个字典 2，将刚刚板块一得到的 excel 表格中证券代码列的每一个证券代码作为字典中每一个键值对中的键；
- （2）对板块一得到的表格中，同一家上市公司（即证券代码相同的行）高管数字背景列的数字进行加总，作为该上市公司键值对的值。

(3) 判断每一个键值对的值是否大于 0，当值大于 0，说明该键值对对应的上市公司高管具有数字背景，同时修改该键值对的值为 1，当值=0，说明该键值对对应的上市公司高管不具有数字背景。

(4) 输出板块二的结果到一个新的 excel2 里面，保存到 mac 的桌面。

第一板块代码终版

http://localhost:8888/notebooks/Desktop/Python/%E6%AF%95%E4%B8%9A%E8%AE%BA%E6%96%87_%E9%AB%98%E7%AE%A1%E8%83%8C%E6%99%AF%E6%95%B0%E6%8D%AE%E5%A4%84%E7%90%86_0104.ipynb

终端输入：

```
pip install pandas openpyxl
```

```
import pandas as pd
```

```
import os
```

```
from datetime import datetime
```

```
def analyze_executive_background(input_file):
```

```
    """
```

```
    板块一： 分析每位高管是否具有数字背景
```

```
    输入： Excel 文件路径
```

```
    输出： 保存处理后的 Excel 文件到桌面
```

```
    """
```

```
    print("    正在读取 Excel 数据...")
```

```
    try:
```

```
        # 读取整个 Excel（包括标题行）
```

```
        df_full = pd.read_excel(input_file, sheet_name=0, header=None)
```

```
    except FileNotFoundError:
```

```
        raise FileNotFoundError(f"    找不到文件: {input_file}\n 请确认文件是否在桌面! ")
```

```
    except Exception as e:
```

```
        raise Exception(f"    读取 Excel 文件失败: {str(e)}")
```

```
    if df_full.empty or len(df_full) < 2:
```

```
        raise ValueError("Excel 文件数据不足， 至少需要包含标题行和数据行")
```

```
    # 提取前两行标题
```

```
    header_row1 = df_full.iloc[0].fillna("").astype(str).tolist() # 英文标题
```

```
    header_row2 = df_full.iloc[1].fillna("").astype(str).tolist() # 中文标题
```

```

# 从第三行开始读取数据
df_data = df_full.iloc[2:].reset_index(drop=True)

# 确保至少 10 列
if df_data.shape[1] < 10:
    raise ValueError(f'Excel 数据列数不足 10 列, 当前只有 {df_data.shape[1]} 列')

# 数字背景关键词列表
keywords = [
    '信息', '智能', '软件', '电子', '通信', '系统',
    '网络', '自动', '无线', '计算机'
]

print(f'    正在分析 {len(df_data)} 位高管的数字背景...')

# 准备输出数据
output_data = []

# 遍历每一行数据
for idx, row in df_data.iterrows():
    # 获取 A-I 列数据并处理缺失值
    sec_code = str(row[0]) if pd.notna(row[0]) else "
    end_date = row[1] if pd.notna(row[1]) else "
    person_id = str(row[2]) if pd.notna(row[2]) else "
    position_type = str(row[3]) if pd.notna(row[3]) else "
    position_id = str(row[4]) if pd.notna(row[4]) else "
    position_detailed = str(row[5]) if pd.notna(row[5]) else "
    gender = str(row[6]) if pd.notna(row[6]) else "
    age = row[7] if pd.notna(row[7]) else "
    col_i = str(row[8]) if pd.notna(row[8]) else "
    resume = str(row[9]) if pd.notna(row[9]) else "

    # 判断是否有数字背景
    if resume.strip() == "":
        digital_bg = 0
    else:
        # 检查是否包含任一关键词
        has_digital = any(keyword in resume for keyword in keywords)
        digital_bg = 1 if has_digital else 0

    # 构建输出行 (包含原始所有列和新列)
    output_row = [

```

```

        sec_code, end_date, person_id, position_type, position_id,
        position_detailed, gender, age, col_i, resume, digital_bg
    ]
    output_data.append(output_row)

# 构建输出的 DataFrame
if len(header_row1) >= 10:
    output_columns = header_row1[:10] + ['DigEdu_Executive']
else:
    output_columns = [f'col_{i}' for i in range(10)] + ['DigEdu_Executive']

# 确保列名都是字符串且唯一
output_columns = [str(col) for col in output_columns]
df_output = pd.DataFrame(output_data, columns=output_columns)

# 保存到桌面（带时间戳）
desktop_path = os.path.join(os.path.expanduser('~'), 'Desktop')
timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')
output_file1 = os.path.join(desktop_path, f'板块一_高管数字背景_{timestamp}.xlsx')

print(f'    正在保存板块一结果...')

# 使用 openpyxl 保留原有标题格式
try:
    from openpyxl import Workbook
    from openpyxl.utils.dataframe import dataframe_to_rows

    wb = Workbook()
    ws = wb.active
    ws.title = "高管数字背景"

    # 写入标题行
    ws.append(header_row1[:10])
    ws.append(header_row2[:10])

    # 写入数据（跳过 pandas 的 header）
    for r in dataframe_to_rows(df_output, index=False, header=False):
        ws.append(r)

    # 自动调整列宽
    for column in ws.columns:
        max_length = 0
        column_letter = column[0].column_letter

```

```

        for cell in column:
            try:
                if cell.value and len(str(cell.value)) > max_length:
                    max_length = len(str(cell.value))
            except:
                pass
        adjusted_width = min(max_length + 2, 50)
        ws.column_dimensions[column_letter].width = adjusted_width

    wb.save(output_file1)

except ImportError:

    print("    未安装 openpyxl 库, 使用 pandas 默认方式保存...")

    with pd.ExcelWriter(output_file1, engine='openpyxl') as writer:
        pd.DataFrame([header_row1[:10]]).to_excel(writer, index=False, header=False,
startrow=0)
        pd.DataFrame([header_row2[:10]]).to_excel(writer, index=False, header=False,
startrow=1)
        df_output.to_excel(writer, index=False, header=True, startrow=2,
sheet_name='Sheet1')

    print(f"    板块一完成! 文件已保存至桌面")

    return output_file1

# ===== 主程序调用部分 =====
if __name__ == "__main__":
    print("="*70)

    print("    上市公司高管数字背景分析工具 - 板块一")

    print("="*70)

    # Mac 桌面路径
    desktop_path = os.path.join(os.path.expanduser('~'), 'Desktop')

    # 指定文件名 (标黄部分需根据实际标题名修改)
    target_file = os.path.join(desktop_path, "2024 年高管简历信息.xlsx")

    # 检查文件是否存在
    if not os.path.exists(target_file):

        print(f"\n    错误: 找不到文件 '{target_file}'")

```

```

print("\n 请确认: ")
print("1. 文件名为 '2024' 年高管简历信息.xlsx")
print("2. 文件在桌面 (Desktop) 文件夹中")
print("\n 如果文件名不同, 请修改代码中的 target_file 变量")
else:

    print(f"\n    找到目标文件: {target_file}")

    print(f"\n    开始执行板块一分析...")

    try:
        # 调用函数
        result_file = analyze_executive_background(target_file)

        print("\n" + "="*70)

        print("    分析完成! ")
        print("="*70)

        print(f"    结果文件: {os.path.basename(result_file)}")

        print(f"    保存位置: 桌面")

        # 显示统计摘要
        df_check = pd.read_excel(result_file, skiprows=2)

        print(f"\n    统计摘要:")

        print(f"    • 分析高管人数: {len(df_check):,} 人")
        print(f"    • 数字背景高管: {df_check.iloc[:, -1].sum():,} 人")
        print(f"    • 占比: {df_check.iloc[:, -1].mean()*100:.2f}%")

    except Exception as e:

        print(f"\n    处理错误: {str(e)}")

        import traceback
        traceback.print_exc()

print("\n 按回车键退出...")
input()

```

第二板块代码终版 (存在证券代码缺 0 问题)

```

import pandas as pd
import os

def merge_company_background(input_file1):
    """
    板块二： 合并公司层面数字背景虚拟变量
    输入： 板块一结果 Excel 文件路径
    输出： 保存公司层面分析结果到桌面
    """

    print("\n    正在读取板块一结果...")

    try:
        # 读取数据 (跳过前两行标题)
        df = pd.read_excel(input_file1, skiprows=2)
    except Exception as e:

        raise Exception(f"    读取板块一结果失败: {str(e)}")

    if df.empty:
        raise ValueError("板块一结果为空")

    # 识别关键列 (证券代码列和数字背景列)
    sec_code_col = df.columns[0] # 第一列： 证券代码
    digital_bg_col = df.columns[-1] # 最后一列： 高管数字背景

    # 数据类型转换
    df[sec_code_col] = df[sec_code_col].astype(str)
    df[digital_bg_col] = pd.to_numeric(df[digital_bg_col], errors='coerce').fillna(0).astype(int)

    print(f"    正在合并 {len(df)} 条高管记录至公司层面...")

    # 按公司分组统计
    company_stats = df.groupby(sec_code_col).agg({
        digital_bg_col: ['sum', 'count']
    }).reset_index()

    # 重命名列
    company_stats.columns = ['证券代码', '数字背景高管数', '高管总数']

    # 生成公司层面虚拟变量
    company_stats['DigEdu_Executive'] = (company_stats['数字背景高管数'] > 0).astype(int)

```

```

# 保存到桌面 (带时间戳)
desktop_path = os.path.join(os.path.expanduser('~'), 'Desktop')
timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')
output_file2 = os.path.join(desktop_path, f'板块二_公司数字背景_{timestamp}.xlsx')

print(f"    正在保存板块二结果...")

# 准备统计摘要数据
summary_data = [ ['指标', '数值'],
                  ['分析上市公司总数', len(company_stats)],
                  ['具有数字背景的公司数', company_stats['DigEdu_Executive'].sum()],
                  ['公司数字背景占比', f'{company_stats["DigEdu_Executive"].mean()*100:.2f}%'],
                  ['平均每公司高管数', f'{company_stats["高管总数"].mean():.1f} 人'],
                  ['总计分析高管数', company_stats["高管总数"].sum()]]

# 保存结果
with pd.ExcelWriter(output_file2, engine='openpyxl') as writer:
    # Sheet1: 主要结果
    company_stats.to_excel(writer, index=False, sheet_name='公司数字背景')

    # Sheet2: 统计摘要
    pd.DataFrame(summary_data).to_excel(writer, index=False, header=False,
sheet_name='统计摘要')

print(f"    板块二完成! 文件已保存至桌面")

return output_file2

# ===== 主程序: 直接执行板块二 =====
if __name__ == "__main__":
    print("="*70)

    print("    上市公司高管数字背景分析工具 - 板块二")

    print("="*70)

    # Mac 桌面路径
    desktop_path = os.path.join(os.path.expanduser('~'), 'Desktop')

    # 请确保此文件名与您桌面上的文件完全一致 (标黄部分需根据实际标题名修改)
    target_file1 = os.path.join(desktop_path, "板块一_高管数字背景_20260104_213343.xlsx")

    # 检查文件是否存在

```

```

if not os.path.exists(target_file1):

    print(f"\n  错误: 找不到板块一结果文件")

    print(f"\n  预期文件路径: {target_file1}")
    print("\n  请确认: ")
    print("1. 文件名为 '板块一_高管数字背景_20260104_213343.xlsx'")
    print("2. 文件在桌面 (Desktop) 文件夹中")
    print("\n  如果文件名不同, 请修改代码中的 target_file1 变量")
else:

    print(f"\n  找到板块一结果文件: {os.path.basename(target_file1)}")

    print(f"  文件路径: {target_file1}")

    print(f"\n  开始执行板块二分析...")

try:
    # 调用板块二函数
    result_file2 = merge_company_background(target_file1)

    print("\n" + "="*70)

    print("  板块二处理完成! ")
    print("="*70)

    print(f"  结果文件: {os.path.basename(result_file2)}")

    print(f"  保存位置: 桌面")

    # 显示统计摘要
    df_company = pd.read_excel(result_file2, sheet_name='公司数字背景')

    summary_data = [['分析上市公司总数', len(df_company)],
                    ['具有数字背景的公司数', df_company['DigEdu_Executive'].sum()],
                    ['  公 司 数 字 背 景 占 比 ',
                     f'{df_company["DigEdu_Executive"].mean()*100:.2f}%'],
                    ['平均每公司高管数', f'{df_company["高管总数"].mean():.1f} 人'],
                    ['总计分析高管数', df_company["高管总数"].sum()]]

    print(f"\n  统计摘要:")

    for item, value in summary_data:
        print(f"    • {item}: {value}")

```

```

# 显示结果示例

print(f"\n  结果示例 (前 5 家公司) :")

print(df_company.head().to_string(index=False))

except Exception as e:

    print(f"\n  处理错误: {str(e)}")

import traceback
traceback.print_exc()

print("\n 按回车键退出...")
input()

```

板块二修正指令:

我现在需要你帮我设计一段 python 代码:

一、现有数据:

第一列是股票代码、数字背景高管数、高管总数、DigEdu_Executive (这是一个虚拟变量, 为 1 时说明这家公司至少一位高管具有数字背景, 为 0 时说明这家公司所有高管均不具备数字背景)

二、我的需求

1、任务目标:

(1) 现在第一列的证券代码中, 存在部分代码前面缺 0 的情况 (正常情况下 A 股证券代码应该是 6 位, 如这张列表中证券代码列为 10 的公司, 正确的证券代码应该是在 10 的左边加 4 个 0 补齐 6 位), 我需要你检测出所有长度没有 6 的证券代码, 并在最左边补齐足量的 0, 使得证券代码的长度为 6 位;

(2) 将所有数据按照证券代码从 000001-9XXXXX 的顺序重新排列

2、代码步骤:

(1) 用 def 操作建立一个函数, 读取输入的 excel 表格, 注意第一列证券代码要用字符串格式, 数字背景高管数、高管总数和 DigEdu_Executive 保留为数字格式;

(2) 对数据一行一行地遍历, 判断该行数据中索引为 0 的字符串 (即证券代码字符串的长度 x 是否为 6, 如果 x=6, 进入下一行, 如果 x<6, 则在这个字符串的前面添(x-6)个 0 (注意保留字符串格式) 补齐 6 位并保存。

(3) 把所有行的遍历结果, 无论是保持原样的还是将证券代码补齐 6 位的, 另存到 mac 桌面上, 命名为: 板块二_公司数字背景格式修正_‘对应的时间戳’。

(4) 调用函数, 读取位于 mac 桌面上的 excel“板块二_公司数字背景_20260104_220914.xlsx”, 对该表中的数据实行 def 函数中的操作, 并将结果储存在桌面上。

第二板块代码格式调整终版

```
import pandas as pd
```

```

import os
from datetime import datetime

def process_stock_code_format(file_path):
    """
    处理 Excel 文件中的证券代码格式:
    1. 将不足 6 位的证券代码前面补 0
    2. 按证券代码排序
    3. 保存到 Mac 桌面并添加时间戳

    参数:
        file_path: str, 输入 Excel 文件的完整路径

    返回:
        DataFrame: 处理后的 DataFrame
    """

    print(f"    正在读取文件: {os.path.basename(file_path)}")

    try:
        # 步骤 1: 读取 Excel 文件, 指定列的数据类型
        df = pd.read_excel(
            file_path,
            dtype={
                '证券代码': str,
                '数字背景高管数': int,
                '高管总数': int,
                'DigEdu_Executive': int
            }
        )

        # 验证数据列是否存在
        required_cols = ['证券代码', '数字背景高管数', '高管总数', 'DigEdu_Executive']
        if not all(col in df.columns for col in required_cols):
            raise ValueError(f"Excel 文件必须包含以下列: {required_cols}")

        # 步骤 2: 逐行检查并补齐证券代码 (使用 zfill 方法自动补 0)
        df['证券代码'] = df['证券代码'].fillna("").astype(str)
        df['证券代码'] = df['证券代码'].apply(lambda x: x.zfill(6) if len(x) < 6 else x)

        # 步骤 3: 按证券代码从小到大排序
        df = df.sort_values('证券代码').reset_index(drop=True)

        # 步骤 4: 生成输出文件路径 (Mac 桌面)

```

```

timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')
desktop_path = os.path.join(os.path.expanduser('~'), 'Desktop')
output_filename = f'板块二_公司数字背景格式修正_{timestamp}.xlsx'
output_path = os.path.join(desktop_path, output_filename)

# 保存处理后的数据
df.to_excel(output_path, index=False, engine='openpyxl')

print(f"    处理成功! 新文件已保存至桌面: {output_filename}")

print(f"    共处理 {len(df)} 条记录")

print(f"    文件路径: {output_path}")

return df

except FileNotFoundError:

    print(f"    错误: 在桌面找不到文件 '{os.path.basename(file_path)}'")
    print("请确保文件已保存在桌面上")
    return None
except Exception as e:

    print(f"    处理过程中发生错误: {str(e)}")
    return None

# ===== 主程序: 无需修改任何路径, 直接运行即可 =====
if __name__ == "__main__":
    # 自动获取 Mac 桌面路径
    desktop_path = os.path.join(os.path.expanduser('~'), 'Desktop')

    # 指定要处理的文件名 (只需修改这里即可)
    target_filename = "板块二_公司数字背景_20260104_220914.xlsx"

    # 组合成完整路径
    input_file = os.path.join(desktop_path, target_filename)

    # 调用函数处理文件
    print("="*60)
    print("开始处理证券代码格式...")
    print("="*60)

```

```
result_df = process_stock_code_format(input_file)
```

```
# 如果处理成功，显示预览结果
```

```
if result_df is not None:
```

```
    print("\n" + "="*60)
```

```
    print("前 5 行处理结果预览:")
```

```
    print("="*60)
```

```
    print(result_df.head())
```

```
    print("\n" + "="*60)
```

```
    print("最后 5 行处理结果预览:")
```

```
    print("="*60)
```

```
    print(result_df.tail())
```

- 1、板块二修正后还存在一个问题，就是 000002 少遍历了一行，导致统计的高管总数比正确的统计数少 1（已手动加回），对虚拟变量判断结果无影响
- 2、除此之外，将 2013-2024 年的高管数字背景数据合并时，手动增添了一列会计年度，便于合并为面板数据

十一、涉诉金额、涉诉次数统计 Python 代码

http://localhost:8888/notebooks/Desktop/Python/%E6%AF%95%E4%B8%9A%E8%AE%BA%E6%96%87_%E8%AF%89%E8%AE%BC%E4%BB%B2%E8%A3%81_20260207.ipynb

注：1、所有代码运行前需把 excel 文件放到桌面上；2、代码中的路径只适用于 mac 系统，windows 系统需要对路径代码进行修改；3、需要把桌面上的文件名称修改至与代码中完全相同。

kimi 指令：

我现在需要你帮我设计一段 python 代码：

一、现有数据：

一张 excel，统计了 2013-2024 年 A 股上市公司诉讼仲裁相关数据。第 1 行为中文标题，从第 2 行开始是数据。第 A-K 列分别为：股票代码、证券简称、事件 ID、公告类型、会计年度、公告日期、事件内容、涉案缘由、涉案金额、诉讼仲裁费用、币种。

二、我的需求

1、任务目标：统计 2013-2024 年各年份各公司的涉诉次数和每年度的涉诉总金额

2、具体操作：

(1) 读取位于 mac 桌面命名为 诉讼仲裁_原始文件.xlsx 的文件。注意，由于股票代码存在左侧数据为 0 的情况，所以一定要始终以字符串的格式储存，保证长度一直为 6 位。

(2) 新建字典 1 和字典 2，分别对各公司各年份的涉诉次数和涉诉金额进行统计。对第 (1) 步读取的文件除标题行外的所有行进行遍历。每读取一行，将该行的（股票代码，会计年度）组成元组，作为一个键值对的键分别写入字典 1 和字典 2。与此同时，将 1 作为字典 1 该键值对的值（计数 1，表示该公司在该会计年度涉诉次数为 1 次），当后续遍历中再次出现该（股票代码，会计年度）组合时，令字典 1 中该键值对的值等于原来的值+1，表示该公司该会计年度的涉诉次数加一；将涉案金额作为键值对的值写入字典 2（如果遍历到的行的涉案金额为空值时，认为是数值 0），当后续遍历中再次出现该（股票代码，会计年度）组合时，令字典 2 中该键值对的值等于原来的值加正在遍历的行的涉案金额。

(3) 输出结果，以 excel 的形式保存到 mac 的桌面，命名为：涉诉金额+涉诉金额_对应的时间戳.xlsx。其中，这个 excel 的从左到右每一列的列标题依次为：股票代码、会计年度、涉诉次数、涉诉金额。并且在表格的最上方添加列标题行。

注意：(1) 鉴于循环会导致代码运行效率低下，你可以给出可行的替代方案优化程序；(2) 我希望这个代码可以实时显示运行进度。

请仔细阅读上述文字，并告诉我你是否已经完全理解所有要求。

```
import pandas as pd
import os
from datetime import datetime
```

```

from tqdm import tqdm

def process_litigation_data():
    """
    处理 A 股上市公司诉讼仲裁数据
    统计各公司各年度的涉诉次数和涉诉总金额
    """
    # ===== 路径配置 =====
    desktop_path = "/Users/air/Desktop"
    input_file = os.path.join(desktop_path, "诉讼仲裁_原始文件.xlsx")

    # 检查文件是否存在
    if not os.path.exists(input_file):
        raise FileNotFoundError(f'未找到文件: {input_file}, 请确认文件已放置在 Mac 桌面')

    # ===== 步骤 1/4: 读取数据 =====

    print(" 步骤 1/4: 正在读取 Excel 文件...")

    # 强制股票代码为字符串类型, 避免 Excel 自动转换为数字导致前导 0 丢失
    df = pd.read_excel(
        input_file,
        dtype={'股票代码': str}, # 关键: 确保读取为字符串
        engine='openpyxl'
    )
    print(f" 成功读取 {len(df):,} 条原始记录")

    # ===== 步骤 2/4: 数据预处理 =====

    print(" 步骤 2/4: 正在清洗数据...")

    # 使用 tqdm 包装 apply 以显示进度
    # 确保股票代码为 6 位字符串, 左侧补 0 (处理各种可能的格式)
    tqdm.pandas(desc=" 格式化股票代码")
    def format_stock_code(x):
        if pd.isna(x):
            return '000000'
        try:
            # 先转 str, 去除可能的空格, 再转为数字去小数, 最后补齐 6 位
            return str(int(float(str(x).strip()))).zfill(6)
        except:
            return str(x).strip().zfill(6)

    df['股票代码'] = df['股票代码'].progress_apply(format_stock_code)

```

```

# 处理涉案金额：强制转换为数值，无法转换的（空值/文本）设为 0
tqdm.pandas(desc=" 处理涉案金额")
def safe_amount(x):
    if pd.isna(x) or str(x).strip() == "":
        return 0.0
    try:
        return float(x)
    except:
        return 0.0

df['涉案金额'] = df['涉案金额'].progress_apply(safe_amount)

# 确保会计年度为整数（去除可能的小数点，如 2023.0 → 2023）
tqdm.pandas(desc=" 处理会计年度")
def safe_year(x):
    if pd.isna(x):
        return 0
    try:
        # 去除可能的"年"字，转为 float 去掉小数位，再转 int
        cleaned = str(x).replace('年', '').strip()
        return int(float(cleaned))
    except:
        return 0

df['会计年度'] = df['会计年度'].progress_apply(safe_year)

# 过滤掉无效年份数据
df = df[df['会计年度'] >= 2013]

# ===== 步骤 3/4: 向量化统计（替代字典循环）
=====

print(" 步骤 3/4: 正在统计涉诉次数和金额（使用向量化 GroupBy 优化）...")

# 使用 GroupBy 进行向量化聚合（避免 Python 层面的逐行循环）
result = df.groupby(['股票代码', '会计年度'], observed=True).agg(
    涉诉次数=('股票代码', 'count'),      # 统计该组记录数
    涉诉金额=('涉案金额', 'sum')        # 涉案金额求和
).reset_index()

print(f" 统计完成: 共 {len(result):,} 个公司-年度组合")

# 按股票代码和年度排序，便于查看

```

```

result = result.sort_values(by=['股票代码', '会计年度']).reset_index(drop=True)

# ===== 步骤 4/4: 输出结果 =====

print("    步骤 4/4: 正在保存结果到 Excel...")

# 生成时间戳文件名
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
output_filename = f'涉诉次数+涉诉金额_{timestamp}.xlsx'
output_path = os.path.join(desktop_path, output_filename)

# 确保列顺序符合要求: 股票代码、会计年度、涉诉次数、涉诉金额
result = result[['股票代码', '会计年度', '涉诉次数', '涉诉金额']]

# 保存到 Excel (不包含行索引)
result.to_excel(output_path, index=False, engine='openpyxl')

# ===== 完成报告 =====
print("\n" + "="*50)

print("    处理成功完成! ")

print(f"    输出文件: {output_path}")

print(f"    统计维度: {result['股票代码'].nunique()} 家公司 × {result['会计年度'].nunique()} 个年度")

print(f"    总记录数: {len(result):,} 条")

# 修正点 1: 添加缺失的括号 ()

print(f"    涉诉金额总计: {result['涉诉金额'].sum():.2f} 元")

print("="*50)

# 显示前 5 行预览
print("\n数据预览 (前 5 行) : ")
print(result.head().to_string(index=False))

return result

if __name__ == "__main__":
    try:
        # 执行处理
        final_data = process_litigation_data()
    except Exception as e:

```

```
print(f"\n  处理过程中出现错误: {str(e)}")  
raise
```

十二、违规处理统计 Python 代码

http://localhost:8889/notebooks/Desktop/Python/%E6%AF%95%E4%B8%9A%E8%AE%BA%E6%96%87_%E8%BF%9D%E8%A7%84%E5%A4%84%E7%90%86_20260207.ipynb

注：1、所有代码运行前需把 excel 文件放到桌面上；2、代码中的路径只适用于 mac 系统，windows 系统需要对路径代码进行修改；3、需要把桌面上的文件名称修改至与代码中完全相同。

kimi 指令：

我现在需要你帮我设计一段 python 代码：

一、现有数据：

一张 excel，统计了 2011-2025 年 A 股上市公司违规处理相关数据。第 1 行为中文标题，从第 2 行开始是数据。第一行中文标题从左到右分别为：违规事件、股票代码、会计年度、公告日期、证券简称、违规类型、违规类型编码、违规年度、处罚总金额（元）、上市公司是否违规、处罚方式-上市公司、处罚方式编码-上市公司、处罚金额-上市公司（元）。

二、我的需求

1、任务目标：分别统计 2011-2025 年各年份各公司的是否受到违规处理和违规处理次数的数据。

2、具体操作：

(1) 读取位于 mac 桌面命名为 违规处理_原始数据.xlsx 的文件。注意，由于股票代码存在左侧数据为 0 的情况，所以一定要始终以字符串的格式储存，保证长度一直为 6 位。

(2) 新建字典 1 和字典 2，分别对各公司各年份是否受到违规处理虚拟变量和违规处理次数的数据进行统计。对第 (1) 步读取的文件除标题行外的所有行进行遍历。每读取一行，将该行的（股票代码，会计年度）组成元组，作为一个键值对的键分别写入字典 1 和字典 2。与此同时，将 1 作为字典 1 该键值对的值（计数 1，表示该公司在该会计年度受到过违规处理），当后续遍历中再次出现该（股票代码，会计年度）组合时，字典 1 中对应键值对的值保持不变；将 1 作为键值对的值写入字典 2（计数 1，表示遍历到该行时，该公司该会计年度已经受到过一次违规处理），当后续遍历中再次出现该（股票代码，会计年度）组合时，令字典 2 中该键值对的值等于原来的值加 1，作为该公司该会计年度受到违规处理次数的最新统计。

(3) 输出结果，将字典 1 和字典 2 以两张 excel 的形式保存到 mac 的桌面，分别命名为：是否受到违规处理_对应的时间戳.xlsx 和违规处理次数_对应的时间戳.xlsx。其中：是否受到违规处理_对应的时间戳.xlsx 的从左到右每一列的列标题依次为：股票代码、会计年度、是否受到违规处理。并且在表格的最上方添加列标题行；违规处理次数_对应的时间戳.xlsx 的从左到右每一列的列标题依次为：股票代码、会计年度、违规处理次数。并且在表格的最上方添加列标题行。

注意：(1) 鉴于循环会导致代码运行效率低下，你可以给出可行的替代方案优化程序；(2) 我希望这个代码可以实时显示运行进度。

请仔细阅读上述文字，并告诉我你是否已经完全理解所有要求。

```

import pandas as pd
import numpy as np
from datetime import datetime
from tqdm import tqdm
import os
import sys

def process_violation_data():
    """
    处理 A 股上市公司违规处理数据
    统计各年份各公司是否受到违规处理和违规处理次数
    """

    # 设置文件路径 (Mac 桌面)
    desktop_path = "/Users/air/Desktop"
    input_file = os.path.join(desktop_path, "违规处理_原始文件.xlsx")

    # 生成时间戳
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    output_file1 = os.path.join(desktop_path, f"是否受到违规处理_{timestamp}.xlsx")
    output_file2 = os.path.join(desktop_path, f"违规处理次数_{timestamp}.xlsx")

    print("=" * 60)
    print("开始处理违规处理数据...")
    print(f"当前时间: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")
    print("=" * 60)

    try:
        # Step 1: 读取 Excel 文件 (带进度条)
        print("\n[1/4] 正在读取 Excel 文件...")
        with tqdm(total=1, desc="读取进度") as pbar:
            # 读取数据, 将所有列作为字符串读取以确保股票代码格式正确
            df = pd.read_excel(
                input_file,
                dtype={'股票代码': str}, # 确保股票代码作为字符串读取
                engine='openpyxl'
            )
            pbar.update(1)

        print(f"✓ 成功读取数据, 共 {len(df)} 行记录")

        # Step 2: 数据预处理 (带进度条)

```

```

print("\n[2/4] 正在预处理数据...")
with tqdm(total=3, desc="预处理进度") as pbar:
    # 确保股票代码为 6 位字符串格式 (左侧补零)
    df['股票代码'] = df['股票代码'].astype(str).str.strip().str.zfill(6)
    pbar.update(1)

    # 确保会计年度为整数
    df['会计年度'] = pd.to_numeric(df['会计年度'], errors='coerce').astype('Int64')
    pbar.update(1)

    # 移除会计年度为空的记录
    initial_rows = len(df)
    df = df.dropna(subset=['会计年度'])
    final_rows = len(df)
    pbar.update(1)

print(f"✓ 数据预处理完成, 有效记录: {final_rows} 行 (删除空值: {initial_rows - final_rows} 行) ")

# Step 3: 统计计算 (使用 groupby 向量化操作, 避免循环)
print("\n[3/4] 正在进行分组统计...")

# 创建进度条包装器用于 groupby 操作
tqdm.pandas(desc="统计进度")

# 按 (股票代码, 会计年度) 分组统计
grouped = df.groupby(['股票代码', '会计年度'])

# 字典 1: 是否受到违规处理 (只要有记录就是 1)
violation_dummy = grouped.size().reset_index(name='计数')
violation_dummy['是否受到违规处理'] = 1 # 只要有记录就是 1

# 字典 2: 违规处理次数 (统计该组合出现的次数)
violation_count = grouped.size().reset_index(name='违规处理次数')

print(f"✓ 统计完成, 共发现 {len(violation_dummy)} 个 (公司-年度) 组合")

# Step 4: 生成完整面板数据 (包含未违规的公司年度组合为 0)
print("\n[4/4] 正在生成完整面板数据...")

with tqdm(total=2, desc="面板生成进度") as pbar:
    # 获取所有唯一的股票代码和年度范围

```

```

all_stocks = df['股票代码'].unique()
all_years = range(2011, 2026) # 2011-2025 年

# 创建完整的 (股票代码, 会计年度) 组合
full_index = pd.MultiIndex.from_product(
    [all_stocks, all_years],
    names=['股票代码', '会计年度']
)
pbar.update(1)

# 将统计结果与完整面板合并, 缺失值填充为 0
result_dummy = violation_dummy.set_index(['股票代码', '会计年度'])['是否受到
违规处理']

result_dummy = result_dummy.reindex(full_index, fill_value=0).reset_index()

result_count = violation_count.set_index(['股票代码', '会计年度'])['违规处理次数
']

result_count = result_count.reindex(full_index, fill_value=0).reset_index()
pbar.update(1)

print(f"✓ 面板数据生成完成, 共 {len(result_dummy)} 条记录")

# Step 5: 保存结果到 Excel
print("\n 正在保存结果文件...")

with pd.ExcelWriter(output_file1, engine='openpyxl') as writer:
    result_dummy.to_excel(writer, index=False, sheet_name='是否违规')

print(f"✓ 已保存: {output_file1}")

with pd.ExcelWriter(output_file2, engine='openpyxl') as writer:
    result_count.to_excel(writer, index=False, sheet_name='违规次数')

print(f"✓ 已保存: {output_file2}")

# 输出统计摘要
print("\n" + "=" * 60)
print("处理完成! 统计摘要: ")
print(f"• 涉及公司数量: {len(all_stocks)} 家")
print(f"• 涉及年度范围: 2011-2025 年")
print(f"• 总记录数: {len(result_dummy)} 条")
print(f"• 有违规记录的公司-年度组合: {(result_dummy['是否受到违规处理'] >
0).sum()} 个")

```

```

print(f'• 总违规次数: {result_count['违规处理次数'].sum()} 次')
print("=" * 60)

return True

except FileNotFoundError:

    print(f'\n✗ 错误: 找不到文件 {input_file}')

    print("请确保文件已放置在 Mac 桌面, 且文件名为: 违规处理_原始文件.xlsx")
    return False

except Exception as e:

    print(f'\n✗ 处理过程中出现错误: {str(e)}')

    import traceback
    traceback.print_exc()
    return False

if __name__ == "__main__":
    # 检查是否安装了必要的库
    try:
        import pandas as pd
        from tqdm import tqdm
    except ImportError:
        print("正在安装必要的依赖库...")
        import subprocess
        subprocess.check_call([sys.executable, "-m", "pip", "install", "pandas", "openpyxl",
                                "tqdm"])
        print("依赖库安装完成, 请重新运行程序")
        sys.exit(0)

    # 运行主程序
    success = process_violation_data()
    if success:
        print("\n 程序执行成功! ")
    else:
        print("\n 程序执行失败, 请检查错误信息。")

```

【只保留披露失实类别】

kimi 指令:

我现在需要你帮我设计一段 python 代码:

一、现有数据:

一张 excel, 统计了 2011-2025 年 A 股上市公司违规处理相关数据。第 1 行为中文标题, 从第 2 行开始是数据。第一行中文标题从左到右分别为: 违规事件、股票代码、会计年度、公

告日期、证券简称、违规类型、违规类型编码、违规年度、处罚总金额（元）、上市公司是否违规、处罚方式-上市公司、处罚方式编码-上市公司、处罚金额-上市公司（元）。

二、我的需求

1、任务目标：只保留“违规类型编码”列包含：P2501, P2502, P2503, P2504, P2505, P2506 中至少一个的所有数据行

2、具体操作：

(1) 读取位于 mac 桌面命名为 违规处理_原始数据.xlsx 的文件。注意，由于股票代码存在左侧数据为 0 的情况，所以一定要始终以字符串的格式储存，保证长度一直为 6 位。

(2) 新建列表，作为保留数据行的白名单：[P2501, P2502, P2503, P2504, P2505, P2506]

(3) 对第 (1) 步读取的表格除标题行之外的所有数据行进行遍历，识别每一行“违规类型编码”列，判断该列是否包含第 (2) 步建立的列表的任意一个违规类型编码，如有，则在最终输出的结果中保留该行，如不包含列表中的任意一种违规类型，则在最终输出的结果中剔除该行。

(4) 输出结果，以 excel 的形式保存到 mac 的桌面，命名为：违规处理_只保留披露失实类别_对应的时间戳.xlsx，其中从左到右每一列的列标题依次为：违规事件、股票代码、会计年度、公告日期、证券简称、违规类型、违规类型编码、违规年度、处罚总金额（元）、上市公司是否违规、处罚方式-上市公司、处罚方式编码-上市公司、处罚金额-上市公司（元）。并且在表格的最上方添加列标题行。

注意：(1) 鉴于循环会导致代码运行效率低下，你可以给出可行的替代方案优化程序；(2) 我希望这个代码可以实时显示运行进度。

请仔细阅读上述文字，并告诉我你是否已经完全理解所有要求。

```
import pandas as pd
import numpy as np
from datetime import datetime
from tqdm import tqdm
import os
import re

def filter_violation_data():
    """
    筛选披露失实类违规数据 (P2501-P2506)
    保留所有原始列，仅做行筛选
    """

    # 设置路径 (Mac 桌面)
    desktop_path = "/Users/air/Desktop"
    input_file = os.path.join(desktop_path, "违规处理_原始数据.xlsx")

    # 生成时间戳
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
```

```

output_file = os.path.join(desktop_path, f'违规处理_只保留披露失实类别_{timestamp}.xlsx')

# 白名单: 披露失实类违规编码
whitelist = ['P2501', 'P2502', 'P2503', 'P2504', 'P2505', 'P2506']

print("=" * 70)
print("开始处理违规数据筛选...")
print(f'目标编码: {','.join(whitelist)}')
print(f'当前时间: {datetime.now().strftime("%Y-%m-%d %H:%M:%S')}")
print("=" * 70)

try:
    # Step 1: 读取数据 (带进度条)
    print("\n[1/4] 正在读取 Excel 文件...")
    with tqdm(total=1, desc="文件读取") as pbar:
        # 指定股票代码为字符串类型, 防止前导零丢失
        df = pd.read_excel(
            input_file,
            dtype={'股票代码': str},
            engine='openpyxl'
        )
        pbar.update(1)

    total_rows = len(df)
    original_columns = df.columns.tolist()

    print(f'✓ 成功读取数据, 原始记录: {total_rows:,} 行, {len(original_columns)} 列')

    print(f' 原始列名: {','.join(original_columns)}')

    # Step 2: 数据预处理 (带进度显示)
    print("\n[2/4] 正在清洗数据格式...")

    # 确保股票代码为 6 位字符串 (左侧补零)
    df['股票代码'] = df['股票代码'].astype(str).str.strip().str.zfill(6)

    # 确保违规类型编码为字符串类型 (去除前后空格)
    df['违规类型编码'] = df['违规类型编码'].astype(str).str.strip()

    print("✓ 数据格式清洗完成")

    print(f' - 涉及公司数: {df['股票代码'].nunique()} 家')
    print(f' - 涉及年度范围: {df['会计年度'].min():.0f} - {df['会计年度'].max():.0f}')

```

```

# Step 3: 向量化筛选 (核心优化: 避免 Python 循环)
print("\n[3/4] 正在筛选披露失实类违规 (向量化处理) ...")

with tqdm(total=1, desc="违规筛选") as pbar:
    # 构建正则表达式模式, 使用单词边界确保精准匹配
    # 例如: 匹配 P2501 但不匹配 P25011 或 AP2501
    pattern = r'\b(' + '|'.join(whitelist) + r')\b'

    # 向量化匹配 (比逐行循环快 10-50 倍)
    # na=False 确保空值不报错, 视为不匹配
    mask = df['违规类型编码'].str.contains(pattern, regex=True, na=False)
    filtered_df = df[mask].copy()
    pbar.update(1)

filtered_rows = len(filtered_df)

print(f"✓ 筛选完成, 保留记录: {filtered_rows:,} 行 (剔除 {total_rows - filtered_rows:,} 行) ")

if filtered_rows == 0:
    print("\n 警告: 未匹配到任何目标违规类型")
    print(f" 请检查'违规类型编码'列是否包含以下编码: {whitelist}")
    print(f" 该列样本值: {df['违规类型编码'].head(10).tolist()}")
    return False

# 显示筛选后的统计信息
print(f" - 保留的公司数: {filtered_df['股票代码'].nunique()} 家")
print(f" - 保留的年度范围: {filtered_df['会计年度'].min():.0f}-{filtered_df['会计年度'].max():.0f}")

# Step 4: 保存结果 (保留原始列顺序和标题)
print("\n[4/4] 正在保存结果文件...")

with tqdm(total=1, desc="文件保存") as pbar:
    # 确保股票代码格式正确 (再次确认)
    filtered_df['股票代码'] = filtered_df['股票代码'].str.zfill(6)

    # 按原始列顺序保存, 保留所有列
    # index=False 不保存行索引, header=True 保留列标题 (第一行)
    filtered_df.to_excel(
        output_file,
        index=False,

```

```

        header=True,
        engine='openpyxl'
    )
    pbar.update(1)

# 输出最终统计摘要
print("\n" + "=" * 70)
print("处理完成! 统计摘要: ")
print(f"• 输入文件: 违规处理_原始数据.xlsx")
print(f"• 筛选编码: {' '.join(whitelist)} (披露失实类) ")
print(f"• 输入记录: {total_rows:,} 行")
print(f"• 输出记录: {filtered_rows:,} 行 (保留率: {filtered_rows/total_rows*100:.2f}%) ")
print(f"• 输出列数: {len(original_columns)} 列 (保持原始顺序) ")
print(f"• 输出文件: {os.path.basename(output_file)}")
print(f"• 保存路径: {desktop_path}")
print("=" * 70)

return True

except FileNotFoundError:

    print(f"\n✘ 错误: 找不到文件 {input_file}")

    print("请确保文件已放置在 Mac 桌面, 且文件名为: 违规处理_原始数据.xlsx")
    return False
except Exception as e:

    print(f"\n✘ 处理过程中出现错误: {str(e)}")

    import traceback
    traceback.print_exc()
    return False

if __name__ == "__main__":
    # 检查依赖库
    try:
        import pandas as pd
        from tqdm import tqdm
    except ImportError:
        print("正在安装必要的依赖库...")
        import subprocess
        import sys
        subprocess.check_call([sys.executable, "-m", "pip", "install", "pandas", "openpyxl",
                                "tqdm"])
    print("依赖库安装完成, 请重新运行程序")

```

```
        exit(0)

# 运行主程序
success = filter_violation_data()
if success:

    print("\n✓ 程序执行成功! 文件已保存至 Mac 桌面。")
else:

    print("\n✗ 程序执行失败, 请检查错误信息。")
```

十三、数字技术相关专利数统计 Python 代码

注：1、所有代码运行前需把 excel 文件放到桌面上；2、代码中的路径只适用于 mac 系统，windows 系统需要对路径代码进行修改；3、需要把桌面上的文件名称修改至与代码中完全相同。

【板块一】

kimi 指令：

我现在需要你帮我设计一段 python 代码：

一、现有数据：

一张 excel，没有标题行，每一行每一个非空的单元格中都是一个字符串，表示一个专利编码。每一行非空的单元格数量不同，但是非空的单元格都集中于表格的左边。

二、我的需求

1、任务目标：将所有单元格代表的专利编码先使用集合去重，再写入到一个 word 文档中。

2、具体操作：

(1) 读取位于 mac 桌面命名为 《关键数字技术分类体系（2023）》国家知识产权局.xlsx 的文件，新建列表 1，对每一行每一个非空的单元格进行遍历，将非空单元格中的专利编码以字符串的格式储存在列表 1 中。

(2) 使用集合对第 (1) 步得到的列表 1 进行去重，得到集合 1。

(3) 输出结果文件，将第 (2) 步得到的集合 1 写入一个 word 文档，命名为 关键数字技术分类体系（2023）国家知识产权局_清洗.doc。

注意：(1) 鉴于循环会导致代码运行效率低下，你可以给出可行的替代方案优化程序；(2) 我希望这个代码可以实时显示运行进度。

请仔细阅读上述文字，并告诉我你是否已经完全理解所有要求。

```
import pandas as pd
import os
from tqdm import tqdm

# ===== 配置路径 =====
input_path = "/Users/air/Desktop/《关键数字技术分类体系（2023）》国家知识产权局.xlsx"
output_path = "/Users/air/Desktop/关键数字技术分类体系（2023）国家知识产权局_清洗.txt"

# 检查文件
if not os.path.exists(input_path):
    raise FileNotFoundError(f"文件不存在: {input_path}")

# ===== 读取与去重 =====
```

```

print("正在读取 Excel 文件...")
df = pd.read_excel(input_path, header=None)

print("正在提取专利编码...")
flat_values = df.to_numpy().ravel()
s = pd.Series(flat_values)
mask = s.notna() & (s.astype(str).str.strip() != "")
valid_codes = s[mask].astype(str).tolist()

unique_codes = sorted(set(valid_codes))

print(f"✓ 原始数量: {len(valid_codes)}, 去重后: {len(unique_codes)}")

# ===== 写入文本文件（带进度条） =====
print("正在生成文本文件...")
with open(output_path, 'w', encoding='utf-8') as f:
    f.write("关键数字技术专利分类体系清洗结果\n")
    f.write(f"原始编码总数: {len(valid_codes)}, 去重后数量: {len(unique_codes)}\n\n")

    # 逐行写入并显示进度
    for code in tqdm(unique_codes, desc="写入进度", unit="条"):
        f.write(code + '\n')

print(f"\n 完成! 文件已保存至: {output_path}")

print("提示: 可直接用 Microsoft Word 打开此 txt 文件, 另存为 .doc 或 .docx 格式")

```

【板块二】

kimi 指令:

我现在需要你帮我设计一段 python 代码:

一、现有数据:

- 1、一张 excel，记录了历年上市公司发明专利细分数量。表格第一行为英文标题，第二行为中文标题。前 4 列的英文标题为: Scode、Year、Ftyp、Aplctm。从第 5 列开始，标题呈现出一定的规律，每两个为一组，其中，每组的第一列的中英文标题分别为 “Inva_xxx” 和 “发明申请 xxx”，第二列的中英文标题分别为 “Inva_xxxnum” 和 “发明申请 xxx 数量”，其中，xxx 为专利细分分类号。
- 2、一个 txt 文件，记录了和数字技术相关的所有专利细分类别的分类号，文件的每一行为一个专利分类号，每一个专利分类号的前三个字母（或数字）即为 excel 标题中的 xxx。

二、我的需求

- 1、任务目标: 将 excel 英文标题与 txt 文件对照，删除 excel 中所有 txt 文件中不存在分类号所在列。
- 2、具体操作:

(1) 读取位于 mac 桌面命名为 上市公司发明专利申请专利细分分类号.xlsx 的文件，按列读取，新建字典 1，将每一列的英文标题（即第一行）作为一个键值对的键，将该列的剩余所有单元格（注意，所有单元格中的内容都按照字符串的格式储存，不要存储为数值型）组合为一个元组（注意，值为空的单元格也需要保留，确保最后得到的数据是只删除了部分列，其他列保持原样），储存为该键值对的值。

(2) 读取位于 mac 桌面命名为 关键数字技术分类体系（2023）国家知识产权局_清洗.txt 的文件。新建列表 1，对 txt 文件中的每一行进行遍历，将每一行的字符串作为列表中的一个元素，储存在列表中。

(3) 使用 lambda 函数，只保留第（2）步得到的列表 1 中的每一个元素（一个字符串）从左到右前 3 位，用于第（4）步与字典 1 对照删除不需要的列。

(4) 对字典 1 除了键为 Scode、Year、Ftyp、Aplctm 这 4 个字符串的其他所有键值对的键进行遍历，若某键值对的键（字符串）中包含列表 1 中的任意一个元素，则保留，反之，删除该键值对。例如，excel 中某列的英文标题为 Inva_A01，即键值对的键为字符串“Inva_A01”，第（3）步处理之后的列表 1 包含元素“A01”，即字符串“Inva_A01”包含字符串“A01”，在最终结果中爆裂该列。

(5) 输出结果文件，将第（4）步得到的删除了部分键值对的字典 1 按列（一个键值对为一列）写入一个 excel 文档，命名为 上市公司发明专利申请专利细分分类号_筛选后.xlsx。其中，excel 从左到右第 1-4 列分别为键为 Scode、Year、Ftyp、Aplctm 的键值对，其余列按照字母和数字的顺序从小到大排列。

注意：（1）鉴于循环会导致代码运行效率低下，你可以给出可行的替代方案优化程序；（2）我希望这个代码可以实时显示运行进度。

请仔细阅读上述文字，并告诉我你是否已经完全理解所有要求。

```
import pandas as pd
import os
from tqdm import tqdm

def process_patent_data():
    """
    处理上市公司发明专利申请专利细分分类号数据
    筛选保留与数字技术相关的专利分类列
    """

    # ===== 路径配置 =====
    desktop_path = "/Users/air/Desktop/"
    excel_path = os.path.join(desktop_path, "上市公司发明专利申请专利细分分类号.xlsx")
    txt_path = os.path.join(desktop_path, "关键数字技术分类体系（2023）国家知识产权局_清洗.txt")
    output_path = os.path.join(desktop_path, "上市公司发明专利申请专利细分分类号_筛选后.xlsx")

    print(">>> 开始执行专利数据筛选任务...")
```

```

# ===== (1) 读取 Excel 并按列构建字典 =====
print("\n[步骤 1/5] 读取 Excel 文件并构建数据字典...")

# 使用 dtype=str 确保所有内容按字符串格式读取
df = pd.read_excel(excel_path, header=None, dtype=str)
print(f"    原始数据维度: {df.shape[0]} 行 × {df.shape[1]} 列")

# 提取英文标题 (第 1 行, 索引 0) 作为键
headers = df.iloc[0].tolist()

# 使用字典推导式构建字典 (向量化操作, 避免逐行循环)
# 值转换为元组, 保留空值 (pandas 的 NaN 会被读作字符串'nan', 符合要求)
data_dict = {}
print("    正在按列构建字典 (保留字符串格式) ...")
for col_idx in tqdm(range(len(headers)), desc="    处理进度", unit="列"):
    col_name = headers[col_idx]
    # 从第 2 行开始取数据 (索引 1 到末尾), 转换为元组
    col_values = tuple(df.iloc[1:, col_idx].astype(str))
    data_dict[col_name] = col_values

print(f"    字典构建完成, 共 {len(data_dict)} 个键值对")

# ===== (2) 读取 TXT 文件 =====
print("\n[步骤 2/5] 读取数字技术分类体系文件...")

with open(txt_path, 'r', encoding='utf-8') as f:
    lines = [line.strip() for line in f if line.strip()]

# ===== (3) 使用 Lambda 处理前 3 位 =====
print("    使用 Lambda 函数提取分类号前 3 位...")

# 严格按照要求: 使用 lambda 函数处理列表, 取前 3 位
list_1 = list(map(lambda x: x[:3], lines))

# 转换为集合 (Set) 以实现 O(1)查找效率, 避免后续循环比对
set_1 = set(list_1)
print(f"    已提取 {len(set_1)} 个唯一分类前缀 (如: {list(set_1)[:3]}...) ")

# ===== (4) 筛选键值对 (核心优化逻辑) =====
print("\n[步骤 3/5] 筛选保留数字技术相关列...")

# 固定保留的 4 列

```

```

fixed_cols = ['Scode', 'Year', 'Ftyp', 'Aplctm']

# 获取除固定列外的所有其他列
other_keys = [k for k in data_dict.keys() if k not in fixed_cols]

# 优化方案：使用集合交集逻辑，避免嵌套循环
# 提取所有可能的 xxx 基础标识（如 A01）
bases_to_keep = set()

for key in tqdm(other_keys, desc="    筛选进度", unit="列"):
    if key.startswith('Inva_'):
        # 判断是 'Inva_xxx' 还是 'Inva_xxxnum' 格式，提取 xxx 部分
        if key.endswith('num'):
            base_code = key[5:-3] # 去掉 'Inva_' 和 'num'
        else:
            base_code = key[5:]   # 只去掉 'Inva_'

        # 集合查找 O(1) 复杂度，无需遍历列表
        if base_code in set_1:
            bases_to_keep.add(base_code)

print(f"    匹配到 {len(bases_to_keep)} 个有效分类号（如：
{sorted(list(bases_to_keep))[:3]}...）")

# 构建最终保留的完整列名列表（包含成对的 Inva_xxx 和 Inva_xxxnum）
final_keys = fixed_cols.copy()

# 为每个匹配的分类号基础标识，添加对应的两个列名
for base in sorted(list(bases_to_keep)): # 先排序确保顺序
    inva_key = f'Inva_{base}'
    invanum_key = f'Inva_{base}num'

    # 检查原始字典中是否存在这两列（防御性编程）
    if inva_key in data_dict and invanum_key in data_dict:
        final_keys.extend([inva_key, invanum_key])

print(f"    最终保留列数: {len(final_keys)} （固定列 4 + 筛选列 {len(final_keys)-4}）")

# ===== (5) 按序重组并输出 Excel =====
print("\n[步骤 4/5] 按字母数字顺序重组数据结构...")

# 从字典中提取数据，按 final_keys 顺序重组为 DataFrame
# 注意：其余列已按 base 排序，因此 Inva_xxx 和 Inva_xxxnum 自然成组并按字母顺序排列

```

```

result_data = {}
for key in tqdm(final_keys, desc="    重组进度", unit="列"):
    # 将元组转回列表, 以便 pandas 处理
    result_data[key] = list(data_dict[key])

# 创建新的 DataFrame
result_df = pd.DataFrame(result_data)

# 确保列顺序严格按照 final_keys (已排序)
result_df = result_df[final_keys]

print(f"    重组后数据维度: {result_df.shape[0]} 行 × {result_df.shape[1]} 列")

print("\n[步骤 5/5] 写入输出文件...")
result_df.to_excel(output_path, index=False)

print(f"\n>>> 任务完成! 输出文件路径: {output_path}")
print(f"    总计处理: {len(data_dict)} 列 → 保留 {len(final_keys)} 列, 删除 {len(data_dict)-len(final_keys)} 列")

if __name__ == "__main__":
    process_patent_data()

```

【板块三】

kimi 指令:

我现在需要你帮忙写一段 python 代码:

一、现有数据:

1、一张 excel, 记录了历年上市公司发明专利细分数量。表格第一行为中文标题, 从左到右分别为: 股票代码、会计年度、公司年度、申请时间、关键数字技术_发明专利申请总数。

二、我的需求

1、任务目标: 将同一股票代码同一会计年度各个公司类型的关键数字技术_发明专利申请总数求和, 在结果文件中只保留求和后的统计数字。

2、具体操作:

(1) 读取位于 mac 桌面上命名为 32-1 关键数字技术_发明专利申请总数.xlsx 的 excel 表格。注意, 由于股票代码存在左侧数据为 0 的情况, 所以一定要始终以字符串的格式储存, 保证长度一直为 6 位。

(2) 使用 pandas 操作, 将 (股票代码, 会计年度) 两列相同的所有行的 “关键数字技术_发明专利申请总数” 列求和, 作为第 (3) 步输出的结果文件中的第 3 列。

(3) 输出结果, 以 excel 的形式保存到 mac 的桌面, 命名为: 32-1 关键数字技术_发明专利申请总数_对应的时间戳.xlsx。其中: 从左到右每一列的列标题依次为: 股票代码、会计年度、关键数字技术_发明专利申请总数。

注意：（1）鉴于循环会导致代码运行效率低下，你可以给出可行的替代方案优化程序；（2）我希望这个代码可以实时显示运行进度。

请仔细阅读上述文字，并告诉我你是否已经完全理解所有要求。

```
pip install pandas openpyxl tqdm
```

```
import pandas as pd
import os
from datetime import datetime
from tqdm import tqdm

# 设置 Mac 桌面路径
desktop_path = os.path.join(os.path.expanduser("~"), "Desktop")
input_file = os.path.join(desktop_path, "32-1 关键数字技术_发明专利申请总数.xlsx")

# 生成带时间戳的输出文件名
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
output_file = os.path.join(desktop_path, f"32-1 关键数字技术_发明专利申请总数_{timestamp}.xlsx")

print("=" * 50)
print("开始执行数据聚合任务")

# 阶段 1: 读取数据（带进度条）
print("\n[1/4] 正在读取 Excel 文件...")
with tqdm(total=1, desc="文件读取", bar_format='{l_bar} {bar}| {n_fmt}/{total_fmt}') as pbar:
    df = pd.read_excel(input_file, dtype=str)
    pbar.update(1)

print(f"✓ 成功读取 {len(df):,} 行原始数据")

# 阶段 2: 数据预处理（带进度条）
print("\n[2/4] 正在处理数据格式...")
with tqdm(total=3, desc="数据清洗", bar_format='{l_bar} {bar}| {n_fmt}/{total_fmt}') as pbar:
    # 确保股票代码为 6 位字符串（左侧补零）
    df['股票代码'] = df['股票代码'].str.zfill(6)
    pbar.update(1)

    # 将会计年度转为字符串（保持格式统一）
    df['会计年度'] = df['会计年度'].astype(str)
    pbar.update(1)
```

```

# 确保专利数量为数值类型
df['关键数字技术_发明专利申请总数'] = pd.to_numeric(df['关键数字技术_发明专利申请
总数'], errors='coerce').fillna(0)
pbar.update(1)

print("✓ 股票代码已标准化为 6 位字符串格式")

# 阶段 3: 分组聚合 (关键修正: 使用 progress_apply 替代 progress_sum)
print("\n[3/4] 正在进行分组聚合 (向量化操作) ...")
print("注意: 此步骤使用高效 groupby 算法, 进度条显示分组处理进度")

# 启用 pandas 进度条
tqdm.pandas(desc="分组求和")

# 修正: 使用 progress_apply 包装 sum 函数, 而非不存在的 progress_sum
result = df.groupby(['股票代码', '会计年度'])['关键数字技术_发明专利申请总数']
.progress_apply(lambda x: x.sum()).reset_index()

# 重命名列 (因为 progress_apply 可能会改变列名结构)
result.columns = ['股票代码', '会计年度', '关键数字技术_发明专利申请总数']

print(f"✓ 聚合完成, 共生成 {len(result):,} 条唯一记录 (股票代码-年度组合)")

# 阶段 4: 保存结果 (带进度条)
print("\n[4/4] 正在保存结果到 Excel...")
with tqdm(total=1, desc="文件写入", bar_format='{l_bar}{bar}| {n_fmt}/{total_fmt}') as pbar:
    result.to_excel(output_file, index=False, engine='openpyxl')
    pbar.update(1)

print("\n" + "=" * 50)

print(f"    处理完成! ")

print(f"    输出文件: {output_file}")

print(f"    数据汇总: ")

print(f"    - 输入记录数: {len(df):,} 行")
print(f"    - 输出记录数: {len(result):,} 行 (已按股票代码-年度去重求和) ")
print(f"    - 压缩比例: {len(result)/len(df)*100:.1f}%")
print("\n" + "=" * 50)

```

十四、数字资产强度计算 Python 代码

http://localhost:8888/notebooks/Desktop/Python/%E6%AF%95%E4%B8%9A%E8%AE%BA%E6%96%87_%E6%95%B0%E5%AD%97%E8%B5%84%E4%BA%A7%E5%BC%BA%E5%BA%A6_20260216.ipynb

注：1、所有代码运行前需把 excel 文件放到桌面上；2、代码中的路径只适用于 mac 系统，windows 系统需要对路径代码进行修改；3、需要把桌面上的文件名称修改至与代码中完全相同。

kimi 指令：

我现在需要你帮我设计一段 python 代码：

一、现有数据：

一张 excel，记录了 2011-2024 年上市公司财务报表附注无形资产账面价值的合计数和软件、数据库、数据资源三类无形资产的账面价值明细。根据现有文献，我们认为将账面价值按照以下公式进行计算可以得到数据资产强度指标，即一家公司数据资产占无形资产的比例，公式如下：（软件+数据库+数据资源）/无形资产合计。

这张表格的第一行为中文标题，从左到右依次为：股票代码、会计年度、报表类型、科目类别、无形资产项目、期初数（元）、本期增加数（元）、本期减少数（元）、其他（元）、期末数（元）、币种、说明。

二、我的需求

1、任务目标：（1）首先对各公司各年份的软件、数据库、数据资源三类无形资产的账面价值求和；（2）其次使用公式计算出表格中存在的每一个（股票代码，会计年度）组合的数据资产强度指标。

2、具体操作：

（1）读取位于 mac 桌面命名为 无形资产账面价值明细（2011-2024）.xlsx 的文件。注意，由于股票代码存在左侧数据为 0 的情况，所以一定要始终以字符串的格式储存，保证长度一直为 6 位。

（2）对无形资产账面价值明细（2011-2024）.xlsx 文件除了标题行的所有行进行遍历。新建字典 1，如果遍历行的无形资产项目列为“合计”，则将该行的（股票代码，会计年度）组成元组，作为键值对的键，将该行的（期初数（元）、本期增加数（元）、本期减少数（元）、其他（元）、期末数（元））组成元组，作为键值对的值。

（3）使用 pandas 分组操作，对同一（股票代码，会计年度）且无形资产项目不是“合计”的数据行的期初数（元）、本期增加数（元）、本期减少数（元）、其他（元）、期末数（元）分别求和。同时新建字典 2，将满足上述条件的所有（股票代码，会计年度）元组作为一个键值对的键，将对应求和得到（期初数（元）、本期增加数（元）、本期减少数（元）、其他（元）、期末数（元））组成元组，作为对应行的键值对的值。

（4）新建字典 3。对字典 2 中的每一个键值对的键进行遍历，找到字典 1 中（股票代码，会计年度与之相同的键值对，用字典 2 该键值对的值中的（期初数（元）、本期增加数（元）、本期减少数（元）、其他（元）、期末数（元））分别除以字典 1 该键值对中的（期初数（元）、本期增加数（元）、本期减少数（元）、其他（元）、期末数（元）），即期初数（元）与期初数（元）相除。注意，请设计报错机制，第一，当字典 1 中没有字典 2 中的键时，跳过，

开始遍历字典 2 中的下一个键；当除法的被除数为 0 时，直接令结果为空值。将两个字典中都存在的（股票代码，会计年度）作为元组存储为字典 3 一个键值对的键；运算结束后，将（期初数、本期增加数、本期减少数、其他、期末数）对应相除得到的 5 个计算结果作为一个元组存储为字典 3 对应的键值对的值。

（5）输出结果文件，将第（4）步的结果写入一个 excel 表格，命名为 数据资产强度.xlsx 。其中：从左到右每一列的列标题依次为：股票代码、会计年度、期初数、本期增加数、本期减少数、其他、期末数。请在最上方添加标题行。

注意：（1）鉴于循环会导致代码运行效率低下，你可以给出可行的替代方案优化程序；（2）我希望这个代码可以实时显示运行进度。

请仔细阅读上述文字，并告诉我你是否已经完全理解所有要求。

```
import pandas as pd
import numpy as np
from tqdm import tqdm
import os

# 初始化 tqdm 的 pandas 支持（用于显示分组进度）
tqdm.pandas(desc="正在分组求和")

# ===== 1. 路径配置 =====
input_path = '/Users/air/Desktop/无形资产账面价值明细（2011-2024）.xlsx'
output_path = '/Users/air/Desktop/数据资产强度.xlsx'

print(f'{'='*50}')
print("开始执行数据资产强度计算程序")
print(f'{'='*50}')

# ===== 2. 数据读取 =====
print("\n[步骤 1/5] 正在读取 Excel 文件...")
print("注意：由于股票代码存在左侧数据为 0 的情况，所以一定要始终以字符串的格式储存，保证长度一直为 6 位。")

try:
    # 读取时即将股票代码强制作为字符串处理，防止前导零丢失
    df = pd.read_excel(input_path, dtype={'股票代码': str})

    # 检查必要列是否存在
    required_cols = ['股票代码', '会计年度', '无形资产项目'] + \
        ['期初数（元）', '本期增加数（元）', '本期减少数（元）', '其他（元）', '期末数（元）']
    missing_cols = [col for col in required_cols if col not in df.columns]
```

```

if missing_cols:

    print(f'✗ 错误: Excel 文件缺少以下列: {missing_cols}')

    exit(1)

# 处理股票代码: 先去除空值, 再确保为 6 位字符串格式
# 原始为 NaN 的保持 NaN, 不为 NaN 的转为 6 位字符串
df['股票代码'] = df['股票代码'].apply(
    lambda x: str(int(float(x))).zfill(6) if pd.notna(x) and x != " else np.nan
)

# 移除股票代码或会计年度为空的行 (无法识别)
original_count = len(df)
df = df.dropna(subset=['股票代码', '会计年度'])
if len(df) < original_count:
    print(f'! 警告: 移除了 {original_count - len(df)} 行 (股票代码或会计年度为空) ")

print(f'✓ 数据读取完成, 原始数据共 {len(df):,} 行, {len(df.columns)} 列")

print(f'✓ 股票代码已格式化为 6 位字符串, 示例: {df['股票代码'].iloc[0]}")

except FileNotFoundError:

    print(f'✗ 错误: 找不到文件 {input_path}')

    exit(1)
except Exception as e:

    print(f'✗ 读取文件时发生错误: {e}')

    exit(1)

# ===== 3. 数据预处理 =====
print("\n[步骤 2/5] 正在清洗数据...")

# 定义关键的五个数值列 (原始名称)
value_cols = ['期初数 (元)', '本期增加数 (元)', '本期减少数 (元)',
              '其他 (元)', '期末数 (元)']

# 转换为数值类型 (无法转换的设为 NaN)
for col in value_cols:
    df[col] = pd.to_numeric(df[col], errors='coerce')

# 确保会计年度为整数类型 (便于后续处理)
df['会计年度'] = pd.to_numeric(df['会计年度'], errors='coerce').astype('Int64')

```

```

# 再次确认无关键空值
df = df.dropna(subset=['股票代码', '会计年度', '无形资产项目'])

# ===== 4. 分离合计与明细 (已修改) =====
print("\n[步骤 3/5] 正在分离合计行与明细行...")

# 字典 1 对应: 合计行 (作为分母)
df_total_raw = df[df['无形资产项目'] == '合计'].copy()

if len(df_total_raw) == 0:
    print("✗ 错误: 未找到任何'合计'行, 请检查'无形资产项目'列内容")
    exit(1)

# 【关键修改】: 对同一公司同一年份的多个合计记录进行求和 (而非取第一个)
print(" 正在对合计行按公司-年份分组求和...")
df_total = df_total_raw.groupby(['股票代码', '会计年度'])[value_cols].sum()

print(f"✓ 合计行 (分母): 原始 {len(df_total_raw):,} 条记录, 去重后 {len(df_total):,} 个组合")

# 字典 2 对应: 非合计行 (软件+数据库+数据资源, 作为分子)
df_details = df[df['无形资产项目'] != '合计'].copy()

if len(df_details) == 0:
    print("✗ 错误: 未找到任何明细行 (非合计) ")
    exit(1)

print(f"✓ 明细行 (分子): {len(df_details):,} 条记录")

group_count = df_details.groupby(['股票代码', '会计年度']).ngroups

print(f"✓ 涉及公司-年份组合: {group_count:,} 个")

# ===== 5. 构建字典并计算 (核心优化部分) =====
print("\n[步骤 4/5] 正在计算数据资产强度指标 (使用向量化运算) ...")

# 【适配修改】: df_total 已通过 groupby 设置多重索引, 无需再 set_index
df_total_indexed = df_total.copy()
# 重命名分母列, 避免冲突
df_total_indexed.columns = [f'{col}_total' for col in value_cols]

```

```

# 使用 progress_apply 显示分组求和进度 (替代低效循环)
# 这是"字典 2"的构建过程: 按 (股票代码, 会计年度) 对明细求和
df_details_grouped = df_details.groupby(['股票代码', '会计年度'])[value_cols].progress_apply(
    lambda x: x.sum(min_count=1) # min_count=1 确保至少有一个非 NaN 才计算
)

print(f'✓ 分组求和完成, 共得到 {len(df_details_grouped)} 个组合')

# 合并字典 1 和字典 2 (inner join 自动实现"当字典 1 中没有该键时跳过"的逻辑)
print(" 正在匹配合计数与明细数...")
merged_data = df_details_grouped.join(df_total_indexed, how='inner')

if len(merged_data) == 0:
    print("✗ 错误: 未能匹配任何有效的公司-年份组合 (请检查合计行与明细行是否有共同
键) ")
    exit(1)

print(f'✓ 成功匹配 {len(merged_data)} 个有效组合 (两边数据均存在) ')

# 计算强度指标 (向量化除法, 自动处理)
result_data = {}
result_data['股票代码'] = merged_data.index.get_level_values('股票代码')
result_data['会计年度'] = merged_data.index.get_level_values('会计年度')

# 定义输出列名 (去掉" (元) ")
output_cols = ['期初数', '本期增加数', '本期减少数', '其他', '期末数']

for orig_col, out_col in zip(value_cols, output_cols):
    numerator = merged_data[orig_col] # 分子: 软件+数据库+数据资源求和
    denominator = merged_data[f'{orig_col}_total'] # 分母: 合计数 (已求和)

    # 处理除零: 分母为 0 或 NaN 时, 结果为 NaN (空值)
    with np.errstate(divide='ignore', invalid='ignore'):
        ratio = np.where(
            (denominator == 0) | (denominator.isna()),
            np.nan,
            numerator / denominator
        )
    result_data[out_col] = ratio

```

```

# 构建最终 DataFrame
result_df = pd.DataFrame(result_data)

# 处理无穷值 (以防万一)
result_df[output_cols] = result_df[output_cols].replace([np.inf, -np.inf], np.nan)

# 确保股票代码仍为 6 位字符串 (防止索引操作改变格式)
result_df['股票代码'] = result_df['股票代码'].astype(str).str.zfill(6)

print(f'✓ 计算完成, 强度指标范围: ")

for col in output_cols:
    valid_data = result_df[col].dropna()
    if len(valid_data) > 0:
        print(f" - {col}: {valid_data.min():.4f} ~ {valid_data.max():.4f}")

# ===== 6. 结果输出 =====
print("\n[步骤 5/5] 正在导出结果...")

# 确保输出顺序符合要求: 股票代码、会计年度、期初数、本期增加数、本期减少数、其他、
# 期末数
final_columns = ['股票代码', '会计年度'] + output_cols
result_df = result_df[final_columns]

try:
    # 保存为 Excel (股票代码作为字符串保存, 防止 Excel 自动去掉前导零)
    with pd.ExcelWriter(output_path, engine='openpyxl') as writer:
        result_df.to_excel(writer, index=False, sheet_name='数据资产强度')

        # 动态获取工作表 (避免 Sheet1 硬编码错误)
        if '数据资产强度' in writer.sheets:
            worksheet = writer.sheets['数据资产强度']
        else:
            worksheet = writer.book.worksheets[0]

        # 设置股票代码列 (第 1 列) 为文本格式, 确保 6 位长度不被 Excel 自动转换
        for row in range(2, worksheet.max_row + 1): # 从第 2 行开始 (跳过标题)
            cell = worksheet.cell(row=row, column=1) # 第 1 列是股票代码
            if cell.value is not None:
                cell.value = str(cell.value).zfill(6)
                cell.number_format = '@' # 设置为文本格式

print(f'✓ 结果已成功保存至: {output_path}")

```

```

print(f'✓ 最终输出共 {len(result_df):,} 行, {len(result_df.columns)} 列")

print(f'✓ 股票代码已强制保存为 6 位文本格式 (示例: 000001) ")

except Exception as e:

    print(f'✗ 保存 Excel 时发生错误: {e}")

    # 尝试以简单方式保存 (不设置格式)
    result_df.to_excel(output_path, index=False)
    print(f"! 已以基础格式保存至: {output_path} (如遇格式问题请手动调整股票代码列)
")

print(f"\n{'='*50}")
print("程序执行完毕")
print(f"\n{'='*50}")

# 显示部分结果预览
print("\n 数据预览 (前 5 行) : ")
print(result_df.head())

```

十六、债务资本成本计算代码

http://localhost:8888/notebooks/Desktop/Python/%E6%AF%95%E4%B8%9A%E8%AE%BA%E6%96%87_%E5%80%BA%E5%8A%A1%E8%B5%84%E6%9C%AC%E6%88%90%E6%9C%AC_20260221.ipynb

注：1、所有代码运行前需把 excel 文件放到桌面上；2、代码中的路径只适用于 mac 系统，windows 系统需要对路径代码进行修改；3、需要把桌面上的文件名称修改至与代码中完全相同。

Kimi 指令：

我现在需要你帮我设计一段 python 代码：

一、现有数据：

我现在有 2 个 excel 文件需要合并。这些文件均位于 mac 桌面上。

第一个 excel 命名为：利息支出.xlsx，记录了历年上市公司的利息支出数据。第一行为中文标题，从左到右分别为：股票代码、证券简称、会计年度、统计截止日期、报表类型、利息支出（元）。

第二个 excel 命名为：平均含息负债.xlsx，记录了历年上市公司的平均含息负债数据。表格的第一行为中文标题，从左到右依次为：股票代码、会计年度、统计截止日期、企业平均含息负债。

二、我的需求

1、任务目标：（1）将利息支出表格中利息支出列的所有空值全部填充为 0；（2）将两张表格按照（股票代码，会计年度）合并；（3）计算各年度各公司的债务资本成本：债务资本成本=利息支出/平均含息负债（注意预防分母为 0 或空的报错）。

2、具体操作：

（1）读取位于 mac 桌面命名为 利息支出.xlsx 和 平均含息负债.xlsx 的文件。注意，由于股票代码存在左侧数据为 0 的情况，所以一定要始终以字符串的格式储存，保证长度一直为 6 位。

（2）对利息支出.xlsx 文件中“利息支出”列为空的所有单元格填充为 0。

（3）将填充 0 后的 利息支出.xlsx 与 平均含息负债.xlsx 按照关键词（股票代码，会计年度）合并。

（4）新建一列“债务资本成本”，利用公式“债务资本成本=利息支出/平均有息负债”，计算每一行对应的（股票代码，会计年度）的债务资本成本。注意，当分母（即平均有息负债）为 0 或者空值时，直接令该行的债务资本成本为空值。

（5）输出结果文件，将第（4）步的结果写入一个 excel 表格，命名为 债务资本成本_对应的时间戳.xlsx。其中：从左到右每一列的列标题依次为：股票代码、会计年度、利息支出、平均含息负债、债务资本成本。请在最上方添加标题行。

注意：（1）鉴于循环会导致代码运行效率低下，你可以给出可行的替代方案优化程序；（2）我希望这个代码可以实时显示运行进度。

请仔细阅读上述文字，并告诉我你是否已经完全理解所有要求。

```

import pandas as pd
import numpy as np
from datetime import datetime
from tqdm import tqdm
import os
import warnings
warnings.filterwarnings('ignore')

# 设置 Mac 桌面路径
desktop_path = "/Users/air/Desktop"

def process_debt_cost():
    """
    处理利息支出与平均含息负债数据，计算债务资本成本
    """
    print("开始处理债务资本成本计算...")

    # 步骤 1: 读取数据 (使用 tqdm 显示进度)
    print("\n[1/5] 正在读取利息支出数据...")
    with tqdm(total=1, desc="读取利息支出") as pbar:
        df_interest = pd.read_excel(
            os.path.join(desktop_path, "利息支出.xlsx"),
            dtype={'股票代码': str} # 先以字符串读取
        )
        # 确保股票代码为 6 位字符串，左侧补 0
        df_interest['股票代码'] = df_interest['股票代码'].astype(str).str.zfill(6)
        pbar.update(1)

    print(f"    利息支出数据读取完成，共 {len(df_interest)} 行")

    print("\n[2/5] 正在读取平均含息负债数据...")
    with tqdm(total=1, desc="读取平均含息负债") as pbar:
        df_debt = pd.read_excel(
            os.path.join(desktop_path, "平均含息负债.xlsx"),
            dtype={'股票代码': str}
        )
        # 确保股票代码为 6 位字符串，左侧补 0
        df_debt['股票代码'] = df_debt['股票代码'].astype(str).str.zfill(6)
        pbar.update(1)

    print(f"    平均含息负债数据读取完成，共 {len(df_debt)} 行")

    # 步骤 2: 处理利息支出空值 (向量化操作，无循环)

```

```

print("\n[3/5] 正在处理利息支出空值填充...")
with tqdm(total=1, desc="空值填充") as pbar:
    # 将利息支出列转换为数值类型（无法转换的变为 NaN）
    df_interest['利息支出（元）'] = pd.to_numeric(df_interest['利息支出（元）'],
errors='coerce')
    # 空值填充为 0
    df_interest['利息支出（元）'] = df_interest['利息支出（元）'].fillna(0)
    pbar.update(1)

print(f"    已填充 {df_interest['利息支出（元）'].eq(0).sum()} 个 0 值（含原空值和原 0
值）")

# 步骤 3：合并数据（向量化 merge 操作）
print("\n[4/5] 正在合并数据...")
with tqdm(total=1, desc="数据合并") as pbar:
    # 使用 inner merge 确保只保留两个表都有的数据
    df_merged = pd.merge(
        df_interest[['股票代码', '会计年度', '利息支出（元）']],
        df_debt[['股票代码', '会计年度', '企业平均含息负债']],
        on=['股票代码', '会计年度'],
        how='inner'
    )
    pbar.update(1)

print(f"    合并后数据共 {len(df_merged)} 行")

# 步骤 4：计算债务资本成本（向量化操作，无循环）
print("\n[5/5] 正在计算债务资本成本...")
with tqdm(total=1, desc="计算成本") as pbar:
    # 将平均含息负债转换为数值类型
    df_merged['企业平均含息负债'] = pd.to_numeric(df_merged['企业平均含息负债'],
errors='coerce')

    # 使用 np.where 进行向量化条件判断：分母为 0 或空值时结果为 NaN，否则计算
    比率
    df_merged['债务资本成本'] = np.where(
        (df_merged['企业平均含息负债'].isna()) | (df_merged['企业平均含息负债'] ==
0),
        np.nan,
        df_merged['利息支出（元）'] / df_merged['企业平均含息负债']
    )
    pbar.update(1)

# 统计计算结果

```

```

valid_count = df_merged['债务资本成本'].notna().sum()
zero_denom_count = (df_merged['企业平均含息负债'] == 0).sum()
na_denom_count = df_merged['企业平均含息负债'].isna().sum()
print(f"    计算完成: {valid_count} 行有效数据, {zero_denom_count} 行分母为 0,
{na_denom_count} 行分母为空")

# 步骤 5: 准备输出数据 (选择指定列并重命名)
print("\n 正在准备输出文件...")
output_df = df_merged[['股票代码', '会计年度', '利息支出 (元)', '企业平均含息负债', '债
务资本成本']].copy()

# 重命名列以符合要求
output_df.columns = ['股票代码', '会计年度', '利息支出', '平均含息负债', '债务资本成本']

# 生成带时间戳的文件名
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
output_filename = f"债务资本成本_{timestamp}.xlsx"
output_path = os.path.join(desktop_path, output_filename)

# 写入 Excel
with pd.ExcelWriter(output_path, engine='openpyxl') as writer:
    output_df.to_excel(writer, index=False, sheet_name='债务资本成本')

print(f"\n    处理完成! ")

print(f"    结果文件已保存至: {output_path}")

print(f"    数据概况: ")
print(f"    - 总记录数: {len(output_df)}")
print(f"    - 有效债务资本成本记录: {output_df['债务资本成本'].notna().sum()}")
print(f"    - 无效记录 (分母为 0 或空): {output_df['债务资本成本'].isna().sum()}")

return output_df

if __name__ == "__main__":
    try:
        result = process_debt_cost()
    except FileNotFoundError as e:
        print(f"\n    错误: 找不到文件, 请确保'利息支出.xlsx'和'平均含息负债.xlsx'已放在
桌面\n{e}")
    except Exception as e:
        print(f"\n    处理过程中出现错误: {e}")

```

raise

十七、现金持有计算代码

http://localhost:8888/notebooks/Desktop/Python/%E6%AF%95%E4%B8%9A%E8%AE%BA%E6%96%87_%E7%8E%B0%E9%87%91%E6%8C%81%E6%9C%89%E8%AE%A1%E7%AE%97%E4%BB%A3%E7%A0%81_20260303.ipynb

注：1、所有代码运行前需把 excel 文件放到桌面上；2、代码中的路径只适用于 mac 系统，windows 系统需要对路径代码进行修改；3、需要把桌面上的文件名称修改至与代码中完全相同。

Kimi 指令：

我现在需要你帮我设计一段 python 代码：

一、现有数据：

我现在有 2 个 excel 文件需要合并。这些文件均位于 mac 桌面上。

第一个 excel 命名为：财务报表明细.xlsx，记录了历年上市公司的货币资金、交易性金融资产、资产总计数据。第一行为中文标题，从左到右分别为：股票代码、证券简称、会计年度、会计年度（滞后前）、统计截止日期、报表类型、货币资金（元）、交易性金融资产（元）、资产总计（元）。

第二个 excel 命名为：现金及现金等价物.xlsx，记录了历年上市公司的现金及现金等价物数据。表格的第一行为中文标题，从左到右依次为：股票代码、证券简称、会计年度、会计年度（滞后前）、统计截止日期、报表类型、期末现金及现金等价物余额（元）。

二、我的需求

1、任务目标：（1）将两张表格按照（股票代码，会计年度）合并；（3）计算各年度各公司现金持有=（货币资金+交易性金融资产）/（总资产-现金及现金等价物）（注意预防分母为 0 或空的报错）。

2、具体操作：

（1）读取位于 mac 桌面命名为 财务报表明细.xlsx 和 现金及现金等价物.xlsx 的文件。注意，由于股票代码存在左侧数据为 0 的情况，所以一定要始终以字符串的格式储存，保证长度一直为 6 位。

（2）将 财务报表明细.xlsx 与 现金及现金等价物.xlsx 按照关键词（股票代码，会计年度）合并。

（3）新建一列“现金持有”，利用公式“现金持有=（货币资金（元）+交易性金融资产（元））/（资产总计（元）-现金及现金等价物（元））”，计算每一行对应的（股票代码，会计年度）的现金持有。注意，当分母为 0 或者空值时，直接令该行的现金持有为空值。

（5）输出结果文件，将第（4）步的结果写入一个 excel 表格，命名为 现金持有_对应的时间戳.xlsx。其中：从左到右每一列的列标题依次为：股票代码、会计年度、货币资金（元）、交易性金融资产（元）、资产总计（元）、现金及现金等价物（元）、现金持有。请在最上方添加标题行。

注意：（1）鉴于循环会导致代码运行效率低下，你可以给出可行的替代方案优化程序；（2）我希望这个代码可以实时显示运行进度。

请仔细阅读上述文字，并告诉我你是否已经完全理解所有要求。

```
pip install pandas numpy tqdm xlswriter openpyxl

import pandas as pd
import numpy as np
import os
from datetime import datetime
from tqdm import tqdm

# 设置 Mac 桌面路径
desktop_path = os.path.expanduser("~/Desktop/")

# 文件路径
file1_path = os.path.join(desktop_path, "财务报表明细.xlsx")
file2_path = os.path.join(desktop_path, "现金及现金等价物.xlsx")

print("=" * 60)
print("开始处理数据...")
print("=" * 60)

# 步骤 1: 读取第一个文件 (带进度条)
print("\n[1/5] 正在读取 财务报表明细.xlsx...")
with tqdm(total=1, desc="读取财务数据") as pbar:
    df_finance = pd.read_excel(file1_path)

    # 关键修复: 处理股票代码, 确保是 6 位字符串 (处理数值型 Excel 中的.0 问题)
    df_finance['股票代码'] = (df_finance['股票代码']
                              .apply(lambda x: str(int(x)).zfill(6)
                                     if pd.notna(x) and str(x).replace('.',1).isdigit()
                                     else str(x).zfill(6)))

    pbar.update(1)

print(f"  财务数据行数: {len(df_finance):,}")
print(f"  股票代码示例: {df_finance['股票代码'].iloc[0]} (类型: {type(df_finance['股票代码'].iloc[0])})")

# 步骤 2: 读取第二个文件 (带进度条)
print("\n[2/5] 正在读取 现金及现金等价物.xlsx...")
with tqdm(total=1, desc="读取现金数据") as pbar:
    df_cash = pd.read_excel(file2_path)
```

```

# 关键修复：同样处理股票代码
df_cash['股票代码'] = (df_cash['股票代码']
                        .apply(lambda x: str(int(x)).zfill(6)
                               if pd.notna(x) and str(x).replace('.', '').isdigit()
                               else str(x).zfill(6)))

pbar.update(1)

print(f"    现金数据行数: {len(df_cash):,}")
print(f"    股票代码示例: {df_cash['股票代码'].iloc[0]} (类型: {type(df_cash['股票代码'].iloc[0])})")

# 步骤 3: 合并数据（向量化操作）
print("\n[3/5] 正在合并数据（按股票代码+会计年度） ...")
with tqdm(total=1, desc="数据合并") as pbar:
    # 使用 inner merge 确保只保留两边都有的数据
    df_merged = pd.merge(
        df_finance,
        df_cash,
        on=['股票代码', '会计年度'],
        how='inner',
        suffixes=('_', '_现金表')
    )
    pbar.update(1)

print(f"    合并后行数: {len(df_merged):,}")

# 步骤 4: 计算现金持有（向量化计算） - 已按需求修改空值处理逻辑
print("\n[4/5] 正在计算现金持有比率...")
with tqdm(total=1, desc="计算指标") as pbar:

    # 提取计算所需列：货币资金、交易性金融资产、现金及现金等价物的空值视为 0
    monetary_funds = pd.to_numeric(df_merged['货币资金（元）'], errors='coerce').fillna(0)
    trading_assets = pd.to_numeric(df_merged['交易性金融资产（元）'], errors='coerce').fillna(0)

    # 注意：第二个表的列名是"期末现金及现金等价物余额（元）"，空值视为 0
    cash_equivalent_col = '期末现金及现金等价物余额（元）'
    cash_equivalent = pd.to_numeric(df_merged[cash_equivalent_col], errors='coerce').fillna(0)

    # 资产总计保持原样（不填充空值），用于判断分母有效性
    total_assets = pd.to_numeric(df_merged['资产总计（元）'], errors='coerce')

    # 计算分子：货币资金 + 交易性金融资产（空值已视为 0）
    numerator = monetary_funds + trading_assets

```

```

# 计算分母: 资产总计 - 现金及现金等价物 (现金等价物空值已视为 0)
denominator = total_assets - cash_equivalent

# 关键修改: 处理分母为 0 或空值的情况
# 条件: 分母为 0 或 资产总计为空值 (分母无效) 或 分母计算结果为空值
invalid_condition = (
    (denominator == 0) |
    (pd.isna(total_assets)) | # 资产总计为空则分母无效
    (pd.isna(denominator))
)

# 计算现金持有: 无效时设为空值, 否则计算比率
df_merged['现金持有'] = np.where(invalid_condition, np.nan, numerator / denominator)

# 处理无穷大值 (分母极接近 0 的情况)
df_merged['现金持有'] = df_merged['现金持有'].replace([np.inf, -np.inf], np.nan)

pbar.update(1)

valid_count = df_merged['现金持有'].notna().sum()
print(f"    有效计算结果数: {valid_count:,}")
print(f"    空值数量: {df_merged['现金持有'].isna().sum():,}")

# 步骤 5: 准备输出数据
print("\n[5/5] 正在准备输出文件...")
with tqdm(total=1, desc="数据整理") as pbar:

    # 重命名现金等价物列以符合输出要求
    df_output = df_merged.rename(columns={
        cash_equivalent_col: '现金及现金等价物 (元) '
    })

    # 选择指定列并按要求的顺序排列
    output_columns = [
        '股票代码',
        '会计年度',
        '货币资金 (元) ',
        '交易性金融资产 (元) ',
        '资产总计 (元) ',
        '现金及现金等价物 (元) ',
        '现金持有'
    ]

    df_result = df_output[output_columns].copy()

```

```

# 确保股票代码为字符串格式（前面已经处理过，这里再次确保）
df_result['股票代码'] = df_result['股票代码'].astype(str)

pbar.update(1)

# 生成带时间戳的文件名
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
output_filename = f'现金持有_{timestamp}.xlsx'
output_path = os.path.join(desktop_path, output_filename)

# 写入 Excel
print(f'\n 正在写入 Excel 文件: {output_filename}...')
with tqdm(total=1, desc="写入 Excel") as pbar:

    # 使用 xlsxwriter 引擎以便更好地控制格式
    with pd.ExcelWriter(output_path, engine='xlsxwriter') as writer:
        df_result.to_excel(writer, index=False, sheet_name='现金持有数据')

        # 获取工作簿和工作表
        workbook = writer.book
        worksheet = writer.sheets['现金持有数据']

        # 设置文本格式（防止 Excel 自动转换股票代码）
        text_format = workbook.add_format({'num_format': '@'})

        # 将 A 列（股票代码）设为文本格式
        worksheet.set_column('A:A', 12, text_format)

        # 将现金持有列设为数字格式（保留 6 位小数）
        num_format = workbook.add_format({'num_format': '0.000000'})
        worksheet.set_column('G:G', 15, num_format) # G 列是现金持有（第 7 列）

    pbar.update(1)

print("\n" + "=" * 60)
print("处理完成! ")
print(f'输出文件路径: {output_path}')
print(f'总行数: {len(df_result):,}')
if valid_count > 0:
    print(f'现金持有均值: {df_result["现金持有"].mean():.6f}')
    print(f'现金持有中位数: {df_result["现金持有"].median():.6f}')
print("=" * 60)

```

十八、固定资产净值占比计算代码

http://localhost:8888/notebooks/Desktop/Python/%E6%AF%95%E4%B8%9A%E8%AE%BA%E6%96%87_%E5%9B%BA%E5%AE%9A%E8%B5%84%E4%BA%A7%E5%87%80%E5%80%BC%E5%8D%A0%E6%AF%94%E8%AE%A1%E7%AE%97%E4%BB%A3%E7%A0%81_20260303.ipynb

注：1、所有代码运行前需把 excel 文件放到桌面上；2、代码中的路径只适用于 mac 系统，windows 系统需要对路径代码进行修改；3、需要把桌面上的文件名称修改至与代码中完全相同。

Kimi 指令：

我现在需要你帮我设计一段 python 代码：

一、现有数据：

我现在有 2 个 excel 文件需要合并。这些文件均位于 mac 桌面上。

第一个 excel 命名为：财务报表明细.xlsx，记录了历年上市公司的货币资金、交易性金融资产、资产总计数据。第一行为中文标题，从左到右分别为：股票代码、证券简称、会计年度、会计年度（滞后前）、统计截止日期、报表类型、货币资金（元）、交易性金融资产（元）、资产总计（元）。

第二个 excel 命名为：固定资产净值.xlsx，记录了历年上市公司的固定资产净值数据。表格的第一行为中文标题，从左到右依次为：股票代码、会计年度、会计年度（滞后前）、统计截止日期、固定资产净值（元）。

二、我的需求

1、任务目标：（1）将两张表格按照（股票代码，会计年度）合并；（3）计算各年度各公司固定资产净值占比=固定资产净值（元）/资产总计（元）（注意预防分母为 0 或空的报错）。

2、具体操作：

（1）读取位于 mac 桌面命名为 财务报表明细.xlsx 和 固定资产净值.xlsx 的文件。注意，由于股票代码存在左侧数据为 0 的情况，所以一定要始终以字符串的格式储存，保证长度一直为 6 位。

（2）将 财务报表明细.xlsx 与 固定资产净值.xlsx 按照关键词（股票代码，会计年度）合并。

（3）新建一列“固定资产净值占比”，利用公式“固定资产净值占比=固定资产净值（元）/资产总计（元）”，计算每一行对应的（股票代码，会计年度）的固定资产净值占比。注意，当分母为 0 或者空值时，直接令该行的固定资产净值占比为空值；当某行的固定资产净值（元）金额为空值时，视为该公司该年份固定资产净值为 0，令该行的固定资产净值占比为 0。

（5）输出结果文件，将第（4）步的结果写入一个 excel 表格，命名为 固定资产净值占比_对应的时间戳.xlsx。其中：从左到右每一列的列标题依次为：股票代码、会计年度、固定资产净值（元）、资产总计（元）、固定资产净值占比。请在最上方添加标题行。

注意：（1）鉴于循环会导致代码运行效率低下，你可以给出可行的替代方案优化程序；（2）我希望这个代码可以实时显示运行进度。

请仔细阅读上述文字，并告诉我你是否已经完全理解所有要求。

```
import pandas as pd
import numpy as np
import os
from datetime import datetime
from tqdm import tqdm

def safe_format_stock_code(x):
    """
    安全格式化股票代码：处理 Excel 读取导致的.0 后缀、文本、空格等问题，
    确保输出为 6 位数字字符串
    """
    if pd.isna(x):
        return np.nan

    # 转为字符串并清理
    s = str(x).strip()

    # 处理 Excel 数值型读取导致的.0 后缀（如 1.0 -> 1）
    if s.endswith('.0') and s[:-2].isdigit():
        s = s[:-2]

    # 提取纯数字部分
    s = "".join(filter(str.isdigit, s))

    # 如果为空返回 nan，否则补零到 6 位
    return s.zfill(6) if s else np.nan

# 设置 Mac 桌面路径
desktop_path = os.path.expanduser("~/Desktop/")

# 文件路径
file1_path = os.path.join(desktop_path, "财务报表明细.xlsx")
file2_path = os.path.join(desktop_path, "固定资产净值.xlsx")

print("=" * 60)
print("开始处理数据...")
print("=" * 60)

# 检查文件是否存在
if not os.path.exists(file1_path):
    raise FileNotFoundError(f"未找到文件: {file1_path}")
```

```

if not os.path.exists(file2_path):
    raise FileNotFoundError(f"未找到文件: {file2_path}")

# 步骤 1: 读取财务报表明细
print("\n[1/4] 正在读取 财务报表明细.xlsx...")
with tqdm(total=1, desc="读取财务数据") as pbar:
    # 读取时指定股票代码为字符串类型
    df_finance = pd.read_excel(file1_path, dtype={'股票代码': str})

    # 格式化股票代码 (6 位字符串)
    df_finance['股票代码'] = df_finance['股票代码'].apply(safe_format_stock_code)

    # 确保会计年度为整数 (统一数据类型以便合并)
    df_finance['会计年度'] = pd.to_numeric(df_finance['会计年度'], errors='coerce')

    # 删除股票代码或会计年度为空的行
    df_finance = df_finance.dropna(subset=['股票代码', '会计年度'])

    pbar.update(1)

print(f"    财务数据行数: {len(df_finance):,}")
print(f"    股票代码示例: {df_finance['股票代码'].iloc[0]}")

# 步骤 2: 读取固定资产净值
print("\n[2/4] 正在读取 固定资产净值.xlsx...")
with tqdm(total=1, desc="读取固定资产数据") as pbar:
    df_fixed = pd.read_excel(file2_path, dtype={'股票代码': str})

    # 同样处理股票代码和会计年度
    df_fixed['股票代码'] = df_fixed['股票代码'].apply(safe_format_stock_code)
    df_fixed['会计年度'] = pd.to_numeric(df_fixed['会计年度'], errors='coerce')

    # 删除空值行
    df_fixed = df_fixed.dropna(subset=['股票代码', '会计年度'])

    pbar.update(1)

print(f"    固定资产数据行数: {len(df_fixed):,}")
print(f"    股票代码示例: {df_fixed['股票代码'].iloc[0]}")

# 步骤 3: 合并数据
print("\n[3/4] 正在合并数据 (按股票代码+会计年度) ...")
with tqdm(total=1, desc="数据合并") as pbar:
    # 使用 inner merge 确保两边都存在的记录

```

```

df_merged = pd.merge(
    df_finance,
    df_fixed,
    on=['股票代码', '会计年度'],
    how='inner',
    suffixes=('_', '_固定')
)
pbar.update(1)

if len(df_merged) == 0:
    raise ValueError("合并后数据为空， 请检查股票代码和会计年度格式是否一致")

print(f" 合并后行数: {len(df_merged):,}")

# 步骤 4: 计算固定资产净值占比（向量化计算）
print("\n[4/4] 正在计算固定资产净值占比...")
with tqdm(total=1, desc="计算指标") as pbar:

    # 处理固定资产净值：转为数值型，空值视为 0
    fixed_assets = pd.to_numeric(df_merged['固定资产净值（元）'], errors='coerce').fillna(0)

    # 处理资产总计：转为数值型，保持空值（用于判断分母有效性）
    total_assets = pd.to_numeric(df_merged['资产总计（元）'], errors='coerce')

    # 计算比率（向量化）
    ratio = fixed_assets / total_assets

    # 定义无效条件：资产总计为 0 或空值时，结果为空值
    invalid_condition = (
        (total_assets == 0) |
        (pd.isna(total_assets))
    )

    # 应用条件：无效时设为 NaN，否则保持计算结果（包括固定资产净值为 0 的情况）
    df_merged['固定资产净值占比'] = np.where(invalid_condition, np.nan, ratio)

    # 处理极端情况：无穷大值（理论上不应出现，但以防万一）
    df_merged['固定资产净值占比'] = df_merged['固定资产净值占比'].replace([np.inf, -np.inf],
np.nan)

    pbar.update(1)

valid_count = df_merged['固定资产净值占比'].notna().sum()
zero_count = (df_merged['固定资产净值占比'] == 0).sum()

```

```

print(f"    有效计算结果数: {valid_count:,}")
print(f"    其中为 0 的结果数: {zero_count:,} (固定资产净值为空或实际为 0) ")
print(f"    空值数量: {df_merged['固定资产净值占比'].isna().sum():,}")

# 步骤 5: 准备输出数据
print("\n[5/5] 正在准备输出文件...")
with tqdm(total=1, desc="数据整理") as pbar:

    # 按用户要求的列顺序选择
    output_columns = [
        '股票代码',
        '会计年度',
        '固定资产净值 (元) ',
        '资产总计 (元) ',
        '固定资产净值占比'
    ]

    # 检查所有必需的列是否存在
    missing_cols = [col for col in output_columns if col not in df_merged.columns]
    if missing_cols:
        raise KeyError(f'缺少必要的列: {missing_cols}')

    df_result = df_merged[output_columns].copy()

    # 确保会计年度显示为整数 (无小数位)
    df_result['会计年度'] = df_result['会计年度'].astype(int)

    # 确保股票代码为字符串格式
    df_result['股票代码'] = df_result['股票代码'].astype(str)

    pbar.update(1)

# 生成带时间戳的文件名
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
output_filename = f"固定资产净值占比_{timestamp}.xlsx"
output_path = os.path.join(desktop_path, output_filename)

# 写入 Excel
print(f"\n正在写入 Excel 文件: {output_filename}...")
with tqdm(total=1, desc="写入 Excel") as pbar:

    # 使用 xlsxwriter 引擎以支持格式设置
    with pd.ExcelWriter(output_path, engine='xlsxwriter') as writer:
        df_result.to_excel(writer, index=False, sheet_name='固定资产净值占比数据')

```

```

# 获取工作簿和工作表对象
workbook = writer.book
worksheet = writer.sheets['固定资产净值占比数据']

# 定义格式: 文本格式 (防止 Excel 自动转换股票代码)
text_format = workbook.add_format({'num_format': '@'})

# 定义格式: 百分比格式 (保留 4 位小数, 便于观察)
percent_format = workbook.add_format({'num_format': '0.0000'})

# 设置列宽和格式
worksheet.set_column('A:A', 12, text_format) # 股票代码列
worksheet.set_column('B:B', 12)             # 会计年度
worksheet.set_column('C:D', 20)             # 固定资产净值、资产总计
worksheet.set_column('E:E', 18, percent_format) # 固定资产净值占比

pbar.update(1)

print("\n" + "=" * 60)
print("处理完成! ")
print(f"输出文件路径: {output_path}")
print(f"总行数: {len(df_result):,}")
if valid_count > 0:
    print(f"固定资产净值占比统计:")
    print(f" - 均值: {df_result['固定资产净值占比'].mean():.4f}")
    print(f" - 中位数: {df_result['固定资产净值占比'].median():.4f}")
    print(f" - 最小值: {df_result['固定资产净值占比'].min():.4f}")
    print(f" - 最大值: {df_result['固定资产净值占比'].max():.4f}")
print("=" * 60)

```

十九、审计费用率计算代码

http://localhost:8888/notebooks/Desktop/Python/%E6%AF%95%E4%B8%9A%E8%AE%BA%E6%96%87_%E5%AE%A1%E8%AE%A1%E8%B4%B9%E7%94%A8%E7%8E%87_20260303.ipynb

注：1、所有代码运行前需把 excel 文件放到桌面上；2、代码中的路径只适用于 mac 系统，windows 系统需要对路径代码进行修改；3、需要把桌面上的文件名称修改至与代码中完全相同。

Kimi 指令：

我现在需要你帮我设计一段 python 代码：

一、现有数据：

我现在有 2 个 excel 文件需要合并。这些文件均位于 mac 桌面上。

第一个 excel 命名为：财务报表明细.xlsx，记录了历年上市公司的货币资金、交易性金融资产、资产总计数据。第一行为中文标题，从左到右分别为：股票代码、证券简称、会计年度、会计年度（滞后前）、统计截止日期、报表类型、货币资金（元）、交易性金融资产（元）、资产总计（元）。

第二个 excel 命名为：审计费用.xlsx，记录了历年上市公司的审计费用数据。表格的第一行为中文标题，从左到右依次为：股票代码、会计年度、会计年度（滞后前）、会计截止日期、审计费用合计（元）、合计审计费用货币单位。

二、我的需求

1、任务目标：（1）将两张表格按照（股票代码，会计年度）合并；（3）计算各年度各公司审计费用率=审计费用合计（元）/资产总计（元）（注意预防分母为 0 或空的报错）。

2、具体操作：

（1）读取位于 mac 桌面命名为 财务报表明细.xlsx 和 审计费用.xlsx 的文件。注意，由于股票代码存在左侧数据为 0 的情况，所以一定要始终以字符串的格式储存，保证长度一直为 6 位。

（2）将 财务报表明细.xlsx 与 审计费用.xlsx 按照关键词（股票代码，会计年度）合并。

（3）新建一列“审计费用率”，利用公式“审计费用率=审计费用合计（元）/资产总计（元）”，计算每一行对应的（股票代码，会计年度）的审计费用率。注意，当分母为 0 或者空值时，直接令该行的审计费用率为空值；当某行的审计费用（元）金额为空值时，视为该公司该年份审计费用为 0，令该行的审计费用率为 0。

（5）输出结果文件，将第（4）步的结果写入一个 excel 表格，命名为 审计费用率_对应的时间戳.xlsx。其中：从左到右每一列的列标题依次为：股票代码、会计年度、审计费用（元）、资产总计（元）、审计费用率。请在最上方添加标题行。

注意：（1）鉴于循环会导致代码运行效率低下，你可以给出可行的替代方案优化程序；（2）我希望这个代码可以实时显示运行进度。

请仔细阅读上述文字，并告诉我你是否已经完全理解所有要求。

```

import pandas as pd
import numpy as np
import os
from datetime import datetime
from tqdm import tqdm

def safe_format_stock_code(x):
    """
    安全格式化股票代码: 处理 Excel 读取导致的.0 后缀、文本、空格等问题,
    确保输出为 6 位数字字符串
    """
    if pd.isna(x):
        return np.nan

    # 转为字符串并清理
    s = str(x).strip()

    # 处理 Excel 数值型读取导致的.0 后缀 (如 1.0 -> 1)
    if s.endswith('.0') and s[:-2].isdigit():
        s = s[:-2]

    # 提取纯数字部分
    s = "".join(filter(str.isdigit, s))

    # 如果为空返回 nan, 否则补零到 6 位
    return s.zfill(6) if s else np.nan

# 设置 Mac 桌面路径
desktop_path = os.path.expanduser("~/Desktop/")

# 文件路径
file1_path = os.path.join(desktop_path, "财务报表明细.xlsx")
file2_path = os.path.join(desktop_path, "审计费用.xlsx")

print("=" * 60)
print("开始处理数据...")
print("=" * 60)

# 检查文件是否存在
if not os.path.exists(file1_path):
    raise FileNotFoundError(f"未找到文件: {file1_path}")
if not os.path.exists(file2_path):
    raise FileNotFoundError(f"未找到文件: {file2_path}")

```

```

# 步骤 1: 读取财务报表明细
print("\n[1/4] 正在读取 财务报表明细.xlsx...")
with tqdm(total=1, desc="读取财务数据") as pbar:
    df_finance = pd.read_excel(file1_path, dtype={'股票代码': str})

    # 格式化股票代码 (6 位字符串)
    df_finance['股票代码'] = df_finance['股票代码'].apply(safe_format_stock_code)

    # 确保会计年度为整数 (统一数据类型以便合并)
    df_finance['会计年度'] = pd.to_numeric(df_finance['会计年度'], errors='coerce')

    # 删除股票代码或会计年度为空的行
    df_finance = df_finance.dropna(subset=['股票代码', '会计年度'])

    pbar.update(1)

print(f"    财务数据行数: {len(df_finance):,}")
print(f"    股票代码示例: {df_finance['股票代码'].iloc[0]}")

# 步骤 2: 读取审计费用
print("\n[2/4] 正在读取 审计费用.xlsx...")
with tqdm(total=1, desc="读取审计费用数据") as pbar:
    df_audit = pd.read_excel(file2_path, dtype={'股票代码': str})

    # 同样处理股票代码和会计年度
    df_audit['股票代码'] = df_audit['股票代码'].apply(safe_format_stock_code)
    df_audit['会计年度'] = pd.to_numeric(df_audit['会计年度'], errors='coerce')

    # 删除空值行
    df_audit = df_audit.dropna(subset=['股票代码', '会计年度'])

    pbar.update(1)

print(f"    审计费用数据行数: {len(df_audit):,}")
print(f"    股票代码示例: {df_audit['股票代码'].iloc[0]}")

# 步骤 3: 合并数据
print("\n[3/4] 正在合并数据 (按股票代码+会计年度) ...")
with tqdm(total=1, desc="数据合并") as pbar:
    # 使用 inner merge 确保两边都存在的记录
    df_merged = pd.merge(
        df_finance,
        df_audit,

```

```

        on=['股票代码','会计年度'],
        how='inner',
        suffixes=('_', '_审计')
    )
    pbar.update(1)

if len(df_merged) == 0:
    raise ValueError("合并后数据为空，请检查股票代码和会计年度格式是否一致")

print(f"    合并后行数: {len(df_merged):,}")

# 步骤 4: 计算审计费用率（向量化计算）
print("\n[4/4] 正在计算审计费用率...")
with tqdm(total=1, desc="计算指标") as pbar:

    # 处理审计费用：转为数值型，空值视为 0
    audit_fee = pd.to_numeric(df_merged['审计费用合计（元）'], errors='coerce').fillna(0)

    # 处理资产总计：转为数值型，保持空值（用于判断分母有效性）
    total_assets = pd.to_numeric(df_merged['资产总计（元）'], errors='coerce')

    # 计算比率（向量化）
    ratio = audit_fee / total_assets

    # 定义无效条件：资产总计为 0 或空值时，结果为空值
    invalid_condition = (
        (total_assets == 0) |
        (pd.isna(total_assets))
    )

    # 应用条件：无效时设为 NaN，否则保持计算结果（包括审计费用为 0 的情况）
    df_merged['审计费用率'] = np.where(invalid_condition, np.nan, ratio)

    # 处理极端情况：无穷大值（理论上不应出现，但以防万一）
    df_merged['审计费用率'] = df_merged['审计费用率'].replace([np.inf, -np.inf], np.nan)

    pbar.update(1)

valid_count = df_merged['审计费用率'].notna().sum()
zero_count = (df_merged['审计费用率'] == 0).sum()
print(f"    有效计算结果数: {valid_count:,}")
print(f"    其中为 0 的结果数: {zero_count:,}（审计费用为空或实际为 0）")
print(f"    空值数量: {df_merged['审计费用率'].isna().sum():,}")

```

```

# 步骤 5: 准备输出数据
print("\n[5/5] 正在准备输出文件...")
with tqdm(total=1, desc="数据整理") as pbar:

    # 重命名审计费用列以符合输出要求 (审计费用合计 (元) -> 审计费用 (元) )
    df_merged = df_merged.rename(columns={
        '审计费用合计 (元) ': '审计费用 (元) '
    })

    # 按用户要求的列顺序选择
    output_columns = [
        '股票代码',
        '会计年度',
        '审计费用 (元) ',
        '资产总计 (元) ',
        '审计费用率'
    ]

    # 检查所有必需的列是否存在
    missing_cols = [col for col in output_columns if col not in df_merged.columns]
    if missing_cols:
        raise KeyError(f'缺少必要的列: {missing_cols}')

    df_result = df_merged[output_columns].copy()

    # 确保会计年度显示为整数 (无小数位)
    df_result['会计年度'] = df_result['会计年度'].astype(int)

    # 确保股票代码为字符串格式
    df_result['股票代码'] = df_result['股票代码'].astype(str)

    pbar.update(1)

# 生成带时间戳的文件名
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
output_filename = f'审计费用率_{timestamp}.xlsx'
output_path = os.path.join(desktop_path, output_filename)

# 写入 Excel
print(f"\n正在写入 Excel 文件: {output_filename}...")
with tqdm(total=1, desc="写入 Excel") as pbar:

    # 使用 xlsxwriter 引擎以支持格式设置
    with pd.ExcelWriter(output_path, engine='xlsxwriter') as writer:

```

```

df_result.to_excel(writer, index=False, sheet_name='审计费用率数据')

# 获取工作簿和工作表对象
workbook = writer.book
worksheet = writer.sheets['审计费用率数据']

# 定义格式：文本格式（防止 Excel 自动转换股票代码）
text_format = workbook.add_format({'num_format': '@'})

# 定义格式：百分比格式（保留 6 位小数，便于观察微小比率）
percent_format = workbook.add_format({'num_format': '0.000000'})

# 设置列宽和格式
worksheet.set_column('A:A', 12, text_format) # 股票代码列
worksheet.set_column('B:B', 12)             # 会计年度
worksheet.set_column('C:D', 20)             # 审计费用、资产总计
worksheet.set_column('E:E', 18, percent_format) # 审计费用率

pbar.update(1)

print("\n" + "=" * 60)
print("处理完成! ")
print(f"输出文件路径: {output_path}")
print(f"总行数: {len(df_result):,}")
if valid_count > 0:
    print(f"审计费用率统计:")
    print(f" - 均值: {df_result['审计费用率'].mean():.6f}")
    print(f" - 中位数: {df_result['审计费用率'].median():.6f}")
    print(f" - 最小值: {df_result['审计费用率'].min():.6f}")
    print(f" - 最大值: {df_result['审计费用率'].max():.6f}")
print("=" * 60)

```

二十、自由现金流计算代码

http://localhost:8888/notebooks/Desktop/Python/%E6%AF%95%E4%B8%9A%E8%AE%BA%E6%96%87_%E8%87%AA%E7%94%B1%E7%8E%B0%E9%87%91%E6%B5%81%E8%AE%A1%E7%AE%97%E4%BB%A3%E7%A0%81_20260308.ipynb

注：1、所有代码运行前需把 excel 文件放到桌面上；2、代码中的路径只适用于 mac 系统，windows 系统需要对路径代码进行修改；3、需要把桌面上的文件名称修改至与代码中完全相同。

Kimi 指令：

我现在需要你帮我设计一段 python 代码：

一、现有数据：

我现在有 2 个 excel 文件需要合并。这些文件均位于 mac 桌面上。

第一个 excel 命名为：经营活动产生的现金流量净额.xlsx。第一行为中文标题，从左到右分别为：股票代码、证券简称、会计年度、统计截止日期、报表类型、经营活动产生的现金流量净额（元）。

第二个 excel 命名为：总资产.xlsx。表格的第一行为中文标题，从左到右依次为：股票代码、证券简称、会计年度、统计截止日期、报表类型、资产总计（元）。

二、我的需求

1、任务目标：（1）将两张表格按照（股票代码，会计年度）合并；（2）计算各年度各公司自由现金流=经营活动产生的现金流量净额（元）/资产总计（元）（注意预防分母为 0 或空的报错）。

2、具体操作：

（1）读取位于 mac 桌面命名为 经营活动产生的现金流量净额.xlsx 和 总资产.xlsx 的文件。注意，由于股票代码存在左侧数据为 0 的情况，所以一定要始终以字符串的格式储存，保证长度一直为 6 位。

（2）将 经营活动产生的现金流量净额.xlsx 与 总资产.xlsx 按照关键词（股票代码，会计年度）合并。

（3）新建一列“自由现金流”，利用公式“自由现金流=经营活动产生的现金流量净额（元）/资产总计（元）”，计算每一行对应的（股票代码，会计年度）的自由现金流。注意，当分母为 0 或者空值时，直接令该行的自由现金流为空值；当某行的经营活动产生的现金流量净额为空值时，视为该公司该年份经营活动产生的现金流量净额为 0，令该行的自由现金流为 0。

（5）输出结果文件，将第（4）步的结果写入一个 excel 表格，命名为 自由现金流_对应的时间戳.xlsx。其中：从左到右每一列的列标题依次为：股票代码、会计年度、自由现金流。请在最上方添加标题行。

注意：（1）鉴于循环会导致代码运行效率低下，你可以给出可行的替代方案优化程序；（2）我希望这个代码可以实时显示运行进度。

请仔细阅读上述文字，并告诉我你是否已经完全理解所有要求。

```

import pandas as pd
import numpy as np
import os
from datetime import datetime
from tqdm import tqdm

# ===== 配置部分 =====
desktop_path = os.path.join(os.path.expanduser("~"), "Desktop")

file_cash = os.path.join(desktop_path, "经营活动产生的现金流量净额.xlsx")
file_asset = os.path.join(desktop_path, "总资产.xlsx")

# 检查文件是否存在
if not os.path.exists(file_cash):
    raise FileNotFoundError(f'未找到文件: {file_cash}')
if not os.path.exists(file_asset):
    raise FileNotFoundError(f'未找到文件: {file_asset}')

timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
output_file = os.path.join(desktop_path, f'自由现金流_{timestamp}.xlsx')

print("=" * 60)
print("开始处理自由现金流计算任务")
print(f'当前时间: {datetime.now().strftime("%Y-%m-%d %H:%M:%S')}")
print("=" * 60)

# ===== 第 1 步: 读取数据 =====
print("\n【步骤 1/4】 读取 Excel 文件...")

# 读取经营现金流数据
df_cash = pd.read_excel(file_cash, engine='openpyxl')
print(f'  原现金流表行数: {len(df_cash)}')

# 安全格式化股票代码 (处理可能的数值型 1.0 -> "000001")
print("  正在处理股票代码格式 (经营现金流表) ...")
df_cash['股票代码'] = df_cash['股票代码'].apply(
    lambda x: str(int(float(x))).zfill(6) if pd.notna(x) else x
)

# 确保会计年度为字符串类型 (统一合并键类型)
df_cash['会计年度'] = df_cash['会计年度'].astype(str)

```

```

print(f" ✓ 经营现金流表处理完成, 共 {len(df_cash)} 行")

# 读取总资产数据
df_asset = pd.read_excel(file_asset, engine='openpyxl')
print(f" 原总资产表行数: {len(df_asset)}")

print(" 正在处理股票代码格式 (总资产表) ...")
df_asset['股票代码'] = df_asset['股票代码'].apply(
    lambda x: str(int(float(x))).zfill(6) if pd.notna(x) else x
)

# 确保会计年度为字符串类型
df_asset['会计年度'] = df_asset['会计年度'].astype(str)

print(f" ✓ 总资产表处理完成, 共 {len(df_asset)} 行")

# ===== 第2步: 合并数据 =====
print("\n 【步骤 2/4】 按 (股票代码, 会计年度) 合并数据...")

# 选择需要的列, 减少内存占用
cash_cols = ['股票代码', '会计年度', '经营活动产生的现金流量净额 (元)']
asset_cols = ['股票代码', '会计年度', '资产总计 (元)']

# 使用 tqdm 显示合并进度 (虽然 merge 很快, 但给用户反馈)
print(" 正在执行数据库合并...")
merged_df = pd.merge(
    df_cash[cash_cols],
    df_asset[asset_cols],
    on=['股票代码', '会计年度'],
    how='left',
    indicator=True
)

match_stats = merged_df['_merge'].value_counts()

print(f" ✓ 合并完成: ")

print(f" - 成功匹配: {match_stats.get('both', 0)} 条")
print(f" - 未匹配 (仅在现金流表): {match_stats.get('left_only', 0)} 条")
print(f" - 总计: {len(merged_df)} 条")

merged_df.drop(columns=['_merge'], inplace=True)

```

```

# ===== 第 3 步：计算自由现金流 =====
print("\n 【步骤 3/4】 计算自由现金流...")

cash_col = '经营活动产生的现金流量净额 (元) '
asset_col = '资产总计 (元) '

print(" 正在处理缺失值和计算 (向量化运算) ...")

# 规则 1: 经营现金流空值 → 视为 0
merged_df[cash_col] = merged_df[cash_col].fillna(0)

# 规则 2: 计算自由现金流, 处理分母异常
denominator = merged_df[asset_col]

# 判断分母是否有效 (不为 0 且不为空)
valid_denominator = (denominator.notna()) & (denominator != 0)

# 向量化计算
merged_df['自由现金流'] = np.where(
    valid_denominator,
    merged_df[cash_col] / denominator,
    np.nan # 分母无效时置空
)

# 统计信息
valid_count = valid_denominator.sum()
zero_or_null = len(merged_df) - valid_count

print(f"  ✓ 计算完成: ")

print(f"  - 有效计算 (分母>0) : {valid_count} 条")
print(f"  - 分母为 0 或空值: {zero_or_null} 条 (已置空) ")
print(f"  - 经营现金流原为空值: {(df_cash[cash_col].isna()).sum()} 条 (已视为 0) ")

# ===== 第 4 步：输出结果 =====
print("\n 【步骤 4/4】 生成结果文件...")

# 选择并整理最终列
result_df = merged_df[['股票代码', '会计年度', '自由现金流']].copy()

# 确保股票代码为 6 位字符串 (再次确认)
result_df['股票代码'] = result_df['股票代码'].astype(str)

print(f" 正在写入 Excel (共 {len(result_df)} 行) ...")

```

```

# 使用 openpyxl 引擎（与读取一致）
with pd.ExcelWriter(output_file, engine='openpyxl') as writer:
    result_df.to_excel(
        writer,
        sheet_name='自由现金流',
        index=False,
        header=True
    )

print(f"  ✓ 文件保存成功: {output_file}")

# ===== 完成统计 =====
print("\n" + "=" * 60)
print("处理完成! ")
print(f"输出文件: 自由现金流_{timestamp}.xlsx")
print(f"总行数: {len(result_df)}")
print(f"有效自由现金流记录: {result_df['自由现金流'].notna().sum()}")
print(f"空值记录（分母异常）: {result_df['自由现金流'].isna().sum()}")
if result_df['自由现金流'].notna().sum() > 0:
    print(f"自由现金流均值: {result_df['自由现金流'].mean():.6f}")
    print(f"自由现金流中位数: {result_df['自由现金流'].median():.6f}")
print("=" * 60)

```

二十一、新增借款/总资产计算代码

http://localhost:8888/notebooks/Desktop/Python/%E6%AF%95%E4%B8%9A%E8%AE%BA%E6%96%87_%E6%96%B0%E5%A2%9E%E5%80%9F%E6%AC%BE%3D%E5%B9%B4%E6%9C%AB%E6%96%B0%E5%A2%9E%E5%80%9F%E6%AC%BE%E5%8D%A0%E5%B9%B4%E6%9C%AB%E6%80%BB%E8%B5%84%E4%BA%A7_20260308.ipynb

注：1、所有代码运行前需把 excel 文件放到桌面上；2、代码中的路径只适用于 mac 系统，windows 系统需要对路径代码进行修改；3、需要把桌面上的文件名称修改至与代码中完全相同。

Kimi 指令：

我现在需要你帮我设计一段 python 代码：

一、现有数据：

我现在有 2 个 excel 文件需要合并。这些文件均位于 mac 桌面上。

第一个 excel 命名为：6 新增借款（按贷款事件统计）.xlsx。第一行为中文标题，从左到右分别为：股票代码、会计年度、本年新增合计（万元）。

第二个 excel 命名为：总资产.xlsx。表格的第一行为中文标题，从左到右依次为：股票代码、证券简称、会计年度、统计截止日期、报表类型、资产总计（元）。

二、我的需求

1、任务目标：（1）将两张表格按照（股票代码，会计年度）合并；（2）计算各年度各公司新增借款=本年新增合计（万元）*10000/资产总计（元）（注意预防分母为 0 或空的报错）。

2、具体操作：

（1）读取位于 mac 桌面命名为 6 新增借款（按贷款事件统计）.xlsx 和 总资产.xlsx 的文件。注意，由于股票代码存在左侧数据为 0 的情况，所以一定要始终以字符串的格式储存，保证长度一直为 6 位。

（2）将 6 新增借款（按贷款事件统计）.xlsx 与 总资产.xlsx 按照关键词（股票代码，会计年度）合并。

（3）新建一列“新增借款”，利用公式“新增借款=本年新增合计（万元）*10000/资产总计（元）”，计算每一行对应的（股票代码，会计年度）的新增借款。注意，当分母为 0 或者空值时，直接令该行的新增借款为空值；当某行的本年新增合计（元）为空值时，视为该公司该年份本年新增合计金额为 0，令该行的新增借款为 0。

（5）输出结果文件，将第（4）步的结果写入一个 excel 表格，命名为 新增借款_对应的时间戳.xlsx。其中：从左到右每一列的列标题依次为：股票代码、会计年度、自由现金流。请在最上方添加标题行。

注意：（1）鉴于循环会导致代码运行效率低下，你可以给出可行的替代方案优化程序；（2）我希望这个代码可以实时显示运行进度。

请仔细阅读上述文字，并告诉我你是否已经完全理解所有要求。

```

import pandas as pd
import numpy as np
import os
from datetime import datetime
from tqdm import tqdm

# ===== 配置部分 =====
desktop_path = os.path.join(os.path.expanduser("~"), "Desktop")

# 输入文件名 (含空格和中文括号)
file_loan = os.path.join(desktop_path, "6 新增借款 (按贷款事件统计) .xlsx")
file_asset = os.path.join(desktop_path, "总资产.xlsx")

# 检查文件存在性
if not os.path.exists(file_loan):
    raise FileNotFoundError(f'未找到文件: {file_loan}\n 请确保文件名完全匹配: '6 新增借款 (按贷款事件统计) .xlsx')
if not os.path.exists(file_asset):
    raise FileNotFoundError(f'未找到文件: {file_asset}')

timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
output_file = os.path.join(desktop_path, f'新增借款_{timestamp}.xlsx')

print("=" * 60)
print("开始处理新增借款计算任务")
print(f'当前时间: {datetime.now().strftime("%Y-%m-%d %H:%M:%S')}")
print("=" * 60)

# ===== 第 1 步: 读取数据 =====
print("\n【步骤 1/4】 读取 Excel 文件...")

# 读取新增借款表, 显示进度
print(f" 正在读取: 6 新增借款 (按贷款事件统计) .xlsx")
df_loan = pd.read_excel(file_loan, engine='openpyxl')

print(f"  ✓ 读取完成, 原表行数: {len(df_loan)}")

# 列名检查 (防止 Excel 列名与预期不符)
expected_loan_cols = ['股票代码', '会计年度', '本年新增合计 (万元) ']
missing_cols = [col for col in expected_loan_cols if col not in df_loan.columns]
if missing_cols:
    raise ValueError(f'新增借款表缺少列: {missing_cols}\n 实际列名: {list(df_loan.columns)}')

```

```

# 安全格式化股票代码（鲁棒版：处理数字、字符串、含字母等情况）
def safe_format_stock_code(x):
    if pd.isna(x):
        return x
    s = str(x).strip()
    # 处理 Excel 浮点数（如 1.0 → 000001）
    if '.' in s and s.replace('.', '').isdigit():
        s = str(int(float(x)))
    # 移除可能的后缀（如 .SZ, .SH）
    s = s.split('.')[0]
    return s.zfill(6)

print(" 正在处理股票代码格式（新增借款表） ...")
tqdm.pandas(desc=" 格式化进度")
df_loan['股票代码'] = df_loan['股票代码'].progress_apply(safe_format_stock_code)

# 确保会计年度为字符串且去除小数（如 2023.0 → 2023）
df_loan['会计年度'] = df_loan['会计年度'].apply(
    lambda x: str(int(x)) if pd.notna(x) and isinstance(x, (int, float)) else str(x)
)

# 确保数值列为数值类型（防止 Excel 读成字符串）
df_loan['本年新增合计（万元）'] = pd.to_numeric(df_loan['本年新增合计（万元）'],
errors='coerce')

print(f" ✓ 新增借款表处理完成，共 {len(df_loan)} 行")

# 读取总资产表
print(f"\n 正在读取：总资产.xlsx")
df_asset = pd.read_excel(file_asset, engine='openpyxl')

print(f" ✓ 读取完成，原表行数：{len(df_asset)}")

# 列名检查
expected_asset_cols = ['股票代码', '会计年度', '资产总计（元）']
missing_cols = [col for col in expected_asset_cols if col not in df_asset.columns]
if missing_cols:
    raise ValueError(f"总资产表缺少列：{missing_cols}\n 实际列名：{list(df_asset.columns)}")

print(" 正在处理股票代码格式（总资产表） ...")
df_asset['股票代码'] = df_asset['股票代码'].progress_apply(safe_format_stock_code)
df_asset['会计年度'] = df_asset['会计年度'].apply(
    lambda x: str(int(x)) if pd.notna(x) and isinstance(x, (int, float)) else str(x)
)

```

```

)

# 确保资产总计为数值类型
df_asset['资产总计 (元)'] = pd.to_numeric(df_asset['资产总计 (元)'], errors='coerce')

print(f"  ✓ 总资产表处理完成, 共 {len(df_asset)} 行")

# ===== 第 2 步: 合并数据 =====
print("\n【步骤 2/4】按 (股票代码, 会计年度) 合并数据...")

loan_cols = ['股票代码', '会计年度', '本年新增合计 (万元)']
asset_cols = ['股票代码', '会计年度', '资产总计 (元)']

print("  正在执行数据库合并...")
merged_df = pd.merge(
    df_loan[loan_cols],
    df_asset[asset_cols],
    on=['股票代码', '会计年度'],
    how='left',
    indicator=True
)

match_stats = merged_df['_merge'].value_counts()

print(f"  ✓ 合并完成: ")

print(f"    - 成功匹配: {match_stats.get('both', 0)} 条")
print(f"    - 未匹配 (仅在借款表): {match_stats.get('left_only', 0)} 条")
print(f"    - 总计: {len(merged_df)} 条")

merged_df.drop(columns=['_merge'], inplace=True)

# ===== 第 3 步: 计算新增借款比率 =====
print("\n【步骤 3/4】计算新增借款比率 (万元→元转换) ...")

loan_col = '本年新增合计 (万元)'
asset_col = '资产总计 (元)'

# 规则 1: 本年新增合计为空值 → 视为 0
null_loan_count = merged_df[loan_col].isna().sum()
merged_df[loan_col] = merged_df[loan_col].fillna(0)

# 单位转换: 万元 × 10000 = 元
loan_in_yuan = merged_df[loan_col] * 10000

```

```

# 规则 2: 分母有效性检查 (不为 0 且不为空)
denominator = merged_df[asset_col]
valid_denominator = (denominator.notna()) & (denominator != 0)
invalid_count = len(merged_df) - valid_denominator.sum()

# 向量化计算
merged_df['新增借款'] = np.where(
    valid_denominator,
    loan_in_yuan / denominator,
    np.nan
)

print(f" ✓ 计算完成: ")

print(f"    - 原新增合计为空的记录: {null_loan_count} 条 (已视为 0) ")
print(f"    - 分母为 0 或空值的记录: {invalid_count} 条 (已置空) ")
print(f"    - 有效计算记录: {valid_denominator.sum()} 条")

# ===== 第 4 步: 输出结果 =====
print("\n【步骤 4/4】生成结果文件...")

# 选择所需列 (注意: 用户描述中误写为"自由现金流", 实际应为"新增借款")
result_df = merged_df[['股票代码', '会计年度', '新增借款']].copy()

# 确保股票代码格式正确 (6 位字符串)
result_df['股票代码'] = result_df['股票代码'].astype(str)

print(f" 正在写入 Excel (共 {len(result_df)} 行) ...")
print(f" 输出路径: {output_file}")

# 使用 openpyxl 引擎写入
with pd.ExcelWriter(output_file, engine='openpyxl') as writer:
    result_df.to_excel(
        writer,
        sheet_name='新增借款',
        index=False,
        header=True
    )

# 调整列宽
worksheet = writer.sheets['新增借款']
worksheet.column_dimensions['A'].width = 12 # 股票代码
worksheet.column_dimensions['B'].width = 12 # 会计年度

```

```

worksheet.column_dimensions['C'].width = 18 # 新增借款

print(f"  ✓ 文件保存成功! ")

# ===== 完成统计 =====
print("\n" + "=" * 60)
print("处理完成摘要")
print("=" * 60)
print(f"输出文件: 新增借款_{timestamp}.xlsx")
print(f"总行数: {len(result_df):,}")
print(f"有效新增借款记录: {result_df['新增借款'].notna().sum():,}")
print(f"空值记录 (分母异常) : {result_df['新增借款'].isna().sum():,}")

if result_df['新增借款'].notna().sum() > 0:
    print(f"\n 新增借款比率统计: ")
    print(f"  均值: {result_df['新增借款'].mean():.6f}")
    print(f"  中位数: {result_df['新增借款'].median():.6f}")
    print(f"  标准差: {result_df['新增借款'].std():.6f}")
    print(f"  最小值: {result_df['新增借款'].min():.6f}")
    print(f"  最大值: {result_df['新增借款'].max():.6f}")

print("=" * 60)

```

二十二、经营效率计算代码

http://localhost:8888/notebooks/Desktop/Python/%E6%AF%95%E4%B8%9A%E8%AE%BA%E6%96%87_%E7%BB%8F%E8%90%A5%E6%95%88%E7%8E%87%E8%AE%A1%E7%AE%97_20260308.ipynb

注：1、所有代码运行前需把 excel 文件放到桌面上；2、代码中的路径只适用于 mac 系统，windows 系统需要对路径代码进行修改；3、需要把桌面上的文件名称修改至与代码中完全相同。

Kimi 指令：

我现在需要你帮我设计一段 python 代码：

一、现有数据：

我现在有 2 个 excel 文件需要合并。这些文件均位于 mac 桌面上。

第一个 excel 命名为：营业收入.xlsx。第一行为中文标题，从左到右分别为：股票代码、证券简称、会计年度、统计截止日期、报表类型、营业收入（元）。

第二个 excel 命名为：总资产.xlsx。表格的第一行为中文标题，从左到右依次为：股票代码、证券简称、会计年度、统计截止日期、报表类型、资产总计（元）。

二、我的需求

1、任务目标：（1）将两张表格按照（股票代码，会计年度）合并；（2）计算各年度各公司经营效率=营业收入（元）/资产总计（元）（注意预防分母为 0 或空的报错）。

2、具体操作：

（1）读取位于 mac 桌面命名为 营业收入.xlsx 和 总资产.xlsx 的文件。注意，由于股票代码存在左侧数据为 0 的情况，所以一定要始终以字符串的格式储存，保证长度一直为 6 位。

（2）将 营业收入.xlsx 与 总资产.xlsx 按照关键词（股票代码，会计年度）合并。

（3）新建一列“经营效率”，利用公式“经营效率=营业收入（元）/资产总计（元）”，计算每一行对应的（股票代码，会计年度）的经营效率。注意，当分母为 0 或者空值时，直接令该行的经营效率为空值；当某行的营业收入（元）为空值时，视为该公司该年份营业收入金额为 0，令该行的经营效率为 0。

（5）输出结果文件，将第（4）步的结果写入一个 excel 表格，命名为 经营效率_对应的时间戳.xlsx。其中：从左到右每一列的列标题依次为：股票代码、会计年度、自由现金流。请在最上方添加标题行。

注意：（1）鉴于循环会导致代码运行效率低下，你可以给出可行的替代方案优化程序；（2）我希望这个代码可以实时显示运行进度。

请仔细阅读上述文字，并告诉我你是否已经完全理解所有要求。

```
import pandas as pd
import numpy as np
import os
from datetime import datetime
```

```

from tqdm import tqdm

# ===== 配置部分 =====
# Mac 桌面路径（自动适配您的用户名"air"）
desktop_path = os.path.join(os.path.expanduser("~"), "Desktop")

# 输入文件名
file_revenue = os.path.join(desktop_path, "营业收入.xlsx")
file_asset = os.path.join(desktop_path, "总资产.xlsx")

# 检查文件存在性（提高鲁棒性）
if not os.path.exists(file_revenue):
    raise FileNotFoundError(f'未找到文件: {file_revenue}\n 请确保文件位于 Mac 桌面且命名为'营业收入.xlsx'')
if not os.path.exists(file_asset):
    raise FileNotFoundError(f'未找到文件: {file_asset}\n 请确保文件位于 Mac 桌面且命名为'总资产.xlsx'')

# 输出文件名（带时间戳）
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
output_file = os.path.join(desktop_path, f'经营效率_{timestamp}.xlsx')

print("=" * 60)
print("开始处理经营效率计算任务（总资产周转率）")
print(f"当前时间: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")
print("=" * 60)

# ===== 第 1 步：读取数据 =====
print("\n【步骤 1/4】读取 Excel 文件...")

# 读取营业收入表，显示进度
print(f" 正在读取: 营业收入.xlsx")
df_revenue = pd.read_excel(file_revenue, engine='openpyxl')

print(f"  ✓ 读取完成，原表行数: {len(df_revenue)}")

# 列名检查（防止 Excel 列名与预期不符）
expected_rev_cols = ['股票代码', '会计年度', '营业收入（元）']
missing_cols = [col for col in expected_rev_cols if col not in df_revenue.columns]
if missing_cols:
    raise ValueError(f"营业收入表缺少列: {missing_cols}\n 实际列名: {list(df_revenue.columns)}")

# 安全格式化股票代码（鲁棒版：处理数字、字符串、含字母等情况）

```

```

def safe_format_stock_code(x):
    if pd.isna(x):
        return x
    s = str(x).strip()
    # 处理 Excel 浮点数 (如 1.0 → 000001)
    if '.' in s and s.replace('.', '').isdigit():
        s = str(int(float(x)))
    # 移除可能的后缀 (如 .SZ, .SH)
    s = s.split('.')[0]
    # 确保 6 位, 左侧补 0
    return s.zfill(6)

print(" 正在处理股票代码格式 (营业收入表) ...")
tqdm.pandas(desc=" 格式化进度")
df_revenue['股票代码'] = df_revenue['股票代码'].progress_apply(safe_format_stock_code)

# 确保会计年度为字符串且去除小数 (如 2023.0 → 2023)
df_revenue['会计年度'] = df_revenue['会计年度'].apply(
    lambda x: str(int(x)) if pd.notna(x) and isinstance(x, (int, float)) else str(x)
)

# 确保数值列为数值类型 (防止 Excel 读成字符串)
df_revenue['营业收入 (元)'] = pd.to_numeric(df_revenue['营业收入 (元)'], errors='coerce')

print(f" ✓ 营业收入表处理完成, 共 {len(df_revenue)} 行")

# 读取总资产表
print(f"\n 正在读取: 总资产.xlsx")
df_asset = pd.read_excel(file_asset, engine='openpyxl')

print(f" ✓ 读取完成, 原表行数: {len(df_asset)}")

# 列名检查
expected_asset_cols = ['股票代码', '会计年度', '资产总计 (元)']
missing_cols = [col for col in expected_asset_cols if col not in df_asset.columns]
if missing_cols:
    raise ValueError(f"总资产表缺少列: {missing_cols}\n 实际列名: {list(df_asset.columns)}")

print(" 正在处理股票代码格式 (总资产表) ...")
df_asset['股票代码'] = df_asset['股票代码'].progress_apply(safe_format_stock_code)
df_asset['会计年度'] = df_asset['会计年度'].apply(
    lambda x: str(int(x)) if pd.notna(x) and isinstance(x, (int, float)) else str(x)
)

```

```

# 确保资产总计为数值类型
df_asset['资产总计 (元)'] = pd.to_numeric(df_asset['资产总计 (元)'], errors='coerce')

print(f" ✓ 总资产表处理完成, 共 {len(df_asset)} 行")

# ===== 第 2 步: 合并数据 =====
print("\n【步骤 2/4】按 (股票代码, 会计年度) 合并数据...")

rev_cols = ['股票代码', '会计年度', '营业收入 (元)']
asset_cols = ['股票代码', '会计年度', '资产总计 (元)']

print(" 正在执行数据库合并...")
merged_df = pd.merge(
    df_revenue[rev_cols],
    df_asset[asset_cols],
    on=['股票代码', '会计年度'],
    how='left', # 以营业收入表为主表
    indicator=True
)

match_stats = merged_df['_merge'].value_counts()

print(f" ✓ 合并完成: ")

print(f"   - 成功匹配: {match_stats.get('both', 0)} 条")
print(f"   - 未匹配 (仅在营业收入表): {match_stats.get('left_only', 0)} 条")
print(f"   - 总计: {len(merged_df)} 条")

merged_df.drop(columns=['_merge'], inplace=True)

# ===== 第 3 步: 计算经营效率 =====
print("\n【步骤 3/4】计算经营效率 (营业收入/总资产) ...")

rev_col = '营业收入 (元)'
asset_col = '资产总计 (元)'

# 规则 1: 营业收入为空值 → 视为 0
null_rev_count = merged_df[rev_col].isna().sum()
merged_df[rev_col] = merged_df[rev_col].fillna(0)

# 规则 2: 分母有效性检查 (不为 0 且不为空)
denominator = merged_df[asset_col]
valid_denominator = (denominator.notna()) & (denominator != 0)

```

```

invalid_count = len(merged_df) - valid_denominator.sum()

# 向量化计算经营效率
merged_df['经营效率'] = np.where(
    valid_denominator,
    merged_df[rev_col] / denominator,
    np.nan # 分母无效时置空
)

print(f" ✓ 计算完成: ")

print(f" - 原营业收入为空的记录: {null_rev_count} 条 (已视为 0, 经营效率=0) ")
print(f" - 分母为 0 或空值的记录: {invalid_count} 条 (已置空) ")
print(f" - 有效计算记录: {valid_denominator.sum()} 条")

# ===== 第 4 步: 输出结果 =====
print("\n 【步骤 4/4】 生成结果文件...")

# 选择所需列 (列名为"经营效率", 非"自由现金流")
result_df = merged_df[['股票代码', '会计年度', '经营效率']].copy()

# 确保股票代码格式正确 (6 位字符串)
result_df['股票代码'] = result_df['股票代码'].astype(str)

print(f" 正在写入 Excel (共 {len(result_df)} 行) ...")
print(f" 输出路径: {output_file}")

# 使用 openpyxl 引擎写入
with pd.ExcelWriter(output_file, engine='openpyxl') as writer:
    result_df.to_excel(
        writer,
        sheet_name='经营效率',
        index=False,
        header=True
    )

# 调整列宽 (美观)
worksheet = writer.sheets['经营效率']
worksheet.column_dimensions['A'].width = 12 # 股票代码
worksheet.column_dimensions['B'].width = 12 # 会计年度
worksheet.column_dimensions['C'].width = 18 # 经营效率

print(f" ✓ 文件保存成功! ")

```

```

# ===== 完成统计 =====
print("\n" + "=" * 60)
print("处理完成摘要")
print("=" * 60)
print(f"输出文件: 经营效率_{timestamp}.xlsx")
print(f"总行数: {len(result_df):,}")
print(f"有效经营效率记录: {result_df['经营效率'].notna().sum():,}")
print(f"空值记录 (分母异常) : {result_df['经营效率'].isna().sum():,}")

if result_df['经营效率'].notna().sum() > 0:
    print(f"\n 经营效率统计指标: ")
    print(f"  均值: {result_df['经营效率'].mean():.6f}")
    print(f"  中位数: {result_df['经营效率'].median():.6f}")
    print(f"  标准差: {result_df['经营效率'].std():.6f}")
    print(f"  最小值: {result_df['经营效率'].min():.6f}")
    print(f"  最大值: {result_df['经营效率'].max():.6f}")

print("=" * 60)

```

二十三、企业相对融资规模计算代码

http://localhost:8888/notebooks/Desktop/Python/%E6%AF%95%E4%B8%9A%E8%AE%BA%E6%96%87_%E4%BC%81%E4%B8%9A%E7%9B%B8%E5%AF%B9%E8%9E%8D%E8%B5%84%E8%A7%84%E6%A8%A1_20260309.ipynb

注：1、所有代码运行前需把 excel 文件放到桌面上；2、代码中的路径只适用于 mac 系统，windows 系统需要对路径代码进行修改；3、需要把桌面上的文件名称修改至与代码中完全相同。

Kimi 指令：

我现在需要你帮我设计一段 python 代码：

一、现有数据：

我现在有 2 个 excel 文件需要合并。这些文件均位于 mac 桌面上。

第一个 excel 命名为：企业融资规模.xlsx。第一行为中文标题，从左到右分别为：股票代码、证券简称、会计年度、统计截止日期、报表类型、取得借款收到的现金（元）、企业融资规模。

第二个 excel 命名为：总资产.xlsx。表格的第一行为中文标题，从左到右依次为：股票代码、证券简称、会计年度、统计截止日期、报表类型、资产总计（元）。

二、我的需求

1、任务目标：（1）将两张表格按照（股票代码，会计年度）合并；（2）计算各年度各公司企业相对融资规模=取得借款收到的现金（元）/资产总计（元）（注意预防分母为 0 或空的报错）。

2、具体操作：

（1）读取位于 mac 桌面命名为 企业融资规模.xlsx 和 总资产.xlsx 的文件。注意，由于股票代码存在左侧数据为 0 的情况，所以一定要始终以字符串的格式储存，保证长度一直为 6 位。

（2）将 企业融资规模.xlsx 与 总资产.xlsx 按照关键词（股票代码，会计年度）合并。

（3）新建一列“企业相对融资规模”，利用公式“企业相对融资规模=取得借款收到的现金（元）/资产总计（元）”，计算每一行对应的（股票代码，会计年度）的经营效率。注意，当分母为 0 或者空值时，直接令该行的经营效率为空值；当某行的取得借款收到的现金为空值时，视为该公司该年份该指标金额为 0，令该行的企业相对融资规模为 0。

（5）输出结果文件，将第（4）步的结果写入一个 excel 表格，命名为 企业相对融资规模_对应的时间戳.xlsx。其中：从左到右每一列的列标题依次为：股票代码、会计年度、企业相对融资规模。请在最上方添加标题行。

注意：（1）鉴于循环会导致代码运行效率低下，你可以给出可行的替代方案优化程序；（2）我希望这个代码可以实时显示运行进度。

请仔细阅读上述文字，并告诉我你是否已经完全理解所有要求。

```
import pandas as pd
```

```

import numpy as np
import os
from datetime import datetime
from tqdm import tqdm
import time

def main():
    # 设置 Mac 桌面路径（自动适配用户名）
    desktop_path = os.path.join(os.path.expanduser("~"), "Desktop")

    print("="*60)
    print("开始执行：取得借款收到的现金与总资产数据合并")
    print("="*60)

    # 1. 读取 Excel 文件（带进度显示）
    print("\n[步骤 1/5] 读取 '企业融资规模.xlsx'...")
    file1_path = os.path.join(desktop_path, "企业融资规模.xlsx")

    with tqdm(total=100, desc="读取进度", ncols=80) as pbar:
        # 关键：指定股票代码为字符串类型读取，防止前导 0 丢失
        # 修改：同时保留'取得借款收到的现金（元）'列用于后续计算
        df_finance = pd.read_excel(file1_path, dtype={'股票代码': str})
        pbar.update(100)
        time.sleep(0.1)

    print(f"成功读取 {len(df_finance):,} 行数据")
    # 验证必要列是否存在
    required_cols = ['股票代码', '会计年度', '取得借款收到的现金（元）']
    missing_cols = [col for col in required_cols if col not in df_finance.columns]
    if missing_cols:
        print(f"[警告] 发现缺失列: {missing_cols}")

    print("\n[步骤 2/5] 读取 '总资产.xlsx'...")
    file2_path = os.path.join(desktop_path, "总资产.xlsx")

    with tqdm(total=100, desc="读取进度", ncols=80) as pbar:
        df_asset = pd.read_excel(file2_path, dtype={'股票代码': str})
        pbar.update(100)
        time.sleep(0.1)

    print(f"成功读取 {len(df_asset):,} 行数据")

    # 2. 格式化股票代码（确保字符串类型，左侧补 0 至 6 位）
    print("\n[步骤 3/5] 格式化股票代码...")

```

```

with tqdm(total=2, desc="格式化进度", ncols=80) as pbar:
    df_finance['股票代码'] = df_finance['股票代码'].astype(str).str.zfill(6)
    pbar.update(1)
    df_asset['股票代码'] = df_asset['股票代码'].astype(str).str.zfill(6)
    pbar.update(1)
print("          股票代码已统一为 6 位字符串格式 (示例: 000001) ")

# 3. 数据合并 (向量化操作, 按股票代码+会计年度)
print("\n[步骤 4/5] 合并数据表...")
with tqdm(total=100, desc="合并进度", ncols=80) as pbar:
    # 修改: 保留'取得借款收到的现金 (元)'列作为计算分子
    df_merge = pd.merge(
        df_finance[['股票代码', '会计年度', '取得借款收到的现金 (元)']],
        df_asset[['股票代码', '会计年度', '资产总计 (元)']],
        on=['股票代码', '会计年度'],
        how='outer' # 保留所有可能的匹配组合
    )
    pbar.update(100)

print(f"          合并后共 {len(df_merge):,} 行记录")

# 4. 计算企业相对融资规模 (完全向量化, 无循环)
print("\n[步骤 5/5] 计算企业相对融资规模...")
with tqdm(total=3, desc="计算进度", ncols=80) as pbar:

    # 子步骤 1: 处理分子 - 取得借款收到的现金 (元) 空值视为 0
    cash_col = '取得借款收到的现金 (元) '
    df_merge[cash_col] = df_merge[cash_col].fillna(0)
    pbar.update(1)

    # 子步骤 2: 向量化计算 (核心逻辑修改点)
    # 公式: 企业相对融资规模 = 取得借款收到的现金 (元) / 资产总计 (元)
    # 条件判断: 分母为 0 或空值 → 结果为 NaN; 否则计算比率
    asset_col = '资产总计 (元) '

    df_merge['企业相对融资规模'] = np.where(
        (df_merge[asset_col].isna()) | (df_merge[asset_col] == 0),
        np.nan, # 分母异常时赋空值
        df_merge[cash_col] / df_merge[asset_col] # 修改: 使用取得借款收到的现金作
为分子
    )
    pbar.update(1)

    # 子步骤 3: 提取最终输出列 (仅保留要求的 3 列)

```

```

df_result = df_merge[['股票代码', '会计年度', '企业相对融资规模']].copy()
pbar.update(1)

# 5. 生成带时间戳的文件名并导出
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
output_filename = f'企业相对融资规模_{timestamp}.xlsx'
output_path = os.path.join(desktop_path, output_filename)

print(f"\n 正在导出结果至: {output_filename}")
with tqdm(total=100, desc="导出进度", ncols=80) as pbar:
    df_result.to_excel(output_path, index=False, engine='openpyxl')
    pbar.update(100)

# 6. 执行结果统计
valid_count = df_result['企业相对融资规模'].notna().sum()
nan_count = df_result['企业相对融资规模'].isna().sum()
zero_count = (df_result['企业相对融资规模'] == 0).sum()

print("\n" + "="*60)
print("任务执行完成! 统计摘要: ")
print("="*60)
print(f'输出文件: {output_path}')
print(f'总记录数: {len(df_result):,} 行')
print(f'有效计算: {valid_count:,} 行 (分母正常) ')
print(f'空值记录: {nan_count:,} 行 (分母为 0 或空) ')
print(f'零值记录: {zero_count:,} 行 (分子原为空值, 已视为 0) ')
print("="*60)

if __name__ == "__main__":
    main()

```

二十四、并表代码

http://localhost:8888/notebooks/Desktop/Python/%E6%AF%95%E4%B8%9A%E8%AE%BA%E6%96%87_%E5%90%88%E5%B9%B6%E6%95%B0%E6%8D%AE_20260126.ipynb

注：1、所有代码运行前需把 excel 文件放到桌面上；2、代码中的路径只适用于 mac 系统，windows 系统需要对路径代码进行修改；3、需要把桌面上的文件名称修改至与代码中完全相同。

Kimi 指令：

我现在需要你帮我设计一段 python 代码：

一、现有数据：

我现在有 29 个 excel 文件需要合并。这些文件均位于 mac 桌面上命名为 用于合并的表格_20260128 的文件夹中。

29 个 excel 文件的名称和内容分别如下：

1、“1 数据资产信息披露水平.xlsx”。记录了 2013-2024 年 A 股上市公司各年份数据资产信息披露水平。表格第一行为中文标题，从左到右依次为：股票代码、会计年度、总关键词频、年报总词频、数据资产关键词占年报总词频百分比、数据资产信息披露水平、替换解释变量

2、“2 信贷可得性.xlsx”。记录了 2013-2024 年 A 股上市公司各年份信贷可得性衡量指标。表格第一行为中文标题，从左到右依次为：股票代码、会计年度、统计截止日期、总资产（元）、借款总额（元）、信贷可得性

3、“3 企业上市年限.xlsx”。记录了 2013-2024 年 A 股上市公司各年份企业上市年限数据。表格第一行为中文标题，从左到右依次为：股票代码、股票简称、会计年度、统计截止日期、行业名称、行业代码、所属城市、上市年限

4、“4 企业规模.xlsx”。记录了 2013-2024 年 A 股上市公司各年份企业规模数据。表格第一行为中文标题，从左到右依次为：股票代码、会计年度、统计截止日期、总资产（元）、借款总额（元）、信贷可得性、企业规模

5、“5 总资产净利润率.xlsx”。记录了 2013-2024 年 A 股上市公司各年份总资产净利润率数据。表格第一行为中文标题，从左到右依次为：股票代码、股票简称、会计年度、统计截止日期、报表类型编码、公告来源、行业代码、行业名称、行业代码 1、行业名称 1、总资产净利润率(ROA)

6、“6 资产负债率.xlsx”。记录了 2013-2024 年 A 股上市公司各年份资产负债率数据。表格第一行为中文标题，从左到右依次为：股票代码、股票简称、会计年度、统计截止日期、报表类型编码、公告来源、行业代码、行业名称、行业代码 1、行业名称 1、资产负债率

7、“7 账面市值比.xlsx”。记录了 2013-2024 年 A 股上市公司各年份账面市值比数据。表格第一行为中文标题，从左到右依次为：股票代码、股票简称、会计年度、统计截止日期、报表类型编码、公告来源、行业代码、行业名称、行业代码 1、行业名称 1、账面市值比 A、账面市值比 B

8、“8 营业收入增长率.xlsx”。记录了 2013-2024 年 A 股上市公司各年份营业收入增长率数据。表格第一行为中文标题，从左到右依次为：股票代码、证券简称、会计年度、统计截止日期、营业收入（元）、营业收入增长率

9、“9 是否亏损.xlsx”。记录了 2013-2024 年 A 股上市公司各年份是否亏损虚拟变量数据。表格第一行为中文标题，从左到右依次为：股票代码、证券简称、会计年度、统计截止日期、

净利润（元）、是否亏损

10、“10 董事会规模.xlsx”。记录了 2013-2024 年 A 股上市公司各年份董事会规模数据。表格第一行为中文标题，从左到右依次为：股票代码、会计年度、截止日期、董事会人数、董事会规模、独立董事人数、独立董事占比

11、“11 第一大股东持股比例.xlsx”。记录了 2013-2024 年 A 股上市公司各年份第一大股东持股比例数据。表格第一行为中文标题，从左到右依次为：股票代码、会计年度、统计截止日期、股权集中指标 1(%)

12、“12 独立董事占比.xlsx”。记录了 2013-2024 年 A 股上市公司各年份独立董事占比数据。表格第一行为中文标题，从左到右依次为：股票代码、会计年度、截止日期、董事会人数、董事会规模、独立董事人数、独立董事占比

13、“13 两职合一.xlsx”。记录了 2013-2024 年 A 股上市公司各年份管理层两职合一数据。表格第一行为中文标题，从左到右依次为：股票代码、会计年度、统计截止日期、两职合一

14、“14 股权性质.xlsx”。记录了 2013-2024 年 A 股上市公司各年份股权性质数据。表格第一行为中文标题，从左到右依次为：股票代码、会计年度、统计截止日期、股权性质、股权性质编码、股权性质虚拟变量

15、“16 是否四大.xlsx”。记录了 2013-2024 年 A 股上市公司各年份审计师来源是否四大虚拟变量数据。表格第一行为中文标题，从左到右依次为：股票代码、会计年度、统计截止日期、证券简称、审计意见类型、是否四大

16、“18 替换解释变量.xlsx”。记录了 2013-2024 年 A 股上市公司各年份数字资产信息披露水平的替代解释变量数据。表格第一行为中文标题，从左到右依次为：股票代码、会计年度、总关键词频、年报总词频、数据资产关键词占年报总词频百分比、数据资产披露水平、替换解释变量

17、“20 新增借款（按贷款事件统计）.xlsx”。记录了 2013-2024 年 A 股上市公司各年份新增借款（按贷款事件统计）数据。表格第一行为中文标题，从左到右依次为：股票代码、会计年度、新增借款（万元）

18、“21-1 新增信用贷款（按贷款事件统计）.xlsx”。记录了 2013-2024 年 A 股上市公司各年份新增信用贷款（按贷款事件统计）数据。表格第一行为中文标题，从左到右依次为：股票代码、会计年度、新增信用贷款（万元）

19、“21-2 新增质押贷款（按贷款事件统计）.xlsx”。记录了 2013-2024 年 A 股上市公司各年份新增质押贷款（按贷款事件统计）数据。表格第一行为中文标题，从左到右依次为：股票代码、会计年度、新增质押贷款（万元）

20、“21-3 新增保证借款（按贷款事件统计）.xlsx”。记录了 2013-2024 年 A 股上市公司各年份新增保证借款（按贷款事件统计）数据。表格第一行为中文标题，从左到右依次为：股票代码、会计年度、新增保证借款（万元）

21、“21-4 新增抵押贷款（按贷款事件统计）.xlsx”。记录了 2013-2024 年 A 股上市公司各年份新增抵押贷款（按贷款事件统计）数据。表格第一行为中文标题，从左到右依次为：股票代码、会计年度、新增抵押贷款（万元）

22、“21-5 新增担保贷款（按贷款事件统计）.xlsx”。记录了 2013-2024 年 A 股上市公司各年份新增担保贷款（按贷款事件统计）数据。表格第一行为中文标题，从左到右依次为：股票代码、会计年度、新增担保贷款（万元）

23、“21-6 新增其他授信项（按贷款事件统计）.xlsx”。记录了 2013-2024 年 A 股上市公司各年份新增其他授信项（按贷款事件统计）数据。表格第一行为中文标题，从左到右依次为：股票代码、会计年度、新增其他授信项（万元）

24、“22-1 新增短期借款.xlsx”。记录了 2013-2024 年 A 股上市公司各年份新增短期借款数

据。表格第一行为中文标题，从左到右依次为：股票代码、会计年度、统计截止日期、本年新增短期借款合计（万元）

25、“22-2 新增长期借款.xlsx”。记录了 2013-2024 年 A 股上市公司各年份新增长期借款数据。表格第一行为中文标题，从左到右依次为：股票代码、会计年度、统计截止日期、本年新增长期借款合计（万元）

26、“23 分析师跟踪.xlsx”。记录了 2013-2024 年 A 股上市公司各年份分析师跟踪数据。表格第一行为中文标题，从左到右依次为：股票代码、证券简称、会计年度、统计截止日期、被分析师关注度

27、“24 资金流动性.xlsx”。记录了 2013-2024 年 A 股上市公司各年份资金流动性数据。表格第一行为中文标题，从左到右依次为：股票代码、证券简称、会计年度、统计截止日期、报表类型、流动资产合计（元）、流动负债合计（元）、所有者权益合计（元）、营运资金（元）、资金流动性

28、“25 HHI.xlsx”。记录了 2013-2024 年 A 股上市公司各年份 HHI 数据。表格第一行为中文标题，从左到右依次为：股票代码、会计年度、所属城市、HHI

29、“26 高管数字背景.xlsx”。记录了 2013-2024 年 A 股上市公司各年份高管数字背景虚拟变量数据。表格第一行为中文标题，从左到右依次为：股票代码、会计年度、数字背景高管数、高管总数、DigEdu_Executive

二、我的需求

1、任务目标：将以上表格的目标列按照（股票代码、会计年度）两列作为索引进行合并，形成一张完整的用于 STATA 回归的表格

2、逻辑：（1）这 29 张原始数据表格记录的都是 A 股所有上市公司 2013-2024 年的数据，所以需要合并的列共用（股票代码，会计年度）作为行索引，最终得到的合并表格中，各个公司、各个年份分别只有一行数据，这一行除了股票代码、会计年度的 $(29+2+1)=32$ 列分别对应 29 张原始表格中我要求你提取出来的列；（2）由于不同原始数据的统计内容不同，所以缺失行也不同，最终的合并表格中的（股票代码、会计年度）两列，需要统计出在 29 张原始表格中凡是出现过的（股票代码，会计年度）组合。即取不重复的（股票代码，会计年度）的并集作为行索引

3、具体操作：

（1）读取位于 mac 桌面上命名为 用于合并的表格_20260128 的文件夹中所有 29 张 excel 里的数据（excel 命名见现有数据部分）。注意，由于股票代码存在左侧数据为 0 的情况，所以一定要始终以字符串的格式储存，保证长度一直为 6 位。

（2）将 29 张表格中的数据以（股票代码，会计年度）为关键词合并。其中 29 张表格中除（股票代码，会计年度）之外，需要并入最终得到的合并表格中的数据如下：

“1 数据资产信息披露水平.xlsx”：数据资产信息披露水平

“2 信贷可得性.xlsx”：信贷可得性

“3 企业上市年限.xlsx”：行业名称、行业代码、上市年限

“4 企业规模.xlsx”：企业规模

“5 总资产净利润率.xlsx”：总资产净利润率(ROA)A

“6 资产负债率.xlsx”：资产负债率

“7 账面市值比.xlsx”：账面市值比 A、账面市值比 B

“8 营业收入增长率.xlsx”：营业收入增长率

“9 是否亏损.xlsx”：是否亏损

“10 董事会规模.xlsx”: 董事会规模
 “11 第一大股东持股比例.xlsx”: 股权集中指标 1(%)
 “12 独立董事占比.xlsx”: 独立董事占比
 “13 两职合一.xlsx”: 两职合一
 “14 股权性质.xlsx”: 股权性质虚拟变量
 “16 是否四大.xlsx”: 是否四大
 “18 替换解释变量.xlsx”: 替换解释变量
 “20 新增借款 (按贷款事件统计) .xlsx”: 新增借款 (万元)
 “21-1 新增信用贷款 (按贷款事件统计) .xlsx ”: 新增信用贷款 (万元)
 “21-2 新增质押贷款 (按贷款事件统计) .xlsx ”: 新增质押贷款 (万元)
 “21-3 新增保证借款 (按贷款事件统计) .xlsx ”: 新增保证借款 (万元)
 “21-4 新增抵押贷款 (按贷款事件统计) .xlsx ”: 新增抵押贷款 (万元)
 “21-5 新增担保贷款 (按贷款事件统计) .xlsx ”: 、新增担保贷款 (万元)
 “21-6 新增其他授信项 (按贷款事件统计) .xlsx ”: 新增其他授信项 (万元)
 “22-1 新增短期借款.xlsx”: 本年新增短期借款合计 (万元)
 “22-2 新增长期借款.xlsx”: 本年新增长期借款合计 (万元)
 “23 分析师跟踪.xlsx ”: 被分析师关注度
 “24 资金流动性.xlsx ”: 资金流动性
 28、“25 HHI.xlsx”: HHI
 “26 高管数字背景.xlsx”: DigEdu_Executive

(3) 输出结果, 以 excel 的形式保存到 mac 的桌面, 命名为: 合并数据_对应的时间戳.xlsx。其中, 这个 excel 的从左到右每一列的列标题依次为: 股票代码、会计年度、数据资产信息披露水平、信贷可得性、行业名称、行业代码、上市年限、企业规模、总资产净利润率(ROA)A、资产负债率、账面市值比 A、账面市值比 B、营业收入增长率、是否亏损、董事会规模、股权集中指标 1 (%)、独立董事占比、两职合一、股权性质虚拟变量、是否四大、替换解释变量、新增借款 (万元)、新增信用贷款 (万元)、新增质押贷款 (万元)、新增保证借款 (万元)、新增抵押贷款 (万元)、新增担保贷款 (万元)、新增其他授信项 (万元)、本年新增短期借款合计 (万元)、本年新增长期借款合计 (万元)、被分析师关注度、资金流动性、HHI、DigEdu_Executive。并且在表格的最上方添加列标题行。

注意: (1) 鉴于使用循环会导致代码运行效率低下, 你可以给出可行的替代方案优化程序;
 (2) 我希望这个代码可以实时显示运行进度。

```

#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""

```

A 股上市公司面板数据合并工具

将 29 个 Excel 文件按(股票代码, 会计年度)合并为 STATA 回归分析表格

```

"""

```

```

#20260217 合并（新增文件）
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
A 股上市公司面板数据合并工具
将 47 个 Excel 文件按(股票代码, 会计年度)合并为 STATA 回归分析表格
"""

import pandas as pd
import numpy as np
from pathlib import Path
from datetime import datetime
from tqdm import tqdm
import warnings

# 忽略可能出现的性能警告
warnings.filterwarnings('ignore')

class PanelDataMerger:
    """面板数据高效合并器"""

    def __init__(self, folder_path):
        """
        初始化合并器

        Parameters:
        -----
        folder_path : str
            Excel 文件所在文件夹路径（Mac 桌面）
        """
        self.folder = Path(folder_path)
        self.merged_df = None

        # 定义文件与目标列的映射关系（键：文件名，值：需要提取的列列表）
        self.file_mappings = {
            "1 数据资产信息披露水平.xlsx": ["股票代码", "会计年度", "数据资产信
息披露水平"],
            "2 信贷可得性.xlsx": ["股票代码", "会计年度", "信贷可得性"],
            "3 企业上市年限.xlsx": ["股票代码", "会计年度", "行业名称", "行业代码",
"上市年限"],
            "4 企业规模.xlsx": ["股票代码", "会计年度", "企业规模"],
            "5 总资产净利润率.xlsx": ["股票代码", "会计年度", "总资产净利润率
(ROA)A"],
            "6 资产负债率.xlsx": ["股票代码", "会计年度", "资产负债率"],

```

值比 B"]],

"7 账面市值比.xlsx": ["股票代码", "会计年度", "账面市值比 A", "账面市值比 B"]],

"8 营业收入增长率.xlsx": ["股票代码", "会计年度", "营业收入增长率"]],

"9 是否亏损.xlsx": ["股票代码", "会计年度", "是否亏损"]],

"10 董事会规模.xlsx": ["股票代码", "会计年度", "董事会规模"]],

"11 第一大股东持股比例.xlsx": ["股票代码", "会计年度", "股权集中指标 1(%)"]],

"12 独立董事占比.xlsx": ["股票代码", "会计年度", "独立董事占比"]],

"13 两职合一.xlsx": ["股票代码", "会计年度", "两职合一"]],

"14 股权性质.xlsx": ["股票代码", "会计年度", "股权性质虚拟变量"]],

"16 是否四大.xlsx": ["股票代码", "会计年度", "是否四大"]],

"18 替换解释变量.xlsx": ["股票代码", "会计年度", "替换解释变量"]],

"20 新增借款（按贷款事件统计）.xlsx": ["股票代码", "会计年度", "新增借款（万元）"]],

"21-1 新增信用贷款（按贷款事件统计）.xlsx": ["股票代码", "会计年度", "新增信用贷款（万元）"]],

"21-2 新增质押贷款（按贷款事件统计）.xlsx": ["股票代码", "会计年度", "新增质押贷款（万元）"]],

"21-3 新增保证借款（按贷款事件统计）.xlsx": ["股票代码", "会计年度", "新增保证借款（万元）"]],

"21-4 新增抵押贷款（按贷款事件统计）.xlsx": ["股票代码", "会计年度", "新增抵押贷款（万元）"]],

"21-5 新增担保贷款（按贷款事件统计）.xlsx": ["股票代码", "会计年度", "新增担保贷款（万元）"]],

"21-6 新增其他授信项（按贷款事件统计）.xlsx": ["股票代码", "会计年度", "新增其他授信项（万元）"]],

"22-1 新增短期借款.xlsx": ["股票代码", "会计年度", "本年新增短期借款合计（万元）"]],

"22-2 新增长期借款.xlsx": ["股票代码", "会计年度", "本年新增长期借款合计（万元）"]],

"23 分析师跟踪.xlsx": ["股票代码", "会计年度", "被分析师关注度"]],

"24 资金流动性.xlsx": ["股票代码", "会计年度", "资金流动性"]],

"25 HHI.xlsx": ["股票代码", "会计年度", "HHI"]],

"26 高管数字背景.xlsx": ["股票代码", "会计年度", "DigEdu_Executive"]],

"28 涉诉风险_涉诉次数_滞后一期.xlsx": ["股票代码", "会计年度", "涉诉次数"]],

"29 涉诉风险_涉诉金额_滞后一期.xlsx": ["股票代码", "会计年度", "涉诉金额"]],

"30-1 是否受到违规处理_全部违规类型_滞后一期.xlsx": ["股票代码", "会计年度", "是否受到违规处理 1"]],

"30-2 违规处理次数_全部违规类型_滞后一期.xlsx": ["股票代码", "会计年度", "违规处理次数 1"]],

"30-3 是否受到违规处理_只保留披露失实类别_滞后一期.xlsx": ["股票代码", "会计年度", "是否受到违规处理 2"]],

```

        "30-4 违规处理次数_只保留披露失实类别_滞后一期.xlsx": ["股票代码",
"会计年度", "违规处理次数 2"],
        "31-1 内部控制质量_内部控制是否有效_滞后一期.xlsx": ["股票代码", "
会计年度", "内部控制是否有效"],
        "31-2 内部控制质量_内部控制是否存在缺陷_滞后一期.xlsx": ["股票代码
", "会计年度", "内部控制是否存在缺陷"],
        "32-1 关键数字技术_发明专利申请总数_滞后一期.xlsx": ["股票代码", "
会计年度", "关键数字技术_发明专利申请总数"],
        "32-2 关键数字技术_发明专利获得总数_滞后一期.xlsx": ["股票代码", "
会计年度", "关键数字技术_发明专利获得总数"],
        "32-3 关键数字技术_实用新型专利申请总数_滞后一期.xlsx": ["股票代码
", "会计年度", "关键数字技术_实用新型专利申请总数"],
        "32-4 关键数字技术_实用新型专利获得总数_滞后一期.xlsx": ["股票代码
", "会计年度", "关键数字技术_实用新型专利获得总数"],
        "34-1 扣非后净利润.xlsx": ["股票代码", "会计年度", "归属于上市公司股
东的扣除非经常性损益的净利润"],
        "34-2 扣非后净利润_滞后一期.xlsx": ["股票代码", "会计年度", "归属于上
市公司股东的扣除非经常性损益的净利润（滞后一期）"],
        "35-1 盈余管理_真实盈余管理_滞后一期.xlsx": ["股票代码", "会计年度",
"真实盈余管理"],
        "35-2 盈余管理_DD 模型_滞后一期.xlsx": ["股票代码", "会计年度", "DD
模型残差绝对值"],
        "35-3 盈余管理_修正 Jones 模型_滞后一期.xlsx": ["股票代码", "会计年度",
"操控性应计利润的绝对值（元）"],
        "35-4 盈余管理_非线性应计模型_滞后一期.xlsx": ["股票代码", "会计年
度", "非线性应计模型残差绝对值"]
    }

```

```

    print(f"    初始化完成")

    print(f"    目标文件夹: {self.folder}")

    print(f"    待处理文件数: {len(self.file_mappings)}\n")

    def _load_and_prepare(self, file_name, columns):
        """
        加载单个 Excel 文件并进行预处理

        Returns:
        -----
        DataFrame: 已设置索引的 DataFrame
        """
        try:

```

```

file_path = self.folder / file_name

# 只读取需要的列，节省内存
df = pd.read_excel(file_path, usecols=columns)

# 关键：确保股票代码为 6 位字符串格式（补零）
df['股票代码'] = df['股票代码'].astype(str).str.zfill(6)

# 确保会计年度为整数类型
df['会计年度'] = pd.to_numeric(df['会计年度'],
errors='coerce').astype('Int64')

# 立即设置索引，为后续合并做准备
df.set_index(['股票代码', '会计年度'], inplace=True)

return df

except Exception as e:

    print(f"    加载失败 {file_name}: {str(e)}")

    return pd.DataFrame()

def merge_all(self):
    """核心合并逻辑 - 高性能向量化操作"""

    print("    开始加载并合并 47 个数据文件...\n")

    dataframes = []

    # 使用 tqdm 显示文件加载进度
    with tqdm(total=len(self.file_mappings),

              desc="    加载进度",

              unit="文件",

              ncols=100,

              bar_format='{l_bar} {bar}|'                                {n_fmt}/{total_fmt}
[ {elapsed}<{remaining} ]) as pbar:

        for file_name, columns in self.file_mappings.items():
            df = self._load_and_prepare(file_name, columns)

            if not df.empty:
                dataframes.append(df)

```

```

        # 更新进度条并显示当前文件名
        pbar.update(1)
        pbar.set_postfix_str(f"当前: {file_name[:20]}")

if not dataframes:

    print("    未成功加载任何数据文件! ")

    return

print("\n    执行高性能合并（使用 outer 连接取并集）...")

# 关键优化：使用 pd.concat 一次性合并所有数据框（向量化）
# 这比循环 merge 快 10 倍以上，且内存效率更高
combined_df = pd.concat(
    dataframes,
    axis=1,          # 水平方向合并（增加列）
    join='outer',    # 取并集（保留所有股票代码-年度组合）
    copy=False       # 避免不必要的数据复制，节省内存
)

# 重置索引为普通列
self.merged_df = combined_df.reset_index()

# 按指定顺序重排列
self._reorder_columns()

# 显示合并统计信息
self._show_merge_stats()

def _reorder_columns(self):
    """按 STATA 格式要求严格重排列顺序"""
    desired_order = [
        '股票代码', '会计年度', '数据资产信息披露水平', '信贷可得性', '行业名称', '
行业代码',
        '上市年限', '企业规模', '总资产净利润率(ROA)A', '资产负债率', '账面市值
比 A',
        '账面市值比 B', '营业收入增长率', '是否亏损', '董事会规模', '股权集中指标
1(%)',
        '独立董事占比', '两职合一', '股权性质虚拟变量', '是否四大', '替换解释变
量',
        '新增借款（万元）', '新增信用贷款（万元）', '新增质押贷款（万元）',
        '新增保证借款（万元）', '新增抵押贷款（万元）', '新增担保贷款（万元）'
    ]

```

```

    '新增其他授信项（万元）','本年新增短期借款合计（万元）',
    '本年新增长期借款合计（万元）','被分析师关注度','资金流动性','HHI',
    'DigEdu_Executive', "涉诉次数", "涉诉金额","是否受到违规处理 1",
    "违规处理次数 1","是否受到违规处理 2","违规处理次数 2","内部控制是
    否有效","内部控制是否存在缺陷",
    "关键数字技术_发明专利申请总数","关键数字技术_发明专利获得总数",
    "关键数字技术_实用新型专利申请总数",
    "关键数字技术_实用新型专利获得总数","归属于上市公司股东的扣除非
    经常性损益的净利润",
    "归属于上市公司股东的扣除非经常性损益的净利润（滞后一期）","真实
    盈余管理",
    "DD 模型残差绝对值","操控性应计利润的绝对值（元）","非线性应计模
    型残差绝对值"
]

```

```

# 只保留实际存在的列
existing_cols = [col for col in desired_order if col in self.merged_df.columns]
self.merged_df = self.merged_df[existing_cols]

```

```

print(f"    列顺序已调整，共 {len(existing_cols)} 列")

```

```

def _show_merge_stats(self):
    """显示合并统计信息"""
    if self.merged_df is not None:
        total_combinations = len(self.merged_df)
        unique_stocks = self.merged_df['股票代码'].nunique()
        year_min = self.merged_df['会计年度'].min()
        year_max = self.merged_df['会计年度'].max()

        print("\n" + "=" * 60)

        print("    合并统计信息")

        print("=" * 60)
        print(f"总行数: {total_combinations},")
        print(f"公司数量: {unique_stocks}")
        print(f"年份跨度: {year_min} - {year_max}")
        print(f"数据维度: {self.merged_df.shape[0]} 行 × {self.merged_df.shape[1]}
        列")

        print("=" * 60)

```

```

def save_to_desktop(self):
    """保存文件到 Mac 桌面"""

```

```

if self.merged_df is None or self.merged_df.empty:

    print("    没有可保存的数据! ")

    return

# 生成时间戳文件名
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
output_name = f"合并数据_{timestamp}.xlsx"
output_path = Path.home() / "Desktop" / output_name

print("\n    正在保存 Excel 文件...")

# 使用 tqdm 显示保存进度
with tqdm(total=1, desc="保存进度", unit="文件") as pbar:
    self.merged_df.to_excel(
        output_path,
        index=False,
        engine='openpyxl',
        sheet_name='合并数据'
    )
    pbar.update(1)

print(f"\n    文件已保存至桌面: {output_name}")

print(f"    完整路径: {output_path}")

return output_path

def main():
    """主函数 - 一键执行"""
    print("=" * 70)

    print("    A 股上市公司面板数据合并工具")
    print("=" * 70)

    # Mac 桌面路径配置
    folder_name = "用于合并的表格_20260217"
    desktop = Path.home() / "Desktop"
    folder_path = desktop / folder_name

    # 检查文件夹是否存在
    if not folder_path.exists():

        print(f"    错误: 桌面上找不到文件夹 '{folder_name}'")

```

```

print(f'请确认文件夹名称是否正确并重新运行')
return

try:
    # 创建合并器实例
    merger = PanelDataMerger(folder_path)

    # 执行高效合并
    merger.merge_all()

    # 保存结果
    output_path = merger.save_to_desktop()

    if output_path:
        print("\n" + "=" * 70)

        print("    合并任务成功完成! ")

        print("=" * 70)

        print(f"    现在您可以用 STATA 打开此文件进行回归分析了")

        print("=" * 70)

    except Exception as e:

        print(f"\n    程序执行出错: {str(e)}")

        import traceback
        traceback.print_exc()

if __name__ == "__main__":
    main()

```

报错;

🔗 执行高性能合并（使用outer连接取并集）...

✗ 程序执行出错: Reindexing only valid with uniquely valued Index objects

```
Traceback (most recent call last):
  File "/var/folders/hs/3j1t8r551rd8v070r98ybjc0000gn/T/ipykernel_24508/1854013938.py", line 232, in main
    merger.merge_all()
  File "/var/folders/hs/3j1t8r551rd8v070r98ybjc0000gn/T/ipykernel_24508/1854013938.py", line 130, in merge_all
    combined_df = pd.concat(
  File "/Users/air/opt/anaconda3/lib/python3.9/site-packages/pandas/util/_decorators.py", line 311, in wrapper
    return func(*args, **kwargs)
  File "/Users/air/opt/anaconda3/lib/python3.9/site-packages/pandas/core/reshape/concat.py", line 360, in concat
    return op.get_result()
  File "/Users/air/opt/anaconda3/lib/python3.9/site-packages/pandas/core/reshape/concat.py", line 591, in get_result
    indexers[ax] = obj_labels.get_indexer(new_labels)
  File "/Users/air/opt/anaconda3/lib/python3.9/site-packages/pandas/core/indexes/base.py", line 3729, in get_indexer
    raise InvalidIndexError(self._requires_unique_msg)
pandas.errors.InvalidIndexError: Reindexing only valid with uniquely valued Index objects
```

原因：部分表格出现重复（股票代码，会计年度）索引

重复行检测代码：

```
#!/usr/bin/env python3
```

```
# -*- coding: utf-8 -*-
```

```
"""
```

A 股数据重复索引诊断工具 v2.1

功能：

1. 实时显示扫描进度（29 个文件）
2. 精确检测重复索引问题
3. 生成详细检测报告

```
"""
```

```
import pandas as pd
```

```
from pathlib import Path
```

```
from collections import Counter
```

```
from datetime import datetime
```

```
from tqdm import tqdm # 导入进度条库
```

```
import warnings
```

```
warnings.filterwarnings('ignore')
```

```
class DuplicateIndexDetector:
```

```
    """重复索引检测器（带进度显示）"""
```

```
    def __init__(self, folder_path):
```

```
        self.folder = Path(folder_path)
```

```
        self.file_mappings = {
```

```
            "1 数据资产信息披露水平.xlsx": ["股票代码", "会计年度", "数据资产信息披露水平"],
```

```
            "2 信贷可得性.xlsx": ["股票代码", "会计年度", "信贷可得性"],
```

```
            "3 企业上市年限.xlsx": ["股票代码", "会计年度", "行业名称", "行业代码", "上
```

```

市年限"],
    "4 企业规模.xlsx": ["股票代码", "会计年度", "企业规模"],
    "5 总资产净利润率.xlsx": ["股票代码", "会计年度", "总资产净利润率(ROA)A"],
    "6 资产负债率.xlsx": ["股票代码", "会计年度", "资产负债率"],
    "7 账面市值比.xlsx": ["股票代码", "会计年度", "账面市值比 A", "账面市值比 B"],
    "8 营业收入增长率.xlsx": ["股票代码", "会计年度", "营业收入增长率"],
    "9 是否亏损.xlsx": ["股票代码", "会计年度", "是否亏损"],
    "10 董事会规模.xlsx": ["股票代码", "会计年度", "董事会规模"],
    "11 第一大股东持股比例.xlsx": ["股票代码", "会计年度", "股权集中指标 1(%)"],
    "12 独立董事占比.xlsx": ["股票代码", "会计年度", "独立董事占比"],
    "13 两职合一.xlsx": ["股票代码", "会计年度", "两职合一"],
    "14 股权性质.xlsx": ["股票代码", "会计年度", "股权性质虚拟变量"],
    "16 是否四大.xlsx": ["股票代码", "会计年度", "是否四大"],
    "18 替换解释变量.xlsx": ["股票代码", "会计年度", "替换解释变量"],
    "20 新增借款（按贷款事件统计）.xlsx": ["股票代码", "会计年度", "新增借款 (万元) "],
    "21-1 新增信用贷款（按贷款事件统计）.xlsx": ["股票代码", "会计年度", "新增信用贷款 (万元) "],
    "21-2 新增质押贷款（按贷款事件统计）.xlsx": ["股票代码", "会计年度", "新增质押贷款 (万元) "],
    "21-3 新增保证借款（按贷款事件统计）.xlsx": ["股票代码", "会计年度", "新增保证借款 (万元) "],
    "21-4 新增抵押贷款（按贷款事件统计）.xlsx": ["股票代码", "会计年度", "新增抵押贷款 (万元) "],
    "21-5 新增担保贷款（按贷款事件统计）.xlsx": ["股票代码", "会计年度", "新增担保贷款 (万元) "],
    "21-6 新增其他授信项（按贷款事件统计）.xlsx": ["股票代码", "会计年度", "新增其他授信项 (万元) "],
    "22-1 新增短期借款.xlsx": ["股票代码", "会计年度", "本年新增短期借款合计 (万元) "],
    "22-2 新增长期借款.xlsx": ["股票代码", "会计年度", "本年新增长期借款合计 (万元) "],
    "23 分析师跟踪.xlsx": ["股票代码", "会计年度", "被分析师关注度"],
    "24 资金流动性.xlsx": ["股票代码", "会计年度", "资金流动性"],
    "25 HHI.xlsx": ["股票代码", "会计年度", "HHI"],
    "26 高管数字背景.xlsx": ["股票代码", "会计年度", "DigEdu_Executive"]
}

def analyze_all_files(self):
    """分析所有文件的重复情况（带进度条）"""
    print("=" * 80)

```

```

print("    开始深度扫描重复索引...")

print("=" * 80)

print(f"    扫描目标: {len(self.file_mappings)} 个 Excel 文件\n")

problem_files = []

# ===== 核心优化: 实时进度显示 =====
# 使用 tqdm 包装迭代器, 实时显示扫描进度
with tqdm(
    total=len(self.file_mappings),

    desc="    扫描进度", # 进度条左侧描述

    unit="文件",         # 单位
    ncols=100,           # 进度条宽度
    bar_format='{l_bar} {bar}|    {n_fmt}/{total_fmt}    [{elapsed}<{remaining},
{rate_fmt}]',
    leave=True
) as pbar:

    for file_name, columns in self.file_mappings.items():
        file_path = self.folder / file_name

        if not file_path.exists():

            print(f"\n    文件不存在: {file_name}")

            pbar.update(1)
            continue

        # 检测重复
        duplicates = self._check_file_duplicates(file_path, columns, file_name)

        if duplicates:
            problem_files.append({
                'file': file_name,
                'duplicates': duplicates
            })

        # 更新进度条并显示当前文件名 (缩短显示)
        pbar.update(1)
        pbar.set_postfix_str(f"当前: {file_name[:15]}...")

# 生成最终报告

```

```

self._generate_report(problem_files)

def _check_file_duplicates(self, file_path, columns, file_name):
    """检测单个文件的重复索引"""
    # 加载数据
    df = pd.read_excel(file_path, usecols=columns)

    # 预处理
    df['股票代码'] = df['股票代码'].astype(str).str.zfill(6)
    df['会计年度'] = pd.to_numeric(df['会计年度'], errors='coerce').astype('Int64')

    # 创建组合键用于统计
    df['组合键'] = df['股票代码'].astype(str) + '_' + df['会计年度'].astype(str)

    # 统计每个组合出现的次数
    counter = Counter(df['组合键'])

    # 找出出现次数>1 的组合
    duplicate_keys = {key: count for key, count in counter.items() if count > 1}

    if not duplicate_keys:
        return None

    # 构建详细报告
    duplicate_details = []
    for key, count in duplicate_keys.items():
        stock_code, year = key.split('_')
        duplicate_rows = df[df['组合键'] == key]

        # 获取重复行的完整信息
        duplicate_details.append({
            '股票代码': stock_code,
            '会计年度': int(year),
            '重复次数': count,
            '行号': duplicate_rows.index.tolist(),
            '所有行数据': duplicate_rows.to_dict('records')
        })

    return duplicate_details

def _generate_report(self, problem_files):
    """生成重复项详细报告"""
    print("\n" + "!" * 80)

    print("  重复索引检测报告")

```

```

print("!" * 80)

if not problem_files:

    print("    恭喜! 所有文件索引唯一, 未发现重复项")

    print("=" * 80)
    return

print(f"    发现 {len(problem_files)} 个文件存在重复索引问题\n")

# 按重复组合数量排序显示
problem_files_sorted = sorted(problem_files, key=lambda x: len(x['duplicates']),
reverse=True)

for idx, problem in enumerate(problem_files_sorted, 1):
    file_name = problem['file']
    duplicates = problem['duplicates']

    print("-" * 80)

    print(f"    问题文件 #{idx}: {file_name}")

    print(f"    受影响组合数: {len(duplicates)} 个")
    print("-" * 80)

    # 显示前 5 个重复项
    for dup in duplicates[:5]:

        print(f"        股票代码: {dup['股票代码']} | 会计年度: {dup['会计年度']} |
重复次数: {dup['重复次数']}")
        print(f"        数据行号: {dup['行号']}")

        # 显示所有重复行的数据对比
        for i, row_data in enumerate(dup['所有行数据']):
            print(f"            行 {dup['行号'][i]}: {row_data}")
        print()

    if len(duplicates) > 5:
        print(f"    ... 还有 {len(duplicates) - 5} 个重复组合未显示\n")

    # 保存 CSV 报告
    self._save_duplicate_details(file_name, duplicates)

print("=" * 80)

```

```

print("    重复项详细信息已保存至桌面 CSV 文件")

self._print_recommendations()

def _save_duplicate_details(self, file_name, duplicates):
    """保存重复详情到 CSV"""
    desktop = Path.home() / "Desktop"
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    output_file = desktop / f'重复项详情_{file_name.replace('.xlsx','')}_ {timestamp}.csv'

    # 转换为 DataFrame 并展开所有行数据
    rows = []
    for dup in duplicates:
        for row_data in dup['所有行数据']:
            row_data['重复标识'] = f'{dup['股票代码']}_ {dup['会计年度']}'
            row_data['重复次数'] = dup['重复次数']
            rows.append(row_data)

    duplicate_df = pd.DataFrame(rows)
    duplicate_df.to_csv(output_file, index=False, encoding='utf-8-sig')

    print(f"        详细信息已保存: {output_file.name}")

def _print_recommendations(self):
    """打印处理建议"""
    print("\n" + "=" * 80)

    print("    处理建议")

    print("=" * 80)
    print("1. 数据清洗: 检查原始数据源, 删除重复录入的行")
    print("2. 数据验证: 确认重复行是数据错误还是需要聚合计算")
    print("3. 处理策略: ")
    print("    - 使用 df.drop_duplicates(keep='last') 保留最后记录")
    print("    - 使用 df.groupby(['股票代码','会计年度']).mean() 聚合计算")
    print("=" * 80)

def main():
    """主函数"""

    print("\n    A 股数据重复索引诊断工具 v2.1")

    print("    功能: 实时扫描 + 精确诊断")
    print("    时间: 预计 30-60 秒\n")

```

```

# Mac 桌面路径配置
folder_name = "用于合并的表格_20260125"
desktop = Path.home() / "Desktop"
folder_path = desktop / folder_name

if not folder_path.exists():

    print(f"    错误: 桌面上找不到文件夹 '{folder_name}')"

    return

detector = DuplicateIndexDetector(folder_path)
detector.analyze_all_files()

if __name__ == "__main__":
    main()

```

运行结果:

```

.....
📊 重复索引检测报告
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
⚠️ 发现 3 个文件存在重复索引问题

-----
🔴 问题文件 #1: 5 总资产净利润率.xlsx
受影响组合数: 46374 个

-----
📌 股票代码: 000002 | 会计年度: 2024 | 重复次数: 4
数据行号: [0, 1, 2, 3]
行 0: {'股票代码': '000002', '会计年度': 2024, '总资产净利润率(ROA)B': -0.034899, '组合键':
'000002_2024'}
行 1: {'股票代码': '000002', '会计年度': 2024, '总资产净利润率(ROA)B': -0.011459, '组合键':
'000002_2024'}
行 2: {'股票代码': '000002', '会计年度': 2024, '总资产净利润率(ROA)B': -0.005821, '组合键':
'000002_2024'}
行 3: {'股票代码': '000002', '会计年度': 2024, '总资产净利润率(ROA)B': 0.000223, '组合键':
'000002_2024'}

```

已人工对三个重复索引表格进行检查,发现一、二、三季度数据未删除,导致同一会计年度出现 4 条数据

已手动删除除 12-31 日的季度数据,重新运行并表代码得到合并数据表格

20260126 并表之后的三个问题:

- 1、数据资产信息披露程度样本选取与其他数据(剔除证券行业和 ST)不同。
解决方案:需要一张截至 20241231 的证券业和 ST、*ST 股票代码列表,将对应的样本行从合并表格中剔除;
- 2、剔除人工错误补齐的行数。
解决方案:用原始数据提取出所有原始数据齐全的行股票代码和会计年度形成一张匹配表,合并表格对照保留原始数据齐全的行,其余统计过程中补齐的行删除。
或:修改代码,删除补齐行数的步骤重新运行。
- 3、数据资产信息披露水平的缺失值处理办法待确定,行业平均水平暂时无法计算。

20260128 实际解决方案：

1、从 csmar 公司研究系列导出 2013-2024 所有上市公司的证券代码、证券简称、统计截止日期和报表类型（为和其他数据保持一致，报表类型均选用 A，即合并报表）

The screenshot displays the CSMAR data query interface. The top navigation bar includes 'CSMAR', '首页', '数据中心', '数据超市', '数据应用', '智能应用', '服务与支持', and a search bar. The left sidebar shows a tree structure for '财务报表' (Financial Statements) with options like '资产负债表' (Balance Sheet), '利润表' (Income Statement), '现金流量表' (Cash Flow Statement), etc. The main content area is titled '资产负债表 - 资产负债表' and shows a list of financial statements. The '时间设置' (Time Settings) section allows selecting a time range from '2013-01-01' to '2024-12-31'. The '代码设置' (Code Settings) section includes options for '代码选择' (Code Selection), '代码导入' (Code Import), and '常用代码' (Common Codes). The '字段设置' (Field Settings) section shows a list of fields to be included in the query, such as '证券代码' (Security Code), '证券简称' (Security Name), '统计截止日期' (Statistical End Date), and '报表类型' (Report Type).

手动删除 B 类报表类型数据后，将原数据资产信息披露水平中除该剔除证券业、ST 之后的 A 股上市公司列表之外的所有行，得到剔除证券业、ST 之后的 A 股上市公司数据资产信息披露水平数据。

代码已补充至本文件第一章。

2、已对对使用贷款事件进行统计的新增借款相关数据（即 20、21-1、21-2、21-3、21-4、21-5、21-6、22-1、22-2 表格）重新处理。

处理代码已补充至本文件第六至八章。

3、待处理。

20250130 新增步骤-对新增借款列补 0

对象：

20-新增借款；21-新增不同种类借款；22-新增不同种类借款

我现在需要你帮我设计一段 python 代码：

一、现有数据：

一张位于 mac 桌面上命名为 **合并数据_20260217_181800.xlsx** 的 excel 表格,统计了 2013-2024 年剔除证券业和 ST 的 A 股上市数据。第一行为中文标题，从左到右分别为：股票代码、会计年度、数据资产信息披露水平、信贷可得性、行业名称、行业代码、上市年限、企业规模、总资产净利润率(ROA)A、资产负债率、账面市值比 A、账面市值比 B、营业收入增长率、是否亏损、董事会规模、股权集中指标 1 (%)、独立董事占比、两职合一、股权性质虚拟变量、是否四大、替换解释变量、新增借款（万元）、新增信用贷款（万元）、新增质押贷款（万元）、新增保证借款（万元）、新增抵押贷款（万元）、新增担保贷款（万元）、新增其他授信项（万元）、本年新增短期借款合计（万元）、本年新增长期借款合计（万元）、被分析师关注度、资金流动性、HHI、DigEdu_Executive。

二、我的需求

1、任务目标：将所有和新增借款相关列的所有缺失值填补为 0；

2、逻辑：凡是新增借款（万元）、新增信用贷款（万元）、新增质押贷款（万元）、新增保证借款（万元）、新增抵押贷款（万元）、新增担保贷款（万元）、新增其他授信项（万元）、本年新增短期借款合计（万元）、本年新增长期借款合计（万元）共 9 列中 1 列及以上存在数据的行，说明该上市公司该年存在某个种类（期限）的新增借款，该公司该年其他种类的新增借款金额为 0。凡是新增借款（万元）、新增信用贷款（万元）、新增质押贷款（万元）、新增保证借款（万元）、新增抵押贷款（万元）、新增担保贷款（万元）、新增其他授信项（万元）、本年新增短期借款合计（万元）、本年新增长期借款合计（万元）9 列全部为空值的行，说明该公司在该年度没有新增借款，即该行这 9 列的所有值应为 0。

3、具体步骤：

（1）读取位于 mac 桌面命名为 **合并数据_20260217_181800.xlsx** 的文件。注意，由于股票代码存在左侧数据为 0 的情况，所以一定要始终以字符串的格式储存，保证长度一直为 6 位。

（2）新增借款缺失值填补：对该表格中除了标题行外的所有行进行遍历，对每一行中新增借款（万元）、新增信用贷款（万元）、新增质押贷款（万元）、新增保证借款（万元）、新增抵押贷款（万元）、新增担保贷款（万元）、新增其他授信项（万元）、本年新增短期借款合计（万元）、本年新增长期借款合计（万元）这 10 列中凡是数值为空值的，填补为数值 0。

（3）输出结果：将填补借款相关缺失值之后所有数据的以 excel 的形式保存到 mac 的桌面，命名为：合并数据（新增借款缺失值处理）_对应的时间戳.xlsx。其中，这个 excel 的从左到

右每一列的列标题依次为：股票代码、会计年度、数据资产信息披露水平、信贷可得性、行业名称、行业代码、上市年限、企业规模、总资产净利润率(ROA)A、资产负债率、账面市值比 A、账面市值比 B、营业收入增长率、是否亏损、董事会规模、股权集中指标 1 (%)、独立董事占比、两职合一、股权性质虚拟变量、是否四大、替换解释变量、新增借款 (万元)、新增信用贷款 (万元)、新增质押贷款 (万元)、新增保证借款 (万元)、新增抵押贷款 (万元)、新增担保贷款 (万元)、新增其他授信项 (万元)、本年新增短期借款合计 (万元)、本年新增长期借款合计 (万元)、被分析师关注度、资金流动性、HHI、DigEdu_Executive。并且在表格的最上方添加列标题行。

注意：（1）鉴于使用循环会导致代码运行效率低下，你可以给出可行的替代方案优化程序；（2）我希望这个代码可以实时显示运行进度。

请仔细阅读上述文字，并告诉我你是否已经完全理解所有要求。

```
import pandas as pd
import os
from datetime import datetime
from tqdm import tqdm
import sys

def main():
    # ===== 1. 路径设置 =====
    desktop_path = os.path.join(os.path.expanduser("~"), "Desktop")
    input_file = os.path.join(desktop_path, "合并数据_20260217_181800.xlsx")

    if not os.path.exists(input_file):
        raise FileNotFoundError(f"未找到文件：{input_file}\n 请确认文件已保存在 Mac 桌面")

    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    output_file = os.path.join(desktop_path, f"合并数据（缺失值处理）_{timestamp}.xlsx")

    print(" 开始执行缺失值填补...")

    # ===== 2. 读取数据 =====

    print("\n 步骤 1/3: 读取原始数据...")

    # 强制股票代码为字符串，防止前导零丢失
    df = pd.read_excel(input_file, dtype={'股票代码': str})
```

```

# 确保股票代码为 6 位字符串（补前导零）
df['股票代码'] = df['股票代码'].astype(str).str.strip().str.zfill(6)

print(f"    ✓ 成功读取 {len(df):,} 行 × {len(df.columns)} 列数据")

print(f"    ✓ 数据时间跨度: {df['会计年度'].min()} - {df['会计年度'].max()}")

# ===== 3. 定义需要处理的 9 列借款相关列
=====
loan_columns = [
    '新增借款（万元）',
    '新增信用贷款（万元）',
    '新增质押贷款（万元）',
    '新增保证借款（万元）',
    '新增抵押贷款（万元）',
    '新增担保贷款（万元）',
    '新增其他授信项（万元）',
    '本年新增短期借款合计（万元）',
    '本年新增长期借款合计（万元）',
    '涉诉次数',
    '涉诉金额',
    '是否受到违规处理 1',
    '违规处理次数 1',
    '是否受到违规处理 2',
    '违规处理次数 2',
    '关键数字技术_发明专利申请总数',
    '关键数字技术_发明专利获得总数',
    '关键数字技术_实用新型专利申请总数',
    '关键数字技术_实用新型专利获得总数'
]

# 检查列名是否存在（严格匹配）
missing_loan_cols = [col for col in loan_columns if col not in df.columns]
if missing_loan_cols:

    print(f"    警告：以下借款列在表中未找到：{missing_loan_cols}")

    loan_columns = [col for col in loan_columns if col in df.columns]

print(f"    待填补缺失值的借款相关列：{len(loan_columns)} 列")

# ===== 4. 缺失值填补（向量化处理）
=====

```

```

print("\n    步骤 2/3: 执行缺失值填补（仅针对新增借款、违规处理、数字技术专利数
据）...")

filled_count = 0
for col in tqdm(loan_columns, desc="    处理进度", unit="列"):
    # 统计该列缺失值数量
    na_before = df[col].isna().sum()

    # 向量化填补：转换为数值类型（无法转换的设为 NaN），然后填补 0
    # 这会处理真正的缺失值(NaN)以及可能的空字符串或无效文本
    df[col] = pd.to_numeric(df[col], errors='coerce').fillna(0)

    if na_before > 0:
        filled_count += na_before

print(f"    ✓ 总计填补 {filled_count:,} 个缺失值单元格")

print(f"    ✓ 其余 {len(df.columns) - len(loan_columns)} 列数据保持原样未修改")

# ===== 5. 严格按指定顺序输出全部 34 列 =====

print("\n    步骤 3/3: 整理并保存完整数据（严格按指定列顺序）...")

# 用户指定的列顺序（从左到右）
target_columns = [
    '股票代码', '会计年度', '数据资产信息披露水平', '信贷可得性', '行业名称', '行业代码',
    '上市年限', '企业规模', '总资产净利润率(ROA)A', '资产负债率', '账面市值比 A',
    '账面市值比 B', '营业收入增长率', '是否亏损', '董事会规模', '股权集中指标 1(%)',
    '独立董事占比', '两职合一', '股权性质虚拟变量', '是否四大', '替换解释变量',
    '新增借款（万元）', '新增信用贷款（万元）', '新增质押贷款（万元）',
    '新增保证借款（万元）', '新增抵押贷款（万元）', '新增担保贷款（万元）',
    '新增其他授信项（万元）', '本年新增短期借款合计（万元）',
    '本年新增长期借款合计（万元）', '被分析师关注度', '资金流动性', 'HHI',
    'DigEdu_Executive', "涉诉次数", "涉诉金额", "是否受到违规处理 1",
    "违规处理次数 1", "是否受到违规处理 2", "违规处理次数 2", "内部控制是否有效",
    "内部控制是否存在缺陷",
    "关键数字技术_发明专利申请总数", "关键数字技术_发明专利获得总数", "关键数
字技术_实用新型专利申请总数",
    "关键数字技术_实用新型专利获得总数", "归属于上市公司股东的扣除非经常性
损益的净利润",

```

```

        "归属于上市公司股东的扣除非经常性损益的净利润（滞后一期）", "真实盈余管理",
    ],
    "DD 模型残差绝对值", "操控性应计利润的绝对值（元）", "非线性应计模型残差绝对值"
]

```

```

# 检查实际数据中存在的列与目标列的差异

```

```

actual_cols = set(df.columns)

```

```

target_cols_set = set(target_columns)

```

```

missing_in_file = target_cols_set - actual_cols

```

```

extra_in_file = actual_cols - target_cols_set

```

```

if missing_in_file:

```

```

    print(f"        输入文件中缺失以下指定列： {sorted(missing_in_file)}")

```

```

if extra_in_file:

```

```

    print(f"        输入文件中存在额外列（将置于最后）： {sorted(extra_in_file)}")

```

```

# 按目标顺序排列，存在的列保留，缺失的列忽略，额外的列置于最后

```

```

existing_target_cols = [col for col in target_columns if col in df.columns]

```

```

remaining_cols = [col for col in df.columns if col not in target_columns]

```

```

final_column_order = existing_target_cols + remaining_cols

```

```

df_output = df[final_column_order]

```

```

print(f"    正在保存至: {output_file}")

```

```

print(f"    输出数据维度: {len(df_output)} 行 × {len(final_column_order)} 列")

```

```

# 保存为 Excel，确保股票代码为文本格式

```

```

with pd.ExcelWriter(output_file, engine='openpyxl') as writer:

```

```

    df_output.to_excel(writer, index=False, sheet_name='Sheet1')

```

```

    # 设置股票代码列（A 列）为文本格式，防止 Excel 吃掉前导零

```

```

    worksheet = writer.sheets['Sheet1']

```

```

    for cell in worksheet['A']:

```

```

        if cell.row > 1: # 跳过标题行

```

```

            cell.number_format = '@'

```

```

print("\n    处理完成！ ")

```

```

print(f"    • 输出文件: {output_file}")

```

```

print(f"    • 总数据量: {len(df_output):,} 行")

```

```

print(f"    • 输出列数: {len(final_column_order)} 列")
print(f"    • 新增借款、涉诉、违规处理、数字技术专利相关列缺失值已填补为 0:
{len(loan_columns)} 列")

if filled_count > 0:
    print(f"    • 共填补 {filled_count:,} 个缺失值")
else:
    print("    • 未发现缺失值（所有借款列已完整）")

print("\n    提示：输出文件包含完整数据，仅对新增借款、涉诉、违规处理、数字技
术专利相关数据进行了 0 值填补，股票代码已保持 6 位文本格式。")

if __name__ == "__main__":
    try:
        main()
    except Exception as e:
        print(f"\n    程序运行出错: {str(e)}")

        import traceback
        traceback.print_exc()
        sys.exit(1)

```

二十五、新增机制分析并表代码 20260303

http://localhost:8888/notebooks/Desktop/Python/%E6%AF%95%E4%B8%9A%E8%AE%BA%E6%96%87_%E6%9C%BA%E5%88%B6%E5%88%86%E6%9E%90%E6%96%B0%E5%A2%9E%E5%8F%98%E9%87%8F%E5%90%88%E5%B9%B6%E4%BB%A3%E7%A0%81_20260303.ipynb

注：1、所有代码运行前需把 excel 文件放到桌面上；2、代码中的路径只适用于 mac 系统，windows 系统需要对路径代码进行修改；3、需要把桌面上的文件名称修改至与代码中完全相同。

Kimi 指令：

我现在需要你帮我设计一段 python 代码：

一、现有数据：

我现在有 7 个 excel 文件需要合并。这些文件均位于 mac 桌面上命名为 31 机制分析新增变量_20260303 的文件夹中。

7 个 excel 文件的名称和内容分别如下：

1、“17 合并数据（缺失值处理）_20260220_145226.xlsx”。记录了 2013-2024 年 A 股上市公司各年份多种数据。表格第一行为中文标题，从左到右依次为：股票代码、会计年度、数据资产信息披露水平、信贷可得性、行业名称、行业代码、上市年限、企业规模、总资产净利润率(ROA)A、资产负债率、账面市值比 A、账面市值比 B、营业收入增长率、是否亏损、董事会规模、股权集中指标 1（%）、独立董事占比、两职合一、股权性质虚拟变量、是否四大、替换解释变量、新增借款（万元）、新增信用贷款（万元）、新增质押贷款（万元）、新增保证借款（万元）、新增抵押贷款（万元）、新增担保贷款（万元）、新增其他授信项（万元）、本年新增短期借款合计（万元）、本年新增长期借款合计（万元）、被分析师关注度、资金流动性、HHI、DigEdu_Executive、涉诉次数、涉诉金额、是否受到违规处理 1、违规处理次数 1、是否受到违规处理 2、违规处理次数 2、内部控制是否有效、内部控制是否存在缺陷、关键数字技术_发明专利申请总数、关键数字技术_发明专利获得总数、关键数字技术_实用新型专利申请总数、关键数字技术_实用新型专利获得总数、期末数字资产强度、归属于上市公司股东的扣除非经常性损益的净利润、归属于上市公司股东的扣除非经常性损益的净利润（滞后一期）、真实盈余管理、DD 模型残差绝对值、操控性应计利润的绝对值（元）、非线性应计模型残差绝对值。

2、“36-1 有形资产比率.xlsx”。记录了 2013-2024 年 A 股上市公司各年份有形资产比率（滞后一期）数据。表格第一行为中文标题，从左到右依次为：股票代码、股票简称、会计年度、会计年度（滞后前）、统计截止日期、报表类型编码、公告来源、流动资产比率、现金资产比率、有形资产比率。

3、“36-2 有形资产比率 2.xlsx”。记录了 2013-2024 年 A 股上市公司各年份有形资产比率 2（滞后一期）数据。表格第一行为中文标题，从左到右依次为：股票代码、证券简称、会计年度、会计年度（滞后前）、统计截止日期、报表类型、存货净额（元）、固定资产净额（元）、资产总计（元）、有形资产比率 2。

4、“37 固定资产净值占比.xlsx”。记录了 2013-2024 年 A 股上市公司各年份固定资产净值占比（滞后一期）数据。表格第一行为中文标题，从左到右依次为：股票代码、会计年度、

固定资产净值（元）、资产总计（元）、固定资产净值占比。

5、“38 现金持有.xlsx”。记录了 2013-2024 年 A 股上市公司各年份现金持有（滞后一期）数据。表格第一行为中文标题，从左到右依次为：股票代码、会计年度、货币资金（元）、交易性金融资产（元）、资产总计（元）、现金及现金等价物（元）、现金持有。

6、“39 流动比率.xlsx”。记录了 2013-2024 年 A 股上市公司各年份流动比率数据。表格第一行为中文标题，从左到右依次为：股票代码、股票简称、会计年度、会计年度（滞后前）、统计截止日期、报表类型编码、公告来源、流动比率、速动比率、现金比率、经营活动产生的现金流量净额 / 流动负债、现金流利息保障倍数、经营活动产生的现金流量净额 / 负债合计、经营活动产生的现金流量净额 / 带息债务。

7、“40 审计费用率.xlsx”。记录了 2013-2024 年 A 股上市公司各年份审计费用率数据。表格第一行为中文标题，从左到右依次为：股票代码、会计年度、审计费用（元）、资产总计（元）、审计费用率。

二、我的需求

1、任务目标：将以上表格的目标列按照（股票代码、会计年度）两列作为索引进行合并，形成一张完整的用于 STATA 回归的表格

2、逻辑：（1）这 7 张原始数据表格记录的都是 A 股所有上市公司 2013-2024 年的数据，所以需要合并的列共用（股票代码，会计年度）作为行索引，最终得到的合并表格中，各个公司、各个年份分别只有一行数据，这一行除了股票代码、会计年度的（51+6）=57 列分别对应 7 张原始表格中我要求你提取出来的列；（2）由于不同原始数据的统计内容不同，所以缺失行也不同，最终的合并表格中的（股票代码、会计年度）两列，需要统计出在 7 张原始表格中凡是出现过的（股票代码，会计年度）组合。即取不重复的（股票代码，会计年度）的并集作为行索引

3、具体操作：

（1）读取位于 mac 桌面上命名为 31 机制分析新增变量_20260303 的文件夹中所有 7 张 excel 里的数据（excel 命名见现有数据部分）。注意，由于股票代码存在左侧数据为 0 的情况，所以一定要始终以字符串的格式储存，保证长度一直为 6 位。

（2）将 7 张表格中的数据以（股票代码，会计年度）为关键词合并。其中 7 张表格中除（股票代码，会计年度）之外，需要并入最终得到的合并表格中的数据如下：

1、“17 合并数据（缺失值处理）_20260220_145226.xlsx”：数据资产信息披露水平、信贷可得性、行业名称、行业代码、上市年限、企业规模、总资产净利润率(ROA)A、资产负债率、账面市值比 A、账面市值比 B、营业收入增长率、是否亏损、董事会规模、股权集中指标 1（%）、独立董事占比、两职合一、股权性质虚拟变量、是否四大、替换解释变量、新增借款（万元）、新增信用贷款（万元）、新增质押贷款（万元）、新增保证借款（万元）、新增抵押贷款（万元）、新增担保贷款（万元）、新增其他授信项（万元）、本年新增短期借款合计（万元）、本年新增长期借款合计（万元）、被分析师关注度、资金流动性、HHI、DigEdu_Executive、涉诉次数、涉诉金额、是否受到违规处理 1、违规处理次数 1、是否受到违规处理 2、违规处理次数 2、内部控制是否有效、内部控制是否存在缺陷、关键数字技术_发明专利申请总数、关键数字技术_发明专利获得总数、关键数字技术_实用新型专利申请总数、关键数字技术_实用新型专利获得总数、期末数字资产强度、归属于上市公司股东的扣除非经常性损益的净利润、归属于上市公司股东的扣除非经常性损益的净利润（滞后一期）、真实盈余管理、DD 模型残差绝对值、操控性应计利润的绝对值（元）、非线性应计

模型残差绝对值。

- 2、“36-1 有形资产比率.xlsx”：有形资产比率。
- 3、“36-2 有形资产比率 2.xlsx”：有形资产比率 2。
- 4、“37 固定资产净值占比.xlsx”：固定资产净值占比。
- 5、“38 现金持有.xlsx”：现金持有。
- 6、“39 流动比率.xlsx”：流动比率。
- 7、“40 审计费用率.xlsx”：审计费用率。

8、(3)输出结果,以 excel 的形式保存到 mac 的桌面,命名为:合并数据_对应的时间戳.xlsx。其中,这个 excel 的从左到右每一列的列标题依次为:股票代码、会计年度、数据资产信息披露水平、信贷可得性、行业名称、行业代码、上市年限、企业规模、总资产净利润率(ROA)A、资产负债率、账面市值比 A、账面市值比 B、营业收入增长率、是否亏损、董事会规模、股权集中指标 1 (%)、独立董事占比、两职合一、股权性质虚拟变量、是否四大、替换解释变量、新增借款(万元)、新增信用贷款(万元)、新增质押贷款(万元)、新增保证借款(万元)、新增抵押贷款(万元)、新增担保贷款(万元)、新增其他授信项(万元)、本年新增短期借款合计(万元)、本年新增长期借款合计(万元)、被分析师关注度、资金流动性、HHI、DigEdu_Executive、涉诉次数、涉诉金额、是否受到违规处理 1、违规处理次数 1、是否受到违规处理 2、违规处理次数 2、内部控制是否有效、内部控制是否存在缺陷、关键数字技术_发明专利申请总数、关键数字技术_发明专利获得总数、关键数字技术_实用新型专利申请总数、关键数字技术_实用新型专利获得总数、期末数字资产强度、归属于上市公司股东的扣除非经常性损益的净利润、归属于上市公司股东的扣除非经常性损益的净利润(滞后一期)、真实盈余管理、DD 模型残差绝对值、操控性应计利润的绝对值(元)、非线性应计模型残差绝对值、有形资产比率、有形资产比率 2、固定资产净值占比、现金持有、流动比率、审计费用率。并且在表格的最上方添加列标题行。

注意: (1) 鉴于使用循环会导致代码运行效率低下,你可以给出可行的替代方案优化程序;
(2) 我希望这个代码可以实时显示运行进度。

```
pip install pandas tqdm openpyxl
```

```
import pandas as pd
from pathlib import Path
from datetime import datetime
from functools import reduce
from tqdm import tqdm
```

```
def merge_mechanism_data():
    """
    合并 7 个机制分析 Excel 文件, 以 (股票代码、会计年度) 为键建立并集
    修正版: 处理列名差异 (会计年度 vs 会计年度 (滞后前))
    """
```

```

# ===== 1. 路径配置 =====
desktop_path = Path.home() / "Desktop"
input_folder = desktop_path / "31 机制分析新增变量_20260303"

if not input_folder.exists():
    raise FileNotFoundError(f'文件夹不存在: {input_folder}\n 请确保文件夹位于 Mac 桌面, 且名称完全匹配 (含空格) ')

timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')
output_file = desktop_path / f'合并数据_{timestamp}.xlsx'

print(f'开始处理数据...')
print(f'输入文件夹: {input_folder}')
print(f'输出文件: {output_file}')

# ===== 2. 定义文件及列配置 =====
# 注意: 文件 2、3、6 的年度列名为"会计年度 (滞后前)", 需特殊处理
file_configs = [
    {
        'file_name': '17 合并数据 (缺失值处理) _20260220_145226.xlsx',
        'year_col': '会计年度', # 原始列名
        'columns': [
            '股票代码', '会计年度', '数据资产信息披露水平', '信贷可得性',
            '行业名称', '行业代码', '上市年限', '企业规模', '总资产净利润率(ROA)A',
            '资产负债率', '账面市值比 A', '账面市值比 B', '营业收入增长率', '是否亏损',
            '董事会规模', '股权集中指标 1(%)', '独立董事占比', '两职合一',
            '股权性质虚拟变量', '是否四大', '替换解释变量', '新增借款 (万元)',
            '新增信用贷款 (万元)', '新增质押贷款 (万元)', '新增保证借款 (万元)',
            '新增抵押贷款 (万元)', '新增担保贷款 (万元)', '新增其他授信项 (万元)',
            '本年新增短期借款合计 (万元)', '本年新增长期借款合计 (万元)',
            '被分析师关注度', '资金流动性', 'HHI', 'DigEdu_Executive', '涉诉次数',
            '涉诉金额', '是否受到违规处理 1', '违规处理次数 1', '是否受到违规处理 2',
            '违规处理次数 2', '内部控制是否有效', '内部控制是否存在缺陷',
            '关键数字技术_发明专利申请总数', '关键数字技术_发明专利获得总数',
            '关键数字技术_实用新型专利申请总数', '关键数字技术_实用新型专利获得总数',
            '期末数字资产强度', '归属于上市公司股东的扣除非经常性损益的净利润',
            '归属于上市公司股东的扣除非经常性损益的净利润 (滞后一期)', '真实盈余管理',
            'DD 模型残差绝对值', '操控性应计利润的绝对值 (元)', '非线性应计模型

```

残差绝对值'

```
]
    },
    {
        'file_name': '36-1 有形资产比率.xlsx',
        'year_col': '会计年度 (滞后前)', # 注意: 原始列为滞后前
        'columns': ['股票代码', '会计年度 (滞后前)', '有形资产比率']
    },
    {
        'file_name': '36-2 有形资产比率 2.xlsx',
        'year_col': '会计年度 (滞后前)',
        'columns': ['股票代码', '会计年度 (滞后前)', '有形资产比率 2']
    },
    {
        'file_name': '37 固定资产净值占比.xlsx',
        'year_col': '会计年度',
        'columns': ['股票代码', '会计年度', '固定资产净值占比']
    },
    {
        'file_name': '38 现金持有.xlsx',
        'year_col': '会计年度',
        'columns': ['股票代码', '会计年度', '现金持有']
    },
    {
        'file_name': '39 流动比率.xlsx',
        'year_col': '会计年度 (滞后前)',
        'columns': ['股票代码', '会计年度 (滞后前)', '流动比率']
    },
    {
        'file_name': '40 审计费用率.xlsx',
        'year_col': '会计年度',
        'columns': ['股票代码', '会计年度', '审计费用率']
    }
]

# ===== 3. 读取数据 (带进度条) =====
print("\n 正在读取 7 个 Excel 文件...")
dataframes = []

for config in tqdm(file_configs, desc="文件读取进度", unit="file"):
    file_path = input_folder / config['file_name']

    try:
        # 只读取需要的列, 减少内存占用
```

```

df = pd.read_excel(file_path, usecols=config['columns'])

# 关键：统一列名 - 将"会计年度（滞后前）"重命名为"会计年度"
if config['year_col'] == '会计年度（滞后前）':
    df.rename(columns={'会计年度（滞后前）': '会计年度'}, inplace=True)

# 关键：股票代码格式化处理（字符串类型，左侧补零至 6 位）
# 先转字符串，去除可能的.xlsx 科学计数法影响，再补零
df['股票代码'] = df['股票代码'].astype(str).str.strip().str.replace('.', '',
regex=False).str.zfill(6)

# 会计年度处理：先转字符串去除小数，再转整数（处理缺失值保护）
df['会计年度'] = pd.to_numeric(df['会计年度'], errors='coerce')
# 检查是否有无效年份
invalid_years = df['会计年度'].isna().sum()
if invalid_years > 0:
    tqdm.write(f" 警告: {config['file_name']} 中发现 {invalid_years} 个无效
年份，已转为缺失值")

# 去重：防止原始数据中存在重复的（股票代码，会计年度）组合
before_drop = len(df)
df = df.drop_duplicates(subset=['股票代码', '会计年度'], keep='first')
after_drop = len(df)
if before_drop != after_drop:
    tqdm.write(f" 提示: 删除 {before_drop - after_drop} 个重复行")

dataframes.append(df)

tqdm.write(f"✓ 已读取 {config['file_name']}: {len(df)} 行, {len(df.columns)}
列")

except ValueError as e:
    # 捕获列名不匹配错误
    if "Usecols do not match columns" in str(e):
        print(f"\n✗ 错误: {config['file_name']} 中的列名与预期不符")
        print(f"  请检查 Excel 中的实际列名是否完全匹配以下列表:")
        print(f"    {config['columns']}")
        raise
    else:
        raise
except Exception as e:
    print(f"\n✗ 读取文件 {config['file_name']} 时出错: {str(e)}")

```

```

raise

# ===== 4. 高效合并（并集） =====
print("\n 正在执行多表合并（外连接/Outer Join）...")
print("合并策略：以（股票代码，会计年度）为键，取 7 张表的并集")

# 使用 reduce 进行链式合并
# 每次 merge 使用 outer join 确保并集逻辑
merged_df = reduce(
    lambda left, right: pd.merge(
        left, right,
        on=['股票代码', '会计年度'],
        how='outer',
        suffixes=('_', '_dup')
    ),
    dataframes
)

# 删除可能的重复后缀列（防御性编程）
dup_cols = [col for col in merged_df.columns if col.endswith('_dup')]
if dup_cols:
    merged_df = merged_df.drop(columns=dup_cols)
    print(f"已清理 {len(dup_cols)} 个重复列")

print(f"合并完成：共 {len(merged_df)} 行，{len(merged_df.columns)} 列")

# ===== 5. 按指定顺序排列列（59 列） =====
# 严格按照您要求的输出顺序：2 个键 + 51 个文件 1 变量 + 6 个机制变量 = 59 列
final_column_order = [
    '股票代码', '会计年度',
    # 文件 1 的 51 个数据列（严格按您指定的顺序）
    '数据资产信息披露水平', '信贷可得性', '行业名称', '行业代码', '上市年限',
    '企业规模', '总资产净利润率(ROA)A', '资产负债率', '账面市值比 A', '账面市值比 B',
    '营业收入增长率', '是否亏损', '董事会规模', '股权集中指标 1(%)', '独立董事占比',
    '两职合一', '股权性质虚拟变量', '是否四大', '替换解释变量', '新增借款（万元）',
    '新增信用贷款（万元）', '新增质押贷款（万元）', '新增保证借款（万元）',
    '新增抵押贷款（万元）', '新增担保贷款（万元）', '新增其他授信项（万元）',
    '本年新增短期借款合计（万元）', '本年新增长期借款合计（万元）', '被分析师关注
度',
    '资金流动性', 'HHI', 'DigEdu_Executive', '涉诉次数', '涉诉金额',
    '是否受到违规处理 1', '违规处理次数 1', '是否受到违规处理 2', '违规处理次数 2',
    '内部控制是否有效', '内部控制是否存在缺陷', '关键数字技术_发明专利申请总数',
    '关键数字技术_发明专利获得总数', '关键数字技术_实用新型专利申请总数',
    '关键数字技术_实用新型专利获得总数', '期末数字资产强度',

```

```

'归属于上市公司股东的扣除非经常性损益的净利润',
'归属于上市公司股东的扣除非经常性损益的净利润（滞后一期）', '真实盈余管理',
'DD 模型残差绝对值', '操控性应计利润的绝对值（元）', '非线性应计模型残差绝对
值',

# 文件 2-7 的 6 个机制分析变量（按指定顺序）
'有形资产比率', '有形资产比率 2', '固定资产净值占比', '现金持有',
'流动比率', '审计费用率'
]

# 验证列名完整性
missing_cols = [col for col in final_column_order if col not in merged_df.columns]
if missing_cols:
    print(f"\n 警告： 以下 {len(missing_cols)} 个列名在合并数据中未找到:")
    for col in missing_cols:
        print(f"    - {col}")
    print("请检查原始 Excel 文件的列名是否与配置完全一致（注意全角/半角括号）")
    # 只保留存在的列
    existing_cols = [col for col in final_column_order if col in merged_df.columns]
    merged_df = merged_df[existing_cols]
else:
    merged_df = merged_df[final_column_order]
    print(f"列顺序整理完成： 共 {len(final_column_order)} 列")

# ===== 6. 数据后处理 =====
# 按股票代码和会计年度排序，便于查看
merged_df = merged_df.sort_values(by=['股票代码', '会计年度']).reset_index(drop=True)

# 显示数据完整性统计（关键变量）
print("\n 数据完整性统计（样本覆盖率）:")
key_vars = ['股票代码', '会计年度', '数据资产信息披露水平', '信贷可得性',
            '有形资产比率', '现金持有', '流动比率']
for col in key_vars:
    if col in merged_df.columns:
        non_null = merged_df[col].notna().sum()
        print(f"                {col}:                {non_null}/{len(merged_df)}
({non_null/len(merged_df)*100:.1f}%)")

# ===== 7. 保存结果 =====
print(f"\n 正在保存结果到 Excel...")
try:
    merged_df.to_excel(output_file, index=False, engine='openpyxl')
    print(f"\n{'='*60}")

    print(f"✓ 处理完成! ")

```

```

    print(f'{'='*60}')
    print(f'输出文件路径: {output_file}')
    print(f'总行数: {len(merged_df):,}')
    print(f'总列数: {len(merged_df.columns)} (股票代码 + 会计年度
+{len(merged_df.columns)-2}个变量)')
    print(f'时间戳: {timestamp}')
    print(f'{'='*60}')
except Exception as e:
    print(f'\n 保存 Excel 时出错: {e}')
    print("尝试保存为 CSV 格式...")
    csv_file = output_file.with_suffix('.csv')
    merged_df.to_csv(csv_file, index=False, encoding='utf-8-sig')
    print(f'已保存为 CSV: {csv_file}')

return merged_df

if __name__ == "__main__":
    try:
        print("=" * 60)
        print("开始执行 Excel 数据合并程序")
        print("=" * 60)
        print("依赖库检查: pandas, tqdm, openpyxl")

        df_result = merge_mechanism_data()

    except ImportError as e:
        print("\n" + "=" * 60)
        print("错误: 缺少必要的 Python 库")
        print("=" * 60)
        print("请运行以下命令安装: ")
        print("pip install pandas tqdm openpyxl")
        print(f'详细错误: {e}')
    except FileNotFoundError as e:
        print(f'\n 路径错误: {e}')
    except Exception as e:
        print(f'\n 程序执行出错: {e}')
        import traceback
        traceback.print_exc()

```

Kimi 指令:

我现在需要你帮我设计一段 python 代码:

一、现有数据:

我现在有 2 个 excel 文件需要合并。这些文件均位于 mac 桌面上。

8、第一个 excel 命名为: 17 合并数据 (缺失值处理) _20260220_145226.xlsx。表格第一行为中文标题, 从左到右分别为: 股票代码、会计年度、数据资产信息披露水平、信贷可得性、行业名称、行业代码、上市年限、企业规模、总资产净利润率(ROA)A、资产负债率、账面市值比 A、账面市值比 B、营业收入增长率、是否亏损、董事会规模、股权集中指标 1(%)、独立董事占比、两职合一、股权性质虚拟变量、是否四大、替换解释变量、新增借款 (万元)、新增信用贷款 (万元)、新增质押贷款 (万元)、新增保证借款 (万元)、新增抵押贷款 (万元)、新增担保贷款 (万元)、新增其他授信项 (万元)、本年新增短期借款合计 (万元)、本年新增长期借款合计 (万元)、被分析师关注度、资金流动性、HHI、DigEdu_Executive、涉诉次数、涉诉金额、是否受到违规处理 1、违规处理次数 1、是否受到违规处理 2、违规处理次数 2、内部控制是否有效、内部控制是否存在缺陷、关键数字技术_发明专利申请总数、关键数字技术_发明专利获得总数、关键数字技术_实用新型专利申请总数、关键数字技术_实用新型专利获得总数、期末数字资产强度、归属于上市公司股东的扣除非经常性损益的净利润、归属于上市公司股东的扣除非经常性损益的净利润 (滞后一期)、真实盈余管理、DD 模型残差绝对值、操控性应计利润的绝对值 (元)、非线性应计模型残差绝对值。

第二个 excel 命名为: 有形资产比率 2.xlsx, 记录了历年上市公司的有形资产比率数据。表格的第一行为中文标题, 从左到右依次为: 股票代码、证券简称、会计年度、统计截止日期、报表类型、存货净额 (元)、固定资产净额 (元)、资产总计 (元)、有形资产比率 2。

二、我的需求

1、任务目标: 将两张表格按照 (股票代码, 会计年度) 合并。

2、具体操作:

(1) 读取位于 mac 桌面命名为 17 合并数据 (缺失值处理) _20260220_145226.xlsx 和 有形资产比率.xlsx 的文件。注意, 由于股票代码存在左侧数据为 0 的情况, 所以一定要始终以字符串的格式储存, 保证长度一直为 6 位。

(2) 将第 (1) 步读取的两张表格按照关键词 (股票代码, 会计年度) 合并。

(3) 输出结果文件, 将第 (4) 步的结果写入一个 excel 表格, 命名为 合并数据_对应的时间戳.xlsx。其中: 从左到右每一列的列标题依次为: 股票代码、会计年度、数据资产信息披露水平、信贷可得性、行业名称、行业代码、上市年限、企业规模、总资产净利润率(ROA)A、资产负债率、账面市值比 A、账面市值比 B、营业收入增长率、是否亏损、董事会规模、股权集中指标 1(%)、独立董事占比、两职合一、股权性质虚拟变量、是否四大、替换解释变量、新增借款 (万元)、新增信用贷款 (万元)、新增质押贷款 (万元)、新增保证借款 (万元)、新增抵押贷款 (万元)、新增担保贷款 (万元)、新增其他授信项 (万元)、本年新增短期借款合计 (万元)、本年新增长期借款合计 (万元)、被分析师关注度、资金流动性、HHI、DigEdu_Executive、涉诉次数、涉诉金额、是否受到违规处理 1、违规处理次数 1、是否受到违规处理 2、违规处理次数 2、内部控制是否有效、内部控制是否存在缺陷、关键数字技术_发明专利申请总数、关键数字技术_发明专利获得总数、关键数字技术_实用新型专利申请总数、关键数字技术_实用新型专利获得总数、期末数字资产强度、归属于上市公司股东的扣除非经常性损益的净利润、归属于上市公司股东的扣除非经常性损益的净利润 (滞后一期)、真实盈余管理、DD 模型残差绝对值、操控性应计利润的绝对值 (元)、非线性应计模型残差绝对值。

后一期)、真实盈余管理、DD 模型残差绝对值、操控性应计利润的绝对值 (元)、非线性应计模型残差绝对值、有形资产比率 2。请在最上方添加标题行。

注意: (1) 鉴于循环会导致代码运行效率低下, 你可以给出可行的替代方案优化程序; (2) 我希望这个代码可以实时显示运行进度。

请仔细阅读上述文字, 并告诉我你是否已经完全理解所有要求。

```
import pandas as pd
import os
from datetime import datetime
from tqdm import tqdm
import numpy as np

# 设置 Mac 桌面路径 (根据您的环境自动适配)
desktop_path = "/Users/air/Desktop"

# ===== 辅助函数: 安全格式化股票代码 =====
def format_stock_code(x):
    """将股票代码格式化为 6 位字符串, 处理浮点数和整数情况"""
    if pd.isna(x):
        return "
    # 先转字符串, 去除可能的.0 后缀, 再补零到 6 位
    return str(int(float(x))) if str(x).replace('.', '', 1).isdigit() else str(x).zfill(6)

def format_year(x):
    """格式化会计年度, 处理浮点数情况"""
    if pd.isna(x):
        return "
    return str(int(float(x))) if str(x).replace('.', '', 1).isdigit() else str(x)

# ===== 进度显示配置 =====

print(" 开始执行数据合并任务...")

print("=" * 60)
start_time = datetime.now()

# ===== 1. 读取第一个 Excel 文件 =====

print("\n 步骤 1/4: 读取主数据文件...")

file1_name = "17 合并数据 (缺失值处理) _20260220_145226.xlsx"
file1_path = os.path.join(desktop_path, file1_name)
```

```

# 检查文件是否存在
if not os.path.exists(file1_path):

    raise FileNotFoundError(f"    文件未找到: {file1_path}\n 请确认文件名是否正确并位于
    桌面")

# 使用 tqdm 显示进度 (模拟进度, 因 pandas 读取不支持实时回调)
with tqdm(total=100, desc=" 读取主数据 ", bar_format='{l_bar}{bar}| {n_fmt}/{total_fmt}
[{elapsed}<{remaining}]') as pbar:
    df_main = pd.read_excel(file1_path, engine='openpyxl')
    pbar.update(100)

    # 关键修正: 处理股票代码格式 (防止浮点数.0 后缀, 确保 6 位)
    df_main['股票代码'] = df_main['股票代码'].apply(format_stock_code)
    df_main['会计年度'] = df_main['会计年度'].apply(format_year)

print(f"    ✓ 主数据读取完成, 共 {len(df_main):,} 行, {len(df_main.columns)} 列")

print(f"    ✓ 股票代码示例: {df_main['股票代码'].iloc[0]} (已格式化为 6 位字符串)")

# ===== 2. 读取第二个 Excel 文件 =====

print("\n    步骤 2/4: 读取有形资产比率数据...")

file2_name = "有形资产比率 2.xlsx"
file2_path = os.path.join(desktop_path, file2_name)

if not os.path.exists(file2_path):

    raise FileNotFoundError(f"    文件未找到: {file2_path}")

with tqdm(total=100, desc=" 读取比率数据 ", bar_format='{l_bar}{bar}| {n_fmt}/{total_fmt}
[{elapsed}<{remaining}]') as pbar:
    df_ratio = pd.read_excel(file2_path, engine='openpyxl')
    pbar.update(100)

    # 同样处理股票代码格式
    df_ratio['股票代码'] = df_ratio['股票代码'].apply(format_stock_code)
    df_ratio['会计年度'] = df_ratio['会计年度'].apply(format_year)

    # 关键: 检查并处理重复键值 (避免合并后行数膨胀)
    duplicates = df_ratio.groupby(['股票代码', '会计年度']).size()
    duplicates = duplicates[duplicates > 1]
    if len(duplicates) > 0:

```

```

        print(f"        警告: 发现 {len(duplicates)} 个重复的股票代码-年度组合, 将保留
第一条记录")
        df_ratio = df_ratio.drop_duplicates(subset=['股票代码', '会计年度'], keep='first')

print(f"    ✓ 比率数据读取完成, 共 {len(df_ratio):,} 行")

# ===== 3. 数据合并 (向量化操作) =====

print("\n    步骤 3/4: 执行数据合并...")
print("    正在按照 (股票代码, 会计年度) 进行左连接合并...")

# 只保留需要的列, 减少内存占用
if '有形资产比率 2' not in df_ratio.columns:

    raise ValueError(f"    第二个文件中未找到列 '有形资产比率 2', 可用列:
{list(df_ratio.columns)}")

ratio_cols = ['股票代码', '会计年度', '有形资产比率 2']
df_ratio_filtered = df_ratio[ratio_cols].copy()

with tqdm(total=100, desc=" 合 并 数 据 ", bar_format='{l_bar}{bar}| {n_fmt}/{total_fmt}
[{elapsed}<{remaining}]]') as pbar:
    # 使用 merge 进行向量化合并 (远快于循环)
    df_merged = df_main.merge(
        df_ratio_filtered,
        on=['股票代码', '会计年度'],
        how='left',
        indicator=False
    )
    pbar.update(100)

# 检查合并结果
matched_count = df_merged['有形资产比率 2'].notna().sum()
match_rate = (matched_count / len(df_merged)) * 100

print(f"    ✓ 合并完成, 结果数据共 {len(df_merged):,} 行")

print(f"    ✓ 成功匹配 {matched_count:,} 条记录 (匹配率: {match_rate:.2f}%) ")

if match_rate < 10:

    print(f"        警告: 匹配率过低, 请检查两个文件的股票代码格式是否一致 (如是否

```

有前导零) ")

===== 4. 调整列顺序 =====

print("\n 步骤 4/4: 调整列顺序并准备输出...")

定义输出列顺序 (严格按照您的要求)

```
output_columns = [  
    '股票代码', '会计年度', '数据资产信息披露水平', '信贷可得性',  
    '行业名称', '行业代码', '上市年限', '企业规模', '总资产净利润率(ROA)A',  
    '资产负债率', '账面市值比 A', '账面市值比 B', '营业收入增长率', '是否亏损',  
    '董事会规模', '股权集中指标 1(%)', '独立董事占比', '两职合一', '股权性质虚拟变量',  
    '是否四大', '替换解释变量', '新增借款 (万元) ', '新增信用贷款 (万元) ',  
    '新增质押贷款 (万元) ', '新增保证借款 (万元) ', '新增抵押贷款 (万元) ',  
    '新增担保贷款 (万元) ', '新增其他授信项 (万元) ', '本年新增短期借款合计 (万元) ',  
    '本年新增长期借款合计 (万元) ', '被分析师关注度', '资金流动性', 'HHI',  
    'DigEdu_Executive', '涉诉次数', '涉诉金额', '是否受到违规处理 1', '违规处理次数 1',  
    '是否受到违规处理 2', '违规处理次数 2', '内部控制是否有效', '内部控制是否存在缺陷',  
    '关键数字技术_发明专利申请总数', '关键数字技术_发明专利获得总数',  
    '关键数字技术_实用新型专利申请总数', '关键数字技术_实用新型专利获得总数',  
    '期末数字资产强度', '归属于上市公司股东的扣除非经常性损益的净利润',  
    '归属于上市公司股东的扣除非经常性损益的净利润 (滞后一期) ', '真实盈余管理',  
    'DD 模型残差绝对值', '操控性应计利润的绝对值 (元) ', '非线性应计模型残差绝对值',  
    '有形资产比率 2'  
]
```

检查列是否存在

```
missing_cols = [col for col in output_columns if col not in df_merged.columns]
```

if missing_cols:

```
    print(f"      警告: 以下列在数据中未找到, 将从输出中排除: {missing_cols}")
```

```
    output_columns = [col for col in output_columns if col in df_merged.columns]
```

确保所有主数据列都在 (防止遗漏)

```
for col in df_main.columns:
```

```
    if col not in output_columns and col != '有形资产比率 2': # 排除即将添加的合并列  
        output_columns.append(col)
```

调整列顺序

```
df_output = df_merged[output_columns]
```

===== 5. 输出结果 =====

```
print("\n 正在写入 Excel 文件...")
```

```

# 生成带时间戳的文件名
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
output_filename = f'合并数据_{timestamp}.xlsx'
output_path = os.path.join(desktop_path, output_filename)

# 分块写入以显示进度（每 1000 行为一个区块）
chunk_size = 1000
total_rows = len(df_output)

with pd.ExcelWriter(output_path, engine='xlsxwriter') as writer:
    # 写入数据（分块处理大文件，虽然 pandas 内部优化已很好，此处为显示进度）
    df_output.to_excel(writer, index=False, sheet_name='合并数据', startrow=0)

    # 获取工作簿和工作表对象
    workbook = writer.book
    worksheet = writer.sheets['合并数据']

    # 设置标题行格式（加粗、背景色、边框）
    header_format = workbook.add_format({
        'bold': True,
        'bg_color': '#D9E1F2',
        'border': 1,
        'align': 'center',
        'valign': 'vcenter',
        'text_wrap': True # 自动换行
    })

    # 应用标题格式并自动调整列宽
    for col_num, value in enumerate(df_output.columns.values):
        worksheet.write(0, col_num, value, header_format)
        # 根据内容长度自动设置列宽（限制最大宽度 30 避免过宽）
        max_len = max(
            df_output[value].astype(str).map(len).max(),
            len(str(value))
        ) + 2
        col_width = min(max_len, 30)
        worksheet.set_column(col_num, col_num, col_width)

    # 使用 tqdm 显示写入进度（模拟）
    with tqdm(total=total_rows, desc="写入 Excel", unit="行",
              bar_format='{l_bar} {bar}| {n_fmt}/{total_fmt} [{elapsed}<{remaining}]') as
pbar:
    pbar.update(total_rows) # pandas 已写完，此处仅为显示

```

```

elapsed_time = datetime.now() - start_time

print(f"\n" + "=" * 60)

print(f"    任务完成! 耗时: {elapsed_time.seconds}秒")

print(f"    输出文件: {output_path}")

print(f"    总行数: {len(df_output):,}")

print(f"    总列数: {len(df_output.columns)}")

print(f"    匹配成功率: {match_rate:.2f}%")

print(f"    提示: 若匹配率异常, 请检查两个 Excel 的'股票代码'是否为文本格式且无前导空
格")
print("=" * 60)

# 输出数据样例供验证

print("\n    数据验证 (前 3 行关键列) :")

print(df_output[['股票代码', '会计年度', '数据资产信息披露水平', '有形资产比率 2']].head(3))

```